



CompTIA

DeepDumps

Premium exam material

Get certification quickly with the **Deep Dumps** Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily.

<https://www.DeepDumps.com>

About DeepDumps

At DeepDumps, we finally had enough of the overpriced, paywall-ridden exam prep industry. Our team of experienced IT professionals spent years navigating the certification landscape, only to realize that most prep materials came with a price tag bigger than the exam itself. We saw aspiring professionals spending more on study guides than on rent, and frankly, it was ridiculous. So, we decided to change the game. DeepDumps was born with one goal: to provide high-quality exam preparation without draining your wallet.

With DeepDumps, you get expert-vetted study materials, real-world insights, and a fair shot at success—without the financial headache..

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://DeepDumps.com>

[Mail us on - support@deepdumps.com](mailto:support@deepdumps.com)



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John



Thank you so much for your help. I scored 972 in my exam today. More than 30 questions were there from your PDFs!

Dana



Thanks a lot for this updated AZ-900 Q&A. I just finished my exam and got 374. I followed both of your AZ-900 videos and the 6 PDF, the PDFs are very accurate, all answers are correct. Could you please create a similar video/PDF for DP900, your content-/PDF's is really awesome. The tem'd a really good job. Thank You 🙏

Ahamod Shibly



Customer support is really fast and friendly, I just finished with me exam I passed it along with the 6 PDF's you

Passed my exam today with 845 marks



Passed my exam today with 845 marks! Out of 52 questions, 51 were from your DeepDumps PDFs including Contoso case study. I will recommend you to my friends. Thank you DeepDumps team!

These questions are real and 100 % valid.



Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria



Simple, easy to understand explanations. To anyone who is writing the exam of AZ900,

Databricks

(Certified Data Engineer Associate)

Certified Data Engineer Associate

Total: **246 Questions**

Link: <https://deepdumps.com/papers/databricks/certified-data-engineer-associate>

Question: 1**Deep Dumps**

A data organization leader is upset about the data analysis team's reports being different from the data engineering team's reports. The leader believes the siloed nature of their organization's data engineering and data analysis architectures is to blame.

Which of the following describes how a data lakehouse could alleviate this issue?

- A. Both teams would autoscale their work as data size evolves
- B. Both teams would use the same source of truth for their work
- C. Both teams would reorganize to report to the same department
- D. Both teams would be able to collaborate on projects in real-time
- E. Both teams would respond more quickly to ad-hoc requests

Answer: B

Explanation:

The correct answer is B: Both teams would use the same source of truth for their work.

A data lakehouse architecture unifies the best aspects of data lakes and data warehouses. Critically, it provides a single, consistent source of truth for both data engineers and data analysts. Currently, the leader notes discrepancies between reports generated by the data analysis and data engineering teams, suggesting these teams are operating on different data sets or transformations. A data lakehouse eliminates this discrepancy by providing a single repository for all data, accessible to both teams.

With a data lakehouse, data is ingested, transformed, and stored in a way that is consistent and accessible to all stakeholders. Data engineers can focus on building pipelines to ingest and transform data, while data analysts can use the same data for reporting and analysis. This eliminates the need for data to be copied or transformed multiple times, reducing the risk of errors and inconsistencies. Options A, C, D, and E are advantages of cloud architectures or organizational structures but don't directly address the core issue of differing reports due to separate data sources. The centralization of data and metadata within a unified platform is the key benefit a data lakehouse offers in resolving the leader's concern.

For further information, explore these resources:

1. Databricks Lakehouse Platform: <https://www.databricks.com/product/data-lakehouse>
2. What is a Data Lakehouse?: <https://aws.amazon.com/what-is/data-lakehouse/>

Question: 2**Deep Dumps**

Which of the following describes a scenario in which a data team will want to utilize cluster pools?

- A. An automated report needs to be refreshed as quickly as possible.
- B. An automated report needs to be made reproducible.
- C. An automated report needs to be tested to identify errors.
- D. An automated report needs to be version-controlled across multiple collaborators.
- E. An automated report needs to be runnable by all stakeholders.

Answer: A

Explanation:

The correct answer is A, "An automated report needs to be refreshed as quickly as possible." Here's why:

Cluster pools significantly reduce the time it takes to start a Databricks cluster. This is because pre-allocated, idle instances are maintained in the pool. When a job, like refreshing an automated report, requires a cluster, it can attach to an instance from the pool almost instantly, bypassing the usual cluster provisioning time. This dramatically improves the speed of the refresh process, meeting the requirement of getting the report refreshed "as quickly as possible."

Option B (reproducible reports) is more about consistent environments and configurations, which can be achieved through Databricks Repos, Delta Lake time travel, and specifying library versions. Option C (testing to identify errors) involves testing frameworks and unit testing best practices within the Databricks environment. Option D (version control) relies on Git integration via Databricks Repos. Option E (runnable by all stakeholders) is more about permissions and access control within Databricks and how you share notebooks/dashboards.

Cluster pools are primarily designed to accelerate cluster startup, focusing on speed rather than the features described in the other options. They offer a performance advantage when cluster initialization time is a bottleneck. Other approaches, like using serverless compute, could also provide rapid startup, but cluster pools are a direct solution offered within the Databricks environment for this specific problem.

Further reading on Databricks Cluster Pools can be found here:

<https://docs.databricks.com/clusters/instance-pools/index.html> and

<https://www.databricks.com/blog/2020/07/08/introducing-instance-pools-reduce-cluster-start-times-and-cloud-costs.html>.

Question: 3

Deep Dumps

Which of the following is hosted completely in the control plane of the classic Databricks architecture?

- A. Worker node
- B. JDBC data source
- C. Databricks web application
- D. Databricks Filesystem
- E. Driver node

Answer: C

Explanation:

The Databricks architecture separates the control plane and the data plane. The control plane manages the overall Databricks environment, while the data plane is where the data processing occurs.

Option A, Worker nodes, reside in the data plane. They are virtual machines that execute the distributed computations.

Option B, JDBC data sources, are external systems, and their location is independent of the Databricks control plane. JDBC connections are configured within Databricks, but the database itself (the data source) resides outside.

Option C, the Databricks web application, is hosted entirely within the control plane. It is the user interface for interacting with Databricks, managing clusters, notebooks, jobs, and other administrative tasks. The control plane handles the front-end and back-end services to manage the Databricks environment. This includes features like workspace administration, security, user management, and job scheduling, which are all core elements of the control plane's responsibility.

Option D, the Databricks File System (DBFS), while accessible through the control plane, has its underlying

storage typically residing within the data plane in cloud storage services such as AWS S3 or Azure Blob Storage. So it is not completely hosted in the control plane.

Option E, the Driver node, although interacting closely with the control plane for job submission and coordination, is part of the data plane as it is responsible for executing the main application code and distributing tasks to worker nodes.

Therefore, only the Databricks web application is fully hosted within the control plane, making option C the correct answer. The control plane is responsible for orchestration, monitoring, and management, and the web application serves as the primary interface for these functions.

Further research:

Databricks Architecture: <https://docs.databricks.com/en/getting-started/concepts.html>

Question: 4

Deep Dumps

Which of the following benefits of using the Databricks Lakehouse Platform is provided by Delta Lake?

- A. The ability to manipulate the same data using a variety of languages
- B. The ability to collaborate in real time on a single notebook
- C. The ability to set up alerts for query failures
- D. The ability to support batch and streaming workloads
- E. The ability to distribute complex data operations

Answer: D

Explanation:

The correct answer is D: The ability to support batch and streaming workloads. Delta Lake is a storage layer that brings reliability to data lakes. A key characteristic of Delta Lake is its ACID (Atomicity, Consistency, Isolation, Durability) transaction support. This enables Delta Lake to handle both batch and streaming data reliably. Batch processing involves large volumes of data processed at once, whereas streaming processes data in near real-time. Delta Lake's ability to handle both types of workloads provides flexibility. Options A, B, and C are features of the Databricks platform but aren't directly enabled by Delta Lake. Option E describes a general function that could be executed on the data itself, not a fundamental benefit provided by Delta Lake. The ability to support both batch and streaming is a direct feature of the Delta Lake storage layer.

Further Research:

Databricks Delta Lake: <https://www.databricks.com/product/delta-lake>
Delta Lake Documentation: <https://docs.databricks.com/delta/index.html>

Question: 5

Deep Dumps

Which of the following describes the storage organization of a Delta table?

- A. Delta tables are stored in a single file that contains data, history, metadata, and other attributes.
- B. Delta tables store their data in a single file and all metadata in a collection of files in a separate location.
- C. Delta tables are stored in a collection of files that contain data, history, metadata, and other attributes.
- D. Delta tables are stored in a collection of files that contain only the data stored within the table.
- E. Delta tables are stored in a single file that contains only the data stored within the table.

Answer: C

Explanation:

The correct answer is C because Delta Lake, the technology behind Delta tables, is built on top of cloud storage like AWS S3, Azure Blob Storage, or Google Cloud Storage. It leverages the distributed nature of these object storage systems for scalability and reliability.

Delta Lake doesn't store all data in a single file. Instead, data is partitioned into multiple data files (typically Parquet format), and these files are stored in the underlying cloud storage. This allows for parallel processing and efficient querying.

The "history" and "metadata" are critical components of Delta Lake's functionality, enabling features like time travel, ACID transactions, and schema evolution. This metadata isn't stored within each data file but rather as a transaction log alongside the data files. The transaction log is an ordered record of every operation performed on the Delta table.

Specifically, each commit to a Delta table creates a new entry (a metadata file) in the `_delta_log` directory, which is typically located alongside the data files in the same storage location. This log tracks changes to the data, including additions, deletions, and updates. These metadata files (often stored in JSON format) contain information about the data files involved in each transaction, enabling Delta Lake to track the table's state over time.

Therefore, a Delta table isn't a single file. It's a directory containing data files and a `_delta_log` directory containing metadata files. Answer C accurately reflects this distributed storage model, encompassing both the data and the necessary metadata for transaction management and data versioning. The distributed storage enables efficient processing and scaling provided by the underlying cloud storage layer. Options A, B, D, and E are incorrect because they imply a single-file storage or an incomplete view of how metadata is managed within a Delta table.

Here are some authoritative links for further research:

1. **Delta Lake Documentation (Databricks):** <https://docs.databricks.com/delta/index.html> - This is the official documentation for Delta Lake and contains detailed information on its architecture and features.
2. **Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores:** <https://databricks.com/blog/2019/04/24/delta-lake-high-performance-acid-table-storage-over-cloud-object-stores.html> - Databricks blog explaining the architecture and benefits of Delta Lake.
3. **Understanding the Delta Lake Transaction Log:** <https://medium.com/datascale/understanding-the-delta-lake-transaction-log-416503e29f64> - A deep dive into how Delta Lake manages transactions using the transaction log.

Question: 6

Deep Dumps

Which of the following code blocks will remove the rows where the value in column age is greater than 25 from the existing Delta table `my_table` and save the updated table?

- A. `SELECT * FROM my_table WHERE age > 25;`
- B. `UPDATE my_table WHERE age > 25;`
- C. `DELETE FROM my_table WHERE age > 25;`
- D. `UPDATE my_table WHERE age <= 25;`
- E. `DELETE FROM my_table WHERE age <= 25;`

Answer: C

Explanation:

The correct answer is C, `DELETE FROM my_table WHERE age > 25;`. This SQL command directly and efficiently removes rows from the `my_table` Delta table where the `age` column's value exceeds 25. Delta Lake, built on Apache Spark, supports standard SQL `DELETE` operations for modifying data within Delta tables. This functionality is crucial for data governance, regulatory compliance (like GDPR), and overall data quality management.

Option A (`SELECT * FROM my_table WHERE age > 25;`) only selects rows meeting the condition; it doesn't modify the table. Options B and D use `UPDATE`, which is designed to modify existing values in specified columns. `UPDATE` isn't used for row deletion. Option E (`DELETE FROM my_table WHERE age <= 25;`) is incorrect because it deletes the wrong rows; it removes rows where the age is less than or equal to 25, which is the opposite of what the question asks for.

The `DELETE` command with the `WHERE` clause provides the precise filtering logic required to remove the specific rows based on the given condition. Using Delta Lake transactions, this `DELETE` operation ensures atomicity, consistency, isolation, and durability (ACID) properties. If the `DELETE` command encounters an error, the entire operation is rolled back, maintaining data integrity. This contrasts sharply with non-ACID data storage systems where partial updates might leave the data in an inconsistent state. Delta Lake also automatically maintains a version history, enabling auditing and rollback to previous versions if needed after the `DELETE` operation.

For further reading on Delta Lake `DELETE` operations, refer to the official Databricks documentation:

Databricks Delta Lake Documentation: <https://docs.databricks.com/delta/index.html>

Delta Lake SQL Commands: <https://docs.databricks.com/sql/language-manual/delta-lake-sql.html>

These links provide comprehensive information on using Delta Lake and its SQL commands, including `DELETE`, with transaction guarantees.

Question: 7

Deep Dumps

A data engineer has realized that they made a mistake when making a daily update to a table. They need to use Delta time travel to restore the table to a version that is 3 days old. However, when the data engineer attempts to time travel to the older version, they are unable to restore the data because the data files have been deleted. Which of the following explains why the data files are no longer present?

- A. The `VACUUM` command was run on the table
- B. The `TIME TRAVEL` command was run on the table
- C. The `DELETE HISTORY` command was run on the table
- D. The `OPTIMIZE` command was run on the table
- E. The `HISTORY` command was run on the table

Answer: A

Explanation:

The reason the data engineer is unable to time travel to a version of the Delta table that is 3 days old is most likely due to the `VACUUM` command having been run on the table. The `VACUUM` command removes data files from the Delta Lake table's underlying storage that are no longer needed by the Delta Lake transaction log. Its primary purpose is to reduce storage costs and improve query performance by eliminating outdated files.

By default, Delta Lake retains historical versions of the data for 30 days, allowing users to time travel to any point within that window. However, if the VACUUM command is executed with a retention period shorter than 3 days (or if the default 7-day retention is already applied automatically), the data files required for the 3-day-old version would have been physically removed from the storage.

The TIME TRAVEL command itself is used to restore a table to a previous version, so it cannot be the cause. The DELETE HISTORY command, while related to removing versions, is not a standard Delta Lake command. Optimizing the table with the OPTIMIZE command reorganizes data files for better performance but doesn't inherently delete historical data. Similarly, the HISTORY command simply displays the table's history and doesn't modify or remove any data files. Therefore, VACUUM, with its function of physically removing data files based on a retention policy, directly explains why the data files are no longer available for time travel to a version older than the retention period. If VACUUM has been executed with a retention interval less than 3 days, restoring the table to a version 3 days old will be impossible, as the data files needed for that version are already purged.

Authoritative Links:

Databricks Delta Lake Vacuum: <https://docs.databricks.com/delta/optimizations/vacuum.html>

Databricks Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Question: 8

Deep Dumps

Which of the following Git operations must be performed outside of Databricks Repos?

- A. Commit
- B. Pull
- C. Push
- D. Clone
- E. Merge

Answer: E

Explanation:

The correct answer is E, Merge. Here's why:

Databricks Repos provide a Git-integrated development environment directly within the Databricks workspace. This allows data engineers to perform common Git operations such as committing (A), pulling (B), pushing (C), and even cloning (D) within the Databricks UI. These actions are directly supported and streamlined by the Databricks Repos functionality.

However, the merge operation is typically more complex and involves resolving potential conflicts between different branches. While Databricks Repos allows you to create merge requests or pull requests, the actual process of resolving conflicts and completing the merge often requires a more comprehensive Git client or integrated development environment (IDE) outside of Databricks. This is because resolving merge conflicts is usually best handled with tools offering visual diffing and conflict resolution capabilities, something not natively built into Databricks Repos.

Specifically, while Databricks repos can visually show you what code differences exist, the tool to resolve those conflicts can be cumbersome. It may involve downloading the impacted file, making the changes locally in a more robust IDE, and uploading the changed file back. It is more common to resolve conflicts in a common git tool like VS Code, or using a code repository like Github or Gitlab. Databricks repos focuses on facilitating collaboration and version control within the Databricks environment, but defers conflict resolution to dedicated Git tools. Therefore, the actual merge operation with conflict resolution is often, and more

efficiently, handled externally.

In summary, commit, pull, push, and clone are core operations handled directly by Databricks Repos, while merge, particularly when involving conflict resolution, is usually performed externally for a more seamless workflow.

Further reading:

Databricks Repos Documentation: <https://docs.databricks.com/repos/index.html> (Explains core functionality)

Git Documentation: <https://git-scm.com/doc> (General Git concepts)

Github Documentation: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github>

Question: 9

Deep Dumps

Which of the following data lakehouse features results in improved data quality over a traditional data lake?

- A. A data lakehouse provides storage solutions for structured and unstructured data.
- B. A data lakehouse supports ACID-compliant transactions.
- C. A data lakehouse allows the use of SQL queries to examine data.
- D. A data lakehouse stores data in open formats.
- E. A data lakehouse enables machine learning and artificial Intelligence workloads.

Answer: B

Explanation:

The correct answer is **B. A data lakehouse supports ACID-compliant transactions.**

Here's why:

Data quality directly benefits from ACID (Atomicity, Consistency, Isolation, Durability) transactions. ACID properties ensure that data operations are reliable and consistent, preventing data corruption and maintaining data integrity.

Atomicity: Guarantees that a transaction is treated as a single, indivisible unit of work. Either all changes within the transaction are applied, or none are. This prevents partial updates that could lead to inconsistent data.

Consistency: Ensures that a transaction moves the database from one valid state to another. Constraints, rules, and validations are enforced, preventing invalid data from being written.

Isolation: Dictates that concurrent transactions do not interfere with each other. Each transaction operates as if it were the only transaction running on the system, preventing data corruption from concurrent updates.

Durability: Ensures that once a transaction is committed, it is permanently recorded and will survive system failures (e.g., power outages, crashes).

Traditional data lakes often lack ACID transaction support, leading to "dirty reads" and inconsistent data during concurrent writes or failures. Data lakehouses, by incorporating ACID transactions, provide a more reliable and consistent foundation for data, thus improving data quality. Features like Delta Lake are used to provide the ACID properties to Databricks Data Lakehouse.

The other options are less directly related to improved data quality compared to a traditional data lake, although they provide benefits to the general functionality. For example, storing data in open formats or using SQL are also typical of a data lake, and they don't ensure the correctness of the data being stored.

Supporting Resources:

Question: 10

Deep Dumps

A data engineer needs to determine whether to use the built-in Databricks Notebooks versioning or version their project using Databricks Repos.

Which of the following is an advantage of using Databricks Repos over the Databricks Notebooks versioning?

- A. Databricks Repos automatically saves development progress
- B. Databricks Repos supports the use of multiple branches
- C. Databricks Repos allows users to revert to previous versions of a notebook
- D. Databricks Repos provides the ability to comment on specific changes
- E. Databricks Repos is wholly housed within the Databricks Lakehouse Platform

Answer: B

Explanation:

The correct answer is B: Databricks Repos supports the use of multiple branches. Here's a detailed justification:

Databricks Notebooks versioning offers a basic level of version control, primarily focused on tracking changes within a single notebook. While it allows users to revert to previous versions (option C) and automatically saves progress, it lacks the robust capabilities of a full-fledged Git-based version control system. Databricks Repos, on the other hand, integrates directly with Git providers like GitHub, GitLab, Azure DevOps, or Bitbucket. This integration enables a full suite of version control features.

The key advantage highlighted in option B, "Databricks Repos supports the use of multiple branches," is crucial for collaborative development workflows. Branching allows data engineers to work on new features or bug fixes in isolation without disrupting the main codebase (e.g., the main or master branch). This is vital for larger projects and teams where multiple developers are working concurrently. Each developer or feature can have its own branch, promoting parallel development and reducing the risk of conflicts. Once changes are tested and approved, the branch can be merged back into the main branch.

Databricks Notebooks versioning does not provide branch management; changes are linear and applied directly to the existing notebook version. While both methods enable reverting (C), Databricks Repos via Git offers a more granular and auditable change history. Neither Notebook Versioning nor Repos provide in-platform commenting on specific changes directly as described in option D. Repos depend on the features offered in the connected git provider, which may include commenting features. Automatic saves (A) also apply to built-in Notebook versioning. Therefore, branching is the distinguishing advantage of Repos. Repos, by leveraging Git, aren't wholly housed within the Databricks Lakehouse Platform (E) but rather integrate with external Git repositories, making it a Git-based version control system integrated into Databricks.

For more information, refer to the official Databricks documentation:

[Databricks Repos](#)
[Notebook Versioning](#)

Question: 11

Deep Dumps

A data engineer has left the organization. The data team needs to transfer ownership of the data engineer's Delta

tables to a new data engineer. The new data engineer is the lead engineer on the data team. Assuming the original data engineer no longer has access, which of the following individuals must be the one to transfer ownership of the Delta tables in Data Explorer?

- A. Databricks account representative
- B. This transfer is not possible
- C. Workspace administrator
- D. New lead data engineer
- E. Original data engineer

Answer: C

Explanation:

The correct answer is C: Workspace administrator. Here's why:

Delta tables in Databricks, like other securable objects, have an owner. Ownership determines who has full control over the object, including the ability to grant permissions to others and, crucially, transfer ownership. When an employee leaves, their access is typically revoked, making them unable to transfer ownership. The new lead data engineer, while designated for future ownership, doesn't automatically inherit the privileges needed to perform the transfer unless they already possess them. Databricks account representatives lack the necessary workspace-level privileges to directly modify object ownership.

The key to resolving this lies with the workspace administrator. Workspace administrators possess elevated privileges within a Databricks workspace. They have the authority to manage users, groups, and, importantly, control access to data objects, including Delta tables. Because of these elevated permissions, a workspace administrator can reassign ownership of the Delta tables to the new lead data engineer. The command used would be `ALTER TABLE <table_name> OWNER TO <new_owner>`. This command requires the `MODIFY` permission on the table, which is implicitly held by the table owner or a workspace administrator.

Transferring ownership is a crucial step in ensuring continued data governance and operational continuity. By assigning ownership to the appropriate person, the organization maintains control over its data assets and can ensure that the new lead data engineer has the necessary permissions to manage and maintain the tables. Thus, the workspace administrator is the only individual with the required privilege after the original owner has departed the organization.

[Databricks Documentation on Privileges](#)[Databricks Documentation on Workspace Admin](#)

Question: 12

Deep Dumps

A data analyst has created a Delta table `sales` that is used by the entire data analysis team. They want help from the data engineering team to implement a series of tests to ensure the data is clean. However, the data engineering team uses Python for its tests rather than SQL.

Which of the following commands could the data engineering team use to access `sales` in PySpark?

- A. `SELECT * FROM sales`
- B. There is no way to share data between PySpark and SQL.
- C. `spark.sql("sales")`
- D. `spark.delta.table("sales")`
- E. `spark.table("sales")`

Answer: E

Explanation:

E. `spark.table("sales")`

A Delta table created in Databricks (or any Spark environment using Delta Lake) is stored in the Metastore.

This means the table can be accessed both:

Through SQL:

```
SELECT * FROM sales;
```

And through PySpark using the SparkSession API.

To load a table in PySpark, the correct command is:

```
df = spark.table("sales")
```

This returns the Delta table as a DataFrame that the data engineering team can use for testing, transformations, or validation in Python.

Why the Other Options Are Incorrect:

A. `SELECT * FROM sales`

This is SQL only, not PySpark. It returns a result within the SQL interpreter, not a PySpark DataFrame.

B. "No way to share data"

Incorrect. Spark provides full interoperability between SQL and PySpark via the Metastore.

C. `spark.sql("sales")`

Incorrect. `spark.sql()` expects a full SQL query. The correct usage would require: `spark.sql("SELECT * FROM sales")`.

D. `spark.delta.table("sales")`

Not valid syntax. The correct form would be: `deltaTable = DeltaTable.forName(spark, "sales")`, which returns a `DeltaTable` object (not a DataFrame).

Question: 13

Deep Dumps

Which of the following commands will return the location of database customer360?

- A. `DESCRIBE LOCATION customer360;`
- B. `DROP DATABASE customer360;`
- C. `DESCRIBE DATABASE customer360;`
- D. `ALTER DATABASE customer360 SET DBPROPERTIES ('location' = '/user' ;`
- E. `USE DATABASE customer360;`

Answer: C

Explanation:

The correct command to retrieve the location of a database in Databricks is `DESCRIBE DATABASE customer360;`. This command provides detailed metadata about the specified database, including its location. Options A, B, D, and E perform different operations. `DESCRIBE LOCATION` is not a valid command in Databricks SQL for retrieving database metadata. `DROP DATABASE` removes the database entirely, which is

destructive and not intended for information retrieval. ALTER DATABASE modifies the database properties; while it can set the location, it doesn't retrieve it, and the example provided sets a new property rather than querying the existing one. USE DATABASE simply sets the current database context for subsequent operations.

DESCRIBE DATABASE provides a comprehensive overview of the database's properties. Databricks leverages a metastore (either the default Hive metastore or Unity Catalog) to manage the metadata for databases, tables, and other data assets. The database location is a crucial piece of this metadata, indicating where the database's managed tables and other associated data are physically stored in cloud storage (e.g., AWS S3, Azure Data Lake Storage, or Google Cloud Storage). The DESCRIBE DATABASE command retrieves this metadata from the metastore. The output includes the database name, description (if any), location (specifying the storage path), and other relevant database properties. By parsing the output of the DESCRIBE DATABASE command, you can programmatically determine the database's storage location.

For further information on Databricks SQL commands and database management, you can refer to the official Databricks documentation:

Databricks SQL Reference: <https://docs.databricks.com/sql/language-manual/index.html>

DESCRIBE DATABASE Command: (This would be in the SQL Reference above, search for "DESCRIBE DATABASE")

Databricks Unity Catalog: <https://docs.databricks.com/data-governance/unity-catalog/index.html> (for understanding metastore concepts).

Question: 14

Deep Dumps

A data engineer wants to create a new table containing the names of customers that live in France. They have written the following command:

```
CREATE TABLE customersInFrance
____ AS
SELECT id,
       firstName,
       lastName,
FROM customerLocations
WHERE country = 'FRANCE';
```

A senior data engineer mentions that it is organization policy to include a table property indicating that the new table includes personally identifiable information (PII).

Which of the following lines of code fills in the above blank to successfully complete the task?

- A. There is no way to indicate whether a table contains PII.
- B. "COMMENT PII"
- C. TBLPROPERTIES PII
- D. COMMENT "Contains PII"
- E. PII

Answer: D

Explanation:

D. COMMENT "Contains PII"

In SQL (especially when working with data platforms like Databricks, Hive, or Spark SQL), metadata can be added to tables using the COMMENT clause to describe the contents or purpose of the table.

If your organization requires tagging tables that contain Personally Identifiable Information (PII), adding a comment like "Contains PII" is a standard and accepted way to do it.

Question: 15

Deep Dumps

Which of the following benefits is provided by the array functions from Spark SQL?

- A. An ability to work with data in a variety of types at once
- B. An ability to work with data within certain partitions and windows
- C. An ability to work with time-related data in specified intervals
- D. An ability to work with complex, nested data ingested from JSON files
- E. An ability to work with an array of tables for procedural automation

Answer: D

Explanation:

The correct answer is D because Spark SQL's array functions are specifically designed to handle complex, nested data structures, commonly encountered when ingesting data from sources like JSON files. JSON often contains arrays of values or nested objects, which can be challenging to process with traditional SQL. Array functions provide the means to manipulate and extract information from these array structures, such as accessing elements by index, transforming array contents, and performing operations on array elements.

Options A, B, and C are less relevant to the primary purpose of array functions. While Spark SQL can handle a variety of data types (A), that's not the exclusive domain of array functions. Window functions (B) handle calculations across sets of rows, not array manipulation, and date/time functions (C) are designed for time-series data, not array processing. Option E refers to handling an array of tables, which can be accomplished via other Spark SQL features like UNION or dynamic views, but doesn't directly represent the capability of array functions.

Therefore, the most direct and accurate benefit offered by Spark SQL's array functions is the ability to efficiently process and transform complex, nested data structures, especially those originating from JSON or similar semi-structured formats. They simplify the task of working with these intricate data structures and provide a more accessible way to extract insights from the data.

Reference:

Databricks documentation on array functions: https://docs.databricks.com/sql/language-manual/functions/array_contains.html (Example - illustrates usage)

Apache Spark SQL documentation: <https://spark.apache.org/docs/latest/sql-ref.html> (Provides a complete overview of all SQL functions, including array functions)

Question: 16

Deep Dumps

Which of the following commands can be used to write data into a Delta table while avoiding the writing of duplicate records?

- A. DROP
- B. IGNORE
- C. MERGE
- D. APPEND
- E. INSERT

Answer: C

Explanation:

The correct answer is **C. MERGE**. Here's a detailed justification:

The MERGE command in Delta Lake is specifically designed for performing upserts - combining update and insert operations based on a specified condition. This is precisely what's needed to avoid writing duplicate records. The MERGE command essentially allows you to specify a source dataset (the new data you're trying to write) and a target Delta table. You then define a condition that determines when a row from the source matches a row in the target.

If a match is found based on this condition, you can specify how to update the existing row in the target table (the "update" part of upsert). If no match is found, you can specify how to insert the new row from the source into the target table (the "insert" part of upsert).

By using MERGE with an appropriate matching condition (e.g., based on a unique key or combination of fields), you can effectively prevent the creation of duplicate records. If a record with the same key already exists, it will be updated; otherwise, a new record will be inserted. This ensures that your Delta table maintains data integrity.

Options A, B, D, and E are incorrect:

DROP: This command is used to delete entire tables or partitions, not to avoid writing duplicates during data ingestion.

IGNORE: While some database systems might have an IGNORE option, it's not a standard Delta Lake or Spark command for handling duplicate records. It wouldn't provide the necessary logic for an upsert operation.

APPEND: This command simply adds new data to the end of the table without checking for duplicates. It's the opposite of what we need.

INSERT: Similar to APPEND, INSERT adds new rows without any de-duplication logic. Using INSERT alone is a guaranteed way to potentially introduce duplicate records.

In summary, MERGE provides the fine-grained control needed to update existing records or insert new ones only when necessary, avoiding duplication and maintaining data integrity within your Delta table.

Further reading on Delta Lake MERGE can be found here:

[Databricks Documentation: MERGE INTO](#)

Question: 17

Deep Dumps

A data engineer needs to apply custom logic to string column city in table stores for a specific use case. In order to apply this custom logic at scale, the data engineer wants to create a SQL user-defined function (UDF). Which of the following code blocks creates this SQL UDF?

```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

A.

```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

B.

```
CREATE UDF combine_nyc(city STRING)
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

C.

```
CREATE FUNCTION combine_nyc(city STRING)
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

D.

```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

E.

Answer: A

Explanation:

The answer E is incorrect. A user defined function is never written as CREATE UDF. The correct way is CREATE FUNCTION. So that leaves us with the choices A and D. Out of that, in D, there is no such thing as RETURN CASE so the correct answer is A.

Question: 18

Deep Dumps

A data analyst has a series of queries in a SQL program. The data analyst wants this program to run every day. They only want the final query in the program to run on Sundays. They ask for help from the data engineering team to complete this task.

Which of the following approaches could be used by the data engineering team to complete this task?

- A. They could submit a feature request with Databricks to add this functionality.
- B. They could wrap the queries using PySpark and use Python's control flow system to determine when to run the final query.
- C. They could only run the entire program on Sundays.
- D. They could automatically restrict access to the source table in the final query so that it is only accessible on Sundays.
- E. They could redesign the data model to separate the data used in the final query into a new table.

Answer: B

Explanation:

The correct answer is B. Here's why:

Option B leverages the flexibility of PySpark combined with Python's control flow capabilities to precisely meet the requirement. The data engineering team can encapsulate the entire SQL program within a PySpark script. Within this script, Python's datetime module can be used to determine the current day of the week. Based on this, a conditional statement (e.g., an if statement checking if the day is Sunday) can be used to decide whether to execute the final SQL query or skip it. The other queries will execute regardless of the day, fulfilling the requirement of running daily. This approach ensures that the entire process is automated, scheduled, and maintains the initial SQL logic provided by the data analyst.

Option A, submitting a feature request, is not a practical immediate solution. It relies on Databricks implementing the feature, which is uncertain and time-consuming. Option C, running the entire program only on Sundays, contradicts the requirement that the initial queries must run every day. Option D, restricting table access, is a poor design choice. It is not recommended to control query execution through data access restrictions, which would be very difficult to manage and could impact other processes. Option E, redesigning the data model, is an unnecessarily complex solution for this requirement. Redesigning data models requires significant effort, testing, and potential changes to other parts of the system. It is not justified by the simple requirement of conditionally executing one query.

Therefore, using PySpark's ability to run SQL queries and Python's control flow mechanisms is the most efficient and direct way to automate the analyst's request without causing unnecessary disruption or adding complex overhead to the system.

Further research:

PySpark SQL: <https://spark.apache.org/docs/latest/sql-getting-started.html>

Python datetime module: <https://docs.python.org/3/library/datetime.html>

Question: 19**Deep Dumps**

A data engineer runs a statement every day to copy the previous day's sales into the table transactions. Each day's sales are in their own file in the location `"/transactions/raw"`.

Today, the data engineer runs the following command to complete this task:

```
COPY INTO transactions
FROM "/transactions/raw"
FILEFORMAT = PARQUET;
```

After running the command today, the data engineer notices that the number of records in table transactions has not changed.

Which of the following describes why the statement might not have copied any new records into the table?

- A. The format of the files to be copied were not included with the `FORMAT_OPTIONS` keyword.
- B. The names of the files to be copied were not included with the `FILES` keyword.
- C. The previous day's file has already been copied into the table.
- D. The `PARQUET` file format does not support `COPY INTO`.
- E. The `COPY INTO` statement requires the table to be refreshed to view the copied rows.

Answer: C**Explanation:**

The previous day's file has already been copied into the table.

Reference:

<https://docs.databricks.com/ingestion/copy-into/tutorial-notebook.html>

Question: 20

Deep Dumps

A data engineer needs to create a table in Databricks using data from their organization's existing SQLite database.

They run the following command:

```
CREATE TABLE jdbc_customer360
USING _____
OPTIONS (
  url "jdbc:sqlite:/customers.db",
  dbtable "customer360"
)
```

Which of the following lines of code fills in the above blank to successfully complete the task?

- A. org.apache.spark.sql.jdbc
- B. autoloader
- C. DELTA
- D. sqlite
- E. org.apache.spark.sql.sqlite

Answer: A

Explanation:

A. org.apache.spark.sql.jdbc

To read from or write to a JDBC-compatible database (such as MySQL, PostgreSQL, SQL Server, etc.) in Apache Spark, you use the format("jdbc") API.

Question: 21

Deep Dumps

A data engineering team has two tables. The first table march_transactions is a collection of all retail transactions in the month of March. The second table april_transactions is a collection of all retail transactions in the month of April. There are no duplicate records between the tables.

Which of the following commands should be run to create a new table all_transactions that contains all records from march_transactions and april_transactions without duplicate records?

- A. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INNER JOIN SELECT * FROM april_transactions;
- B. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
UNION SELECT * FROM april_transactions;
- C. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
OUTER JOIN SELECT * FROM april_transactions;
- D. CREATE TABLE all_transactions AS


```
SELECT * FROM march_transactions  
INTERSECT SELECT * FROM april_transactions;
```

```
E. CREATE TABLE all_transactions AS  
SELECT * FROM march_transactions  
MERGE SELECT * FROM april_transactions;
```

Answer: B

Explanation:

The correct command to combine all records from `march_transactions` and `april_transactions` into a new table `all_transactions` without duplicates is option B: `CREATE TABLE all_transactions AS SELECT FROM march_transactions UNION SELECT FROM april_transactions;`

Here's why:

UNION: The UNION operator in SQL is specifically designed to combine the result-sets of two or more SELECT statements. Importantly, UNION automatically removes duplicate rows from the final result, ensuring each record is unique. This directly addresses the requirement of creating a table without duplicate records from the two source tables.

INNER JOIN, OUTER JOIN, INTERSECT, MERGE: These other options are not suitable for combining records from two separate tables in the way described in the prompt.

INNER JOIN and OUTER JOIN are used to combine rows from two tables based on a related column. Since the goal is to combine all rows regardless of a shared key, these are incorrect.

INTERSECT returns only the rows that are present in both tables, not all rows from both.

MERGE is typically used for complex data manipulations, such as synchronizing two tables based on a condition, and it is not a standard SQL command in all database systems. Databricks may support it through variations in spark SQL, but it is not the simplest solution.

CREATE TABLE AS SELECT (CTAS): The `CREATE TABLE all_transactions AS SELECT ...` syntax allows creating a new table based on the results of a SELECT query. This is an efficient way to directly populate the new table with the combined and de-duplicated data.

Therefore, the UNION operator within a `CREATE TABLE AS SELECT` statement is the most appropriate way to create the `all_transactions` table with unique records from both input tables.

Supporting Documentation:

Databricks SQL Reference - UNION: <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-qry-select-union.html>

Databricks SQL Reference - CREATE TABLE AS SELECT: <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-as-select.html>

Question: 22

Deep Dumps

A data engineer only wants to execute the final block of a Python program if the Python variable `day_of_week` is equal to 1 and the Python variable `review_period` is True.

Which of the following control flow statements should the data engineer use to begin this conditionally executed code block?

- A. `if day_of_week = 1 and review_period:`
- B. `if day_of_week = 1 and review_period = "True":`
- C. `if day_of_week == 1 and review_period == "True":`

- D. if day_of_week == 1 and review_period:
E. if day_of_week = 1 & review_period: = "True":

Answer: D

Explanation:

The correct answer is D: if day_of_week == 1 and review_period:.

The if statement in Python is used to execute a block of code only if a certain condition is met. This condition is evaluated as a boolean (True or False). In this scenario, the data engineer wants the code to execute only when day_of_week is equal to 1 and review_period is True.

Let's break down why other options are incorrect:

A. if day_of_week = 1 and review_period:: The = operator is an assignment operator, not a comparison operator. Using it in an if statement like this would attempt to assign the value 1 to day_of_week, which is syntactically incorrect and will raise an error. You need the comparison operator == to check for equality.

B. if day_of_week = 1 and review_period = "True":: This suffers from the same assignment error as option A. Additionally, even if corrected to use ==, comparing a boolean variable review_period to the string "True" is generally incorrect and will likely not produce the desired result. A boolean variable is already True or False.

C. if day_of_week == 1 and review_period == "True":: Correctly uses the == operator, however this incorrectly compares a boolean variable review_period to a string "True". This will only evaluate True if review_period is the string value "True" which is unlikely to be the case.

E. if day_of_week = 1 & review_period: = "True":: Contains both the assignment error (=) and uses the bitwise AND operator (&) instead of the logical AND operator (and). Additionally, has the same boolean comparison issue as option B and C.

Option D, if day_of_week == 1 and review_period:, correctly uses the equality operator == to compare day_of_week to 1. It also correctly checks if review_period is True. Because review_period is already a boolean variable (either True or False), simply stating its name after the and implicitly checks if it's True. The whole condition will only be true if both conditions are satisfied.

In short, using and ensures that both conditions, day_of_week == 1 and review_period, must be True for the conditional code block to execute.

Relevant links for more information:

Python if statement: https://www.w3schools.com/python/python_conditions.asp

Python comparison operators: https://www.w3schools.com/python/python_operators.asp

Python boolean data type: https://www.w3schools.com/python/python_booleans.asp

Question: 23

Deep Dumps

A data engineer is attempting to drop a Spark SQL table my_table. The data engineer wants to delete all table metadata and data.

They run the following command:

```
DROP TABLE IF EXISTS my_table -
```

While the object no longer appears when they run SHOW TABLES, the data files still exist.

Which of the following describes why the data files still exist and the metadata files were deleted?

- A. The table's data was larger than 10 GB

- B. The table's data was smaller than 10 GB
- C. The table was external
- D. The table did not have a location
- E. The table was managed

Answer: C

Explanation:

The reason the data files still exist despite dropping the table is that the table was an external table. In Spark SQL, when you drop a managed table (also known as an internal table), Spark removes both the metadata and the underlying data from the metastore and storage location, respectively. However, when you drop an external table, only the metadata is removed from the metastore. The data files themselves, residing in the external storage location (e.g., cloud object storage like AWS S3 or Azure Blob Storage), remain untouched. The DROP TABLE command only removes the table's definition from the metastore, disconnecting the table name from the actual data location. The IF EXISTS clause simply prevents an error if the table does not exist, and it doesn't affect the behavior regarding data deletion. The size of the data (options A and B) is irrelevant. Option D, "The table did not have a location," is contradictory, as external tables are defined by their location, unlike managed tables that default to the metastore's managed location. In summary, the defining characteristic of external tables is that their data lives outside the control of the metastore and is unaffected by table dropping operations.

Further research can be done here: <https://spark.apache.org/docs/latest/sql-data-sources-hive-tables.html> <https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-drop-table.html>

Question: 24

Deep Dumps

A data engineer wants to create a data entity from a couple of tables. The data entity must be used by other data engineers in other sessions. It also must be saved to a physical location. Which of the following data entities should the data engineer create?

- A. Database
- B. Function
- C. View
- D. Temporary view
- E. Table

Answer: E

Explanation:

The correct answer is **E. Table**. Here's why:

The requirement is to create a data entity accessible across multiple sessions and persistently stored in a physical location. Let's examine each option:

A. Database: A database is a container for schemas, tables, views, and functions. While necessary for organizing data, it doesn't directly represent a data entity derived from other tables. It's more like the overall storage structure.

B. Function: Functions are reusable code blocks, not data entities derived from tables. They perform operations, not store data.

C. View: Views are virtual tables based on the result-set of a query. While they can combine data from

multiple tables, they are **not physically stored**. Views are simply stored queries.

D. Temporary View: Temporary views are session-scoped. This means they exist only for the duration of the Databricks session in which they were created. This directly contradicts the requirement of being accessible across multiple sessions.

E. Table: Tables are physical representations of data stored in a structured format. They persist across sessions and are stored in a defined physical location (e.g., cloud storage like AWS S3 or Azure Data Lake Storage). A table can be created by selecting data from other tables, fulfilling the data engineer's requirement to combine data. Thus, other data engineers can access it in other sessions. This makes it the correct option. A managed table stores the data and metadata in the metastore while an unmanaged table only stores metadata in the metastore, and references the physical storage location that is manually defined.

In summary, only a table provides the required combination of persistence, cross-session availability, and physical storage for the derived data entity.

Further Research:

Databricks Tables: <https://docs.databricks.com/en/tables/index.html>

Databricks Views: <https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-aux-show-views.html>

Question: 25

Deep Dumps

A data engineer is maintaining a data pipeline. Upon data ingestion, the data engineer notices that the source data is starting to have a lower level of quality. The data engineer would like to automate the process of monitoring the quality level.

Which of the following tools can the data engineer use to solve this problem?

- A. Unity Catalog
- B. Data Explorer
- C. Delta Lake
- D. Delta Live Tables
- E. Auto Loader

Answer: D

Explanation:

The correct answer is **D. Delta Live Tables (DLT)**. Here's why:

Delta Live Tables is the best choice for automating data quality monitoring in a Databricks data pipeline. DLT allows you to define data quality constraints directly within your data pipeline code using expectations. Expectations are boolean expressions that, when evaluated against your data, determine whether the data meets a certain quality threshold. DLT automatically monitors these expectations during pipeline execution and tracks data quality metrics over time. This allows data engineers to create a continuous data quality monitoring system without manual intervention.

DLT provides features like `expect` and `expect_or_drop` that can enforce data quality. You can set actions based on expectation results, such as dropping records that fail a quality check or simply recording the violations. This active monitoring and enforcement of data quality rules during the transformation process ensures that only clean, reliable data proceeds through the pipeline.

Furthermore, DLT's UI provides detailed metrics and dashboards that visualize data quality trends. You can readily identify data quality issues, track improvements after implementing fixes, and set alerts to be notified when data quality metrics fall below acceptable levels. The pipeline's lineage and data flow are visualized,

making it easier to understand the impact of data quality issues.

While other options might play roles in a data pipeline, they don't directly provide the automated data quality monitoring features of DLT. Unity Catalog (A) focuses on data governance and discovery. Data Explorer (B) is more for interactive data browsing and exploration. Delta Lake (C) provides reliable storage but doesn't actively monitor data quality during pipeline execution. Auto Loader (E) efficiently ingests data but doesn't enforce or monitor data quality.

Therefore, Delta Live Tables is purpose-built for automating data quality monitoring within a Databricks data pipeline, making it the optimal solution.

Relevant links:

[Delta Live Tables documentation](#)

[Declarative Data Quality with Delta Live Tables](#)

Question: 26

Deep Dumps

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.

The table is configured to run in Production mode using the Continuous Pipeline Mode.

Assuming previously unprocessed data exists and all definitions are valid, what is the expected outcome after clicking Start to update the pipeline?

- A. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.
- B. All datasets will be updated once and the pipeline will persist without any processing. The compute resources will persist but go unused.
- C. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.
- D. All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.
- E. All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.

Answer: C

Explanation:

The correct answer is C because of the combination of STREAMING LIVE TABLE, LIVE TABLE, Continuous pipeline mode, and Production mode.

Let's break down why:

STREAMING LIVE TABLE: This keyword signifies datasets that continuously ingest and process new data as it arrives. They are designed for incremental updates.

LIVE TABLE: These datasets are also updated by the DLT pipeline, but may or may not be updated incrementally.

Continuous Pipeline Mode: This mode instructs the pipeline to run continuously, processing data as it becomes available. In contrast to Triggered mode, which runs once and then shuts down, Continuous mode aims for near real-time data processing.

Production Mode: In production mode, the DLT pipeline is optimized for efficiency and cost. This means compute resources are deployed when needed for updates and then terminated when the pipeline is stopped. This differs from development mode, where compute may persist for faster iterations.

Therefore, in this scenario, both the streaming and non-streaming tables will be updated continuously at set intervals as governed by the DLT pipeline's internal mechanisms. Because it is production mode, the

infrastructure will shut down when you hit the stop button.

Here are some relevant resources to enhance your understanding:

Delta Live Tables Modes: <https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-concepts.html#pipeline-modes>

STREAMING LIVE TABLE: <https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-syntax.html#streaming-live-table>

Delta Live Tables Concepts: <https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-concepts.html>

Question: 27

Deep Dumps

In order for Structured Streaming to reliably track the exact progress of the processing so that it can handle any kind of failure by restarting and/or reprocessing, which of the following two approaches is used by Spark to record the offset range of the data being processed in each trigger?

- A. Checkpointing and Write-ahead Logs
- B. Structured Streaming cannot record the offset range of the data being processed in each trigger.
- C. Replayable Sources and Idempotent Sinks
- D. Write-ahead Logs and Idempotent Sinks
- E. Checkpointing and Idempotent Sinks

Answer: A

Explanation:

The correct answer is **A. Checkpointing and Write-ahead Logs**. Here's a detailed justification:

Structured Streaming's fault tolerance and ability to guarantee exactly-once or at-least-once processing relies heavily on tracking the progress of data ingestion and processing in a reliable manner. This involves recording the offset range of data processed in each trigger. This enables the system to recover from failures and resume processing from the correct point.

Checkpointing: Checkpointing involves periodically saving the state of the streaming application to a reliable storage system, such as cloud storage (e.g., S3, ADLS). This state includes information about the processed offsets, aggregations, and any other relevant information. In case of a failure, the application can restart from the last checkpoint, effectively rewinding the state to a known good point. Checkpointing is used to store metadata about the streaming query itself, including information about the progress of data consumption.

Write-Ahead Logs (WAL): The Write-Ahead Log is a persistent record of all changes made to the state. Before any changes are applied, they are first written to the WAL. This ensures that even if a failure occurs during processing, the system can replay the WAL to reconstruct the exact state of the data processing, including the offsets that have been processed. The WAL provides durability for the per-micro-batch changes.

Together, checkpointing and WAL guarantee fault tolerance. The WAL provides immediate recovery in case of minor failures, while checkpoints allow for recovery from more significant failures. Without checkpointing or write-ahead logs, Structured Streaming would be unable to reliably resume processing after a failure, potentially leading to data loss or duplication.

The other options are incorrect for the following reasons:

B: Structured Streaming does record offset ranges for fault tolerance.

C: Replayable Sources and Idempotent Sinks are related to fault tolerance but do not, on their own, provide the mechanism for tracking offsets. While ideal, not all sources are replayable. Sinks being idempotent

prevents duplicate writes, but this is not directly related to tracking offsets.

D: Write-ahead Logs are important, and Idempotent Sinks are beneficial to avoid duplicates, but checkpoints are essential for long-term fault tolerance.

E: Checkpointing alone is insufficient; a Write-Ahead Log provides a finer-grained level of fault tolerance between checkpoints.

Authoritative Links:

Spark Structured Streaming Programming Guide - Fault Tolerance Semantics:

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#fault-tolerance-semantics>

Databricks - Fault tolerance in Structured Streaming: <https://www.databricks.com/glossary/structured-streaming> (Search for the term Fault tolerance)

Question: 28

Deep Dumps

Which of the following describes the relationship between Gold tables and Silver tables?

- A. Gold tables are more likely to contain aggregations than Silver tables.
- B. Gold tables are more likely to contain valuable data than Silver tables.
- C. Gold tables are more likely to contain a less refined view of data than Silver tables.
- D. Gold tables are more likely to contain more data than Silver tables.
- E. Gold tables are more likely to contain truthful data than Silver tables.

Answer: A

Explanation:

The correct answer is **A. Gold tables are more likely to contain aggregations than Silver tables.** This relates to the Medallion Architecture (Bronze, Silver, Gold), a common data lake design pattern used in Databricks and other cloud data platforms.

Bronze tables represent the raw data ingested from source systems, often without transformation. Silver tables are refined versions of Bronze tables, typically cleansed, filtered, and joined. Gold tables sit at the top of the pyramid, designed for specific business use cases, reporting, and analysis.

Therefore, Gold tables often involve aggregations, such as calculating daily sales totals, customer lifetime value, or inventory summaries. Silver tables, while refined, are more likely to be at a row-level detail, providing the foundation for these aggregations. Options B, D, and E aren't correct because Silver tables contain refined and valuable data, though perhaps not tailored to a specific end-user use case, so they would not necessarily contain "more data" or "more truthful data" than Silver tables. While Gold tables are valuable, the type of value is different. Option C is incorrect because Gold tables provide a more refined view of the data tailored to answering specific business questions. The Medallion architecture is all about progressive refinement as data moves from Bronze to Gold.

Here are some authoritative links for further research:

Databricks Medallion Architecture: <https://www.databricks.com/glossary/medallion-architecture>

Understanding the Medallion Architecture: <https://towardsdatascience.com/understanding-the-medallion-architecture-d566f8364164>

Delta Lake Documentation (related to table structure): <https://docs.delta.io/latest/index.html>

Question: 29**Deep Dumps**

Which of the following describes the relationship between Bronze tables and raw data?

- A. Bronze tables contain less data than raw data files.
- B. Bronze tables contain more truthful data than raw data.
- C. Bronze tables contain aggregates while raw data is unaggregated.
- D. Bronze tables contain a less refined view of data than raw data.
- E. Bronze tables contain raw data with a schema applied.

Answer: E**Explanation:**

The correct answer is E: Bronze tables contain raw data with a schema applied. Let's break down why.

Bronze tables in the Databricks Lakehouse architecture represent the first layer of data ingestion. Think of them as landing zones for your data. The primary purpose of a Bronze table is to reliably and incrementally ingest raw data from various sources into your data lake.

Importantly, data in Bronze tables is not transformed or cleaned. It is persisted in its original form, ensuring a complete and auditable record of the incoming information. The key difference between the raw files themselves and the Bronze table is the imposition of a schema. This schema provides structure to the raw data, making it queryable and discoverable within the Databricks environment. Without a schema, the raw data files would be difficult to query directly using SQL or other data processing tools.

Options A, B, C, and D are incorrect because they describe transformations or refinements that occur in later stages of the medallion architecture (Silver and Gold layers). Bronze tables should not be reducing data volume (A), improving data quality (B), aggregating data (C), or refining the view of the data (D). These are all responsibilities of subsequent layers in the data pipeline.

Therefore, the defining characteristic of a Bronze table is that it provides a structured view (through schema enforcement) of raw, untransformed data. It forms the foundation for building a robust and reliable data pipeline within the Databricks Lakehouse.

For further research, consult the official Databricks documentation on the Lakehouse architecture and Delta Lake:

Databricks Lakehouse Architecture: <https://www.databricks.com/product/lakehouse-architecture>

Delta Lake Documentation: <https://docs.databricks.com/delta/index.html>

Medallion Architecture: Search Databricks Documentation for "Medallion Architecture"

Question: 30**Deep Dumps**

Which of the following tools is used by Auto Loader process data incrementally?

- A. Checkpointing
- B. Spark Structured Streaming
- C. Data Explorer
- D. Unity Catalog
- E. Databricks SQL

Answer: B

Explanation:

The correct answer is **B. Spark Structured Streaming**.

Auto Loader leverages Spark Structured Streaming as its underlying engine for processing data incrementally. This means instead of processing entire datasets at once, Auto Loader reads new data as it arrives, offering a near real-time data ingestion pipeline. Checkpointing (A) is a mechanism used by Spark Structured Streaming to provide fault tolerance and recover state, but it's not the tool driving the incremental processing itself. Data Explorer (C) and Databricks SQL (E) are tools for data discovery, visualization, and querying of data stored in a Databricks workspace. Unity Catalog (D) provides centralized data governance, security, and access control across Databricks workspaces, but is not directly involved in the incremental processing mechanism.

Spark Structured Streaming defines a stream, which is a continuously updating table that represents new data arriving. Auto Loader uses this stream to automatically discover new files in the source data location. As new files are discovered, Structured Streaming processes them and appends the data to the target table. This continuous processing model is what allows Auto Loader to process data incrementally. The entire process is designed to be fault-tolerant and scalable thanks to the capabilities inherited from Spark Structured Streaming. Without Spark Structured Streaming, Auto Loader would not be able to offer its core functionality of incremental data ingestion. In essence, Auto Loader is a specialized application built on top of Spark Structured Streaming for simplified incremental data loading from cloud storage.

For further research, consider exploring these resources:

Databricks Auto Loader Documentation: <https://docs.databricks.com/ingestion/auto-loader/index.html>

Spark Structured Streaming Programming Guide: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Question: 31

Deep Dumps

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.table("sales")
  .withColumn("avg_price", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  ._____
  .table("new_sales")
)
```

If the data engineer only wants the query to execute a micro-batch to process data every 5 seconds, which of the following lines of code should the data engineer use to fill in the blank?

- A. trigger("5 seconds")
- B. trigger()
- C. trigger(once="5 seconds")
- D. trigger(processingTime="5 seconds")
- E. trigger(continuous="5 seconds")

Answer: D

Explanation:

D. trigger(processingTime="5 seconds")

In Structured Streaming with Apache Spark, if a data engineer wants to run the stream as a micro-batch every 5 seconds, they need to specify a processing time trigger.

Question: 32**Deep Dumps**

A dataset has been defined using Delta Live Tables and includes an expectations clause:

CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW

What is the expected behavior when a batch of data containing data that violates these constraints is processed?

- A. Records that violate the expectation are dropped from the target dataset and loaded into a quarantine table.
- B. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.
- C. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.
- D. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.
- E. Records that violate the expectation cause the job to fail.

Answer: C**Explanation:**

The correct answer is C because it accurately describes the behavior of Delta Live Tables (DLT) when handling expectation violations with the ON VIOLATION DROP ROW clause. When this clause is specified, any record that fails to meet the defined expectation (in this case, timestamp > '2020-01-01') is removed from the target dataset during the DLT pipeline execution. Crucially, DLT provides a way to track these violations without failing the entire pipeline. It does this by logging the details of the dropped records in the DLT event log. This log provides valuable insights into data quality issues and allows for monitoring and auditing of data transformations.

Option A is incorrect because DLT does not automatically load violating records into a quarantine table when ON VIOLATION DROP ROW is used. The records are simply dropped. Option B is incorrect as DLT does not add records to the target dataset and flag them as invalid when using DROP ROW. Option D is partially correct as invalid records are indeed recorded in the event log. However, it incorrectly suggests the records are added to the target dataset, which is not the case when using DROP ROW. Finally, option E is incorrect because the DROP ROW clause is specifically designed to prevent the job from failing due to data quality issues. Instead, DLT gracefully handles the violations by dropping the problematic records. The overall goal of expectations within DLT is to maintain data quality without interrupting the continuous flow of data processing.

For further research on DLT expectations, you can refer to the official Databricks documentation:

Expectations in Delta Live Tables: <https://docs.databricks.com/delta-live-tables/expectations.html>

Monitor Delta Live Tables Pipelines: <https://docs.databricks.com/delta-live-tables/monitor.html>

Question: 33**Deep Dumps**

Which of the following describes when to use the CREATE STREAMING LIVE TABLE (formerly CREATE INCREMENTAL LIVE TABLE) syntax over the CREATE LIVE TABLE syntax when creating Delta Live Tables (DLT) tables using SQL?

- A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.
- B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.
- C. CREATE STREAMING LIVE TABLE is redundant for DLT and it does not need to be used.
- D. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.
- E. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.

Answer: B

Explanation:

The correct answer is **B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.**

Here's a detailed justification:

Delta Live Tables (DLT) is a framework for building reliable, maintainable, and testable data pipelines on Databricks. It automatically manages data dependencies, optimizes execution, and handles data quality. Within DLT, the CREATE LIVE TABLE and CREATE STREAMING LIVE TABLE commands serve distinct purposes.

CREATE LIVE TABLE is used for batch processing. It's suitable when you need to process the entire input dataset at once. The entire table is recomputed upon each update. If the underlying data changes, the entire table will be reprocessed during the next pipeline update. This is ideal for scenarios where you want to ensure the table always reflects the complete and accurate view of the underlying data, irrespective of incremental changes.

CREATE STREAMING LIVE TABLE, on the other hand, is specifically designed for incremental data processing. This is crucial when dealing with continuously arriving data streams or when you only want to process new data since the last update, avoiding full refreshes. The command leverages Structured Streaming for continuous data ingestion and processing. It maintains state and only processes new records as they arrive, significantly improving efficiency and reducing processing time for streaming data or large datasets with frequent updates.

When using CREATE STREAMING LIVE TABLE, DLT understands that the source data is continuously changing. Hence, it's optimized to incrementally update the table based on the changes in the source data. Using this with batch data that doesn't incrementally change will lead to inefficient pipeline execution. The statement correctly relates to processing of continuously arriving data stream or when only processing new data since the last update.

Option A is incorrect because the nature of the subsequent step (static or not) doesn't dictate whether to use CREATE STREAMING LIVE TABLE. The key factor is whether the input is a continuous stream.

Option C is incorrect because CREATE STREAMING LIVE TABLE has a very specific and important function in incremental data processing.

Option D is incorrect. Complicated aggregations can be performed with both CREATE LIVE TABLE and CREATE STREAMING LIVE TABLE, so this isn't a defining factor. CREATE STREAMING LIVE TABLE can handle aggregations too, albeit in a streaming fashion.

Option E is incorrect. The nature of the previous step is not the primary determinant. The main consideration is whether the data feeding into the current table is a stream.

Therefore, CREATE STREAMING LIVE TABLE excels when the underlying data is constantly evolving and you only need to process the new or changed data incrementally to update the table.

Further Reading:

Delta Live Tables: <https://docs.databricks.com/delta-live-tables/index.html>

CREATE LIVE TABLE: <https://docs.databricks.com/delta-live-tables/delta-live-tables-sql-ref.html#create-live-table>

CREATE STREAMING LIVE TABLE: <https://docs.databricks.com/delta-live-tables/delta-live-tables-sql-ref.html#create-streaming-live-table>

Question: 34

Deep Dumps

A data engineer is designing a data pipeline. The source system generates files in a shared directory that is also used by other processes. As a result, the files should be kept as is and will accumulate in the directory. The data engineer needs to identify which files are new since the previous run in the pipeline, and set up the pipeline to only ingest those new files with each run.

Which of the following tools can the data engineer use to solve this problem?

- A. Unity Catalog
- B. Delta Lake
- C. Databricks SQL
- D. Data Explorer
- E. Auto Loader

Answer: E

Explanation:

The correct answer is E, Auto Loader, because it's specifically designed for incrementally and efficiently ingesting new files as they arrive in a cloud storage location (like a shared directory). Auto Loader automatically detects new files without requiring you to manually track which files have already been processed. It does this by leveraging either directory listing, file notification, or a combination of both, optimizing for different scale and latency requirements.

Here's a breakdown of why the other options are not suitable:

A. Unity Catalog: Unity Catalog is a data governance solution for managing data assets across Databricks workspaces. While it's important for data discovery and access control, it doesn't directly address the problem of identifying and ingesting new files.

B. Delta Lake: Delta Lake is a storage layer that brings reliability to data lakes by providing ACID transactions, scalable metadata handling, and unified streaming and batch data processing. While Delta Lake is excellent for storing and managing ingested data, it doesn't solve the initial problem of identifying new files for ingestion.

C. Databricks SQL: Databricks SQL is a serverless data warehouse on Databricks that allows you to run SQL queries on data stored in your data lake. It's useful for analyzing the data after it's been ingested, but it's not a tool for incremental file ingestion.

D. Data Explorer: Data Explorer allows users to explore the Databricks filesystem, display table metadata, and display sample data. This does not solve the ingestion challenge.

Auto Loader's ability to handle the continuous arrival of new files in a shared directory, especially in scenarios where other processes are also writing to the same location, makes it the ideal choice for this data pipeline requirement. It's built to handle the scale and complexity of such scenarios without requiring manual intervention or custom scripting to track file ingestion status.

Question: 35**Deep Dumps**

Which of the following Structured Streaming queries is performing a hop from a Silver table to a Gold table?

- A.
- ```
(spark.readStream.load(rawSalesLocation)
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("append")
 .table("newSales")
)
```
- B.
- ```
(spark.read.load(rawSalesLocation)
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("newSales")
)
```
- C.
- ```
(spark.table("sales")
 .withColumn("avgPrice", col("sales") / col("units"))
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("append")
 .table("newSales")
)
```
- D.
- ```
(spark.table("sales")
  .filter(col("units") > 0)
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("newSales")
)
```
- E.


```
(spark.table("sales")
  .groupBy("store")
  .agg(sum("sales"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("newSales")
)
```

Answer: E

Explanation:

E is the right answer. The "gold layer" is used to store aggregated clean data, E is the only answer in which aggregation is performed.

Question: 36

Deep Dumps

A data engineer has three tables in a Delta Live Tables (DLT) pipeline. They have configured the pipeline to drop invalid records at each table. They notice that some data is being dropped due to quality concerns at some point in the DLT pipeline. They would like to determine at which table in their pipeline the data is being dropped. Which of the following approaches can the data engineer take to identify the table that is dropping the records?

- A. They can set up separate expectations for each table when developing their DLT pipeline.
- B. They cannot determine which table is dropping the records.
- C. They can set up DLT to notify them via email when records are dropped.
- D. They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics.
- E. They can navigate to the DLT pipeline page, click on the "Error" button, and review the present errors.

Answer: D

Explanation:

The correct approach for identifying the table in a Delta Live Tables (DLT) pipeline where records are being dropped due to quality concerns is option D: "They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics."

Here's why this is the best solution and why the other options are less suitable:

Delta Live Tables (DLT) Observability: DLT provides built-in monitoring and observability features specifically designed to track data quality metrics. This is a core benefit of using DLT for data transformation pipelines. The DLT UI provides a visual representation of the pipeline's data flow, including information about data quality.

Data Quality Statistics per Table: DLT automatically tracks data quality statistics for each table in the pipeline. These statistics include metrics like the number of records processed, the number of records that passed quality checks (expectations), and the number of records that were dropped or failed expectations.

Navigation and Inspection: By navigating to the DLT pipeline UI in Databricks and clicking on a specific table node in the graph, the data engineer can access detailed statistics for that table. The "Data Quality" section displays metrics related to expectations that were defined and how many records satisfied or violated each expectation. This allows direct inspection of the data quality results at each stage of the pipeline.

Option A (Separate Expectations): Setting up separate expectations is a good practice when developing the pipeline, but it doesn't directly help after the fact if you just observe that data is being dropped. Separate expectations are necessary to enable the tracking that option D relies on. Having defined specific expectations for each table's data quality will then allow you to see the resulting statistics in the DLT UI.

Option B (Cannot Determine): This is incorrect. DLT is designed to provide this type of data lineage and quality tracking information.

Option C (Email Notifications): While DLT does offer the ability to configure email notifications based on pipeline status, it wouldn't directly point to which table is dropping the data. Custom code would be needed to generate the proper notification content.

Option E (Error Button): The "Error" button in the DLT UI typically displays errors that prevented the pipeline from running or errors caused by syntax issues. Errors related to data quality are not necessarily indicated by the "Error" button, especially when configured to drop invalid records using constraint clause or expectations using `expect_or_drop` or `expect_all_or_drop` functions. Data quality issues are generally tracked as data quality statistics related to expectation results.

Authoritative Links:

Delta Live Tables documentation: <https://docs.databricks.com/delta-live-tables/index.html>

Monitor Delta Live Tables pipelines: <https://docs.databricks.com/delta-live-tables/monitor.html>

Manage data quality with Delta Live Tables: <https://docs.databricks.com/delta-live-tables/expectations.html>

Question: 37

Deep Dumps

A data engineer has a single-task Job that runs each morning before they begin working. After identifying an upstream data issue, they need to set up another task to run a new notebook prior to the original task. Which of the following approaches can the data engineer use to set up the new task?

- A.They can clone the existing task in the existing Job and update it to run the new notebook.
- B.They can create a new task in the existing Job and then add it as a dependency of the original task.
- C.They can create a new task in the existing Job and then add the original task as a dependency of the new task.
- D.They can create a new job from scratch and add both tasks to run concurrently.
- E.They can clone the existing task to a new Job and then edit it to run the new notebook.

Answer: B

Explanation:

The correct answer is B: "They can create a new task in the existing Job and then add it as a dependency of the original task." Here's why:

The scenario requires executing a new notebook before the existing one within the same automated daily schedule. Jobs in Databricks provide the capability to define dependencies between tasks, enabling a directed acyclic graph (DAG) workflow.

Option B leverages this dependency feature perfectly. Creating a new task that executes the new notebook, and then configuring the original task to depend on the new task, ensures the new notebook finishes successfully before the original one starts. This achieves the required sequential execution.

Option A is incorrect because cloning and updating the existing task would simply replace the notebook it executes, not run one before it. Option C has the dependency relationship reversed. The original task depending on the new task means the original task would run after the new one, contradicting the

requirement. Option D proposes a new job, which is unnecessary. Creating a new job would decouple the original task and introduce scheduling complexities. Option E is also incorrect because moving a task to a new job does not solve the dependency problem. The data engineer aims to execute two notebooks sequentially within a single workflow.

In summary, Databricks Jobs' task dependency feature is specifically designed to handle sequential task execution within a workflow. Option B correctly applies this feature to meet the scenario's requirements.

For more information, refer to the Databricks documentation on Jobs:

<https://docs.databricks.com/workflows/jobs/index.html> and task dependencies:

<https://docs.databricks.com/workflows/jobs/jobs-concepts.html#dependencies>.

Question: 38

Deep Dumps

An engineering manager wants to monitor the performance of a recent project using a Databricks SQL query. For the first week following the project's release, the manager wants the query results to be updated every minute. However, the manager is concerned that the compute resources used for the query will be left running and cost the organization a lot of money beyond the first week of the project's release.

Which of the following approaches can the engineering team use to ensure the query does not cost the organization any money beyond the first week of the project's release?

- A.They can set a limit to the number of DBUs that are consumed by the SQL Endpoint.
- B.They can set the query's refresh schedule to end after a certain number of refreshes.
- C.They cannot ensure the query does not cost the organization money beyond the first week of the project's release.
- D.They can set a limit to the number of individuals that are able to manage the query's refresh schedule.
- E.They can set the query's refresh schedule to end on a certain date in the query scheduler.

Answer: E

Explanation:

The correct answer is **E: They can set the query's refresh schedule to end on a certain date in the query scheduler.**

Here's a detailed justification:

Databricks SQL provides a query scheduler that allows users to define how frequently a query should be executed and its results updated. The key to controlling costs in this scenario is to ensure that the query stops running after the first week.

Option E directly addresses this requirement. By setting an end date in the query scheduler, you can ensure that the scheduled refreshes will automatically stop after the specified date, preventing further compute resource consumption and associated costs. This provides a precise and automated way to stop the query from running beyond the intended period. The Databricks SQL UI enables users to specify the end date during schedule creation/modification.

Options A, B, C, and D are not suitable solutions:

A: Limiting DBUs consumed by the SQL Endpoint may prevent a single query from consuming excessive resources, but it does not guarantee that the query will stop running after the first week. It only controls the resource usage per run, not the overall duration of the query's execution.

B: Setting a limit on the number of refreshes might work if the query is refreshed at exact one-minute intervals. However, it is not a robust solution because intervals may vary and will cause the query to stop early or run late and cost additional money.

C: This statement is incorrect. The query execution can be precisely controlled.

D: Limiting the individuals that manage the refresh schedule doesn't automatically ensure that the query costs are limited.

Ultimately, the best way to limit query costs is to schedule the query to stop running automatically after the needed period, which is achieved by specifying an end date in the scheduler.

Here are some useful links for further information:

[Databricks SQL scheduling](#)

[Optimize Databricks SQL costs](#)

Question: 39

Deep Dumps

A data analysis team has noticed that their Databricks SQL queries are running too slowly when connected to their always-on SQL endpoint. They claim that this issue is present when many members of the team are running small queries simultaneously. They ask the data engineering team for help. The data engineering team notices that each of the team's queries uses the same SQL endpoint.

Which of the following approaches can the data engineering team use to improve the latency of the team's queries?

- A.They can increase the cluster size of the SQL endpoint.
- B.They can increase the maximum bound of the SQL endpoint's scaling range.
- C.They can turn on the Auto Stop feature for the SQL endpoint.
- D.They can turn on the Serverless feature for the SQL endpoint.
- E.They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy to "Reliability Optimized."

Answer: B

Explanation:

The problem described involves slow Databricks SQL query performance due to concurrent small queries overwhelming a single SQL endpoint. The goal is to improve query latency under high concurrency.

Option B, increasing the maximum bound of the SQL endpoint's scaling range, is the most appropriate solution. A SQL endpoint is a compute resource, and if multiple users simultaneously submit queries, the endpoint may become overloaded. By increasing the maximum bound of the scaling range, the endpoint can automatically scale up to a larger size (more compute resources) to handle the increased load. This allows more queries to be processed concurrently, reducing latency.

Option A, increasing the cluster size, could help temporarily but doesn't address the dynamic nature of the workload. Increasing the cluster size directly sets a fixed size and doesn't automatically adapt to fluctuations in query load. A larger fixed size might lead to underutilization and unnecessary costs during periods of lower activity.

Option C, turning on Auto Stop, is counterproductive. Auto Stop is designed to shut down the endpoint after a period of inactivity to save costs. This would only exacerbate the problem because the endpoint would need to restart frequently, adding significant latency to the initial queries after periods of inactivity.

Option D, turning on the Serverless feature is also a very good answer and, depending on Databricks account configuration, may be the best answer. Databricks Serverless SQL Endpoints (if available) automatically scale resources based on workload demands and are managed entirely by Databricks. This removes the need to manually configure and manage cluster sizes. Serverless SQL is designed for exactly this type of scenario: concurrent small queries from many users.

Option E is redundant. Turning on Serverless inherently involves Databricks managing the infrastructure. Changing the Spot Instance Policy is a more fine-grained optimization option, but not the primary solution for this issue. While "Reliability Optimized" can improve things, Serverless as a whole is more impactful.

The most appropriate solution is therefore either B or D, depending on the context and the specific architecture of the Data Engineering Team's Databricks platform. B is acceptable as the answer if Serverless is not an option, or considered too risky to enable immediately.

Here are some resources:

Databricks SQL Endpoints: <https://docs.databricks.com/sql/admin/sql-endpoints.html>

Databricks Serverless SQL: <https://docs.databricks.com/sql/admin/serverless.html>

Autoscaling Databricks Clusters: <https://docs.databricks.com/clusters/auto-scaling.html>

Question: 40

Deep Dumps

A data engineer wants to schedule their Databricks SQL dashboard to refresh once per day, but they only want the associated SQL endpoint to be running when it is necessary.

Which of the following approaches can the data engineer use to minimize the total running time of the SQL endpoint used in the refresh schedule of their dashboard?

- A.They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.
- B.They can set up the dashboard's SQL endpoint to be serverless.
- C.They can turn on the Auto Stop feature for the SQL endpoint.
- D.They can reduce the cluster size of the SQL endpoint.
- E.They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.

Answer: C

Explanation:

The correct answer is C, enabling the Auto Stop feature for the SQL endpoint. Here's a detailed justification:

The goal is to minimize the total running time (and thus cost) of the SQL endpoint used by the dashboard's refresh schedule. The most direct way to achieve this is to automatically shut down the endpoint when it's not actively processing queries for the dashboard refresh. The "Auto Stop" feature does precisely that. When enabled, the SQL endpoint will automatically terminate after a specified period of inactivity. This prevents the endpoint from running unnecessarily when the dashboard doesn't need to be refreshed, drastically reducing compute costs.

Option A is incorrect because matching SQL endpoints of queries to the dashboard doesn't inherently minimize runtime. The endpoint still needs to run to execute the queries.

Option B (serverless) is not applicable to Databricks SQL. Serverless compute for dashboards and SQL workloads is a feature available in other cloud platforms but is not a direct option in Databricks SQL as of the current available knowledge base.

Option D (reducing cluster size) might reduce costs per hour of runtime, but it doesn't guarantee minimizing total runtime. A smaller cluster might take longer to execute the queries, potentially offsetting the cost savings. It can also cause performance issues and query timeouts if the cluster is too small to handle the workload.

Option E is incorrect. There's no performance reason for the dashboard's SQL endpoint not to match the queries.

In summary, the Auto Stop feature directly addresses the requirement to minimize the running time of the

SQL endpoint by automatically shutting it down when not in use, leading to significant cost savings.

Authoritative Link:

While a direct link focusing solely on the 'Auto Stop' feature within Databricks documentation is limited (the interface and location of that setting can change), the core concept of auto-scaling and auto-stopping compute resources is a fundamental principle in cloud computing cost optimization. Search the Databricks documentation for "SQL endpoint configuration" and "cost management" for relevant details, as well as looking for topics on "Auto Stop." The UI for auto-stopping is generally located in the SQL endpoint settings configuration.

Question: 41

Deep Dumps

A data engineer has been using a Databricks SQL dashboard to monitor the cleanliness of the input data to an ELT job. The ELT job has its Databricks SQL query that returns the number of input records containing unexpected NULL values. The data engineer wants their entire team to be notified via a messaging webhook whenever this value reaches 100.

Which of the following approaches can the data engineer use to notify their entire team via a messaging webhook whenever the number of NULL values reaches 100?

- A. They can set up an Alert with a custom template.
- B. They can set up an Alert with a new email alert destination.
- C. They can set up an Alert with a new webhook alert destination.
- D. They can set up an Alert with one-time notifications.
- E. They can set up an Alert without notifications.

Answer: C

Explanation:

The correct answer is C: They can set up an Alert with a new webhook alert destination.

Here's why: The requirement is to notify the team via a messaging webhook when a specific threshold (100 NULL values) is reached. Databricks SQL Alerts are specifically designed to trigger notifications based on query results meeting certain conditions. Alert destinations define where and how these notifications are sent.

Option A is incorrect because while custom templates allow formatting the alert message, they don't dictate where the alert is sent. A webhook destination is needed for that.

Option B is incorrect because email alerts are designed for email notifications, not for sending data to messaging webhooks.

Option D is incorrect because "one-time notifications" is not a standard feature within Databricks SQL Alerts. Alerts are designed to trigger repeatedly based on the specified condition being met.

Option E is incorrect because the goal is to notify the team; therefore, notifications are essential.

Webhooks are a standard way for applications to send real-time information to other applications. In this context, Databricks SQL can send data (specifically, the query result showing NULL values) to a messaging platform like Slack, Microsoft Teams, or any other system that accepts webhooks. Setting up a "new webhook alert destination" configures Databricks SQL to post data to a specific URL whenever the query result triggers the alert condition (NULL values ≥ 100). The messaging service listens at this URL and processes the data to send a notification to the team.

Therefore, the most appropriate solution is to configure an Alert with a new webhook alert destination to achieve real-time notifications to the team when the threshold is met.

Further research:

Databricks SQL Alerts: <https://docs.databricks.com/sql/user/alerts/index.html>

Databricks SQL Alert Destinations: <https://docs.databricks.com/sql/user/alerts/destinations.html>

Webhooks: Search for "webhooks" on the Databricks website or general cloud computing resources to learn more about them and how they are used for integration.

Question: 42

Deep Dumps

A single Job runs two notebooks as two separate tasks. A data engineer has noticed that one of the notebooks is running slowly in the Job's current run. The data engineer asks a tech lead for help in identifying why this might be the case.

Which of the following approaches can the tech lead use to identify why the notebook is running slowly as part of the Job?

- A. They can navigate to the Runs tab in the Jobs UI to immediately review the processing notebook.
- B. They can navigate to the Tasks tab in the Jobs UI and click on the active run to review the processing notebook.
- C. They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.
- D. There is no way to determine why a Job task is running slowly.
- E. They can navigate to the Tasks tab in the Jobs UI to immediately review the processing notebook.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most suitable approach for identifying the cause of a slow-running notebook in a Databricks Job, along with explanations for why the other options are less effective or incorrect:

Justification for Option C:

The Databricks Jobs UI is designed to provide comprehensive monitoring and debugging capabilities for job executions. When a data engineer observes a slow-running notebook within a Job, the most direct approach is to leverage the Jobs UI to inspect the active run. Navigating to the "Runs" tab displays a list of all job executions, including the active one. By clicking on the active run, the tech lead can drill down into specific details of that execution. Within the run details, Databricks provides metrics, logs, and execution details that are invaluable for diagnosing performance bottlenecks. Specifically, the tech lead can examine the following:

1. **Task duration:** Identify if the notebook execution time is unexpectedly long compared to historical runs.
2. **Spark UI integration:** Databricks Jobs seamlessly integrate with the Spark UI. From the run details, you can access the Spark UI to analyze the underlying Spark jobs, stages, and tasks associated with the notebook execution. This allows you to pinpoint bottlenecks like data skew, inefficient transformations, or resource contention.
3. **Driver and executor logs:** The logs provide detailed insights into the execution of the notebook's code. Error messages, warnings, or performance-related logging statements can provide valuable clues to the cause of the slowdown.
4. **Resource utilization:** Databricks monitors CPU, memory, and disk I/O usage for the driver and executors involved in the notebook execution. High resource utilization might indicate that the cluster is undersized or that the notebook's code is inefficient.

5. **Data volume:** Analyze the amount of data being processed by the notebook. Increased data volume compared to previous runs could be a contributing factor to the slowdown.

By using this information, the tech lead can isolate the cause of the slowdown, determine whether it's related to code inefficiencies, resource limitations, or external factors, and propose appropriate solutions.

Why other options are not as good:

Option A: While the Runs tab is the starting point, it's not the immediate review of the notebook. It requires an extra step of clicking on the specific run.

Option B & E: The "Tasks" tab might offer some high-level information about the tasks within the job, it does not provide the necessary context for analyzing performance bottlenecks within a notebook's execution, especially not without first navigating to the active run. The task tab is more suitable for multi-task jobs where individual task status needs monitoring.

Option D: This is incorrect. Databricks provides extensive tools and features for monitoring and debugging Job executions.

Authoritative links:

[Databricks Jobs](#)

[Monitor Databricks Jobs](#)

[Spark UI](#)

Question: 43

Deep Dumps

A data engineer has a Job with multiple tasks that runs nightly. Each of the tasks runs slowly because the clusters take a long time to start.

Which of the following actions can the data engineer perform to improve the start up time for the clusters used for the Job?

- A. They can use endpoints available in Databricks SQL
- B. They can use jobs clusters instead of all-purpose clusters
- C. They can configure the clusters to be single-node
- D. They can use clusters that are from a cluster pool
- E. They can configure the clusters to autoscale for larger data sizes

Answer: D

Explanation:

The correct answer is D: They can use clusters that are from a cluster pool. Here's why:

Cluster startup time is a common bottleneck in Databricks Jobs, especially when clusters are created from scratch for each job run. Cluster pools address this issue by pre-allocating a set of idle instances ready for use. When a job requests a cluster that's configured to use a pool, Databricks attempts to allocate an instance from the pool first. This is significantly faster than provisioning a completely new cluster.

Options A and B are incorrect. Endpoints in Databricks SQL are for serving SQL queries, not for accelerating job startup times. Jobs clusters versus all-purpose clusters doesn't affect startup time, but rather lifecycle management (job clusters are automatically terminated after the job completes).

Option C is incorrect. Single-node clusters might be faster to provision, but they are generally inadequate for data engineering workloads as they lack the distributed processing capabilities needed to efficiently handle large datasets. Moreover, reducing resources doesn't directly address the initial startup delay.

Option E is related to scaling an already-running cluster. Autoscaling helps adjust the cluster size during a job based on workload demands, but doesn't affect the initial cluster startup time.

Cluster pools optimize the allocation of resources, significantly reducing the time required for jobs to begin execution. By reusing pre-warmed instances, the overhead associated with provisioning and configuring new clusters is minimized.

For more information on cluster pools, refer to the official Databricks documentation:

<https://docs.databricks.com/clusters/instance-pools/index.html>

Question: 44

Deep Dumps

A new data engineering team has been assigned to an ELT project. The new data engineering team will need full privileges on the database customers to fully manage the project.

Which of the following commands can be used to grant full permissions on the database to the new data engineering team?

- A. GRANT USAGE ON DATABASE customers TO team;
- B. GRANT ALL PRIVILEGES ON DATABASE team TO customers;
- C. GRANT SELECT PRIVILEGES ON DATABASE customers TO teams;
- D. GRANT SELECT CREATE MODIFY USAGE PRIVILEGES ON DATABASE customers TO team;
- E. GRANT ALL PRIVILEGES ON DATABASE customers TO team;

Answer: E

Explanation:

The correct answer is E: GRANT ALL PRIVILEGES ON DATABASE customers TO team;

Here's why:

The goal is to grant the data engineering team full privileges on the customers database. This implies the ability to perform all possible operations on the database, including creating, reading, updating, deleting, and managing its objects. The GRANT ALL PRIVILEGES command is specifically designed for this purpose.

Option A, GRANT USAGE ON DATABASE customers TO team;, only grants the ability to access the database, not to create or modify objects within it. It's a necessary but insufficient privilege for full management.

Option B, GRANT ALL PRIVILEGES ON DATABASE team TO customers;, reverses the subject and object. It attempts to give privileges to the customers database based on the existence of a team database, which is incorrect syntax and doesn't fulfill the requirement.

Option C, GRANT SELECT PRIVILEGES ON DATABASE customers TO teams;, grants only read access (SELECT) and misuses the word teams (plural). It does not provide the other necessary privileges for managing the database.

Option D, GRANT SELECT CREATE MODIFY USAGE PRIVILEGES ON DATABASE customers TO team;, grants several important permissions but does not grant all possible permissions. For example, it excludes DROP, ALTER, or OWNERSHIP capabilities, which are essential for full management control.

Therefore, only option E grants all the privileges necessary for the data engineering team to fully manage the customers database. ALL PRIVILEGES is a shorthand to specify all possible permissions allowed on the specified securable (in this case, a database).

For further information on Databricks SQL privileges and GRANT syntax, consult the official Databricks

documentation:

[Databricks SQL Privileges and Securable Objects](#)
[GRANT statement \(Databricks SQL\)](#)

Question: 45

Deep Dumps

A new data engineering team has been assigned to work on a project. The team will need access to database customers in order to see what tables already exist. The team has its own group team. Which of the following commands can be used to grant the necessary permission on the entire database to the new team?

- A. GRANT VIEW ON CATALOG customers TO team;
- B. GRANT CREATE ON DATABASE customers TO team;
- C. GRANT USAGE ON CATALOG team TO customers;
- D. GRANT CREATE ON DATABASE team TO customers;
- E. GRANT USAGE ON DATABASE customers TO team;

Answer: E

Explanation:

The correct answer is **E. GRANT USAGE ON DATABASE customers TO team;**

Here's why:

The question asks how to grant the team the permission to see what tables exist within the customers database. The USAGE privilege on a database (in Databricks, which operates on the Apache Spark SQL engine and leverages its metastore) is the key to allowing a principal (in this case, the team group) to access the database's metadata. Without USAGE, even if a user or group has specific permissions on tables within the database, they will not be able to list the tables present. USAGE is a prerequisite for accessing any objects within the database.

Option A, GRANT VIEW ON CATALOG customers TO team; is incorrect because there is no VIEW privilege on a catalog or database.

Option B, GRANT CREATE ON DATABASE customers TO team; is wrong because CREATE permission would allow the team to create new tables in the database, which isn't needed only to see existing tables.

Option C, GRANT USAGE ON CATALOG team TO customers; is incorrect because it reverses the objects involved in the grant statement. It also tries to grant access on catalog named team to the object called customers, which is likely not what you intended.

Option D, GRANT CREATE ON DATABASE team TO customers; is also incorrect because it is trying to grant permissions on a database named team to something called customers. The logic of the objects involved also makes no sense.

Essentially, USAGE on a database enables the principal to traverse the database and see its contents (tables, views, etc.). Other permissions, like SELECT or MODIFY, control what the principal can do with the data within those objects. To explore the database structure without modifying data, the USAGE privilege is sufficient and necessary.

For more information, review the Databricks SQL documentation on GRANT statements and access control:

[Databricks SQL GRANT statement](#)

Question: 46**Deep Dumps**

A data engineer is running code in a Databricks Repo that is cloned from a central Git repository. A colleague of the data engineer informs them that changes have been made and synced to the central Git repository. The data engineer now needs to sync their Databricks Repo to get the changes from the central Git repository.

Which of the following Git operations does the data engineer need to run to accomplish this task?

- A.Merge
- B.Push
- C.Pull
- D.Commit
- E.Clone

Answer: C

Explanation:

The correct answer is **C. Pull**. Here's a detailed justification:

When a Databricks Repo is cloned from a central Git repository, it becomes a local copy of the remote repository. The data engineer's goal is to update their local Repo with the latest changes from the central (remote) repository. Git provides several operations for managing these changes, and the appropriate one depends on the desired outcome.

Pull: The git pull command is used to fetch changes from a remote repository and then automatically merge them into the current branch. It's essentially a combination of git fetch (retrieving the remote changes) and git merge (integrating those changes into the local branch). In this scenario, the data engineer needs to get the changes made by their colleague from the central Git repository and integrate them into their local Databricks Repo. Therefore, git pull is the correct operation.

Merge: The git merge command integrates changes from one branch into another. While merging is part of the pull operation, it's not the sole command needed to retrieve changes from the remote repository in the first place. A pull automatically does a fetch + merge.

Push: The git push command is used to upload local repository content to a remote repository. It's the opposite of pulling; the data engineer is trying to receive changes, not contribute them in this scenario.

Commit: The git commit command records changes to the local repository. It prepares changes to be pushed to the remote repo later. However, it doesn't retrieve any remote changes.

Clone: The git clone command creates a copy of a remote repository. The data engineer already has a cloned Repo and only needs to sync it with the latest changes. Cloning is a one-time operation to initially create a local copy.

In summary, the data engineer should use git pull to download and merge the changes from the central Git repository into their Databricks Repo.

Authoritative Links for further research:

Git Basics - Getting a Git Repository: <https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>

Git Branching - Basic Branching and Merging: <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>

Question: 47

Deep Dumps

Which of the following is a benefit of the Databricks Lakehouse Platform embracing open source technologies?

- A. Cloud-specific integrations
- B. Simplified governance
- C. Ability to scale storage
- D. Ability to scale workloads
- E. Avoiding vendor lock-in

Answer: E

Explanation:

The correct answer is **E. Avoiding vendor lock-in**.

Here's a detailed justification:

Databricks' embrace of open-source technologies like Delta Lake, Apache Spark, and MLflow is a cornerstone of its Lakehouse platform. Vendor lock-in occurs when using proprietary technologies that tightly couple users to a specific vendor's ecosystem, making it difficult and costly to migrate data, code, and processes to another platform.

By leveraging open-source technologies, Databricks allows users to maintain greater control over their data and processing. They are not locked into proprietary formats or APIs. Users can leverage their skills and knowledge across different environments, increasing portability. Open standards promote interoperability and reduce the risk of being dependent on a single vendor's roadmap or pricing.

Specifically:

Delta Lake, an open-source storage layer, provides ACID transactions, scalable metadata handling, and unified streaming and batch data processing. Using Delta Lake avoids proprietary data formats.

Apache Spark, an open-source distributed processing engine, is the core compute framework of the Lakehouse. Its wide adoption and open nature mean that data engineers have a large pool of skills and can run Spark workloads on various infrastructures.

MLflow is an open-source platform to manage the ML lifecycle, promoting reproducibility and collaboration across different platforms.

Options A, B, C, and D are relevant benefits of the Databricks platform in general, but they are not direct consequences of embracing open-source technologies. While the Lakehouse platform leverages cloud-specific integrations (A) and allows for scaling storage (C) and workloads (D), these capabilities are provided via Databricks' infrastructure. Simplified governance (B) can be facilitated by specific open-source tools, but is also a function of the governance tools Databricks incorporates, not the open-source nature.

The key differentiator is that the open-source component directly addresses vendor lock-in because it provides alternatives to completely proprietary solutions. The other options are more about the platform's functionalities.

Further resources:

Delta Lake: <https://delta.io/>

Apache Spark: <https://spark.apache.org/>

MLflow: <https://mlflow.org/>

Question: 48

Deep Dumps

A data engineer needs to use a Delta table as part of a data pipeline, but they do not know if they have the appropriate permissions.

In which of the following locations can the data engineer review their permissions on the table?

- A.Databricks Filesystem
- B.Jobs
- C.Dashboards
- D.Repos
- E.Data Explorer

Answer: E

Explanation:

The correct answer is **E. Data Explorer**. Here's why:

The Data Explorer in Databricks is the primary user interface for discovering, exploring, and managing data objects within the Databricks environment. It provides a centralized location to view tables, databases, and their associated metadata. Crucially, this includes access control information. Specifically, Data Explorer displays the permissions that a user or group has on a particular table, including SELECT, MODIFY, CREATE, etc. A data engineer can easily navigate to the specific Delta table in Data Explorer and review the "Permissions" tab or similar section to understand their access rights.

The other options are incorrect because:

A. Databricks Filesystem (DBFS): DBFS is a distributed filesystem, and while Delta tables are stored as data files within DBFS, the filesystem itself doesn't inherently display access control lists (ACLs) related to Databricks SQL permissions or Delta table-specific permissions. While POSIX-style permissions exist on the underlying files, these are not the Databricks-level permissions that control access to the table via Spark SQL or Delta Lake APIs.

B. Jobs: Jobs are for scheduling and running notebooks or other tasks; they don't provide information about data object permissions.

C. Dashboards: Dashboards are visualization tools and do not expose table-level permissions.

D. Repos: Repos are used for version control of code and have nothing to do with data permissions.

Data Explorer, specifically, is designed to provide a graphical user interface to manage and inspect metadata, including security and access control. It provides a centralized place to discover, explore, and grant/revoke access to data assets managed by Databricks.

Authoritative links for further research:

Databricks Data Explorer documentation: <https://docs.databricks.com/data-governance/data-explorer/index.html>

Databricks Permissions Model: <https://docs.databricks.com/security/access-control/index.html>

Question: 49

Deep Dumps

Which of the following describes a scenario in which a data engineer will want to use a single-node cluster?

- A. When they are working interactively with a small amount of data
- B. When they are running automated reports to be refreshed as quickly as possible
- C. When they are working with SQL within Databricks SQL
- D. When they are concerned about the ability to automatically scale with larger data
- E. When they are manually running reports with a large amount of data

Answer: A

Explanation:

The correct answer is A: "When they are working interactively with a small amount of data." Here's why:

Single-node clusters in Databricks are designed for development, testing, and interactive data exploration with small datasets. They are not intended for production workloads or scenarios requiring high availability, fault tolerance, or scalability. The driver node acts as both the driver and worker, streamlining resource allocation for smaller tasks.

Option B is incorrect because automated reports often benefit from parallel processing for speed, which a single-node cluster lacks. Option C pertains to Databricks SQL, which leverages optimized engines and doesn't necessarily dictate the need for a single-node cluster; it's more about query performance. Option D is the opposite of what single-node clusters offer; they explicitly don't scale. Option E is wrong because large datasets necessitate distributed processing to be handled efficiently, which is beyond the capabilities of a single-node setup.

Single-node clusters offer a cost-effective and simple environment for initial data exploration, prototyping, and learning Databricks features. They're ideal for smaller datasets where the overhead of distributed computing would outweigh the performance gains. Interactivity is improved as there is no need to distribute the data across multiple nodes for simple analysis.

For further reading on Databricks clusters and their use cases:

Databricks documentation on cluster configuration: <https://docs.databricks.com/clusters/index.html>

Databricks documentation on single-node clusters: <https://docs.databricks.com/clusters/single-node.html>

Question: 50

Deep Dumps

A data engineer has been given a new record of data:

```
id STRING = 'a1'  
rank INTEGER = 6  
rating FLOAT = 9.4
```

Which of the following SQL commands can be used to append the new record to an existing Delta table my_table?

- A. INSERT INTO my_table VALUES ('a1', 6, 9.4)
- B. my_table UNION VALUES ('a1', 6, 9.4)
- C. INSERT VALUES ('a1', 6, 9.4) INTO my_table
- D. UPDATE my_table VALUES ('a1', 6, 9.4)
- E. UPDATE VALUES ('a1', 6, 9.4) my_table

Answer: A

Explanation:

The correct answer is A: INSERT INTO my_table VALUES ('a1', 6, 9.4). This is because it follows the standard

SQL syntax for inserting new rows into a table.

Here's a breakdown:

INSERT INTO my_table: This specifies the target table (my_table) where the new record will be inserted. The INSERT INTO clause is the foundation for adding data to a table.

VALUES ('a1', 6, 9.4): This provides the values for the new record. The values are specified in the same order as the columns defined in the my_table schema: id (STRING), rank (INTEGER), and rating (FLOAT). String values are enclosed in single quotes, while numeric values are not. The number of provided values must match the number of columns you are populating in your table (unless default values are defined for some columns).

Let's examine why the other options are incorrect:

B. my_table UNION VALUES ('a1', 6, 9.4): UNION is used to combine the result sets of two or more SELECT statements. It does not insert data directly into a table. It also requires SELECT statements on both sides of the UNION operator.

C. INSERT VALUES ('a1', 6, 9.4) INTO my_table: The SQL syntax is incorrect. The INTO my_table part should come directly after INSERT.

D. UPDATE my_table VALUES ('a1', 6, 9.4): UPDATE is used to modify existing rows in a table, not to insert new rows. Furthermore, the UPDATE statement requires a WHERE clause to specify which rows should be updated. VALUES keyword is not used with UPDATE statement.

E. UPDATE VALUES ('a1', 6, 9.4) my_table: The syntax is completely incorrect for an UPDATE statement.

Delta Lake and SQL:

Delta Lake, which Databricks uses extensively, supports standard SQL commands. The INSERT INTO command works seamlessly with Delta tables for appending new data. Delta Lake extends the capabilities of Apache Spark with ACID transactions, scalable metadata handling, and unified streaming and batch data processing. It ensures data reliability when writing and reading data. Using the correct SQL syntax ensures that the data insertion operation is consistent and reliable within the Delta Lake environment.

Further Research:

Delta Lake Documentation: <https://docs.databricks.com/delta/index.html>

SQL INSERT Statement: https://www.w3schools.com/sql/sql_insert.asp

Apache Spark SQL Documentation: <https://spark.apache.org/docs/latest/sql-ref.html>

Question: 51

Deep Dumps

A data engineer has realized that the data files associated with a Delta table are incredibly small. They want to compact the small files to form larger files to improve performance.

Which of the following keywords can be used to compact the small files?

- A. REDUCE
- B. OPTIMIZE
- C. COMPACTION
- D. REPARTITION
- E. VACUUM

Answer: B

Explanation:

The correct answer is **B. OPTIMIZE**. Here's a detailed justification:

The primary purpose of the OPTIMIZE command in Databricks Delta Lake is to compact small files within a Delta table into larger files. This process, often referred to as bin-packing, aims to reduce the number of files that the query engine needs to read when querying the table. Fewer, larger files generally result in improved query performance, particularly for workloads that involve scanning large portions of the dataset.

Delta Lake stores data in Parquet format, and having numerous small Parquet files can lead to increased metadata overhead and slower query execution. The OPTIMIZE command addresses this issue by rewriting these small files into fewer, larger ones, thereby optimizing the data layout for better performance.

The REDUCE keyword isn't a standard Delta Lake command. COMPACTION is a general term describing the process, but OPTIMIZE is the specific command used in Databricks Delta Lake to perform file compaction. REPARTITION is used to change the number of partitions in a DataFrame, and while it can affect file size, it's not the direct tool for compacting existing small files. VACUUM removes files that are no longer part of the current table state (old versions), it doesn't compact files.

Therefore, OPTIMIZE is the most suitable keyword to compact small files in a Databricks Delta table, enhancing query performance by reducing the overhead associated with numerous small files.

Here are some authoritative links for further research:

Databricks Documentation on OPTIMIZE: <https://docs.databricks.com/optimizations/optimize.html>
Delta Lake Documentation: <https://delta.io/>

Question: 52**Deep Dumps**

In which of the following file formats is data from Delta Lake tables primarily stored?

- A.Delta
- B.CSV
- C.Parquet
- D.JSON
- E.A proprietary, optimized format specific to Databricks

Answer: C**Explanation:**

The correct answer is Parquet (C). Delta Lake leverages Parquet as its underlying storage format for data within its tables. While Delta Lake introduces a transactional layer and other features on top, the actual data itself is stored in Parquet files.

Parquet is a columnar storage format. This characteristic offers significant advantages for analytical workloads common in data engineering. Columnar storage enables efficient data retrieval because only the necessary columns for a query need to be read, minimizing I/O operations. This significantly accelerates query performance compared to row-oriented formats like CSV or JSON, where entire rows must be read even if only a few columns are relevant.

Delta Lake's transactional guarantees, versioning, and schema evolution are implemented by storing metadata alongside the Parquet data files. This metadata, including a transaction log, tracks changes to the data and allows for ACID properties. However, the core data persistence relies on Parquet's efficient storage

and retrieval capabilities. Delta Lake doesn't create its own proprietary data format; it optimizes and extends the functionality of Parquet for data lake use cases. While Delta Lake can ingest data from other formats (like CSV or JSON), when that data is stored in a Delta Lake table, it is converted to Parquet. The Delta format then wraps the data for transactional integrity.

Therefore, while Delta Lake provides enhanced capabilities, Parquet remains the fundamental file format for data storage within Delta Lake tables.

For more information, refer to the official Databricks documentation:

Delta Lake Overview: <https://docs.databricks.com/delta/index.html>

Parquet Format: Understanding Parquet will involve researching various resources, as the Databricks documentation is more centered on Delta Lake usage than on a deep dive into Parquet itself. General resources on Parquet are beneficial, such as those from Apache.

Question: 53

Deep Dumps

Which of the following is stored in the Databricks customer's cloud account?

- A. Databricks web application
- B. Cluster management metadata
- C. Repos
- D. Data
- E. Notebooks

Answer: D

Explanation:

The correct answer is D (Data). Here's a detailed justification:

Databricks operates within the customer's cloud account (AWS, Azure, or GCP), employing a control plane/data plane architecture. The control plane, managed by Databricks, handles cluster management, job orchestration, and the web application. This includes components like cluster metadata (B), Repos (C), and Notebooks (E) which are essentially metadata or code pointers, not the raw data itself. These items are stored securely and efficiently in Databricks-managed infrastructure, enabling scalable computation.

The crucial distinction is where the actual data resides. Databricks doesn't store customer's data in its managed control plane. Instead, Databricks clusters are configured to access data stored in the customer's own cloud storage services, such as AWS S3, Azure Blob Storage, or Google Cloud Storage. This means the underlying data (D) remains entirely within the customer's cloud account, providing them with control over data governance, security, and compliance. Databricks only processes and analyzes this data based on the instructions given in the code (Notebooks, for example), but it doesn't permanently store copies of it. The web application (A) is also part of the Databricks control plane and therefore not residing within the customer's cloud account. The benefit of this architecture is enhanced data security and governance because data never leaves the customer's environment. It also ensures cost optimization because the customers can leverage their existing cloud storage investments.

For more information, refer to the Databricks documentation on the Databricks architecture and security model:

Databricks Architecture: <https://docs.databricks.com/en/getting-started/overview.html>

Databricks Security: <https://www.databricks.com/trust>

Question: 54**Deep Dumps**

Which of the following can be used to simplify and unify siloed data architectures that are specialized for specific use cases?

- A. None of these
- B. Data lake
- C. Data warehouse
- D. All of these
- E. Data lakehouse

Answer: E**Explanation:**

The correct answer is **E. Data lakehouse**. Here's a detailed justification:

Siloed data architectures are common when organizations use specialized systems for different tasks. For example, a data warehouse might handle structured reporting, while a data lake stores raw, unstructured data for data science. This creates data duplication, complexity in data pipelines, and difficulties in consistent data governance.

A data lake can store diverse data types in a central repository, but it often lacks strong transactional support and governance features needed for reliable analytical use cases, leading to "data swamps." A data warehouse can provide structured, governed data, but typically struggles with the volume and variety of data found in modern enterprise environments.

A data lakehouse architecture aims to combine the best of both worlds. It offers the low-cost storage and scalability of a data lake with the data management and performance capabilities of a data warehouse. This unification allows organizations to work with all their data – structured, semi-structured, and unstructured – within a single system. Key features of a data lakehouse include:

Schema enforcement and governance: Metadata management and schema evolution ensure data quality and consistency.

ACID transactions: Support for transactional consistency across multiple data updates.

Direct access to data: Enables diverse workloads, including SQL analytics, data science, and machine learning, directly on the data in the lakehouse.

End-to-end streaming: Real-time data ingestion and processing.

Open formats: Usually based on open file formats like Parquet and Delta Lake to avoid vendor lock-in.

Therefore, a data lakehouse can significantly simplify and unify data architectures that are specialized for specific use cases by eliminating data silos and providing a single source of truth for all data-driven initiatives.

Authoritative links for further research:

Databricks Data Lakehouse: <https://www.databricks.com/glossary/data-lakehouse>

Microsoft: What is a data lakehouse?: <https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/data-lakehouse>

Question: 55**Deep Dumps**

A data architect has determined that a table of the following format is necessary:

employeeId	startDate	avgRating
a1	2009-01-06	5.5
a2	2018-11-21	7.1
...	...	...

Which of the following code blocks uses SQL DDL commands to create an empty Delta table in the above format regardless of whether a table already exists with this name?

- A.
- ```
CREATE TABLE IF NOT EXISTS table_name (
 employeeId STRING,
 startDate DATE,
 avgRating FLOAT
)
```
- B.
- ```
CREATE OR REPLACE TABLE table_name AS
SELECT
    employeeId STRING,
    startDate DATE,
    avgRating FLOAT
USING DELTA
```
- C.
- ```
CREATE OR REPLACE TABLE table_name WITH COLUMNS (
 employeeId STRING,
 startDate DATE,
 avgRating FLOAT
) USING DELTA
```
- D.



```
CREATE TABLE table_name AS
SELECT
 employeeId STRING,
 startDate DATE,
 avgRating FLOAT
```

E.

```
CREATE OR REPLACE TABLE table_name (
 employeeId STRING,
 startDate DATE,
 avgRating FLOAT
)
```

**Answer: E**

**Explanation:**

```
CREATE OR REPLACE TABLE table_name (
 employeeId STRING,
 startDate DATE,
 avgRating FLOAT
)
```

### Question: 56

**Deep Dumps**

A data engineer has a Python notebook in Databricks, but they need to use SQL to accomplish a specific task within a cell. They still want all of the other cells to use Python without making any changes to those cells.

Which of the following describes how the data engineer can use SQL within a cell of their Python notebook?

- A. It is not possible to use SQL in a Python notebook
- B. They can attach the cell to a SQL endpoint rather than a Databricks cluster
- C. They can simply write SQL syntax in the cell
- D. They can add %sql to the first line of the cell
- E. They can change the default language of the notebook to SQL

**Answer: D**

**Explanation:**

The correct answer is **D. They can add %sql to the first line of the cell.**

Here's a detailed justification:

Databricks notebooks support polyglot programming, meaning you can use multiple languages (Python, SQL, R, Scala) within the same notebook. To achieve this, Databricks uses "magic commands" or "language magics". These are special commands that start with a percent sign (%) and tell the notebook to interpret the cell's content in a specific language.

In this scenario, the data engineer wants to use SQL within a Python notebook without altering the other cells. Prefixing a cell with %sql instructs Databricks to interpret the subsequent content of that specific cell as SQL code. The Databricks runtime then executes the SQL query against the attached Spark cluster, leveraging Spark SQL capabilities. This allows the data engineer to seamlessly incorporate SQL operations (like querying tables, creating views, or performing aggregations) within their predominantly Python-based workflow. The Python cells remain unaffected and continue to execute as Python code.

Option A is incorrect because Databricks notebooks are designed to support multiple languages. Option B is incorrect because attaching a cell to a SQL endpoint would require fundamental changes to the notebook structure and not allow for mixed language usage in other cells. Option C is wrong because writing SQL syntax directly in a Python cell will result in syntax errors, as the Python interpreter will not recognize the SQL code. Option E is incorrect because you change the cell to SQL and not the notebook.

Therefore, using the %sql magic command provides a localized and efficient way to integrate SQL into a Python Databricks notebook, enabling the data engineer to complete their task without disrupting the existing Python code.

#### Authoritative links:

**Databricks Documentation on Magic Commands:** <https://docs.databricks.com/notebooks/notebooks-use.html#mix-languages>

**Spark SQL:** <https://spark.apache.org/sql/>

#### Question: 57

Deep Dumps

Which of the following SQL keywords can be used to convert a table from a long format to a wide format?

- A. TRANSFORM
- B. PIVOT
- C. SUM
- D. CONVERT
- E. WHERE

**Answer: B**

#### Explanation:

The correct SQL keyword to convert a table from a long (narrow and tall) format to a wide (short and wide) format is PIVOT.

PIVOT is a relational operator that transforms rows into columns. In the context of data warehousing and data transformation, pivoting is a common technique used to reshape data for analysis and reporting. A long format table typically has multiple rows representing different measurements or attributes for the same entity, while a wide format table consolidates these measurements into a single row per entity, with each measurement becoming a separate column.

The PIVOT operator takes values from one column and turns them into new columns. It then aggregates

values from another column based on these new columns. For example, if a table has columns category, product, and sales, pivoting on category would create new columns for each category, and the sales column would be aggregated (e.g., summed) for each category.

The other options are incorrect because:

TRANSFORM is not a standard SQL keyword for reshaping data. It's more commonly associated with data transformation tools but doesn't directly perform pivoting in SQL.

SUM is an aggregate function used to calculate the sum of values but does not restructure the table. It's often used within a PIVOT query for aggregation.

CONVERT is used for data type conversions, not for reshaping data.

WHERE is used to filter rows based on specified conditions and doesn't change the table structure.

Databricks' implementation of PIVOT adheres to the standard SQL functionality, allowing for efficient and scalable data reshaping within their environment. This is crucial for preparing data for analysis and machine learning tasks on Databricks. Pivoting is a common data manipulation technique in Spark SQL, frequently used when preparing data frames for analysis.

For further research, refer to the official Databricks documentation on Spark SQL, specifically examples related to data transformation and pivoting. Additionally, consult SQL tutorials focusing on the PIVOT operator for various database systems.

Authoritative Link:

Apache Spark SQL Programming Guide: <https://spark.apache.org/docs/latest/sql-programming-guide.html>  
(Look for examples utilizing PIVOT for data transformation)

### Question: 58

Deep Dumps

Which of the following describes a benefit of creating an external table from Parquet rather than CSV when using a CREATE TABLE AS SELECT statement?

- A. Parquet files can be partitioned
- B. CREATE TABLE AS SELECT statements cannot be used on files
- C. Parquet files have a well-defined schema
- D. Parquet files have the ability to be optimized
- E. Parquet files will become Delta tables

**Answer: C**

**Explanation:**

The correct answer is C: Parquet files have a well-defined schema. Here's why:

When creating an external table using CREATE TABLE AS SELECT (CTAS), the source data's structure is critical. CSV files are inherently schema-less. While you can use CTAS on a CSV file (contradicting option B), you must explicitly define the schema during the table creation process. The CTAS statement will infer data types, but it relies on assumptions that may not always be accurate, leading to potential data type mismatches or incorrect data interpretation.

Parquet, on the other hand, is a columnar storage format that stores its schema within the file itself. When you use CTAS to create an external table from Parquet files, the Databricks runtime automatically infers the schema from the Parquet metadata. This eliminates the need to manually define the schema, reduces the risk of schema inconsistencies, and streamlines the table creation process.

While option A (Parquet files can be partitioned) is true in general, partitioning is a separate concern from the immediate benefit when using CTAS. Partitioning can be applied to both Parquet and CSV tables after creation. Option D (Parquet files have the ability to be optimized) is also true but less directly related to the benefit of CTAS. Parquet's columnar nature lends itself to optimization techniques, but the primary benefit in the context of CTAS is the automatic schema inference. Option E (Parquet files will become Delta tables) is incorrect; you would need to specifically create a Delta table, it doesn't happen automatically from a Parquet source with CTAS.

In essence, the self-describing nature of Parquet files, with their embedded schema, makes CREATE TABLE AS SELECT operations simpler, more reliable, and less prone to errors compared to using schema-less formats like CSV. Here are some resources for further reading:

**Apache Parquet:** <https://parquet.apache.org/>

**Databricks CREATE TABLE AS SELECT:** <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-as.html>

### Question: 59

Deep Dumps

A data engineer wants to create a relational object by pulling data from two tables. The relational object does not need to be used by other data engineers in other sessions. In order to save on storage costs, the data engineer wants to avoid copying and storing physical data.

Which of the following relational objects should the data engineer create?

- A. Spark SQL Table
- B. View
- C. Database
- D. Temporary view
- E. Delta Table

**Answer: D**

#### Explanation:

Here's a detailed justification for why the correct answer is D, Temporary View, along with explanations of why the other options are not the best choice:

The scenario requires creating a relational object based on two tables without physically storing or copying data and limiting its scope to the current session.

**Temporary Views (D)** perfectly satisfy these requirements. A temporary view is a view that exists only for the duration of the SparkSession in which it was created. It doesn't persist data to storage. It dynamically generates results based on the underlying tables each time it's queried. Moreover, it is not accessible to other sessions, aligning with the requirement that the object shouldn't be used by other data engineers.

Why other options are incorrect:

**Spark SQL Table (A):** Tables, by default, persist data to storage (e.g., cloud storage like AWS S3, Azure Data Lake Storage, or Google Cloud Storage). This contradicts the need to avoid physical storage and incur unnecessary cost.

**View (B):** While a regular view is also a logical representation without data duplication, it persists in the metastore and is available to other users and sessions. This contradicts the single-session requirement.

**Database (C):** A database is a container for tables, views, and functions. It doesn't directly address the requirement of creating a relational object by pulling data from two tables in a session-specific manner.

without storing data. Creating a database is not directly relevant to the described problem.

**Delta Table (E):** Delta Tables are an enhanced version of Spark tables and provide ACID transactions, scalable metadata handling, and time travel capabilities. However, they fundamentally persist data to storage as well and are not inherently session-specific.

In summary, a temporary view allows the data engineer to create a relational object derived from two tables without incurring storage costs or making it accessible to other sessions. This aligns perfectly with the problem's constraints.

Relevant Documentation:

**Spark SQL Views:** <https://spark.apache.org/docs/latest/sql-ref-syntax-ddl-create-view.html>

**Databricks Views:** <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html>

### Question: 60

### Deep Dumps

A data analyst has developed a query that runs against Delta table. They want help from the data engineering team to implement a series of tests to ensure the data returned by the query is clean. However, the data engineering team uses Python for its tests rather than SQL.

Which of the following operations could the data engineering team use to run the query and operate with the results in PySpark?

- A. `SELECT * FROM sales`
- B. `spark.delta.table`
- C. `spark.sql`
- D. There is no way to share data between PySpark and SQL.
- E. `spark.table`

**Answer: C**

**Explanation:**

The correct answer is C, `spark.sql`. Here's why:

The scenario requires running an SQL query (developed by the data analyst) within a PySpark environment for data quality testing. PySpark, which is the Python API for Apache Spark, provides several ways to interact with Spark's SQL engine.

Option A, `SELECT * FROM sales`, is a raw SQL query string. While syntactically correct SQL, it doesn't execute anything within a PySpark context on its own. It would need to be passed to a Spark execution function.

Option B, `spark.delta.table`, is specific to interacting with Delta tables. While the query operates on a Delta table, this option doesn't provide a mechanism to execute the analyst's arbitrary SQL query. It's used for specific Delta Lake operations like versioning or time travel.

Option C, `spark.sql`, is the most direct and appropriate way to run SQL queries from within PySpark. The `spark` object represents the `SparkSession`, and the `sql` method allows you to execute any valid SQL query against the Spark environment. The result of the query is a PySpark `DataFrame`, which can then be easily manipulated and tested using Python code. This directly allows the engineering team to work with results in PySpark.

Option D is incorrect because PySpark is specifically designed to bridge Python and SQL.

Option E, `spark.table`, is used to access an existing table registered in the Spark metastore as a `DataFrame`. While you could access the Delta table this way, it doesn't directly execute the arbitrary SQL the data analyst wrote. You'd need to write additional `DataFrame` transformations to recreate the query's logic. `spark.sql` is

much simpler.

Therefore, `spark.sql(query_string)` allows the data engineering team to take the analyst's SQL query (represented as `query_string`), execute it within the Spark environment, and receive the results as a PySpark `DataFrame`, which is readily usable for further processing and testing using Python.

#### Authoritative Links:

**Apache Spark SQL Programming Guide:** <https://spark.apache.org/docs/latest/sql-programming-guide.html>

**SparkSession:** <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.Session.html>

**Delta Lake documentation:** <https://docs.delta.io/latest/>

### Question: 61

Deep Dumps

Which of the following commands will return the number of null values in the `member_id` column?

- A. `SELECT count(member_id) FROM my_table;`
- B. `SELECT count(member_id) - count_null(member_id) FROM my_table;`
- C. `SELECT count_if(member_id IS NULL) FROM my_table;`
- D. `SELECT null(member_id) FROM my_table;`
- E. `SELECT count_null(member_id) FROM my_table;`

#### Answer: C

#### Explanation:

The correct answer is C, `SELECT count_if(member_id IS NULL) FROM my_table;` because it accurately counts the number of null values within the `member_id` column.

Let's break down why each option is or isn't correct:

**A. `SELECT count(member_id) FROM my_table;`** The `count()` function ignores null values. This command will return the number of non-null values in the `member_id` column, not the number of nulls.

**B. `SELECT count(member_id) - count_null(member_id) FROM my_table;`** The `count_null()` function is not a standard SQL or Spark SQL function. Therefore, this command will likely result in an error. Furthermore, even if such a function existed, this logic subtracts something that might not be directly equivalent to the total count of non-null values.

**C. `SELECT count_if(member_id IS NULL) FROM my_table;`** The `count_if()` function evaluates a boolean expression for each row and counts the number of times the expression is true. In this case, `member_id IS NULL` is a boolean expression that returns true if `member_id` is null and false otherwise. Therefore, `count_if(member_id IS NULL)` correctly returns the number of null values in the `member_id` column. This aligns with Databricks' and Spark SQL's functionality.

**D. `SELECT null(member_id) FROM my_table;`** The `null()` function does not exist in SQL or Spark SQL. This would cause an error. Even conceptually, the `null()` keyword is used to represent a null value, not to count them.

**E. `SELECT count_null(member_id) FROM my_table;`** Again, the `count_null()` function is not a standard SQL or Spark SQL function. It will likely cause an error.

Therefore, `count_if(member_id IS NULL)` is the only correct option because it uses a valid Spark SQL function (`count_if`) and a valid SQL comparison operator (`IS NULL`) to achieve the desired outcome of counting null



values. This directly addresses the problem of identifying rows where member\_id is missing. Here are some relevant links for further research:

**Apache Spark SQL Functions:** <https://spark.apache.org/docs/latest/api/sql/> (Look for count\_if in the documentation).

**SQL IS NULL operator:** [https://www.w3schools.com/sql/sql\\_isnull.asp](https://www.w3schools.com/sql/sql_isnull.asp)

**Databricks SQL Documentation:** <https://docs.databricks.com/sql/language-manual/functions.html> (Search for count\_if)

## Question: 62

## Deep Dumps

A data engineer needs to apply custom logic to identify employees with more than 5 years of experience in array column employees in table stores. The custom logic should create a new column exp\_employees that is an array of all of the employees with more than 5 years of experience for each row. In order to apply this custom logic at scale, the data engineer wants to use the FILTER higher-order function.

Which of the following code blocks successfully completes this task?

A.

```
SELECT
 store_id,
 employees,
 FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

B.

```
SELECT
 store_id,
 employees,
 FILTER (exp_employees, years_exp > 5) AS exp_employees
FROM stores;
```

C.

```
SELECT
 store_id,
 employees,
 FILTER (employees, years_exp > 5) AS exp_employees
FROM stores;
```

D.

```
SELECT
 store_id,
 employees,
 CASE WHEN employees.years_exp > 5 THEN employees
 ELSE NULL
 END AS exp_employees
FROM stores;
```

E.

```
SELECT
 store_id,
 employees,
 FILTER (exp_employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

**Answer: A**

**Explanation:**

```
SELECT
 store_id,
 employees,
 FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

### Question: 63

**Deep Dumps**

A data engineer has a Python variable `table_name` that they would like to use in a SQL query. They want to construct a Python code block that will run the query using `table_name`.

They have the following incomplete code block:

```
____(f"SELECT customer_id, spend FROM table_name ")
```

Which of the following can be used to fill in the blank to successfully complete the task?

- A.spark.delta.sql
- B.spark.delta.table
- C.spark.table
- D.dbutils.sql
- E.spark.sql

**Answer: E**

**Explanation:**

Here's a detailed justification for why option E, `spark.sql`, is the correct answer and why the other options are incorrect when constructing a dynamic SQL query using a Python variable in Databricks:

The core requirement is to execute a SQL query string that incorporates a Python variable, `table_name`. Databricks leverages Apache Spark for data processing. Spark provides several methods for interacting with data, including its own SQL engine.

**`spark.sql(query_string)`** is the standard and most direct way to execute a SQL query within a `SparkSession` (represented by `spark` in Databricks notebooks). It takes a string as input, which can be dynamically constructed using Python's f-strings to embed the `table_name` variable. The result of `spark.sql` is a `Spark DataFrame`, which can then be further processed. [Apache Spark SQL Programming Guide](#) - Look for the section describing running SQL queries programmatically.

**spark.delta.sql** is specific to Delta Lake functionality. While Delta Lake uses SQL syntax, **spark.delta.sql** isn't a standard function for executing arbitrary SQL queries with variable interpolation.

**spark.delta.table** is used to create or refer to a Delta table object, not to execute general SQL queries. It's used when you want to perform Delta Lake-specific operations (like time travel or schema evolution).

**spark.table** is used to access an existing table or view as a Spark DataFrame. It does not execute SQL queries.

**dbutils.sql** is not a valid method provided by Databricks **dbutils** module. **dbutils** provides a variety of utility functions within Databricks notebooks, mostly related to interacting with the Databricks environment (e.g., file system, secrets).

Therefore, using `spark.sql(f"SELECT customer_id, spend FROM table_name ")` correctly constructs a dynamic SQL query using the value of `table_name` and executes it using Spark's SQL engine. This ensures the query will run against the appropriate table and return a Spark DataFrame.

### Question: 64

Deep Dumps

A data engineer has created a new database using the following command:

```
CREATE DATABASE IF NOT EXISTS customer360;
```

In which of the following locations will the `customer360` database be located?

- A. `dbfs:/user/hive/database/customer360`
- B. `dbfs:/user/hive/warehouse`
- C. `dbfs:/user/hive/customer360`
- D. More information is needed to determine the correct response
- E. `dbfs:/user/hive/database`

**Answer: B**

**Explanation:**

The correct answer is **B. `dbfs:/user/hive/warehouse`**. Here's why:

When a database is created in Databricks without explicitly specifying a `LOCATION`, it defaults to using the Hive metastore's managed location. The Hive metastore stores metadata about your tables and databases, including their location. The default location for managed tables and databases in Databricks is `dbfs:/user/hive/warehouse`.

The `CREATE DATABASE IF NOT EXISTS customer360;` command creates a database named `customer360` if one doesn't already exist. Because no `LOCATION` clause is included in the command, the database's data files will be stored in the default Hive warehouse location.

Option A (`dbfs:/user/hive/database/customer360`) is incorrect because this path is usually where the database directory would be if the location were explicitly specified during the database creation. However, without a `LOCATION` clause, the database itself isn't created inside a 'database' subfolder.

Option C (`dbfs:/user/hive/customer360`) is incorrect because the default path does not directly append the database name to `/user/hive/`.

Option D is incorrect because sufficient information is provided in the command to determine the default location. The lack of a `LOCATION` clause is key.

Option E (dbfs:/user/hive/database) is incorrect because the tables will exist within the default directory(/user/hive/warehouse).

In summary, in absence of the LOCATION clause in the CREATE DATABASE statement, Databricks utilizes the default Hive warehouse location for storing metadata.

Further research:

**Databricks documentation on Hive Metastore:** <https://docs.databricks.com/data/metastores/index.html>

**Databricks documentation on CREATE DATABASE:** <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-database.html> - Check the section on the LOCATION clause.

### Question: 65

Deep Dumps

A data engineer is attempting to drop a Spark SQL table my\_table and runs the following command:

```
DROP TABLE IF EXISTS my_table;
```

After running this command, the engineer notices that the data files and metadata files have been deleted from the file system.

Which of the following describes why all of these files were deleted?

- A. The table was managed
- B. The table's data was smaller than 10 GB
- C. The table's data was larger than 10 GB
- D. The table was external
- E. The table did not have a location

**Answer: A**

**Explanation:**

The correct answer is A: The table was managed.

When a DROP TABLE command is executed in Spark SQL, the behavior differs significantly depending on whether the table is a managed (or internal) table or an external table. A managed table's data and metadata are completely under Spark SQL's control. This implies that Spark SQL manages the data's storage location and its schema. When a managed table is dropped, Spark SQL not only removes the table's metadata from the metastore (e.g., Hive metastore or Databricks metastore) but also deletes the underlying data files from the storage location (usually cloud storage like AWS S3, Azure Blob Storage, or ADLS Gen2).

In contrast, an external table points to data stored in an external location. The DROP TABLE command for an external table only removes the metadata from the metastore, leaving the data in the external location untouched. This allows for scenarios where you want to remove the table definition from Spark SQL without losing the underlying data. This is useful when the data is managed by another system or you want to retain the data for other purposes.

The question specifically states that the data files and metadata files were deleted. This behavior definitively points to the table being a managed table. Options B and C concerning table size are irrelevant; the size of the table data does not affect whether the data files are deleted upon dropping the table. Option D is incorrect because if the table was external, only the metadata would be dropped, leaving the data files untouched. Option E is incorrect because a table always has a defined type, so saying "the table did not have a location" doesn't accurately specify the table type. The key factor is that DROP TABLE deletes the data for a managed table.

Further reading:

**Apache Spark SQL Documentation on Tables:** <https://spark.apache.org/docs/latest/sql-data-sources-hive-tables.html>

**Databricks documentation on creating tables:** <https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-create-table.html> - Pay attention to the CREATE TABLE syntax and the difference between specifying LOCATION and not specifying it.

### Question: 66

Deep Dumps

A data engineer that is new to using Python needs to create a Python function to add two integers together and return the sum?

Which of the following code blocks can the data engineer use to complete this task?

A. 

```
function add_integers(x, y):
 return x + y
```

B. 

```
function add_integers(x, y):
 x + y
```

C. 

```
def add_integers(x, y):
 print(x + y)
```

D. 

```
def add_integers(x, y):
 return x + y
```

E. 

```
def add_integers(x, y):
 x + y
```

**Answer: D**

**Explanation:**

```
def add_integers(x, y):
 return x + y
```

### Question: 67

Deep Dumps

In which of the following scenarios should a data engineer use the MERGE INTO command instead of the INSERT INTO command?

- A. When the location of the data needs to be changed
- B. When the target table is an external table
- C. When the source table can be deleted
- D. When the target table cannot contain duplicate records

E. When the source is not a Delta table

**Answer: D**

**Explanation:**

The correct answer is **D. When the target table cannot contain duplicate records.**

The MERGE INTO command in Databricks is specifically designed for performing upsert operations, which are combinations of update and insert operations. Upsert is crucial when maintaining data integrity and ensuring uniqueness in a target table. It allows you to update existing records in the target table based on matching conditions with the source data and insert new records from the source if no match is found in the target. This inherently addresses the scenario where the target table cannot contain duplicate records by either updating existing matching records or inserting only non-existing ones.

In contrast, the INSERT INTO command simply appends new records to the target table without checking for duplicates. If the source data contains records that already exist in the target table, INSERT INTO will create duplicates. Option A, changing data location, is typically handled through ALTER TABLE commands or data lake management features. Option B, concerning external tables, doesn't directly relate to the choice between MERGE INTO and INSERT INTO; both can be used with external tables. Option C, deleting the source table, is irrelevant to whether to use MERGE INTO or INSERT INTO. The decision hinges on the need to maintain data uniqueness in the target table.

Therefore, the key difference lies in MERGE INTO's ability to handle both updates and inserts while preventing duplicates in the target table, making it ideal when data uniqueness is a requirement. INSERT INTO only handles inserts and does not consider existing data, leading to potential duplication issues.

For more information, refer to the Databricks documentation on the MERGE INTO command: <https://docs.databricks.com/sql/language-manual/delta-merge-into.html> and for general DML commands: <https://docs.databricks.com/sql/language-manual/delta-dml.html>. These resources offer comprehensive details on the command's syntax, behavior, and best practices.

### Question: 68

**Deep Dumps**

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

## sales

| customer_id | spend  | units |
|-------------|--------|-------|
| a1          | 28.94  | 7     |
| a3          | 874.12 | 23    |
| a4          | 8.99   | 1     |

## favorite\_stores

| customer_id | store_id |
|-------------|----------|
| a1          | s1       |
| a2          | s1       |
| a4          | s2       |

The data engineer runs the following query to join these tables together:

```
SELECT
 sales.customer_id,
 sales.spend,
 favorite_stores.store_id
FROM sales
LEFT JOIN favorite_stores
ON sales.customer_id = favorite_stores.customer_id;
```

Which of the following will be returned by the above query?

A.

| customer_id | spend | store_id |
|-------------|-------|----------|
| a1          | 28.94 | s1       |
| a4          | 8.99  | s2       |

B.

| customer_id | spend | units | store_id |
|-------------|-------|-------|----------|
| a1          | 28.94 | 7     | s1       |
| a4          | 8.99  | 1     | s2       |

C.



| customer_id | spend  | store_id |
|-------------|--------|----------|
| a1          | 28.94  | s1       |
| a3          | 874.12 | NULL     |
| a4          | 8.99   | s2       |

D.

| customer_id | spend  | store_id |
|-------------|--------|----------|
| a1          | 28.94  | s1       |
| a2          | NULL   | s1       |
| a3          | 874.12 | NULL     |
| a4          | 8.99   | s2       |

E.

| customer_id | spend | store_id |
|-------------|-------|----------|
| a1          | 28.94 | s1       |
| a2          | NULL  | s1       |
| a4          | 8.99  | s2       |

**Answer: C**

**Explanation:**

A typical LEFT JOIN scenario.

The LEFT JOIN keyword returns all records from the left table, even if there are no matches in the right table.

### Question: 69

Deep Dumps

A data engineer needs to create a table in Databricks using data from a CSV file at location /path/to/csv.

They run the following command:

```
CREATE TABLE new_table
```

```

OPTIONS (
 header = "true",
 delimiter = "|")
LOCATION "path/to/csv"
```

Which of the following lines of code fills in the above blank to successfully complete the task?

- A. None of these lines of code are needed to successfully complete the task
- B. USING CSV
- C. FROM CSV
- D. USING DELTA
- E. FROM "path/to/csv"

**Answer: B**

**Explanation:**

Correct answer is B:USING CSV.

### Question: 70

Deep Dumps

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.readStream
 .table("sales")
 .withColumn("avg_price", col("sales") / col("units"))
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("complete")
 ._____
 .table("new_sales")
)
```

If the data engineer only wants the query to process all of the available data in as many batches as required, which of the following lines of code should the data engineer use to fill in the blank?

- A.processingTime(1)
- B.trigger(availableNow=True)
- C.trigger(parallelBatch=True)
- D.trigger(processingTime="once")
- E.trigger(continuous="once")

**Answer: B**

**Explanation:**

Correct answer is B:trigger(availableNow=True).

### Question: 71

Deep Dumps

A data engineer has developed a data pipeline to ingest data from a JSON source using Auto Loader, but the engineer has not provided any type inference or schema hints in their pipeline. Upon reviewing the data, the data engineer has noticed that all of the columns in the target table are of the string type despite some of the fields only including float or boolean values.

Which of the following describes why Auto Loader inferred all of the columns to be of the string type?

- A. There was a type mismatch between the specific schema and the inferred schema
- B. JSON data is a text-based format
- C. Auto Loader only works with string data
- D. All of the fields had at least one null value
- E. Auto Loader cannot infer the schema of ingested data

**Answer: B**

**Explanation:**

The correct answer is **B. JSON data is a text-based format.**

Here's a detailed justification:

Auto Loader in Databricks is designed to automatically infer the schema of incoming data, simplifying data ingestion pipelines. However, schema inference isn't always perfect, especially when dealing with semi-structured data like JSON.

JSON (JavaScript Object Notation) is inherently a text-based data format. While JSON supports various data types like numbers, booleans, and arrays, these are all represented as text within the JSON structure. When Auto Loader encounters a JSON file without explicit schema hints, it analyzes the data to determine the most appropriate data type for each field.

Because everything in the original JSON is represented as text and without further information or configurations, Auto Loader plays it safe and interprets fields as strings. This is a common and expected behavior when dealing with schemaless or schema-on-read scenarios. Auto Loader doesn't "know" that a particular field should be an integer or a boolean simply by seeing textual representations like "123" or "true".

Option A is incorrect because there's no pre-defined schema in this scenario. The entire point is that Auto Loader is attempting to infer the schema. A type mismatch would only occur if you provide a schema that conflicts with the data.

Option C is incorrect because Auto Loader certainly works with data types other than string. The inference process aims to recognize various data types if possible.

Option D is incorrect as while null values can influence data type inference, they do not automatically force all columns to be strings. Typically, a column with null values will be nullable but of its inferred type if other non-null values can be used to ascertain the right type.

Option E is incorrect because Auto Loader does attempt to infer the schema, it's a core feature. This question specifically deals with a situation where schema inference leads to an unexpected result.

To ensure correct data types, users should use schema hints or schema evolution features of Auto Loader. Schema hints can be passed in as options when creating your stream. Using the `.option("cloudFiles.schemaHints", "col1 string, col2 int")` can define specific column types when the inferred types are not what is desired. You can enable schema evolution with `.option("cloudFiles.schemaLocation", "<path-to-checkpoint>")` which will allow it to use a schema from a location that contains the current structure.

#### Further Research:

**Databricks Auto Loader Documentation:** [<https://docs.databricks.com/delta/autos loader.html>] (<https://docs.databricks.com/delta/autos loader.html>)

**JSON Data Format:** <https://www.json.org/>

### Question: 72

### Deep Dumps

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.

The table is configured to run in Development mode using the Continuous Pipeline Mode.

Assuming previously unprocessed data exists and all definitions are valid, what is the expected outcome after clicking Start to update the pipeline?

- A. All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.
- B. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist until the pipeline is shut down.
- C. All datasets will be updated once and the pipeline will persist without any processing. The compute resources will persist but go unused.
- D. All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.
- E. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.

#### Answer: E

#### Explanation:

Here's a detailed justification for why option E is the correct answer:

The question specifies a Delta Live Tables (DLT) pipeline operating in Development mode with Continuous Pipeline Mode. This configuration significantly affects how the pipeline processes data and manages compute resources.

First, let's understand Development mode. In this mode, DLT is designed to facilitate iterative development and testing. This means it prioritizes speed and resource efficiency for quick feedback loops. A key characteristic is the intention to retain compute resources for interactive analysis and modifications.

Second, Continuous Pipeline Mode dictates that the pipeline will continuously process new data as it arrives, rather than running as a batch job. The STREAMING LIVE TABLE declarations further solidify this continuous processing behavior, as these tables are specifically designed to ingest and transform streaming data

incrementally. Since the pipeline is set up as continuous, all datasets, regardless of their origin (streaming or static), will be updated at set intervals.

Now, let's address the different dataset types. STREAMING LIVE TABLE datasets will continuously ingest and transform data as it becomes available. LIVE TABLE datasets defined against Delta Lake sources will also be updated incrementally in Continuous mode, picking up any changes made to the source tables.

Therefore, the pipeline will update all datasets at set intervals. Because it's in Development mode, the compute resources will persist after the initial update, allowing data engineers to inspect the results, make adjustments to the pipeline, and re-run it as needed without incurring the overhead of spinning up new clusters each time. This enables interactive development and rapid iteration. When the pipeline is no longer needed, it should be manually stopped to deallocate the compute resources.

Other options are incorrect for the following reasons:

**A and D:** These options suggest that the pipeline shuts down after a single update, which is not the behavior of Continuous mode.

**B:** Although datasets update at set intervals, the mention of indefinite operation is not correct. Continuous pipeline will persist unless shut down.

**C:** Suggests the pipeline will persist without processing, is inconsistent with continuous mode.

In summary, Development mode in Continuous pipeline mode enables rapid iteration and testing, involving continuous updates, and the preservation of compute resources for interactive analysis and testing.

For further reading, refer to the official Databricks documentation on Delta Live Tables:

[Delta Live Tables concepts](#)

[Delta Live Tables settings](#)

### Question: 73

Deep Dumps

Which of the following data workloads will utilize a Gold table as its source?

- A. A job that enriches data by parsing its timestamps into a human-readable format
- B. A job that aggregates uncleaned data to create standard summary statistics
- C. A job that cleans data by removing malformed records
- D. A job that queries aggregated data designed to feed into a dashboard
- E. A job that ingests raw data from a streaming source into the Lakehouse

**Answer: D**

**Explanation:**

The correct answer is **D. A job that queries aggregated data designed to feed into a dashboard.**

Here's why: Gold tables in the medallion architecture (Bronze, Silver, Gold) represent the highest quality, most refined, and business-ready data. They are designed for consumption by end-users or applications that require highly reliable and aggregated information. Dashboards typically visualize summarized, aggregated data. Therefore, a dashboard directly benefits from the refined data provided by a Gold table.

Let's eliminate the other options:

**A. A job that enriches data by parsing its timestamps into a human-readable format:** Enrichment typically occurs in the Silver layer after initial cleansing and standardization.

**B. A job that aggregates uncleaned data to create standard summary statistics:** Aggregating uncleaned

data doesn't belong in the Gold layer. Aggregation should happen after cleaning and transformation, using Silver data.

**C. A job that cleans data by removing malformed records:** Cleaning is primarily done in the Silver layer (transforming Bronze data).

**E. A job that ingests raw data from a streaming source into the Lakehouse:** Ingestion of raw data goes directly into the Bronze layer.

In summary, Gold tables are the final stage in the data refinement process, providing clean, aggregated, and business-ready data specifically designed for consumption, such as feeding a dashboard. The medallion architecture prioritizes incremental refinement of data as it flows from Bronze (raw) to Silver (cleaned and transformed) to Gold (aggregated and business-ready). This makes Gold tables the ideal source for reporting and visualization tools that require pre-processed, reliable information.

Authoritative Links:

**Databricks Medallion Architecture:** <https://www.databricks.com/blog/2019/08/15/diving-into-delta-lake-unpacking-the-medallion-architecture.html> (This blog post provides a detailed explanation of the Medallion Architecture)

**Delta Lake Documentation:** <https://docs.delta.io/latest/index.html> (Official documentation for Delta Lake, the underlying technology for the Medallion Architecture)

## Question: 74

## Deep Dumps

Which of the following must be specified when creating a new Delta Live Tables pipeline?

- A. A key-value pair configuration
- B. The preferred DBU/hour cost
- C. A path to cloud storage location for the written data
- D. A location of a target database for the written data
- E. At least one notebook library to be executed

**Answer: C**

**Explanation:**

The correct answer is **C. A path to cloud storage location for the written data.**

Delta Live Tables (DLT) pipelines require a storage location to materialize and persist the data processed through the pipeline. This storage location acts as the root directory for all tables and metadata generated by the pipeline. Without specifying this location, DLT would have no place to store the refined, cleaned, and transformed data, rendering the entire pipeline useless.

Option A is incorrect because while key-value pair configurations can be added to a DLT pipeline for various configurations, it is not a mandatory requirement.

Option B is incorrect. DBU costs are automatically managed by Databricks based on the cluster configuration you select, and you don't need to specify a preferred DBU/hour cost directly when defining the pipeline. Databricks optimizes the resource allocation based on the data being processed.

Option D is incorrect. Although you can define a target database for the written data, it's not strictly necessary when defining the pipeline. DLT can also write to paths in cloud storage without being specifically associated with a database. You do define the target schema (database) in the notebook, so it is less associated with the pipeline than the storage location.

Option E is technically correct, but incomplete. While a pipeline must have at least one notebook library, you

also need the storage location to actually do anything with the transformed data. The notebook provides the logic, but the storage location is the fundamental requirement for persisting data.

In summary, a cloud storage location is the fundamental and essential requirement for any DLT pipeline to function, as it provides the designated location to store all generated and transformed data. It is required for the pipeline to write and persist the result of any table transformations. The other options are optional or implicitly managed by Databricks.

For further details, consult the official Databricks documentation:

**Delta Live Tables Concepts:** <https://docs.databricks.com/delta-live-tables/index.html>

**Configure Delta Live Tables Pipelines:** <https://docs.databricks.com/delta-live-tables/delta-live-tables-ui.html#configure-your-pipeline-settings>

### Question: 75

Deep Dumps

A data engineer has joined an existing project and they see the following query in the project repository:

```
CREATE STREAMING LIVE TABLE loyal_customers AS
```

```
SELECT customer_id -
FROM STREAM(LIVE.customers)
WHERE loyalty_level = 'high';
```

Which of the following describes why the STREAM function is included in the query?

- A. The STREAM function is not needed and will cause an error.
- B. The table being created is a live table.
- C. The customers table is a streaming live table.
- D. The customers table is a reference to a Structured Streaming query on a PySpark DataFrame.
- E. The data in the customers table has been updated since its last run.

**Answer: C**

**Explanation:**

The correct answer is C: The customers table is a streaming live table.

Here's why:

The CREATE STREAMING LIVE TABLE syntax in Databricks SQL signifies that we are defining a table as part of a Delta Live Tables (DLT) pipeline designed for incremental data processing. DLT pipelines are specifically built to handle streaming data, allowing continuous data ingestion and transformation.

The STREAM function in STREAM(LIVE.customers) is crucial because it tells the DLT engine that LIVE.customers is a streaming live table itself, meaning it continuously receives updates. If customers were a regular, batch-oriented live table, then using STREAM would not be appropriate. The LIVE. prefix indicates the table is managed within the DLT environment.

In a DLT pipeline, data flows from one streaming live table to another. In this case, loyal\_customers is reading updates (streams) from LIVE.customers to identify high-loyalty customers, instead of reading all available LIVE.customers data at once.

Option A is incorrect because the STREAM function is essential when reading from a streaming live table within another streaming live table definition in a DLT pipeline. Removing it would disrupt the streaming data flow, which is the core function of DLT.



Option B is partially correct in that the table being created is a live table; however, it doesn't explain the usage of the STREAM function. The function's presence is directly linked to the input table's streaming nature.

Option D is incorrect because while Structured Streaming is related to streaming data processing in Spark, this syntax STREAM(LIVE.customers) specifically refers to interacting with DLT managed streaming live tables, not directly to Spark DataFrames.

Option E might be a characteristic of streaming data, but the STREAM function's main purpose isn't to indicate updates; instead, it's to access the continuous stream of updates from a streaming live table. The DLT engine internally manages the update handling.

In short, the STREAM function is used to read the streaming data from a source streaming live table within a Delta Live Tables pipeline, ensuring that the destination table (loyal\_customers) is updated incrementally as new data arrives in the source table (customers).

For authoritative information, refer to the official Databricks documentation on Delta Live Tables:

[Delta Live Tables](#)

[Create a Live Table](#)

[Streaming Live Tables](#)

## Question: 76

Deep Dumps

Which of the following describes the type of workloads that are always compatible with Auto Loader?

- A.Streaming workloads
- B.Machine learning workloads
- C.Serverless workloads
- D.Batch workloads
- E.Dashboard workloads

**Answer: A**

**Explanation:**

Auto Loader in Databricks is specifically designed to efficiently and incrementally ingest new data as it arrives in cloud storage (like AWS S3, Azure Data Lake Storage Gen2, or Google Cloud Storage). This "as it arrives" nature makes it fundamentally suited for streaming workloads. Streaming workloads involve the continuous flow of data, where new data points are constantly being added. Auto Loader automatically detects new files as they land in the storage location and processes them in a streaming fashion, without requiring manual triggering or scheduling.

Batch workloads, in contrast, involve processing a large, finite set of data all at once. While Auto Loader can be used with batch workloads initially, its strengths lie in its ability to handle the continuous arrival of new data, not in processing a pre-existing, fixed dataset. Serverless workloads are more about the infrastructure aspect of not managing servers, and not directly related to the type of data ingestion. Machine learning workloads and dashboard workloads are application areas that may consume data ingested by Auto Loader, but Auto Loader itself isn't intrinsically tied to these workload types. The core capability of Auto Loader is optimized for the continuous processing of data streams, making it inherently compatible with streaming workloads. Its schema inference and evolution capabilities are particularly valuable in streaming scenarios where data schemas might change over time. Its incremental processing ensures that the data remains fresh and up-to-date, which is crucial in real-time applications.

Therefore, the characteristic of incremental and continuous data ingestion directly aligns with the definition

of Streaming workloads.

Further reading on Auto Loader and its use cases:

Databricks Auto Loader documentation: <https://docs.databricks.com/delta/auto-loader/index.html>

### Question: 77

### Deep Dumps

A data engineer and data analyst are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.

Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

- A. None of these changes will need to be made
- B. The pipeline will need to stop using the medallion-based multi-hop architecture
- C. The pipeline will need to be written entirely in SQL
- D. The pipeline will need to use a batch source in place of a streaming source
- E. The pipeline will need to be written entirely in Python

**Answer: A**

**Explanation:**

The correct answer is A, "None of these changes will need to be made." Here's why:

Delta Live Tables (DLT) is designed to simplify the development and deployment of reliable data pipelines. It embraces and enhances existing best practices, not replaces them.

**Medallion Architecture:** DLT fully supports the medallion architecture (bronze, silver, gold). This approach of progressively refining data is fundamental to building robust data lakes and data warehouses. DLT streamlines the implementation by handling dependency management and data quality checks, but it doesn't force you to abandon the pattern.

**Language Support:** DLT pipelines can be built using both Python and SQL. This allows data engineers and data analysts to continue using their preferred languages and skillsets within the same pipeline. DLT simply provides a declarative framework for orchestrating these transformations.

**Streaming Support:** DLT natively supports both streaming and batch data sources. It automatically manages the complexities of incremental data processing for streaming sources. This means the existing streaming source can be seamlessly integrated into the DLT pipeline. DLT handles the necessary state management and checkpointing implicitly.

Therefore, the existing pipeline already aligns with the core principles of DLT. Migrating to DLT would primarily involve refactoring the existing code into DLT-compatible notebooks or files and defining the data dependencies within the DLT framework. The underlying logic and data flow can remain largely unchanged.

For further research, consult the official Databricks Delta Live Tables documentation:

**Delta Live Tables Introduction:** <https://docs.databricks.com/delta-live-tables/index.html>

**Delta Live Tables Python API:** <https://docs.databricks.com/delta-live-tables/delta-live-tables-python.html>

**Delta Live Tables SQL API:** <https://docs.databricks.com/delta-live-tables/delta-live-tables-sql.html>

### Question: 78

### Deep Dumps

A data engineer is using the following code block as part of a batch ingestion pipeline to read from a composable table:

```
transactions_df = (spark.read
 .schema(schema)
 .format("delta")
 .table("transactions")
)
```

Which of the following changes needs to be made so this code block will work when the transactions table is a stream source?

- A. Replace predict with a stream-friendly prediction function
- B. Replace schema(schema) with option("maxFilesPerTrigger", 1)
- C. Replace "transactions" with the path to the location of the Delta table
- D. Replace format("delta") with format("stream")
- E. Replace spark.read with spark.readStream

**Answer: E**

**Explanation:**

Replace spark.read with spark.readStream.

### Question: 79

Deep Dumps

Which of the following queries is performing a streaming hop from raw data to a Bronze table?

A.

```
(spark.table("sales")
 .groupBy("store")
 .agg(sum("sales"))
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("complete")
 .table("newSales")
)
```

B.

```
(spark.table("sales")
 .filter(col("units") > 0)
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("append")
 .table("newSales")
)
```

C.

```
(spark.table("sales")
 .withColumn("avgPrice", col("sales") / col("units"))
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("append")
 .table("newSales")
)
```

D.

```
(spark.read.load(rawSalesLocation)
 .write
 .mode("append")
 .table("newSales")
)
```

E.

```
(spark.readStream.load(rawSalesLocation)
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("append")
 .table("newSales")
)
```

**Answer: E**

**Explanation:**

```
(spark.readStream.load(rawSalesLocation)
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("append")
 .table("newSales")
)
```

### Question: 80

Deep Dumps

A dataset has been defined using Delta Live Tables and includes an expectations clause:

CONSTRAINT valid\_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION FAIL UPDATE

What is the expected behavior when a batch of data containing data that violates these constraints is processed?

- A. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.
- B. Records that violate the expectation cause the job to fail.
- C. Records that violate the expectation are dropped from the target dataset and loaded into a quarantine table.
- D. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.
- E. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.

### Answer: B

#### Explanation:

The correct answer is B because the ON VIOLATION FAIL UPDATE clause in the Delta Live Tables (DLT) pipeline specifies the action to be taken when an expectation (constraint) is violated. Let's break down why the other options are incorrect:

**Option A:** This behavior corresponds to ON VIOLATION DROP ROW, where violating records are dropped and logged.

**Option C:** Quarantine tables are not a standard feature for expectation failures in DLT. While you could manually implement similar logic with more complex transformations, it's not the default behavior for built-in expectations.

**Option D:** Adding invalid records and logging them goes against the purpose of data quality checks, especially when FAIL UPDATE is specified.

**Option E:** DLT doesn't automatically add an "invalid" flag field to the dataset.

The ON VIOLATION FAIL UPDATE setting is designed for scenarios where data quality is critical, and any violation of the specified expectation should halt the pipeline's progress. This ensures that downstream processes are not affected by potentially incorrect or incomplete data. When a violation is detected, the DLT pipeline stops processing the current batch and raises an error, preventing further updates to the target dataset until the data quality issue is resolved. This approach emphasizes data correctness and allows data engineers to investigate and fix the problem before proceeding. This approach is useful for situations where the data quality must be guaranteed for downstream processes and decision-making.

#### Authoritative Links:

**Delta Live Tables Expectations:** <https://docs.databricks.com/delta-live-tables/expectations.html>

(Specifically, see the section on "Handle expectation failures")

**Delta Live Tables Error Handling:** (Refer to general DLT documentation on error handling and pipeline monitoring on the Databricks website.)

The FAIL UPDATE option provides the strongest guarantee of data quality within a DLT pipeline.

### Question: 81

Deep Dumps

Which of the following statements regarding the relationship between Silver tables and Bronze tables is always true?

- A. Silver tables contain a less refined, less clean view of data than Bronze data.
- B. Silver tables contain aggregates while Bronze data is unaggregated.
- C. Silver tables contain more data than Bronze tables.
- D. Silver tables contain a more refined and cleaner view of data than Bronze tables.
- E. Silver tables contain less data than Bronze tables.

**Answer: D**

**Explanation:**

The correct answer is D: Silver tables contain a more refined and cleaner view of data than Bronze tables. This is fundamentally based on the Medallion Architecture, a common data engineering paradigm.

In the Medallion Architecture (Bronze, Silver, Gold), data progresses through layers of increasing refinement and business readiness. Bronze tables, also known as raw tables, store data directly from source systems with minimal transformation. They essentially provide a historical, immutable record of the data received. They are often implemented using Delta Lake for its ACID properties and schema evolution capabilities.

Silver tables sit in the middle layer. The purpose of this layer is to clean, transform, and conform the data from the Bronze layer. This includes tasks such as data cleansing (handling missing values, correcting errors), data standardization (ensuring consistent formats), data deduplication, and joining data from multiple Bronze tables.

Therefore, Silver tables inherently hold a more refined and cleaner view compared to Bronze. Data quality checks and enforcement are implemented in the transition from Bronze to Silver. Silver tables represent data ready for further processing and analysis, having been scrubbed of inconsistencies present in the raw Bronze data.

Options A, B, C, and E are incorrect because they misrepresent the relationships and characteristics of Bronze and Silver layers within a medallion architecture. Option A states the opposite of the truth: Silver tables are more refined. Option B is incorrect because Silver tables don't necessarily contain aggregates; aggregation usually happens at the Gold level. Options C and E's claims about the relative data size are not guaranteed; Silver tables might have fewer rows due to deduplication or filtering but could also have more columns due to joins and transformations. The crucial difference is the data quality and refinement aspect, making option D the only correct choice.

For more information, you can refer to these resources:

**Databricks Medallion Architecture:** <https://www.databricks.com/glossary/medallion-architecture>

**Delta Lake Documentation:** <https://docs.delta.io/latest/index.html> - while Delta Lake is an implementation detail it is the most common method for building the architecture.



**Question: 82****Deep Dumps**

A data engineering team has noticed that their Databricks SQL queries are running too slowly when they are submitted to a non-running SQL endpoint. The data engineering team wants this issue to be resolved.

Which of the following approaches can the team use to reduce the time it takes to return results in this scenario?

- A.They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy to "Reliability Optimized."
- B.They can turn on the Auto Stop feature for the SQL endpoint.
- C.They can increase the cluster size of the SQL endpoint.
- D.They can turn on the Serverless feature for the SQL endpoint.
- E.They can increase the maximum bound of the SQL endpoint's scaling range.

**Answer: E**

**Explanation:**

The problem is slow query execution when submitted to a non-running SQL endpoint. This clearly indicates a cold start issue, where the endpoint needs to power up before it can process the query. Options focusing on pre-existing endpoint settings (like cluster size - C) don't address the cold start. Option B (Auto Stop) would exacerbate the problem by ensuring the endpoint shuts down when idle, leading to more cold starts.

Option D, turning on Serverless, would indeed improve cold start times because it utilizes Databricks' managed compute infrastructure, which is designed to be readily available. However, option E, increasing the maximum bound of the SQL endpoint's scaling range, is a more direct way to improve cold start times. By allowing the endpoint to scale to a larger number of clusters more quickly, it can handle the initial query load more efficiently, reducing the delay caused by the initial startup. Furthermore, increasing the scaling range means the endpoint can acquire more resources during the initial spin-up phase. A larger scaling range allows the endpoint to potentially allocate more compute resources during the startup, thus speeding up the process.

Option A combines Serverless with "Reliability Optimized" spot instance policy. While Serverless improves cold starts, the spot instance policy primarily affects cost and resilience to instance interruptions; it has minimal direct impact on cold start duration itself. While spot instances can be cheaper, they can also be preempted, leading to interruptions. Focusing on increasing the maximum scaling range directly tackles the problem of initial resource allocation when the endpoint starts up.

Therefore, the most effective solution is to increase the maximum bound of the SQL endpoint's scaling range. This allows the endpoint to allocate more compute resources during startup, thus speeding up the process and reducing the initial query execution time.

Further research:

**Databricks SQL Endpoints Scaling:** <https://docs.databricks.com/sql/admin/sql-endpoints.html#configure-the-scaling-range>

**Databricks SQL Serverless:** <https://docs.databricks.com/sql/admin/serverless.html>

**Question: 83****Deep Dumps**

A data engineer has a Job that has a complex run schedule, and they want to transfer that schedule to other Jobs.

Rather than manually selecting each value in the scheduling form in Databricks, which of the following tools can the data engineer use to represent and submit the schedule programmatically?



- A.pyspark.sql.types.DateType
- B.datetime
- C.pyspark.sql.types.TimestampType
- D.Cron syntax
- E.There is no way to represent and submit this information programmatically

**Answer: D**

**Explanation:**

The correct answer is **D. Cron syntax**. Here's why:

Databricks Jobs can be scheduled to run automatically based on a defined schedule. While the Databricks UI offers a visual scheduling form, more complex or repetitive schedules are better managed programmatically. Cron syntax provides a standardized, concise way to express recurring time-based schedules. It's a widely used convention supported by various scheduling systems, including those found in cloud platforms. Cron expressions consist of fields representing minutes, hours, days of the month, months, and days of the week, allowing for precise schedule definitions (e.g., "Run every day at 5 AM").

Options A, B, and C represent data types used for representing dates and timestamps within PySpark. These are useful for handling date and time data within a Spark application but not for defining the schedule of the Databricks Job itself. Option E is incorrect because Databricks Jobs do support programmatic scheduling, particularly through the use of Cron expressions.

By using Cron syntax, the data engineer can define the complex schedule in a text-based format, allowing for easy transfer to other Jobs via scripts, APIs, or infrastructure-as-code deployments. This promotes repeatability and reduces the risk of manual configuration errors. The Databricks Jobs API also supports the use of Cron expressions, making it the preferred method for programmatic scheduling.

Further research:

**Databricks Jobs API:** <https://docs.databricks.com/api/workspace/jobs>

**Cron Expressions:** <https://en.wikipedia.org/wiki/Cron> or search for "Cron syntax" for numerous online generators and explanations.

## Question: 84

**Deep Dumps**

Which of the following approaches should be used to send the Databricks Job owner an email in the case that the Job fails?

- A.Manually programming in an alert system in each cell of the Notebook
- B.Setting up an Alert in the Job page
- C.Setting up an Alert in the Notebook
- D.There is no way to notify the Job owner in the case of Job failure
- E.MLflow Model Registry Webhooks

**Answer: B**

**Explanation:**

The correct answer is B: Setting up an Alert in the Job page. Here's why:

Databricks Jobs offer built-in alerting mechanisms directly within the Job configuration. These alerts are specifically designed to notify job owners or other designated recipients about job status changes, including

failures. This approach provides a centralized and easily configurable way to manage job monitoring and notifications.

Option A (Manually programming an alert system in each cell) is highly inefficient, error-prone, and difficult to maintain. Embedding alert logic within individual notebook cells would scatter monitoring responsibilities across the codebase, making it hard to manage and update. Furthermore, this would significantly bloat the notebook code and make it less readable.

Option C (Setting up an Alert in the Notebook) is not the intended use of notebooks. While notebooks can generate outputs and logs, the core purpose is for data engineering tasks, not centralized job monitoring and notification. While some notebook functionality might be used to trigger external alerting, the native Databricks Jobs interface is the preferred method.

Option D (There is no way to notify the Job owner in the case of Job failure) is factually incorrect, as Databricks Jobs directly supports email alerts.

Option E (MLflow Model Registry Webhooks) is also incorrect. MLflow Model Registry Webhooks are designed to trigger actions (like retraining or deployment) based on changes in the model registry, such as a new model version being created. They are not directly tied to the status of Databricks Jobs that might use those models. While it is possible to create workflows that would detect a failed job and then trigger a model redeployment via Webhooks, this is an indirect and more complicated solution.

Therefore, using the Databricks Job page alerts provides a straightforward, native, and maintainable solution for notifying job owners of failures.

Here are some helpful resources:

**Databricks Jobs documentation:** <https://docs.databricks.com/workflows/jobs/index.html>

**Configure email alerts for Databricks Jobs:** <https://docs.databricks.com/workflows/jobs/jobs-alerts.html>

## Question: 85

## Deep Dumps

An engineering manager uses a Databricks SQL query to monitor ingestion latency for each data source. The manager checks the results of the query every day, but they are manually rerunning the query each day and waiting for the results.

Which of the following approaches can the manager use to ensure the results of the query are updated each day?

- A.They can schedule the query to refresh every 1 day from the SQL endpoint's page in Databricks SQL.
- B.They can schedule the query to refresh every 12 hours from the SQL endpoint's page in Databricks SQL.
- C.They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.
- D.They can schedule the query to run every 1 day from the Jobs UI.
- E.They can schedule the query to run every 12 hours from the Jobs UI.

**Answer: C**

**Explanation:**

The correct answer is **C. They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.**

Here's why:

**Databricks SQL Queries and Scheduling:** Databricks SQL provides the ability to schedule queries directly from the query editor. This allows for automated execution and refreshing of results. This functionality is designed specifically for Databricks SQL queries and makes it the most straightforward method for the

manager's goal. The schedule can be set to a daily interval, addressing the requirement that the query results be updated each day.

**Refreshing vs. Running:** Scheduling a query to refresh is different from scheduling a job to run the query. Refreshing a query in Databricks SQL uses caching mechanisms and optimized execution plans to efficiently update the results. Running the query as a new job might incur more overhead. The question implied that it is enough for the results to be updated, making the "refresh" option in Databricks SQL the more suited option.

**Location of Scheduling:** The scheduling option for Databricks SQL queries is found within the query's page. This makes management and monitoring easier since the schedule is directly associated with the query itself.

**Why other options are incorrect:**

**A & B:** While scheduling might be available from the SQL endpoint, it would be less direct. The scheduling is intrinsically tied to the query's execution logic, making it most appropriate to manage the schedule from the query's page.

**D & E:** While you could execute the query from the Jobs UI, this would be more suited to ad-hoc tasks. It is also less convenient because the query is already saved in Databricks SQL. Databricks SQL is made to manage the running and scheduling of SQL queries. Using the Jobs UI adds unnecessary complexity.

**Relevant Links for further research:**

Databricks SQL Query Scheduling: <https://docs.databricks.com/sql/user/queries/query-schedule.html>

Databricks Jobs: <https://docs.databricks.com/workflows/jobs/index.html> (For contrasting with Databricks SQL Query Scheduling).

## Question: 86

## Deep Dumps

In which of the following scenarios should a data engineer select a Task in the Depends On field of a new Databricks Job Task?

- A. When another task needs to be replaced by the new task
- B. When another task needs to fail before the new task begins
- C. When another task has the same dependency libraries as the new task
- D. When another task needs to use as little compute resources as possible
- E. When another task needs to successfully complete before the new task begins

**Answer: E**

**Explanation:**

The correct answer, E, "When another task needs to successfully complete before the new task begins," precisely describes the purpose of the Depends On field in Databricks Jobs. This field establishes a task dependency, meaning the designated "new task" will only initiate execution if the task it depends on has finished with a successful (or, depending on settings, with specific success/failure configurations) status.

This mechanism allows for orchestrating a sequence of operations within a job, where the output or state of one task is a prerequisite for another. Task dependencies are a fundamental concept in workflow management and data pipelines. Cloud computing platforms such as Databricks provide such features for dependency management to efficiently handle complex workflows. Option A is incorrect because the Depends On field does not replace tasks. Option B is wrong as the new task won't trigger if the dependent task fails without configuring it to do so. Option C is irrelevant since dependency libraries are typically managed at the job or cluster level, not individual task dependencies. Option D has no relationship to task dependencies; resource management is separate.

For example, imagine a data pipeline where you first extract data from a source (Task 1) and then transform it (Task 2). Task 2 should only begin after Task 1 has successfully extracted the data. By setting Task 1 as a dependency for Task 2, you guarantee this order of execution. If Task 1 fails, Task 2 won't start, preventing potential errors caused by missing input data. This ensures data integrity and efficient resource utilization. Databricks Jobs uses directed acyclic graphs (DAGs) to handle dependencies, where each task is a node, and the dependencies create edges.

Further reading can be found in the Databricks documentation on Jobs:

<https://docs.databricks.com/workflows/jobs/index.html> and Task dependencies specifically:

<https://docs.databricks.com/workflows/jobs/jobs-concepts.html#task-dependencies>.

### Question: 87

### Deep Dumps

A data engineer has been using a Databricks SQL dashboard to monitor the cleanliness of the input data to a data analytics dashboard for a retail use case. The job has a Databricks SQL query that returns the number of store-level records where sales is equal to zero. The data engineer wants their entire team to be notified via a messaging webhook whenever this value is greater than 0.

Which of the following approaches can the data engineer use to notify their entire team via a messaging webhook whenever the number of stores with \$0 in sales is greater than zero?

- A.They can set up an Alert with a custom template.
- B.They can set up an Alert with a new email alert destination.
- C.They can set up an Alert with one-time notifications.
- D.They can set up an Alert with a new webhook alert destination.
- E.They can set up an Alert without notifications.

### Answer: D

### Explanation:

Here's a detailed justification for why option D is the correct answer, along with supporting explanations and links:

The core requirement is to trigger a notification to the team via a messaging webhook whenever a specific condition in the data (number of stores with zero sales) is met. Databricks SQL Alerts are designed to monitor query results and trigger actions when those results meet predefined thresholds.

Option D, setting up an Alert with a new webhook alert destination, directly addresses this need. Webhooks allow Databricks to send automated HTTP requests to external services (like messaging platforms such as Slack, Microsoft Teams, etc.) when an alert condition is triggered. This enables real-time notification to the team whenever the "number of stores with \$0 sales" query returns a value greater than zero. The data engineer can configure the webhook payload to include relevant information about the alert, allowing the team to quickly understand the issue.

Let's examine why the other options are not ideal:

**A. They can set up an Alert with a custom template:** While custom templates are useful for formatting alert messages, they don't address the fundamental requirement of sending the notification to a messaging webhook. Custom templates enhance the content of the notification, but they don't determine the delivery mechanism.

**B. They can set up an Alert with a new email alert destination:** Email notifications might be a viable option in some scenarios, but the question specifically requests a messaging webhook. Email is less immediate and collaborative compared to messaging platforms commonly used by teams.

**C. They can set up an Alert with one-time notifications:** One-time notifications would only trigger the alert once, which is not suitable for continuous monitoring of data cleanliness. The requirement is to be notified every time the condition is met.

**E. They can set up an Alert without notifications:** Setting up an alert without notifications defeats the entire purpose. The goal is to notify the team when the specified condition is met.

Therefore, leveraging Databricks SQL Alerts with a webhook destination is the optimal solution. It enables automated, real-time notification to the team's messaging platform whenever the query result (number of stores with zero sales) exceeds zero, ensuring prompt attention to data quality issues.

#### Supporting Links:

**Databricks SQL Alerts:** <https://docs.databricks.com/sql/user/alerts/index.html>

**Databricks SQL Alerts - Webhooks:** <https://docs.databricks.com/sql/user/alerts/webhooks.html>

### Question: 88

### Deep Dumps

A data engineer wants to schedule their Databricks SQL dashboard to refresh every hour, but they only want the associated SQL endpoint to be running when it is necessary. The dashboard has multiple queries on multiple datasets associated with it. The data that feeds the dashboard is automatically processed using a Databricks Job.

Which of the following approaches can the data engineer use to minimize the total running time of the SQL endpoint used in the refresh schedule of their dashboard?

- A.They can turn on the Auto Stop feature for the SQL endpoint.
- B.They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.
- C.They can reduce the cluster size of the SQL endpoint.
- D.They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.
- E.They can set up the dashboard's SQL endpoint to be serverless.

#### Answer: A

#### Explanation:

The correct answer is A: They can turn on the Auto Stop feature for the SQL endpoint.

Here's why: The goal is to minimize the total running time (and therefore cost) of the Databricks SQL endpoint while still ensuring the dashboard refreshes hourly. Let's analyze the options:

**A. They can turn on the Auto Stop feature for the SQL endpoint.** This is the most effective approach. The Auto Stop feature automatically shuts down the SQL endpoint after a period of inactivity. Since the dashboard only needs the endpoint to be running during the refresh, setting Auto Stop ensures it shuts down after the refresh is complete, minimizing idle run time and associated costs. The endpoint will automatically restart when the next scheduled refresh is triggered. This approach directly addresses the requirement of only running the endpoint when necessary. [Databricks documentation on SQL endpoint auto-stop](#) confirms this functionality.

**B. They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.** This is incorrect. The dashboard needs to be pointed at a single SQL endpoint so that its queries can be executed. This suggestion conflicts with the core architecture of Databricks SQL Dashboards.

**C. They can reduce the cluster size of the SQL endpoint.** Reducing the cluster size can lower the cost per hour while the endpoint is running, but it doesn't address the problem of minimizing total running time. A smaller cluster might also increase query execution time, potentially negating cost savings and impacting dashboard responsiveness.

**D. They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.** This is generally not possible or desirable. It implies having multiple, independent SQL endpoints for each query, which complicates management and defeats the purpose of having a consolidated endpoint for the dashboard. Dashboards are typically connected to a single endpoint to execute all of their queries.

**E. They can set up the dashboard's SQL endpoint to be serverless.** While serverless SQL endpoints offer automatic scaling and resource management, the key benefit here is cost optimization when the endpoint is idle. However, serverless SQL endpoints are only available on AWS and only on specific Databricks Workspaces. If the environment does not support serverless, this option isn't applicable, making option A a more reliable overall approach. Even if serverless is an option, Auto Stop would still be beneficial to further optimize cost. Databricks serverless compute automatically scales and shuts down when not in use (similar to auto-stop), reducing operational burden and costs.

Therefore, Auto Stop is the most direct and universally applicable solution to minimize the SQL endpoint's total running time and associated costs when using a scheduled refresh. It's a standard feature in Databricks SQL endpoints designed for this purpose.

### Question: 89

Deep Dumps

A data engineer needs access to a table `new_table`, but they do not have the correct permissions. They can ask the table owner for permission, but they do not know who the table owner is.

Which of the following approaches can be used to identify the owner of `new_table`?

- A. Review the Permissions tab in the table's page in Data Explorer
- B. All of these options can be used to identify the owner of the table
- C. Review the Owner field in the table's page in Data Explorer
- D. Review the Owner field in the table's page in the cloud storage solution
- E. There is no way to identify the owner of the table

**Answer: C**

**Explanation:**

The correct answer is **C. Review the Owner field in the table's page in Data Explorer.**

Here's why:

Databricks Data Explorer provides a user interface to browse and manage data objects in the metastore. A key function is displaying metadata about tables, including information about the owner. The "Owner" field directly displays the user or service principal who owns the table. This is the most straightforward and accessible method for a data engineer to identify the owner when working within the Databricks environment.

Option A, "Review the Permissions tab in the table's page in Data Explorer," is incorrect because while the Permissions tab shows who has various permissions (SELECT, MODIFY, etc.), it doesn't explicitly label one user as the "owner." The owner generally has implicit full permissions, but they're not identified as the owner on this tab.

Option D, "Review the Owner field in the table's page in the cloud storage solution," is incorrect because the ownership in the underlying cloud storage (like AWS S3 or Azure Data Lake Storage) is separate from the ownership managed by the Databricks metastore. While the storage location has its own permissions and ownership settings, those don't necessarily translate directly to the metastore owner. Databricks manages access control on top of the underlying storage.

Option E, "There is no way to identify the owner of the table," is incorrect because, as discussed above, Data



Explorer provides this functionality.

Option B, "All of these options can be used to identify the owner of the table," is incorrect because options A and D are not direct or reliable methods to find the Databricks metastore owner of the table.

In summary, the most direct and appropriate method to determine the owner of a table within a Databricks workspace is to check the "Owner" field displayed in the table's page within Data Explorer. This feature is specifically designed to provide metadata, including ownership information, for managed tables.

For further research, you can refer to the official Databricks documentation on Data Explorer and Access Control:

**Databricks Data Explorer:** <https://docs.databricks.com/data-governance/data-explorer/index.html>

**Databricks Access Control:** <https://docs.databricks.com/security/access-control/index.html>

### Question: 90

Deep Dumps

A new data engineering team has been assigned to an ELT project. The new data engineering team will need full privileges on the table sales to fully manage the project.

Which of the following commands can be used to grant full permissions on the database to the new data engineering team?

- A. GRANT ALL PRIVILEGES ON TABLE sales TO team;
- B. GRANT SELECT CREATE MODIFY ON TABLE sales TO team;
- C. GRANT SELECT ON TABLE sales TO team;
- D. GRANT USAGE ON TABLE sales TO team;
- E. GRANT ALL PRIVILEGES ON TABLE team TO sales;

**Answer: A**

**Explanation:**

The correct answer is A, GRANT ALL PRIVILEGES ON TABLE sales TO team;. This command provides the team with comprehensive control over the sales table. Here's why the other options are incorrect:

**B. GRANT SELECT CREATE MODIFY ON TABLE sales TO team;** This grants specific privileges (SELECT, CREATE, MODIFY) but may not cover all necessary permissions for full management, such as DROP, OWNER, or potentially other administrative tasks.

**C. GRANT SELECT ON TABLE sales TO team;** This only allows the team to read data from the sales table and nothing else. It's insufficient for full management.

**D. GRANT USAGE ON TABLE sales TO team;** The USAGE privilege is typically used for schemas (databases), not tables directly, to grant access to objects within the schema. It doesn't grant permissions for manipulating the data within the table.

**E. GRANT ALL PRIVILEGES ON TABLE team TO sales;** This is syntactically incorrect. The GRANT statement should grant privileges on the object (in this case, the sales table) to the principal (the team). The order is reversed. Furthermore, "team" is assumed to be a user or group, not an object.

GRANT ALL PRIVILEGES provides the new data engineering team the ability to select, insert, update, delete, alter, and drop the sales table. This level of access enables them to perform all the operations necessary for their ELT project. This command grants the team the highest level of access, which aligns with the requirement of having "full privileges" to fully manage the project. Databricks SQL uses ANSI SQL syntax for



GRANT and REVOKE statements. Therefore, the statement grants all object-specific privileges associated with table sales to the group or user team. The concept of assigning privileges to teams (groups) streamlines permission management, ensuring consistent access control. \* [Databricks SQL Privileges and Grantable Privileges](#) \* [Databricks SQL GRANT statement](#)

### Question: 91

Deep Dumps

Which data lakehouse feature results in improved data quality over a traditional data lake?

- A. A data lakehouse stores data in open formats.
- B. A data lakehouse allows the use of SQL queries to examine data.
- C. A data lakehouse provides storage solutions for structured and unstructured data.
- D. A data lakehouse supports ACID-compliant transactions.

**Answer: D**

#### Explanation:

The correct answer is D, a data lakehouse supports ACID-compliant transactions. This feature significantly improves data quality compared to a traditional data lake because it ensures data consistency and reliability.

Traditional data lakes often lack ACID properties (Atomicity, Consistency, Isolation, Durability). This means that concurrent read and write operations can lead to data corruption or inconsistent query results. For example, if one process is updating a table while another is reading it, the reader might see partially updated data, leading to inaccurate analytics.

ACID transactions, on the other hand, guarantee that database operations are performed reliably. Atomicity ensures that a transaction is treated as a single, indivisible unit of work - either all changes are applied, or none. Consistency ensures that a transaction only brings the database from one valid state to another. Isolation ensures that concurrent transactions do not interfere with each other. Durability guarantees that once a transaction is committed, it remains committed even in the event of a system failure.

By supporting ACID transactions, a data lakehouse prevents data corruption and ensures that all users see a consistent view of the data, leading to improved data quality and more reliable analytics. This is crucial for business intelligence, machine learning, and other data-driven applications. Features like Delta Lake, which provide ACID transactions on top of data lakes, are key to making data lakehouses a valuable tool for data engineering.

Further Research:

[Databricks Delta Lake Documentation](#)  
[What is a Data Lakehouse? - Databricks](#)

### Question: 92

Deep Dumps

In which scenario will a data team want to utilize cluster pools?

- A. An automated report needs to be version-controlled across multiple collaborators.
- B. An automated report needs to be runnable by all stakeholders.
- C. An automated report needs to be refreshed as quickly as possible.
- D. An automated report needs to be made reproducible.

**Answer: C**

**Explanation:**

The correct answer, C (An automated report needs to be refreshed as quickly as possible), is the most fitting scenario for utilizing Databricks cluster pools. Cluster pools are designed primarily to accelerate cluster startup times. In a traditional Databricks setup, creating a new cluster requires allocating resources, installing libraries, and initializing the environment, which can take several minutes. This delay becomes a bottleneck when you have automated reports that need to be generated frequently, such as hourly or even more often.

Cluster pools pre-allocate and maintain a set of idle instances, ready to be used by new clusters. When an automated report job requests a cluster, Databricks can quickly grab an instance from the pool, drastically reducing the cluster startup time. This directly translates to a faster report refresh cycle. This speed improvement is crucial for time-sensitive reporting applications.

Option A is incorrect because version control is typically managed using tools like Git, not cluster pools. Option B is incorrect because running reports for all stakeholders is more about access control and sharing permissions than cluster startup speed. Option D is incorrect because reproducibility is addressed through defining consistent environments (Databricks Repos, Databricks CLI with job definitions) and managing dependencies, not primarily through cluster pools. While reproducibility could be indirectly assisted by using a cluster pool that consistently provides the same environment, it's not the primary reason to use pools.

Therefore, the speed enhancement offered by cluster pools is the key factor driving their use for quickly refreshing automated reports. The ability to rapidly allocate resources minimizes delays and ensures reports are generated and distributed in a timely manner, enabling faster decision-making based on near real-time data.

Refer to Databricks documentation for detailed information on cluster pools and their functionalities:

[Databricks Cluster Pools](#)

[Best practices: Using cluster pools](#)

**Question: 93**

**Deep Dumps**

What is hosted completely in the control plane of the classic Databricks architecture?

- A. Worker node
- B. Databricks web application
- C. Driver node
- D. Databricks Filesystem

**Answer: B**

**Explanation:**

The correct answer is B, the Databricks web application. The control plane in the classic Databricks architecture is responsible for managing and coordinating the Databricks environment. This includes aspects like authentication, authorization, workspace management, job scheduling, and the web interface that users interact with.

The Databricks web application is the primary user interface for interacting with the Databricks service. It's where users manage their workspaces, notebooks, jobs, clusters, and data. This interface resides entirely within the control plane, managed and operated by Databricks. It doesn't run on the user's infrastructure.

On the other hand, worker nodes and driver nodes (A and C) are part of the data plane. These nodes are the computational resources where the actual data processing and Spark jobs execute. They are typically

provisioned within the user's cloud environment (AWS, Azure, GCP) and are responsible for processing data and running Spark applications.

The Databricks File System (DBFS) (D) is a distributed file system mounted into a Databricks workspace and available within Databricks clusters. While DBFS has elements managed by the control plane (metadata and access control), the actual storage of data resides in the user's cloud storage account (e.g., AWS S3, Azure Blob Storage, or Google Cloud Storage). Therefore, DBFS is not completely in the control plane.

The control plane being managed entirely by Databricks allows them to handle infrastructure management, upgrades, and security patching for the web application seamlessly. This separation of concerns (control plane managed by Databricks, data plane in the user's cloud) is a core principle of the Databricks architecture, providing a managed service experience while giving users control over their data and compute resources.

For further reading:

**Databricks Architecture:** <https://docs.databricks.com/en/getting-started/concepts.html#architecture>

**Control Plane vs Data Plane:** (While not specific to Databricks, understanding the general concept is helpful): <https://www.cloudflare.com/learning/ddos/what-is-the-ddos-control-plane/>

### Question: 94

Deep Dumps

A data engineer needs to determine whether to use the built-in Databricks Notebooks versioning or version their project using Databricks Repos.

What is an advantage of using Databricks Repos over the Databricks Notebooks versioning?

- A. Databricks Repos allows users to revert to previous versions of a notebook
- B. Databricks Repos is wholly housed within the Databricks Data Intelligence Platform
- C. Databricks Repos provides the ability to comment on specific changes
- D. Databricks Repos supports the use of multiple branches

**Answer: D**

**Explanation:**

The correct answer is D: Databricks Repos supports the use of multiple branches. Here's why:

Databricks Notebooks versioning, while helpful for basic version control, offers limited functionality compared to Git-based version control systems like those used in Databricks Repos. The crucial advantage lies in **branching**. Branching allows data engineers to work on different features, bug fixes, or experiments in isolation without affecting the main codebase (usually the main or master branch). This is a fundamental principle of modern software development and facilitates parallel development, collaboration, and easier rollback of changes. Notebook versioning, on the other hand, primarily provides a linear history of changes to a single notebook.

Databricks Repos leverages Git for version control, thus inheriting Git's powerful branching capabilities. This means developers can create new branches (e.g., feature/new-data-pipeline) to develop new features, and subsequently merge these changes back into the main branch via pull requests after review. This workflow promotes code quality and collaboration.

Option A is incorrect because Databricks Notebook versioning does allow reverting to previous versions of a notebook. Option B is incorrect because both are housed within the Databricks platform, and this isn't a distinguishing advantage. Option C is inaccurate, as code review and commenting are possible via pull requests with Databricks Repos, but not built into native Notebook versioning. Databricks Repos, through Git integration, provides more comprehensive collaboration and code management features.

In summary, the ability to create and manage multiple branches for parallel development and feature isolation is a significant advantage of Databricks Repos compared to the built-in Notebook versioning.

Supporting Links:

**Databricks Repos:** <https://docs.databricks.com/repos/index.html>

**Git Branching:** <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>

### Question: 95

Deep Dumps

What is a benefit of the Databricks Lakehouse Architecture embracing open source technologies?

- A.Avoiding vendor lock-in
- B.Simplified governance
- C.Ability to scale workloads
- D.Cloud-specific integrations

**Answer: A**

**Explanation:**

The correct answer is A, avoiding vendor lock-in. The Databricks Lakehouse Architecture leverages open-source technologies like Apache Spark, Delta Lake, and MLflow. Choosing open-source formats and frameworks allows organizations to avoid being tied to a single vendor's proprietary solutions. This gives them the freedom to switch providers or adopt best-of-breed tools without extensive data migration or code refactoring. Vendor lock-in can lead to increased costs, limited flexibility, and reduced innovation as businesses become dependent on a particular provider's services and pricing. Open source reduces dependency and enables broader community support and improvements. Simplified governance (B) is a potential benefit but isn't directly inherent to open-source. Scaling workloads (C) is possible due to Spark, but open-source alone doesn't guarantee scalability. Cloud-specific integrations (D) may be present in Databricks, but this isn't a primary benefit of using open-source technologies within the Lakehouse Architecture itself. The freedom of choice and control offered by open-source are key advantages for long-term data strategy. By using open standards and technologies, organizations can ensure that their data and analytical pipelines remain portable and interoperable across different environments. Here are a few resources for further research:

**Apache Spark:** <https://spark.apache.org/>

**Delta Lake:** <https://delta.io/>

**MLflow:** <https://mlflow.org/>

**Databricks Lakehouse:** <https://www.databricks.com/product/lakehouse-architecture>

### Question: 96

Deep Dumps

A data engineer needs to use a Delta table as part of a data pipeline, but they do not know if they have the appropriate permissions.

In which location can the data engineer review their permissions on the table?

- A.Jobs
- B.Dashboards
- C.Catalog Explorer
- D.Repos

**Answer: C**

**Explanation:**

The correct answer is C, Catalog Explorer. Here's a detailed justification:

Catalog Explorer within Databricks is the primary tool for discovering and managing data assets, including Delta tables. A core function of Catalog Explorer is to provide users with detailed information about tables, including their schema, metadata, and importantly, access control lists (ACLs). These ACLs define the permissions granted to users and groups on the table. A data engineer can navigate to the specific Delta table within Catalog Explorer and then review the "Permissions" tab or section. This section clearly lists who has what type of access (e.g., SELECT, MODIFY, CREATE).

Jobs (A) are used for scheduling and managing data pipelines, not for inspecting table-level permissions. While a job might fail due to insufficient permissions, it wouldn't be the place to review what those permissions are. Dashboards (B) are for visualizing data and don't provide information about table permissions. Repos (D) are for version controlling code, which is unrelated to data permissions on Delta tables.

Therefore, Catalog Explorer is the only tool of the options presented that directly addresses the data engineer's need to understand their permissions on a Delta table within the Databricks environment. The permissions information displayed enables them to understand if they can read, write, or modify the table.

For further information, you can refer to the Databricks documentation on Catalog Explorer and data governance:

**Databricks Documentation on Unity Catalog:** <https://docs.databricks.com/en/unity-catalog/index.html> (Note: Unity Catalog is used to manage permissions centrally)

**Catalog Explorer:** <https://docs.databricks.com/en/data-governance/catalog-explorer/index.html>

**Question: 97**

**Deep Dumps**

A data engineer is running code in a Databricks Repo that is cloned from a central Git repository. A colleague of the data engineer informs them that changes have been made and synced to the central Git repository. The data engineer now needs to sync their Databricks Repo to get the changes from the central Git repository.

Which Git operation does the data engineer need to run to accomplish this task?

- A.Clone
- B.Pull
- C.Merge
- D.Push

**Answer: B**

**Explanation:**

The correct Git operation for the data engineer to synchronize their Databricks Repo with the central Git repository after changes have been made remotely is **B. Pull**.

Here's why:

Git repositories maintain a local copy and a remote copy of the codebase. When a repository is initially set up, Clone is used to copy the remote repository to the local environment. Subsequent updates from the remote repository require a different operation.

**Clone:** This is used to create a local copy of a remote repository. It is a one-time operation usually done at the

start. Since the data engineer already has a cloned repo, Clone isn't appropriate.

**Pull:** This operation fetches changes from a remote repository and merges them into the current branch of the local repository. It's essentially a combination of fetch (downloading the remote changes) and merge (integrating them into the local branch). This is exactly what the data engineer needs to do – get the latest changes from the central repository.

**Merge:** While merge is part of the pull operation, it specifically integrates changes from one branch into another. A simple merge operation, without first fetching, will not retrieve updates from the remote repository.

**Push:** This operation uploads local changes to the remote repository. The data engineer wants to receive changes, not send them.

Therefore, to get the latest changes made by the colleague in the central Git repository and integrate those changes into the local Databricks Repo, the data engineer needs to execute a pull operation. This ensures their local copy is synchronized with the latest version in the central repository. A pull keeps local repo up-to-date. Databricks repos use standard git commands under the hood, so standard git workflows apply.

[Git Basics - Getting a Git Repository](#)[Git Branching - Basic Branching and Merging](#)

## Question: 98

Deep Dumps

Which file format is used for storing Delta Lake Table?

- A.CSV
- B.Parquet
- C.JSON
- D.Delta

**Answer: B**

**Explanation:**

Delta Lake, an open-source storage layer, brings ACID (Atomicity, Consistency, Isolation, Durability) transactions to Apache Spark and big data workloads. The foundation of a Delta Lake table lies in its underlying storage format. The primary format used for storing data in a Delta Lake table is **Parquet**.

While Delta Lake provides additional features like versioning, time travel, and schema evolution, it leverages the efficient storage capabilities of Parquet. Parquet is a columnar storage format that optimizes for query performance, especially when dealing with large datasets, as it enables efficient encoding and compression, leading to faster data retrieval and reduced storage costs. CSV and JSON are row-oriented formats that are not optimized for the analytical workloads that Delta Lake targets. The "Delta" format itself isn't a data storage format but rather refers to the Delta Lake transaction log which stores metadata about changes to the Parquet files. The Delta log provides a versioned history of the Delta Lake table, enabling features like time travel and rollback capabilities.

Therefore, the actual data within a Delta Lake table resides in Parquet files, while the Delta Lake transaction log manages the metadata and changes to those files. Other file formats like CSV or JSON are not natively integrated for storing the actual underlying data in a Delta Lake Table. Using Parquet allows Delta Lake to provide ACID compliance and enhanced query performance, making it ideal for data warehousing and large-scale data processing scenarios. Choosing Parquet as the storage format ensures efficient data storage, retrieval, and processing, which is essential for building reliable and performant data pipelines on platforms like Databricks.



Here's a useful link for further reading:

[What is Delta Lake?](#)

### Question: 99

Deep Dumps

A data architect has determined that a table of the following format is necessary:

```
employeeId startDate avgRating
```

```
a1 2009-01-06 5.5
```

```
a2 2018-11-21 7.1
```

```
...
```

Which code block is used by SQL DDL command to create an empty Delta table in the above format regardless of whether a table already exists with this name?

- A.CREATE OR REPLACE TABLE table\_name ( employeeId STRING, startDate DATE, avgRating FLOAT )
- B.CREATE OR REPLACE TABLE table\_name WITH COLUMNS ( employeeId STRING, startDate DATE, avgRating FLOAT ) USING DELTA
- C.CREATE TABLE IF NOT EXISTS table\_name ( employeeId STRING, startDate DATE, avgRating FLOAT )
- D.CREATE TABLE table\_name AS SELECT employeeId STRING, startDate DATE, avgRating FLOAT

**Answer: A**

**Explanation:**

```
CREATE OR REPLACE TABLE table_name (employeeId STRING, startDate DATE, avgRating FLOAT)
```

### Question: 100

Deep Dumps

A data engineer has been given a new record of data:

```
id STRING = 'a1'
```

```
rank INTEGER = 6
```

```
rating FLOAT = 9.4
```

Which SQL commands can be used to append the new record to an existing Delta table my\_table?

- A.INSERT INTO my\_table VALUES ('a1', 6, 9.4)
- B.INSERT VALUES ('a1', 6, 9.4) INTO my\_table
- C.UPDATE my\_table VALUES ('a1', 6, 9.4)
- D.UPDATE VALUES ('a1', 6, 9.4) my\_table

**Answer: A**

**Explanation:**

The correct SQL command to append a new record to an existing Delta table my\_table is INSERT INTO my\_table VALUES ('a1', 6, 9.4). Let's break down why and why the others are incorrect.

INSERT INTO is the standard SQL command for adding new rows to a table. It's fundamental to database manipulation. The syntax is INSERT INTO table\_name VALUES (value1, value2, value3, ...). In our case, my\_table is the table we're inserting into, and ('a1', 6, 9.4) represents the values for the id, rank, and rating columns, respectively. These values directly correspond to the data types defined for those columns (STRING, INTEGER, and FLOAT).

Option B, INSERT VALUES ('a1', 6, 9.4) INTO my\_table, reverses the expected syntax. The INTO clause should precede the table name. SQL syntax requires a specific order, and reversing it will cause a syntax error.

Options C and D use UPDATE. UPDATE commands are used to modify existing rows in a table, not to create new ones. You use UPDATE when you need to change the values in certain columns of a row that already exists based on some condition (e.g., UPDATE my\_table SET rating = 9.5 WHERE id = 'a1'). Attempting to use UPDATE without a WHERE clause to add a new row is incorrect and will likely result in errors or unexpected behavior, as it's not designed for record insertion. UPDATE also always needs a SET clause.

Delta Lake, built on top of Apache Spark, fully supports standard SQL commands, including INSERT INTO. Delta Lake extends the functionality of Spark SQL, providing features like ACID transactions, schema enforcement, and time travel, but it does not alter the basic SQL syntax. Therefore, the familiar INSERT INTO command works seamlessly with Delta tables.

Therefore, INSERT INTO my\_table VALUES ('a1', 6, 9.4) is the only option that aligns with correct SQL syntax and the purpose of adding a new row to a Delta table.

For further research, you can consult the following resources:

**Databricks SQL Reference:** <https://docs.databricks.com/sql/language-manual/index.html>

**Apache Spark SQL Documentation:** <https://spark.apache.org/docs/latest/sql-ref.html>

**Delta Lake Documentation:** <https://docs.delta.io/latest/index.html>

## Question: 101

## Deep Dumps

A data engineer has realized that the data files associated with a Delta table are incredibly small. They want to compact the small files to form larger files to improve performance.

Which keyword can be used to compact the small files?

- A.OPTIMIZE
- B.VACUUM
- C.COMPACTION
- D.REPARTITION

**Answer: A**

**Explanation:**

The correct keyword for compacting small files in a Delta table is OPTIMIZE. Here's why:

OPTIMIZE is a Delta Lake command specifically designed to consolidate small files within a Delta table into larger files. This process directly addresses the "small file problem," a common issue in data lakes that can degrade query performance. Having numerous small files increases metadata management overhead for the Spark engine and leads to excessive I/O operations as it must open, read, and close many small files instead of fewer, larger ones.

OPTIMIZE improves read performance by reducing the number of files that need to be accessed during queries. It achieves this by rewriting existing data into fewer, larger Parquet files, which are the standard

format for Delta tables. You can optionally specify a partitioning column with OPTIMIZE, enabling the command to perform bin-packing and co-locate data within partitions based on specified criteria, further enhancing query performance.

VACUUM, on the other hand, is used for removing files that are no longer referenced by the Delta table's transaction log (older versions of the data). It is essential for cost optimization by deleting obsolete data files but does not consolidate existing small files.

COMPACTION isn't a standard Delta Lake keyword, though the concept of compaction is what OPTIMIZE achieves. The documentation uses the term "compaction" to describe what OPTIMIZE does.

REPARTITION is a Spark operation used to change the number of partitions in a DataFrame, which can influence the number of files written when the DataFrame is saved as a Delta table. While using REPARTITION before writing a Delta table can prevent small file issues initially, it's not the solution for compacting existing small files within an already created Delta table. OPTIMIZE is specifically created to tackle already created small file issue.

In summary, OPTIMIZE is the only keyword designed to address the stated problem of compacting small files in an existing Delta table. Therefore, using OPTIMIZE is the most effective way to improve performance in this scenario.

[Delta Lake Optimize Command](#)[Delta Lake Vacuum Command](#)

## Question: 102

Deep Dumps

A data engineer wants to create a data entity from a couple of tables. The data entity must be used by other data engineers in other sessions. It also must be saved to a physical location.

Which of the following data entities should the data engineer create?

- A. Table
- B. Function
- C. View
- D. Temporary view

**Answer: A**

**Explanation:**

The correct answer is A (Table) because it best addresses all the requirements outlined in the problem. Here's a breakdown of why:

**Data Entity:** Tables, views, functions, and temporary views are all valid data entities.

**Used by Other Data Engineers in Other Sessions:** This rules out temporary views. Temporary views are session-scoped. They are only available within the session in which they were created and disappear once the session ends. Other users/sessions cannot access them. Tables, views, and functions persist across sessions.

**Saved to a Physical Location:** This is a key differentiator. Tables are physically stored in a cloud storage location (like Azure Blob Storage or AWS S3, configured in the Databricks workspace). When a table is created, the data is persistently written to the defined storage path. Views, while persistent metadata-wise, are logical representations of data from one or more tables/views. They don't store data on their own; they are queries stored as objects. Functions, similarly, are code blocks that perform a specific task and don't contain actual data. The physical data continues to be available in the underlying storage and not in the functions.

**Benefits of Tables:** Creating a table allows other data engineers to query and manipulate the data in a structured format. The table acts as a source of truth and enforces a schema, ensuring data consistency.

Therefore, a table satisfies all the requirements: it's a data entity, can be accessed by other users in different sessions, and is physically saved to a storage location.

Authoritative Links for Further Research:

**Databricks Tables:** <https://docs.databricks.com/en/data/tables.html>

**Databricks Views:** <https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-create-view.html>

**Databricks Temporary Views:** <https://spark.apache.org/docs/latest/sql-ref-syntax-ddl-create-view.html>

**Databricks User-Defined Functions (UDFs):** <https://docs.databricks.com/en/sql/udf/index.html>

### Question: 103

Deep Dumps

A data engineer runs a statement every day to copy the previous day's sales into the table transactions. Each day's sales are in their own file in the location "/transactions/raw".

Today, the data engineer runs the following command to complete this task:

```
COPY INTO transactions
FROM "/transactions/raw"
FILEFORMAT = PARQUET;
```

After running the command today, the data engineer notices that the number of records in table transactions has not changed.

What explains why the statement might not have copied any new records into the table?

- A.The format of the files to be copied were not included with the FORMAT\_OPTIONS keyword.
- B.The COPY INTO statement requires the table to be refreshed to view the copied rows.
- C.The previous day's file has already been copied into the table.
- D.The PARQUET file format does not support COPY INTO.

**Answer: C**

**Explanation:**

The previous day's file has already been copied into the table.

### Question: 104

Deep Dumps

Which command can be used to write data into a Delta table while avoiding the writing of duplicate records?

- A.DROP
- B.INSERT
- C.MERGE
- D.APPEND

**Answer: C**

**Explanation:**

The correct answer is MERGE because it provides a powerful mechanism for updating Delta tables while handling duplicate records intelligently.

The MERGE command allows you to conditionally insert, update, or delete rows in a target Delta table based on conditions evaluated against a source table or DataFrame. It avoids the writing of duplicate records because it can identify existing records based on a JOIN condition and either update them or ignore new records if they are duplicates. This operation is idempotent, meaning it can be run multiple times without creating further duplicates beyond the initial run.

Compared to other options, DROP is used to remove tables or views entirely, not to manage duplicate records during data writing. INSERT blindly appends new records, leading to duplicates if the same data is inserted repeatedly. APPEND is essentially the same as INSERT, and therefore also contributes to the problem of duplicates.

MERGE offers flexibility in how duplicate records are handled. You can specify conditions to update existing records that match the criteria or insert new records only if they don't already exist, effectively preventing duplicates. You can also handle updates and inserts simultaneously within a single MERGE statement. This is crucial for scenarios where data sources can produce updates to existing records or entirely new records, some of which might already be present. It's also important for slowly changing dimensions.

The atomicity and transactionality of Delta Lake guarantee that the MERGE operation either completes fully, with all changes applied, or rolls back entirely in case of failures, ensuring data consistency and preventing partial updates that could lead to inconsistencies related to duplicate records.

Further reading:

**Delta Lake MERGE:** <https://docs.databricks.com/delta/delta-update.html>

**Databricks documentation on Delta Lake:** <https://docs.databricks.com/delta/index.html>

## Question: 105

## Deep Dumps

A data analyst has created a Delta table sales that is used by the entire data analysis team. They want help from the data engineering team to implement a series of tests to ensure the data is clean. However, the data engineering team uses Python for its tests rather than SQL.

Which command could the data engineering team use to access sales in PySpark?

- A. `SELECT * FROM sales`
- B. `spark.table("sales")`
- C. `spark.sql("sales")`
- D. `spark.delta.table("sales")`

**Answer: B**

**Explanation:**

The correct way to access a Delta table in PySpark is using `spark.table("sales")`. Here's why:

Option A, `SELECT * FROM sales`, is SQL syntax. While you can execute SQL queries in PySpark using `spark.sql()`, directly using a SQL query string isn't the preferred method for simply accessing an existing table as a DataFrame.

Option C, `spark.sql("sales")`, is incorrect. `spark.sql()` is used to execute a SQL query string. Passing just the table name "sales" as a string won't retrieve the table's content. It would be trying to interpret "sales" as a query, which is invalid.

Option D, `spark.delta.table("sales")`, is also incorrect. While `spark.delta` is related to Delta Lake, `spark.delta.table()` is primarily used when you want to get a `DeltaTable` object, not a Spark DataFrame. The

DeltaTable object provides Delta Lake-specific operations like versioning and time travel. If the goal is merely to read the data for testing purposes, retrieving a DataFrame is simpler and more efficient.

`spark.table("sales")` directly loads the Delta table named "sales" into a PySpark DataFrame, which is the standard and most straightforward way to access it within PySpark. This creates a DataFrame that the data engineering team can then use with Python-based data quality testing libraries like Great Expectations or Deequ. The DataFrame representation makes the data accessible through PySpark's API, enabling various transformations and analysis, essential for implementing their data validation procedures. Using `spark.table()` ensures that the tests operate directly on the data, ensuring the accuracy of the data validation process.

Relevant documentation:

**SparkSession.table() documentation:**

<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.Session.html>

**Delta Lake documentation:** <https://docs.delta.io/latest/index.html>

### Question: 106

Deep Dumps

A data engineer has created a new database using the following command:

```
CREATE DATABASE IF NOT EXISTS customer360;
```

In which location will the customer360 database be located?

- A.dbfs:/user/hive/database/customer360
- B.dbfs:/user/hive/warehouse
- C.dbfs:/user/hive/customer360
- D.dbfs:/user/hive/database

**Answer: B**

**Explanation:**

The correct answer is B. `dbfs:/user/hive/warehouse`. Here's why:

When a database is created in Databricks without specifying a `LOCATION`, the database's metadata and associated tables are stored in a default location managed by the Hive metastore. The default location for Hive metastore managed databases and tables is `dbfs:/user/hive/warehouse`. This directory acts as the central repository for all managed data in the Hive metastore.

Option A, `dbfs:/user/hive/database/customer360`, is incorrect. While it might seem logical that a database named `customer360` would be placed in a similarly named directory under `/database`, the standard Hive metastore convention places databases directly under the warehouse directory. The specific location is not under the `"/database"` folder.

Option C, `dbfs:/user/hive/customer360`, is incorrect. Again, this is a plausible path, but is not the default managed location. The database and its tables will be located under `/user/hive/warehouse`.

Option D, `dbfs:/user/hive/database` is incomplete. When creating a database, specifying the `LOCATION` is optional, but the default location should be `/user/hive/warehouse`.

The `IF NOT EXISTS` clause in the `CREATE DATABASE` statement ensures that the command will not fail if a database with the same name already exists.

Understanding the Hive metastore and how Databricks leverages it for managing data and metadata is critical

for data engineers working on the platform. It allows for consistent data discovery and management.

For more information, you can refer to the official Databricks documentation on database management and Hive metastore integration.

Databricks documentation for CREATE DATABASE: <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-database.html>

Databricks documentation on managed tables: <https://docs.databricks.com/data/tables.html#managed-and-unmanaged-tables>

### Question: 107

Deep Dumps

A data engineer is attempting to drop a Spark SQL table `my_table` and runs the following command:

```
DROP TABLE IF EXISTS my_table;
```

After running this command, the engineer notices that the data files and metadata files have been deleted from the file system.

What is the reason behind the deletion of all these files?

- A. The table was managed
- B. The table's data was smaller than 10 GB
- C. The table did not have a location
- D. The table was external

**Answer: A**

**Explanation:**

Here's a detailed justification for why the correct answer is A (The table was managed) when a `DROP TABLE IF EXISTS my_table` command results in the deletion of both data and metadata files.

When a Spark SQL table is created, it can be either a managed table (also known as an internal table) or an external table. The crucial difference lies in how the data is managed and where it resides.

**Managed Tables:** In the case of a managed table, Spark SQL controls both the data and the metadata. The data files physically reside within the Spark warehouse directory (e.g., `/user/hive/warehouse` by default) or a location managed by Spark. Crucially, when you `DROP` a managed table, Spark completely removes the table, including the data files and the metadata that defines the table's schema and properties within the metastore.

**External Tables:** Conversely, an external table's data resides in a location outside the Spark warehouse. You explicitly define the location of the data files when creating the external table (using the `LOCATION` clause in the `CREATE TABLE` statement). When you `DROP` an external table, Spark only removes the metadata from the metastore. The underlying data files in the specified location remain untouched. This is because Spark only "points" to the external data; it does not own or manage the storage.

Therefore, the `DROP TABLE` command's behavior of deleting both data and metadata definitively indicates that the table in question, `my_table`, was a managed table. The other options are not relevant: table size (option B) and the absence of a location (option C) would not lead to data deletion on a managed table. In fact, managed tables do not require a user-specified location; Spark manages their location internally.

Because the engineer observed the deletion of both data and metadata, it provides conclusive evidence that `my_table` was a managed table. Here is supporting documentation:



**Apache Hive Documentation (Concepts of Managed vs. External Tables):** While the concept originated in Hive, it's fundamental to Spark SQL's table management. You can search for "managed vs external tables" in the Apache Hive documentation to understand the core distinction. For instance, consult:

<https://cwiki.apache.org/confluence/display/Hive/Managed+vs+External+Tables>

**Databricks Documentation (Create Table):** Look for sections detailing how to create managed and external tables, noting the significance of the LOCATION keyword. Search for "create table external" in Databricks documentation. <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table.html>

### Question: 108

Deep Dumps

A data engineer needs to create a table in Databricks using data from a CSV file at location /path/to/csv.

They run the following command:

```
CREATE TABLE new_table

OPTIONS (
 header = "true",
 delimiter = "|"
)
LOCATION "path/to/csv"
```

Which of the following lines of code fills in the above blank to successfully complete the task?

- A.FROM "path/to/csv"
- B.USING CSV
- C.FROM CSV
- D.USING DELTA

**Answer: B**

**Explanation:**

Correct answer is B:USING CSV.

### Question: 109

Deep Dumps

What is a benefit of creating an external table from Parquet rather than CSV when using a CREATE TABLE AS SELECT statement?

- A.Parquet files can be partitioned
- B.Parquet files will become Delta tables
- C.Parquet files have a well-defined schema
- D.Parquet files have the ability to be optimized

**Answer: C**

**Explanation:**

Here's a detailed justification for why option C, "Parquet files have a well-defined schema," is the most accurate benefit when creating an external table from Parquet versus CSV using a CREATE TABLE AS SELECT (CTAS) statement in Databricks.

When using CTAS, you are essentially creating a new table based on the result of a SELECT query. The source data for this query can be either CSV or Parquet files. CSV (Comma Separated Values) is a plain text format where data fields are separated by commas. While widely used, CSV lacks a built-in schema definition. This means the data types for each column are not explicitly specified within the file itself. The schema must be inferred or externally defined.

Parquet, on the other hand, is a columnar storage format optimized for data warehousing and analytics. A key advantage of Parquet is that it stores schema information (data types for each column, column names) directly within the file's metadata. This self-describing nature is particularly beneficial when creating tables using CTAS.

When creating an external table from Parquet using CTAS, Databricks can directly read and leverage the schema embedded within the Parquet files. This ensures that the created table accurately reflects the data types and structure of the underlying data. In contrast, when using CSV files, the schema must be explicitly defined in the CTAS statement or inferred by Databricks, which can sometimes lead to incorrect data type assignments or parsing errors. The schema evolution capabilities also facilitate seamless upgrades.

Option A is incorrect because while partitioning can improve query performance, it's a separate feature and not inherent only to Parquet with CTAS. Partitioning can be applied regardless of the file format. Option B is incorrect because Parquet files will not automatically transform to Delta tables unless you explicitly convert them. Option D is incorrect because optimization is also separate and not automatically applied, nor is it a feature that solely benefits Parquet when used with CTAS.

Therefore, having a well-defined schema embedded within Parquet files simplifies the CTAS process, reduces the risk of data type mismatches, and ensures data integrity, making it the most significant benefit in this context.

For further reading and reference, please refer to the official Databricks documentation on:

**Data Formats:** <https://docs.databricks.com/en/delta/optimizations/file-selection-optimization.html>

**CREATE TABLE AS SELECT:** <https://spark.apache.org/docs/latest/sql-ref-syntax-dml-insert.html>

**Apache Parquet:** <https://parquet.apache.org/>

**Question: 110**

**Deep Dumps**

Which SQL keyword can be used to convert a table from a long format to a wide format?

- A.TRANSFORM
- B.PIVOT
- C.SUM
- D.CONVERT

**Answer: B**

**Explanation:**

The correct answer is **B. PIVOT**.

The PIVOT operator in SQL is specifically designed to transform data from a long (or narrow) format, where data is stacked vertically, to a wide format, where data is spread horizontally. Essentially, PIVOT rotates rows into columns.

Here's why the other options are incorrect:

**A. TRANSFORM:** This is not a standard SQL keyword for data transformation from long to wide formats. While transformations are common in data engineering (often using Spark's transformations), "TRANSFORM" isn't a specific SQL command for pivoting.

**C. SUM:** SUM is an aggregate function used to calculate the sum of values within a group. It doesn't inherently change the structure of the table from long to wide. It might be used in conjunction with a PIVOT operation, but it isn't the keyword responsible for the transformation itself.

**D. CONVERT:** CONVERT is generally used for data type conversions (e.g., converting a string to an integer). It does not manipulate the structure of the table to pivot rows into columns.

The PIVOT operation reorganizes data based on values in one or more columns, turning those values into new column headers. The data associated with those values is then distributed across these newly created columns. This is the fundamental characteristic of a long-to-wide transformation. Within Databricks, which leverages Apache Spark SQL, the PIVOT keyword functions as it would in standard SQL databases, providing a straightforward way to reshape data. Data engineers frequently employ PIVOT to create summary reports or prepare data for machine learning models, where a wide format is often preferred. It reduces data redundancy by representing multiple measurements for a single entity within a single row, enhancing readability and facilitating analysis.

For further research, you can refer to:

**Databricks SQL documentation on PIVOT:** <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-qry-select-pivot.html>

**General SQL PIVOT documentation (applies conceptually):** <https://www.sqlservertutorial.net/sql-server-pivot/>

**Question: 111****Deep Dumps**

A data engineer has a Python variable `table_name` that they would like to use in a SQL query. They want to construct a Python code block that will run the query using `table_name`.

They have the following incomplete code block:

```
____(f"SELECT customer_id, spend FROM table_name ")
```

What can be used to fill in the blank to successfully complete the task?

- A.spark.delta.sql
- B.spark.sql
- C.spark.table
- D.dbutils.sql

**Answer: B****Explanation:**

The correct answer is **B. spark.sql**. Here's why:

The core objective is to execute a SQL query constructed dynamically using a Python variable (`table_name`) within a Databricks environment, which leverages Apache Spark. `spark` here represents the `SparkSession`, the entry point to Spark functionality. To run SQL queries against data in a Spark environment, you use the `spark.sql()` method. This method takes a SQL query string as input and returns a Spark `DataFrame` representing the result of that query.

The provided code snippet `____(f"SELECT customer_id, spend FROM table_name ")` indicates that we need a function or method that accepts a SQL query string. Using an f-string allows us to seamlessly inject the `table_name` variable into the query. `spark.sql()` perfectly fits this requirement.

Option A, `spark.delta.sql`, is incorrect because it's more specific to Delta Lake functionalities. While you might be querying a Delta table, `spark.sql` is the general method for executing SQL queries, even on Delta tables. Using `spark.delta.sql` might not work universally for all table types.

Option C, `spark.table`, is also incorrect. `spark.table()` is used to retrieve a table as a `DataFrame`, not to execute arbitrary SQL queries against it. It returns an existing table or view as a `DataFrame`.

Option D, `dbutils.sql`, is incorrect. While `dbutils` provides utility functions for interacting with Databricks, it doesn't directly offer a method to execute SQL queries and return `DataFrames` in the same way as `spark.sql`. Furthermore, `dbutils` typically focuses on file system operations, secret management, and notebook utilities rather than core Spark SQL functionality.

In summary, `spark.sql()` is the direct and appropriate method in Spark for executing SQL queries constructed as strings, making it the correct choice to complete the given code block.

For further reading, refer to the official Apache Spark documentation:

Spark SQL, DataFrames and Datasets Guide: <https://spark.apache.org/docs/latest/sql-getting-started.html>

SparkSession: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.SparkSession.html>

## Question: 112

Deep Dumps

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

sales

| customer_id | spend | units |
|-------------|-------|-------|
|-------------|-------|-------|

|    |       |   |
|----|-------|---|
| a1 | 28.94 | 7 |
|----|-------|---|

|    |        |    |
|----|--------|----|
| a3 | 874.12 | 23 |
|----|--------|----|

|    |      |   |
|----|------|---|
| a4 | 8.99 | 1 |
|----|------|---|

favorite\_stores

| customer_id | store_id |
|-------------|----------|
|-------------|----------|

|    |    |
|----|----|
| a1 | s1 |
|----|----|

|    |    |
|----|----|
| a2 | s1 |
|----|----|

|    |    |
|----|----|
| a4 | s2 |
|----|----|

The data engineer runs the following query to join these tables together:

```
SELECT
```

```
 sales.customer_id,
```

```
 sales.spend,
```

```
 favorite_stores.store_id
```

```
FROM sales
```

```
LEFT JOIN favorite_stores
```

```
ON sales.customer_id = favorite_stores.customer_id;
```

| customer_id | spend | store_id |
|-------------|-------|----------|
|-------------|-------|----------|

|    |       |    |
|----|-------|----|
| a1 | 28.94 | s1 |
|----|-------|----|

|    |      |    |
|----|------|----|
| a2 | NULL | s1 |
|----|------|----|

|    |      |    |
|----|------|----|
| a4 | 8.99 | s2 |
|----|------|----|

A.

customer\_idspendstore\_id

a1 28.94 s1

a4 8.99 s2

B.

customer\_idspend store\_id

a1 28.94 s1

a3 874.12NULL

a4 8.99 s2

C.

customer\_idspend store\_id

a1 28.94 s1

a2 NULL s1

a3 874.12NULL

a4 8.99 s2

D.

Answer: C

Explanation:

customer\_idspend store\_id

a1 28.94 s1

a3 874.12NULL

a4 8.99 s2

### Question: 113

Deep Dumps

A data engineer needs to apply custom logic to identify employees with more than 5 years of experience in array column employees in table stores. The custom logic should create a new column exp\_employees that is an array of all of the employees with more than 5 years of experience for each row. In order to apply this custom logic at scale, the data engineer wants to use the FILTER higher-order function.

Which code block successfully completes this task?

A.

```
SELECT
 store_id,
 employees,
 FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

B.

```
SELECT
 store_id,
 employees,
 FILTER (exp_employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

C.

```
SELECT
 store_id,
 employees,
 FILTER (employees, years_exp > 5) AS exp_employees
FROM stores;
```

D.

```
SELECT
 store_id,
 employees,
 CASE WHEN employees.years_exp > 5 THEN employees
 ELSE NULL
 END AS exp_employees
FROM stores;
```

**Answer: A**

**Explanation:**

```
SELECT
 store_id,
 employees,
 FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

#### Question: 114

**Deep Dumps**

A data engineer that is new to using Python needs to create a Python function to add two integers together and return the sum?



Which code block can the data engineer use to complete this task?

- A. 

```
function add_integers(x, y):
 return x + y
```
- B. 

```
def add_integers(x, y):
 print(x + y)
```
- C. 

```
def add_integers(x, y):
 x + y
```
- D. 

```
def add_integers(x, y):
 return x + y
```

**Answer: D**

**Explanation:**

```
def add_integers(x, y):
 return x + y
```

#### Question: 115

Deep Dumps

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.table("sales")
 .withColumn("avg_price", col("sales") / col("units"))
 .writeStream
 .option("checkpointLocation", checkpointPath)
 .outputMode("complete")
 .
 .table("new_sales")
)
```

Which line of code should the data engineer use to fill in the blank if the data engineer only wants the query to execute a micro-batch to process data every 5 seconds?

- A. `trigger("5 seconds")`
- B. `trigger(continuous="5 seconds")`
- C. `trigger(once="5 seconds")`
- D. `trigger(processingTime="5 seconds")`

**Answer: D**

**Explanation:**

trigger(processingTime="5 seconds").

**Question: 116****Deep Dumps**

A data engineer is maintaining a data pipeline. Upon data ingestion, the data engineer notices that the source data is starting to have a lower level of quality. The data engineer would like to automate the process of monitoring the quality level.

Which of the following tools can the data engineer use to solve this problem?

- A.Auto Loader
- B.Unity Catalog
- C.Delta Lake
- D.Delta Live Tables

**Answer: D****Explanation:**

The correct answer is D, Delta Live Tables (DLT). Here's why:

Delta Live Tables is a framework designed specifically for building and managing reliable data pipelines with automated data quality monitoring. It allows data engineers to define data quality expectations using constraints (e.g., expect\_or\_fail, expect\_or\_drop) directly within their data pipelines. These expectations are evaluated during the pipeline's execution. If data doesn't meet the defined quality thresholds, DLT can take actions like failing the pipeline, dropping the bad records, or quarantining the problematic data for further investigation. This built-in data quality management is a core feature of DLT.

Auto Loader (Option A) is an efficient way to incrementally and automatically ingest new data files as they arrive in cloud storage, but it doesn't inherently provide data quality monitoring. While Auto Loader can be part of a pipeline feeding into a system that does check data quality, it doesn't do it itself.

Unity Catalog (Option B) is Databricks' unified governance solution for data and AI assets. It provides centralized access control, auditing, and data discovery. It helps in organizing and securing data but is not a tool for actively monitoring data quality during pipeline execution. It helps manage lineage, which indirectly aids understanding potential quality issues.

Delta Lake (Option C) is a storage layer that brings reliability to data lakes. It provides ACID transactions, scalable metadata handling, and unified streaming and batch data processing. While Delta Lake improves data reliability in general, it doesn't have built-in automated data quality monitoring like DLT. DLT pipelines often use Delta Lake as the underlying storage format.

Therefore, DLT's features for defining data quality expectations and automatically monitoring data quality during pipeline execution make it the best solution for the data engineer's problem.

Further Research:

**Delta Live Tables Overview:** <https://docs.databricks.com/delta-live-tables/index.html>

**Delta Live Tables expectations:** <https://docs.databricks.com/delta-live-tables/expectations.html>

**Question: 117****Deep Dumps**

A data engineer has three tables in a Delta Live Tables (DLT) pipeline. They have configured the pipeline to drop invalid records at each table. They notice that some data is being dropped due to quality concerns at some point in the DLT pipeline. They would like to determine at which table in their pipeline the data is being dropped.

Which approach can the data engineer take to identify the table that is dropping the records?

- A.They can set up separate expectations for each table when developing their DLT pipeline.
- B.They can navigate to the DLT pipeline page, click on the "Error" button, and review the present errors.
- C.They can set up DLT to notify them via email when records are dropped.
- D.They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics.

**Answer: D**

**Explanation:**

Here's a detailed justification for why option D is the best approach to identify the table in a Delta Live Tables (DLT) pipeline where records are being dropped due to quality concerns:

Option D, navigating to the DLT pipeline page, clicking on each table, and viewing the data quality statistics, provides the most direct and granular way to identify where data is being dropped. DLT inherently captures and displays detailed data quality metrics for each table in the pipeline. This includes metrics like the number of records processed, the number of records that passed expectations, and the number of records that failed expectations (dropped records). By inspecting these statistics for each table, the data engineer can immediately pinpoint the specific table where a significant number of records are being dropped.

Option A, setting up separate expectations, is good practice but primarily focuses on preventing and detecting data quality issues from the beginning. It doesn't directly help diagnose where existing drops are occurring within the current pipeline execution. It's a proactive approach, but not reactive for pinpointing the source of existing drops. Expectations help control the data quality, but the pipeline already has the drop invalid records configuration which is what is leading to the dropping and is being investigated.

Option B, clicking on the "Error" button, provides information about pipeline failures or exceptions, which might not always be directly related to data quality drops specifically. Errors can arise from other pipeline issues besides expectation failures leading to dropped records. "Error" button may show the end result error from the data not present at the end table, but not which table is dropping them along the way.

Option C, email notifications, are a reactive approach that can alert the data engineer when drops occur, but it doesn't automatically identify the problematic table. The notification would still require the data engineer to investigate the pipeline to determine the source of the drops.

In summary, analyzing the data quality statistics for each table in the DLT pipeline provides the most direct and detailed information needed to identify the table where records are being dropped due to quality concerns. This approach leverages the built-in monitoring capabilities of DLT for data quality.

Relevant Resources:

**Databricks Documentation on Delta Live Tables:** <https://docs.databricks.com/delta-live-tables/index.html>

**Databricks Documentation on DLT Monitoring:** <https://docs.databricks.com/delta-live-tables/monitor.html>

**Databricks Blog on DLT:** <https://www.databricks.com/blog/2021/06/15/simplify-etl-with-delta-live-tables.html>

**Question: 118**

**Deep Dumps**

What is used by Spark to record the offset range of the data being processed in each trigger in order for Structured Streaming to reliably track the exact progress of the processing so that it can handle any kind of failure by restarting and/or reprocessing?

- A.Checkpointing and Write-ahead Logs
- B.Replayable Sources and Idempotent Sinks
- C.Write-ahead Logs and Idempotent Sinks
- D.Checkpointing and Idempotent Sinks

**Answer: A**

**Explanation:**

The correct answer is A: Checkpointing and Write-ahead Logs. Let's break down why.

Structured Streaming in Spark ensures fault tolerance and exactly-once semantics through a combination of mechanisms, with **Checkpointing** and **Write-ahead Logs (WAL)** being core to this capability.

**Checkpointing:** This mechanism periodically saves the state of the streaming query to reliable storage (like cloud storage such as AWS S3, Azure Blob Storage, or DBFS). This state includes information about the offset ranges of the input data that have been processed up to a certain point in time. In case of failure, the streaming application can restart from the last checkpoint and resume processing. This allows Spark to remember where it left off. Without checkpointing, Spark would lose track of its progress and potentially reprocess data from the beginning, leading to data duplication.

**Write-ahead Logs (WAL):** Before any data is actually processed, the system writes the details of the intended operation (including the input data offset) to a persistent log. This log ensures that even if a failure occurs during processing, the system can replay the operations from the log and recover the state, preventing data loss. These logs provide fine-grained recovery capabilities in addition to the broader checkpoints.

Together, checkpointing and WAL enable Spark to reliably track the progress of streaming jobs, ensuring fault tolerance and consistent results. The offset ranges of the data being processed are specifically recorded and recovered using these two components. The combination ensures that Spark can handle any kind of failure by restarting and/or reprocessing from a known, consistent state.

Options B, C, and D are not as complete or accurate because:

**Replayable Sources and Idempotent Sinks:** While important, they don't address the fundamental need to track the progress of the streaming query itself. Replayable sources (like Kafka) allow re-reading data, and idempotent sinks ensure that writing the same data multiple times has the same effect as writing it once. But without checkpointing and WAL, the streaming job wouldn't know which data to replay or write.

**Idempotent Sinks** Idempotency is a desirable property for sinks, ensuring that writing the same data multiple times has the same effect as writing it once. This prevents data duplication in case of restarts. However, it does not address the tracking of processing progress; Checkpointing handles this task.

**Write-ahead Logs and Idempotent Sinks:** While WAL is crucial, it typically works in tandem with checkpointing for robust recovery in Structured Streaming. Idempotent sinks are helpful, but without checkpointing to determine what data needs to be written, idempotency alone is insufficient for complete fault tolerance.

Therefore, option A provides the most comprehensive and accurate explanation of how Spark Structured Streaming achieves fault tolerance in streaming data processing.

Relevant Documentation:

[Apache Spark Structured Streaming Programming Guide](#)  
[Databricks Documentation on Structured Streaming](#)

What describes the relationship between Gold tables and Silver tables?

- A. Gold tables are more likely to contain aggregations than Silver tables.
- B. Gold tables are more likely to contain valuable data than Silver tables.
- C. Gold tables are more likely to contain a less refined view of data than Silver tables.
- D. Gold tables are more likely to contain truthful data than Silver tables.

**Answer: A**

**Explanation:**

The correct answer, A, accurately reflects the typical relationship between Gold and Silver tables in a Medallion architecture (Bronze, Silver, Gold). Gold tables are indeed more likely to contain aggregations compared to Silver tables.

The Medallion architecture organizes data processing into distinct layers representing data quality and transformation stages. Bronze tables serve as the raw landing zone for data, mirroring the source data format. Silver tables represent data that has been cleaned, conformed, and enriched with joins/lookups, providing a refined and reliable dataset. Gold tables, at the final stage, house data that's been transformed for specific business needs, often involving aggregations, summaries, and business-oriented views to power dashboards, reports, and downstream applications.

Silver tables focus on standardizing and cleaning the data, establishing a single source of truth with cleaned and validated records. Gold tables focus on deriving business insights. For example, a Silver table may contain individual transaction records. A corresponding Gold table could aggregate those transactions to show total sales by region or product category, representing a more summarized and insightful view. Gold tables are designed for efficient querying by business users, so aggregations and pre-calculated metrics are common. Options B, C, and D are incorrect because while Gold tables might contain more valuable or truthful data in some contexts, the primary difference from Silver tables is the level of data aggregation and business-specific transformation performed. Silver tables contain clean, validated data which is essential for both the Silver and Gold layer, meaning 'truthful data' is found in both layers.

Further research can be done on these topics:

**Databricks Medallion Architecture:** <https://www.databricks.com/blog/2019/08/15/delta-lake-medallion-architecture.html>

**Delta Lake:** <https://delta.io/>

## Question: 120

**Deep Dumps**

What describes when to use the CREATE STREAMING LIVE TABLE (formerly CREATE INCREMENTAL LIVE TABLE) syntax over the CREATE LIVE TABLE syntax when creating Delta Live Tables (DLT) tables using SQL?

- A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.
- B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.
- C. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.
- D. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.

**Answer: B**

**Explanation:**

The correct answer is **B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.**

Let's break down why:

**Delta Live Tables (DLT) purpose:** DLT simplifies building and managing reliable data pipelines on Databricks. It automatically manages infrastructure, scaling, data quality, and error handling. The core idea is to declare the desired end-state of your tables, and DLT figures out how to get there.

**CREATE LIVE TABLE vs. CREATE STREAMING LIVE TABLE:** The key difference lies in how they handle data updates. CREATE LIVE TABLE is designed for batch processing. It recomputes the entire table each time the pipeline runs. CREATE STREAMING LIVE TABLE, however, is designed for incremental processing. It processes only the new data that has arrived since the last pipeline update.

**Incremental Processing:** Incremental processing is critical when dealing with large, continuously updating datasets (streaming data). Instead of reprocessing the entire dataset every time new data arrives, only the new records are processed and appended to the existing table. This significantly improves efficiency and reduces processing time.

**Why other options are incorrect:**

**A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.** The type of the subsequent step is not the primary deciding factor. The need for incremental processing of the current step is what dictates the use of CREATE STREAMING LIVE TABLE.

**C. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.** Complex aggregations can be performed on either type of table. The method of processing (batch vs. incremental) is the deciding factor. While aggregations on streaming data often benefit from incremental updates, that's a consequence, not the defining characteristic.

**D. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.** Again, the nature of the previous step is not the key factor. A static previous step can feed data into either a batch or streaming table.

In summary, CREATE STREAMING LIVE TABLE is essential when you need to process continuously arriving data in an efficient, incremental manner. By only processing new data, it minimizes processing time and resources, making it ideal for streaming data scenarios within DLT pipelines.

**Authoritative Links for further research:**

**Delta Live Tables documentation:** <https://docs.databricks.com/delta-live-tables/index.html>

**Creating a Streaming Live Table:** <https://docs.databricks.com/delta-live-tables/delta-live-tables-tutorial-sql.html#create-a-streaming-live-table>

**Delta Live Tables SQL Reference:** <https://docs.databricks.com/sql/language-manual/delta-live-tables/index.html>

## Question: 121

Deep Dumps

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.

The table is configured to run in Production mode using the Continuous Pipeline Mode.

What is the expected outcome after clicking Start to update the pipeline assuming previously unprocessed data exists and all definitions are valid?

A.All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.

B.All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.



C.All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.

D.All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.

**Answer: C**

**Explanation:**

The correct answer is C. Here's a detailed justification:

Delta Live Tables (DLT) in Production mode with Continuous pipeline mode is designed for continuous data processing. The key lies in the Continuous pipeline mode setting. This mode means the pipeline will run indefinitely, processing new data as it arrives.

STREAMING LIVE TABLE indicates streaming sources. DLT pipelines with streaming sources are designed to run continuously, ingesting and processing new data as it becomes available. These tables are incrementally updated as new data arrives in the source. LIVE TABLE sources based on Delta Lake tables will also be processed.

The question specifies that the pipeline is running in Production mode and utilizing Continuous pipeline mode. Continuous mode ensures the pipeline actively monitors the source data for changes and triggers updates automatically. This is in contrast to Triggered mode which only runs once until specifically re-triggered.

As the question explicitly states that "previously unprocessed data exists," the pipeline will ingest and process this initial backlog. Because it's configured in Continuous mode, it won't stop after processing this initial data. Instead, it will remain active, continually monitoring the source for further updates.

The compute resources are deployed when the pipeline is started and continue to run as long as the pipeline is active in Continuous mode. When the pipeline is stopped manually, the compute resources are then terminated. The option that compute resources will persist for additional testing after the pipeline stops is incorrect.

Therefore, the expected outcome is that all datasets, both streaming and batch, will be updated at set intervals until the pipeline is manually shut down, and compute resources will exist only during pipeline runtime.

Further reading on Delta Live Tables and their various configurations can be found here:

[What is Delta Live Tables?](#)

[Continuous vs. Triggered Execution](#)

## Question: 122

Deep Dumps

Which type of workloads are compatible with Auto Loader?

- A.Streaming workloads
- B.Machine learning workloads
- C.Serverless workloads
- D.Batch workloads

**Answer: A**

**Explanation:**

The correct answer is A, Streaming workloads. Here's a detailed justification:



Auto Loader is a Databricks feature specifically designed to incrementally and efficiently process new data files as they arrive in a cloud storage location (like AWS S3, Azure Data Lake Storage Gen2, or Google Cloud Storage). Its core functionality revolves around continuously detecting and ingesting new files as they become available. This real-time or near real-time ingestion characteristic directly aligns with the requirements of streaming workloads.

Streaming workloads, by definition, involve the continuous processing of data streams. Auto Loader addresses the challenge of managing the continuous influx of new data files that often characterize streaming applications. It automatically handles schema inference, schema evolution, and handles corrupted records, simplifying the data ingestion process for streaming pipelines.

Option B, Machine learning workloads, can utilize data ingested by Auto Loader, but Auto Loader itself isn't a machine learning-specific tool. Machine learning pipelines often consume data, regardless of how it was ingested.

Option C, Serverless workloads, refers to the deployment model of the compute infrastructure. While Auto Loader can be part of a serverless data pipeline (e.g., triggered by a serverless function when new files arrive), its primary role isn't being a serverless workload itself.

Option D, Batch workloads, are typically processed on a fixed schedule or on-demand, processing all available data at once. While Auto Loader can be used to load data for batch processing if the data is arriving in a continuous stream, it's primarily optimized for streaming. Batch processing is not the optimal use case because Auto Loader is meant for incremental loading.

In summary, Auto Loader's continuous, incremental processing nature makes it ideally suited for streaming workloads, where timely data ingestion is crucial. It automatically detects and processes new files, making it a key component in building robust and efficient streaming pipelines.

For more information, consult the official Databricks documentation:

**Auto Loader introduction:** <https://docs.databricks.com/ingestion/auto-loader/index.html>

### Question: 123

### Deep Dumps

A data engineer has developed a data pipeline to ingest data from a JSON source using Auto Loader, but the engineer has not provided any type inference or schema hints in their pipeline. Upon reviewing the data, the data engineer has noticed that all of the columns in the target table are of the string type despite some of the fields only including float or boolean values.

Why has Auto Loader inferred all of the columns to be of the string type?

- A. Auto Loader cannot infer the schema of ingested data
- B. JSON data is a text-based format
- C. Auto Loader only works with string data
- D. All of the fields had at least one null value

**Answer: B**

**Explanation:**

The correct answer is **B. JSON data is a text-based format.**

Auto Loader's schema inference behavior is crucial to understand. While Auto Loader excels at evolving schemas, its initial type inference in the absence of schema hints leans towards safety and minimal data loss. JSON is inherently a text-based format. Without explicit schema information, Auto Loader interprets all values initially as strings to prevent data loss. This is because a string representation can accommodate any possible

value, be it a number, boolean, or complex object.

Consider the example: [ "voted\_answers": "B", "vote\_count": 1, "is\_most\_voted": false ]. While "vote\_count" appears to be an integer and "is\_most\_voted" appears to be a boolean, Auto Loader sees them as strings "1" and "false" respectively, due to JSON's representation and the lack of schema hints. If the column was inferred as a numeric or boolean type, it might lead to errors if a non-numeric or non-boolean value were encountered later, causing the pipeline to break.

Option A is incorrect. Auto Loader can infer schemas, but its default behavior without guidance is to err on the side of safety. Option C is also incorrect. Auto Loader supports various data types, not just strings. Option D is incorrect. While null values can influence schema inference, they are not the sole reason for all columns to be strings. Auto Loader handles null values appropriately during schema inference and will only promote to nullable types when appropriate. The primary reason is the textual nature of JSON and the need for a safe initial schema that avoids data loss. If the schema is known beforehand it's highly recommended to specify the schema, this avoids potential data loss and can allow you to optimize the loading of your data. By providing a schema, you're able to avoid the schema inference and optimize the whole process.

For further reading on Auto Loader and schema inference, refer to the official Databricks documentation:

[Auto Loader Schema Inference and Evolution](#)  
[Auto Loader](#)

### Question: 124

Deep Dumps

Which statement regarding the relationship between Silver tables and Bronze tables is always true?

- A. Silver tables contain a less refined, less clean view of data than Bronze data.
- B. Silver tables contain aggregates while Bronze data is unaggregated.
- C. Silver tables contain more data than Bronze tables.
- D. Silver tables contain less data than Bronze tables.

**Answer: B**

#### Explanation:

The correct answer, stating that Silver tables contain aggregates while Bronze tables are unaggregated, is rooted in the Medallion Architecture employed within Databricks. This architecture provides a structured approach to data lake management, classifying data into three distinct layers: Bronze, Silver, and Gold.

Bronze tables, also known as raw data tables, ingest data directly from source systems without significant transformation. They maintain the data's original format and granularity, acting as an immutable record of the data's journey into the lake. Therefore, Bronze tables contain unaggregated data.

Silver tables, in contrast, represent a refined and enriched version of the data found in the Bronze layer. They are often created through initial transformations, cleaning, and standardization processes. A common transformation applied at the Silver layer is aggregation, allowing for summaries and roll-ups of the raw data, hence storing aggregated data. Data deduplication, data cleansing, and standardization also occur at this stage.

Gold tables then utilize Silver tables to create data products designed for business intelligence (BI), reporting, and analytics. These will provide further transformation to the data.

Options A, C, and D are incorrect because they do not reflect the purpose or standard usage of the Medallion Architecture. Silver tables are more refined than Bronze tables (contradicting A). Silver tables generally contain less granular data than Bronze tables (contradicting C). Lastly, raw data will generally be more than

what has been aggregated, meaning D is also incorrect. The key distinction lies in the data transformation phase and the aggregation that occurs when moving from raw (Bronze) to a refined stage (Silver).

Further Research:

**Databricks Medallion Architecture:** <https://www.databricks.com/glossary/medallion-architecture>

**Delta Lake Documentation:** Consult the official Delta Lake documentation, as it's the underpinning technology for the Medallion Architecture implementation in Databricks.

### Question: 125

Deep Dumps

Which query is performing a streaming hop from raw data to a Bronze table?

- A.
- ```
(spark.table("sales")
  .groupBy("store")
  .agg(sum("sales"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("newSales")
)
```
- B.
- ```
(spark.read.load(rawSalesLocation)
 .write
 .mode("append")
 .table("newSales")
)
```
- C.
- ```
(spark.table("sales")
  .withColumn("avgPrice", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("newSales")
)
```
- D.

```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

Answer: D

Explanation:

```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

Question: 126

Deep Dumps

A dataset has been defined using Delta Live Tables and includes an expectations clause:

CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW

What is the expected behavior when a batch of data containing data that violates these constraints is processed?

- A. Records that violate the expectation cause the job to fail.
- B. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.
- C. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.
- D. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.

Answer: C

Explanation:

The correct answer is C: Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.

Here's a detailed justification:

Delta Live Tables (DLT) is a declarative framework for building reliable, maintainable, and testable data pipelines. Expectations in DLT allow you to define data quality constraints. The ON VIOLATION clause specifies what should happen when an expectation is violated. In this case, the expectation CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW dictates that rows where the

timestamp is not greater than '2020-01-01' should be dropped from the output table.

When a DLT pipeline processes data, it evaluates these expectations. If a record violates the `valid_timestamp` constraint, the `DROP ROW` action is triggered, and the row is excluded from the target dataset. Furthermore, DLT tracks data quality metrics and expectation results. Rows dropped due to failed expectations are recorded as invalid data, and information about these violations is captured in the DLT event log. This event log provides valuable insights into data quality issues and helps with monitoring and debugging the pipeline. The event log allows you to understand the extent of data quality issues and implement corrective actions. Options A, B, and D are incorrect because they do not accurately describe the behavior of the `DROP ROW` action in DLT expectations. Option A is incorrect because the job does not fail; instead, violating records are dropped. Options B and D are wrong because the violating rows are dropped, not added to the target dataset.

Authoritative Links:

Delta Live Tables Documentation: <https://docs.databricks.com/delta-live-tables/index.html>

Expectations in Delta Live Tables: <https://docs.databricks.com/delta-live-tables/expectations.html>

Question: 127

Deep Dumps

A data engineer has a Job with multiple tasks that runs nightly. Each of the tasks runs slowly because the clusters take a long time to start.

Which action can the data engineer perform to improve the start up time for the clusters used for the Job?

- A.They can use endpoints available in Databricks SQL
- B.They can use jobs clusters instead of all-purpose clusters
- C.They can configure the clusters to autoscale for larger data sizes
- D.They can use clusters that are from a cluster pool

Answer: D

Explanation:

The correct answer is **D. They can use clusters that are from a cluster pool.**

Here's a detailed justification:

The problem describes slow task execution due to long cluster startup times. Cluster startup time is a known bottleneck in Databricks, especially for frequent or short-lived jobs. Cluster pools address this issue directly by pre-allocating and maintaining a pool of idle instances. When a job requests a cluster, it can be provisioned from the pool much faster than creating a new cluster from scratch. This is because the instances are already up and running, bypassing the typical provisioning process (VM allocation, software installation, configuration).

Option A is incorrect because Databricks SQL endpoints are designed for SQL workloads, not general-purpose data engineering tasks within a Databricks Job.

Option B is incorrect because both job clusters and all-purpose clusters can benefit from cluster pools. Switching to job clusters alone won't inherently improve cluster startup time if new clusters are still being created for each job run. While job clusters are often a good practice for managed execution, they don't directly address the startup delay.

Option C is incorrect because autoscaling affects the runtime scaling of a cluster based on workload, not its initial startup time. Autoscaling adjusts the number of instances after the cluster is already running.

Cluster pools drastically reduce the time it takes to acquire a cluster because the instances are already

available. The key benefit is reduced latency from cold starts. For nightly jobs, this can significantly decrease overall job duration and free up resources. Configuring clusters to draw from a cluster pool ensures that instances are readily available, leading to faster job startup times. The pool maintains a set of idle instances ready to be assigned to jobs, thereby avoiding the lengthy cluster creation process. The job is then scheduled on this quickly provisioned cluster and executed.

For further research, refer to the official Databricks documentation on cluster pools:

[Databricks Cluster Pools](#)

Question: 128

Deep Dumps

A data engineer has a single-task Job that runs each morning before they begin working. After identifying an upstream data issue, they need to set up another task to run a new notebook prior to the original task.

Which approach can the data engineer use to set up the new task?

- A.They can clone the existing task in the existing Job and update it to run the new notebook.
- B.They can create a new task in the existing Job and then add it as a dependency of the original task.
- C.They can create a new task in the existing Job and then add the original task as a dependency of the new task.
- D.They can create a new job from scratch and add both tasks to run concurrently.

Answer: B

Explanation:

The correct answer is B: They can create a new task in the existing Job and then add it as a dependency of the original task. Here's why:

The goal is to run a new notebook before the original task. This implies a sequential execution order within the existing daily job. Modifying the existing job is preferable to creating a new job, as it maintains the daily schedule and avoids unnecessary complexity.

Option A is incorrect because cloning the existing task and updating it would create a second, independent task that doesn't address the dependency requirement. The new notebook needs to run before the original one.

Option C correctly creates a new task in the existing job. However, it incorrectly makes the original task dependent on the new task. This would cause the original task to run after the new task, which is the reverse of the desired outcome.

Option D is inefficient. Creating a new job from scratch duplicates scheduling and job management unnecessarily. The objective is to integrate the new notebook into the existing workflow seamlessly, not to create a completely separate process. Running the tasks concurrently would also defeat the purpose of ensuring the new notebook runs before the original. Concurrent execution would introduce race conditions, which should be avoided when data dependencies exist.

By creating a new task running the new notebook within the existing job and designating it as a dependency of the original task, the job execution engine (Databricks in this case) will ensure the new task completes successfully before initiating the original task. This achieves the objective of running the data issue resolution notebook first, followed by the regularly scheduled process. This approach leverages the existing job's scheduling and environment, promoting code reuse and simplifying maintenance. The dependency relationship guarantees the necessary pre-processing completes before the original task begins.

Further research on Databricks Jobs and Task Dependencies can be found at:

Question: 129

Deep Dumps

A single Job runs two notebooks as two separate tasks. A data engineer has noticed that one of the notebooks is running slowly in the Job's current run. The data engineer asks a tech lead for help in identifying why this might be the case.

Which approach can the tech lead use to identify why the notebook is running slowly as part of the Job?

- A.They can navigate to the Runs tab in the Jobs UI to immediately review the processing notebook.
- B.They can navigate to the Tasks tab in the Jobs UI and click on the active run to review the processing notebook.
- C.They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.
- D.They can navigate to the Tasks tab in the Jobs UI to immediately review the processing notebook.

Answer: C

Explanation:

The correct answer is **C. They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.**

Here's a detailed justification:

The Databricks Jobs UI is the central place to manage and monitor the execution of jobs. When a job is running, it creates a "run" for that specific execution. To investigate a slow-running notebook within a job, the tech lead needs to access the details of the active, or currently executing, run.

The **Runs tab** provides a history of all job executions, including the active one. By selecting the active run from the Runs tab, the tech lead can drill down into the specific details of that execution.

Once inside the active run's details, Databricks provides granular information about each task within that run. This includes the status of the task (e.g., running, succeeded, failed), start and end times, duration, and most importantly, links to the notebook or task being executed.

Clicking on the notebook link will take the tech lead directly to the specific execution of that notebook within the context of the job run. From there, they can examine cell-level execution times, Spark UI logs (if the notebook is using Spark), and any error messages that might be causing the slowdown. They can also observe the resource utilization (CPU, memory) during the problematic cells' execution to identify potential bottlenecks.

Option A is incorrect because while the Runs tab is the correct starting point, merely reviewing the processing notebook "immediately" might not offer sufficient insight. Examining it within the context of the active run is crucial.

Option B and D are incorrect because the **Tasks tab** generally shows a more aggregate view of the tasks across all runs of the job. While useful for identifying trends or persistent issues, it's not the optimal first step to diagnose a slowdown in a specific active run. The Runs tab allows you to navigate to a specific run and see detailed information about its tasks.

Therefore, navigating to the Runs tab, selecting the active run, and then drilling into the specific notebook execution within that run provides the most direct and detailed information for troubleshooting the slow

notebook. Here are some relevant links for further research:

Databricks Jobs documentation: <https://docs.databricks.com/workflows/jobs/index.html>

Monitor Databricks Jobs: <https://docs.databricks.com/workflows/jobs/monitor-jobs.html>

Databricks UI: <https://docs.databricks.com/getting-started/index.html#databricks-ui>

Question: 130

Deep Dumps

A data analysis team has noticed that their Databricks SQL queries are running too slowly when connected to their always-on SQL endpoint. They claim that this issue is present when many members of the team are running small queries simultaneously. They ask the data engineering team for help. The data engineering team notices that each of the team's queries uses the same SQL endpoint.

Which approach can the data engineering team use to improve the latency of the team's queries?

- A. They can increase the cluster size of the SQL endpoint.
- B. They can increase the maximum bound of the SQL endpoint's scaling range.
- C. They can turn on the Auto Stop feature for the SQL endpoint.
- D. They can turn on the Serverless feature for the SQL endpoint.

Answer: B

Explanation:

The correct answer is **B: They can increase the maximum bound of the SQL endpoint's scaling range.**

Here's a detailed justification:

The problem described is that numerous small queries are running concurrently against a single SQL endpoint, leading to performance bottlenecks and increased latency. This suggests that the existing SQL endpoint is resource-constrained and unable to handle the concurrent workload efficiently.

Option A, increasing the cluster size, could help, but it's a more static approach. While it provides more resources, it doesn't dynamically adjust to varying workloads. The SQL endpoint might be oversized during periods of low activity, wasting resources.

Option B, increasing the maximum bound of the SQL endpoint's scaling range, is the most appropriate solution. SQL endpoints in Databricks can be configured to automatically scale up (and down) the number of clusters based on the workload. By increasing the maximum bound, you allow the endpoint to dynamically allocate more resources (clusters) when a large number of concurrent queries are running, thus improving latency. The endpoint will scale out to meet demand during periods of high concurrency, and then scale back in during periods of low activity, optimizing cost and performance. This dynamic scaling directly addresses the issue of concurrent queries overloading the endpoint.

Option C, turning on Auto Stop, is counterproductive. Auto Stop shuts down the endpoint after a period of inactivity, meaning users would experience cold starts (increased latency) whenever they tried to run a query after the endpoint had been stopped. This would exacerbate the problem, not alleviate it.

Option D, turning on Serverless, could be beneficial but isn't necessarily the best immediate solution. Serverless SQL endpoints automatically manage compute resources, potentially improving efficiency. However, switching to Serverless might involve configuration changes and could introduce unforeseen complexities. Increasing the scaling range is a simpler, less disruptive change to address the existing problem.

Therefore, increasing the maximum bound of the SQL endpoint's scaling range allows it to dynamically adjust resources to handle the concurrent queries effectively, reducing latency and improving the overall

performance of the data analysis team's queries.

Further reading on Databricks SQL endpoints and auto-scaling:

Databricks SQL endpoints: <https://docs.databricks.com/sql/admin/sql-endpoints.html>

Configure SQL endpoint compute: <https://docs.databricks.com/sql/admin/configure-endpoints.html#configure-compute>

Question: 131

Deep Dumps

A data engineer has been using a Databricks SQL dashboard to monitor the cleanliness of the input data to an ELT job. The ELT job has its Databricks SQL query that returns the number of input records containing unexpected NULL values. The data engineer wants their entire team to be notified via a messaging webhook whenever this value reaches 100.

Which approach can the data engineer use to notify their entire team via a messaging webhook whenever the number of NULL values reaches 100?

- A.They can set up an Alert with a custom template.
- B.They can set up an Alert with a new email alert destination.
- C.They can set up an Alert with a new webhook alert destination.
- D.They can set up an Alert with one-time notifications.

Answer: C

Explanation:

C. They can set up an Alert with a new webhook alert destination.

In Databricks SQL, Alerts can be used to monitor query results and notify when specific conditions are met — in this case, when the number of records with unexpected NULL values reaches 100.

Question: 132

Deep Dumps

A data engineer wants to schedule their Databricks SQL dashboard to refresh once per day, but they only want the associated SQL endpoint to be running when it is necessary.

Which approach can the data engineer use to minimize the total running time of the SQL endpoint used in the refresh schedule of their dashboard?

- A.They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.
- B.They can set up the dashboard's SQL endpoint to be serverless.
- C.They can turn on the Auto Stop feature for the SQL endpoint.
- D.They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.

Answer: C

Explanation:

The correct answer is **C: They can turn on the Auto Stop feature for the SQL endpoint.** Here's a detailed justification:

The core requirement is to minimize the SQL endpoint's running time when it's only needed for scheduled dashboard refreshes. The Auto Stop feature is specifically designed to automatically shut down an SQL

endpoint after a period of inactivity. When a dashboard refresh schedule triggers a query, the endpoint will automatically start up, execute the query, and refresh the dashboard. Once the refresh is complete and no further queries are being executed, the Auto Stop mechanism will kick in after the configured inactivity period and shut down the endpoint. This ensures that the endpoint isn't needlessly running when it's not actively serving requests.

Options A and D are related to query performance and SQL endpoint selection, which may have an impact on how long a query runs, but they don't directly address the problem of when the endpoint is running. Matching or mismatching SQL endpoints among queries might impact caching or query compilation, but it doesn't manage the endpoint's uptime.

Option B, serverless SQL endpoints, could potentially minimize costs by only charging for query execution time, but this is more about cost optimization than minimizing running time. Even with a serverless setup, the dashboard could trigger the queries at unexpected times leading to endpoint uptime longer than wanted. The Auto Stop feature is a more direct and controllable way to minimize the running time by ensuring the SQL endpoint is running only during the refresh activity and for a short period after. It provides a clear mechanism to govern uptime to the activity.

Therefore, enabling Auto Stop provides a direct and effective method to minimize the total running time of the SQL endpoint by automatically shutting it down when not in use, directly addressing the data engineer's requirement.

Further Reading:

Databricks SQL Endpoint Auto Stop: <https://docs.databricks.com/sql/admin/sql-endpoints.html#configure-an-sql-endpoint>

Question: 133

Deep Dumps

An engineering manager wants to monitor the performance of a recent project using a Databricks SQL query. For the first week following the project's release, the manager wants the query results to be updated every minute. However, the manager is concerned that the compute resources used for the query will be left running and cost the organization a lot of money beyond the first week of the project's release.

Which approach can the engineering team use to ensure the query does not cost the organization any money beyond the first week of the project's release?

- A.They can set a limit to the number of DBUs that are consumed by the SQL Endpoint.
- B.They can set the query's refresh schedule to end after a certain number of refreshes.
- C.They can set the query's refresh schedule to end on a certain date in the query scheduler.
- D.They can set a limit to the number of individuals that are able to manage the query's refresh schedule.

Answer: C

Explanation:

The correct answer is C, "They can set the query's refresh schedule to end on a certain date in the query scheduler." This is because Databricks SQL query scheduling allows for precise control over when a query runs. Setting an end date ensures that the query will automatically stop refreshing after that date, thereby preventing unnecessary compute resource usage and associated costs beyond the intended monitoring period. Options A, B, and D are incorrect because they do not directly stop the query from running after a specific date. Limiting DBUs (Option A) might reduce cost but doesn't guarantee the query will stop running after the first week. Setting a limit to the number of refreshes (Option B) may work if the refresh interval and duration are known precisely, but an end date is more straightforward and reliable. Limiting user access (Option D) does not affect the query's runtime or cost. Therefore, using the query scheduler to set a specific

end date for the refresh schedule is the most direct and effective way to prevent unnecessary costs after the first week.

Further research:

[Databricks SQL Query Scheduling](#)

Question: 134

Deep Dumps

A new data engineering team has been assigned to work on a project. The team will need access to database customers in order to see what tables already exist. The team has its own group team.

Which command can be used to grant the necessary permission on the entire database to the new team?

- A. GRANT VIEW ON CATALOG customers TO team;
- B. GRANT CREATE ON DATABASE customers TO team;
- C. GRANT USAGE ON CATALOG team TO customers;
- D. GRANT USAGE ON DATABASE customers TO team;

Answer: D

Explanation:

D. GRANT USAGE ON DATABASE customers TO team;

When granting permissions in a data lake or database system like AWS Lake Formation, Databricks Unity Catalog, or BigQuery, certain permissions are needed just to see or browse tables in a database (without necessarily accessing the data).

Question: 135

Deep Dumps

A new data engineering team team has been assigned to an ELT project. The new data engineering team will need full privileges on the table sales to fully manage the project.

Which command can be used to grant full permissions on the database to the new data engineering team?

- A. GRANT ALL PRIVILEGES ON TABLE sales TO team;
- B. GRANT SELECT CREATE MODIFY ON TABLE sales TO team;
- C. GRANT SELECT ON TABLE sales TO team;
- D. GRANT ALL PRIVILEGES ON TABLE team TO sales;

Answer: A

Explanation:

The correct answer is A: GRANT ALL PRIVILEGES ON TABLE sales TO team; because it directly addresses the requirement of granting full privileges on the sales table to the specified team.

Here's a detailed justification:

1. **Problem Statement Alignment:** The question explicitly states that the data engineering team requires "full privileges" on the sales table. Therefore, the chosen command must grant all available permissions.

2. **GRANT ALL PRIVILEGES Clause:** The GRANT ALL PRIVILEGES clause is the standard SQL construct for granting all possible permissions on a database object. This encompasses actions like selecting data, inserting new records, updating existing data, deleting data, altering the table structure, and potentially even granting permissions to others.
3. **ON TABLE sales Clause:** This part of the command specifies the target object for which privileges are being granted, which aligns with the requirement to grant permissions on the sales table.
4. **TO team Clause:** This specifies the recipient of the granted privileges. This ensures the data engineering team receives the privileges. Note: In Databricks, "team" would typically refer to a group principal.
5. **Why other options are incorrect:**

B. GRANT SELECT CREATE MODIFY ON TABLE sales TO team; This grants only a subset of privileges. It explicitly allows reading (SELECT), creating (CREATE - likely implying creating a table based on the sales table structure, but this is context dependent) and modifying (MODIFY - likely implies UPDATE and DELETE), but it might miss other important permissions like ALTER (to modify the schema), DROP, or the ability to grant these same permissions to other users (GRANT). Therefore, it is not the complete solution.

C. GRANT SELECT ON TABLE sales TO team; This only grants read access. It is insufficient for the team's stated requirement of managing the project.

D. GRANT ALL PRIVILEGES ON TABLE team TO sales; This reverses the subject and object. It attempts to give the table sales full privileges on a table or object named team, which is semantically incorrect and doesn't meet the project's requirements. Databases generally don't allow tables to hold privileges.

6. **Database Security Best Practices:** While GRANT ALL PRIVILEGES might seem like a straightforward solution, it's crucial to note that granting such broad permissions should be done with caution. It's vital to understand precisely what privileges are included and whether they are all genuinely necessary. In some situations, granting specific privileges using GRANT SELECT, INSERT, UPDATE, DELETE ... may be preferable for enhanced security.

Authoritative Resources:

Databricks SQL Data Control: <https://docs.databricks.com/en/sql/language-manual/security.html>

SQL GRANT Statement: (General SQL, concepts apply): <https://www.sqltutorial.org/sql-grant/>

Question: 136

Deep Dumps

Differentiate between all-purpose clusters and jobs clusters.

A data engineering team has created a python notebook to load data from cloud storage, this job has been tested and now needs to be scheduled in production.

Which would be the best cluster to be used in this case?

- A.All purpose cluster
- B.Any Unity Catalog-enabled cluster
- C.Jobs Cluster
- D.Serverless SQL warehouse

Answer: C

Explanation:

The correct answer is C: Jobs Cluster. Here's a detailed justification:

All-Purpose clusters are designed for interactive development, collaborative data exploration, and ad-hoc queries. They are typically long-running and shared among multiple users. In contrast, Jobs clusters are designed for running automated, non-interactive workloads such as ETL pipelines, batch processing, and scheduled tasks. Jobs clusters are ephemeral, meaning they are automatically created when a job starts and terminated when the job completes. This ensures efficient resource utilization and cost optimization.

Since the data engineering team has a tested Python notebook ready for production scheduling, a Jobs cluster is the most appropriate choice. Using an All-Purpose cluster would be wasteful, as it would remain active even when the job is not running, incurring unnecessary costs. Jobs clusters spin up quickly and are specifically designed to execute defined tasks.

A Unity Catalog-enabled cluster (option B) refers to a cluster configured to use Databricks Unity Catalog for data governance and security. While Unity Catalog is a valuable feature, it's not the primary differentiator for choosing between cluster types. The crucial distinction is the cluster's purpose: interactive development (All-Purpose) vs. automated execution (Jobs). Serverless SQL warehouses (option D) are specialized for SQL-based workloads and would not be suitable for running a Python notebook.

Jobs clusters offer benefits like automatic scaling, error handling, and retry mechanisms, making them ideal for production workloads. They provide isolation, ensuring that the job's resources are dedicated and not impacted by other activities. This improves stability and predictability of the scheduled task. Because they are automatically terminated after the job completes, Jobs clusters minimize costs, which is critical in production environments.

Furthermore, Databricks Jobs API and UI are designed to work seamlessly with Jobs clusters, allowing for easy scheduling, monitoring, and management of automated workflows.

For further reading, refer to the official Databricks documentation on Clusters and Jobs:

Databricks Clusters: <https://docs.databricks.com/clusters/index.html>

Databricks Jobs: <https://docs.databricks.com/workflows/jobs/index.html>

Question: 137

Deep Dumps

Identify how the count_if function and the count where x is null can be used

Consider a table random_values with below data.

What would be the output of below query?

```
select count_if(col > 1) as count_a, count(*) as count_b, count(col1) as count_c from random_values col1
```

0

1

2

NULL -

2

3

A.3 6 5

B.4 6 5

C.3 6 6

D.4 6 6

Answer: A

Explanation:

Let's break down how each part of the SQL query `select count_if(col1 > 1) as count_a, count(*) as count_b, count(col1) as count_c from random_values` operates on the `random_values` table.

The `random_values` table has the following data:

col1
0
1
2
NULL
-2
3

count_if(col1 > 1) as count_a: This function counts the number of rows where the condition `col1 > 1` is true. `count_if` (or similar conditional counting functions) are common in data warehousing tools. We can evaluate row by row in the `col1` column as follows:

Row 1: `0 > 1` is false.

Row 2: `1 > 1` is false.

Row 3: `2 > 1` is true.

Row 4: `NULL > 1` is false. SQL NULL values are treated as unknown, and comparisons with NULL generally evaluate to false.

Row 5: `-2 > 1` is false.

Row 6: `3 > 1` is true.

Therefore, `count_a` will be 3 (rows 3 and 6).

count() as count_b: The `count()` function counts all rows in the table, including rows where `col1` is NULL. This is a fundamental SQL aggregate function. The table has 6 rows, so `count_b` will be 6.

count(col1) as count_c: The `count(col1)` function counts the number of rows where `col1` is not NULL. Crucially, this function ignores NULL values. It's another essential SQL aggregate function. In this case, there are 5 non-NULL values in `col1` (0, 1, 2, -2, 3). Therefore, `count_c` will be 5.

Combining the results, `count_a = 3`, `count_b = 6`, and `count_c = 5`. This corresponds to option A.

Authoritative Links:

SQL COUNT() function: https://www.w3schools.com/sql/sql_count.asp

SQL NULL values: https://www.w3schools.com/sql/sql_null_values.asp (This is an important concept for understanding how SQL handles missing or unknown data, and is essential for understanding why comparison with NULL is neither true nor false, but NULL).

Question: 138

Deep Dumps

Which two components function in the DB platform architecture's control plane? (Choose two.)

- A.Virtual Machines
- B.Compute Orchestration
- C.Serverless Compute
- D.Compute
- E.Unity Catalog

Answer: BE

Explanation:

The correct answer identifies the two components of the Databricks platform architecture that reside within the control plane: Compute Orchestration and Unity Catalog.

The control plane is responsible for managing and orchestrating the various resources within the Databricks environment. Compute orchestration falls squarely into this category, as it handles the creation, management, and scaling of compute resources (clusters) needed for data processing. It dictates how and where workloads are executed. Unity Catalog, a unified governance solution for data and AI, also lives in the control plane. It defines and enforces access policies, manages data lineage, and facilitates data discovery across different workspaces and compute clusters, effectively controlling how data is accessed and utilized.

Virtual Machines and Compute itself (without the "Orchestration" aspect) are part of the data plane. The data plane is where the actual data processing and computations occur. The virtual machines serve as the infrastructure where the Databricks Runtime and user code execute, performing the computations orchestrated by the control plane. 'Compute' alone is too general; the orchestration of compute is what distinctly positions it within the control plane. Serverless Compute, while a possible compute deployment model on Databricks, represents an execution environment, making it also part of the data plane rather than the control plane directly. The control plane sets the stage for how and what computations will run, while the data plane executes them.

For further reading, consult Databricks' official documentation:

Databricks Architecture: <https://www.databricks.com/glossary/databricks-architecture>

Unity Catalog: <https://www.databricks.com/product/unity-catalog>

Question: 139

Deep Dumps

In a healthcare provider organization using Delta Lake to store electronic health records (EHRs), a data analyst needs to analyze a snapshot of the patient_records table from two weeks ago before some recent data corrections were applied.

What approach should the Data Engineer take to allow the analyst to query that specific prior version?

- A.Truncate the table to remove all data, then reload the data from two weeks ago into the truncated table for the analyst to query.
- B.Identify the version number corresponding to two weeks ago from the Delta transaction log, share that version number with the analyst to query using VERSION AS OF syntax, or export that version to a new Delta table for the analyst to query.
- C.Restore the table to the version from two weeks ago using the RESTORE command, and have the analyst query the restored table.
- D.Use the VACUUM command to remove all versions of the table older than two weeks, then the analyst can query the remaining version.

Answer: B

Explanation:

The correct approach is **B: Identify the version number corresponding to two weeks ago from the Delta transaction log, share that version number with the analyst to query using VERSION AS OF syntax, or export that version to a new Delta table for the analyst to query.**

Here's why:

Delta Lake's time travel feature allows querying previous versions of a table. Option B leverages this capability directly. The Delta transaction log maintains a history of all changes made to the table. By examining this log, you can determine the version number that represents the table's state two weeks prior. The VERSION AS OF syntax allows querying the table as it existed at that specific version. Alternatively, you could create a new table with the historical data which is more suitable if the analyst wants to make persistent modification without affecting the source data.

Option A is incorrect because truncating and reloading destroys the current state of the table and could lead to data loss or inconsistency. Furthermore, finding the exact data from two weeks ago for reloading might be complex and time-consuming, especially when you have the time travel features.

Option C, using the RESTORE command, modifies the current table, making it represent the state from two weeks ago. While it allows the analyst to query the past state, it overwrites the current state. The analyst will no longer be able to see the updated data after the restore. Moreover, other processes depending on the current state would be affected.

Option D is incorrect because the VACUUM command removes old versions, which is the opposite of what's needed to analyze a past state. It's used for optimizing storage by deleting outdated files after a retention period. Using vacuum would prevent you from running queries for previous versions, and it would be destructive.

In summary, Option B is the most appropriate because it directly utilizes Delta Lake's time travel feature, preserving the current table state while allowing the analyst to access and analyze the desired historical data. Further Reading:

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake Restore: <https://docs.databricks.com/sql/language-manual/delta-restore.html>

Delta Lake Vacuum: <https://docs.databricks.com/delta/optimizations/optimize-vacuum.html>

Question: 140

Deep Dumps

What can be used to simplify and unify siloed data architectures that are specialized for specific use cases?

- A. Delta Lake
- B. Data lake
- C. Data warehouse
- D. Data lakehouse

Answer: D

Explanation:

The best answer is D, the Data Lakehouse, because it directly addresses the problem of siloed data architectures. Traditional data lakes lack the transactional consistency and data governance capabilities needed for reliable analytics and AI. Conversely, data warehouses offer strong governance and consistency but struggle with the variety and velocity of modern data.

A data lakehouse combines the best aspects of both. It provides the cost-effectiveness and scalability of a data lake for storing all types of data (structured, semi-structured, and unstructured) while adding the data

management and transaction support features of a data warehouse. This allows organizations to use a single, unified platform for various use cases, including business intelligence, reporting, advanced analytics, and machine learning, thereby breaking down silos.

Delta Lake (A) is an open-source storage layer that brings ACID transactions to Apache Spark and big data workloads. While Delta Lake is a key technology within a data lakehouse and enhances its capabilities, it is not the complete architecture itself. Similarly, a Data Lake (B) is a storage repository holding a vast amount of raw data in its native format until it is needed, but lacks the necessary data management features. A Data Warehouse (C) is optimized for structured data and analytical workloads, often struggling to handle diverse data types efficiently.

Therefore, the data lakehouse architecture is the most suitable solution for unifying specialized and isolated data architectures, offering a single source of truth for all data-related activities. A data lakehouse provides capabilities such as schema enforcement, version control, and audit trails, ensuring data quality and compliance across all use cases. This integration reduces redundancy, improves data governance, and simplifies data access for various teams. It's the architecture designed specifically to converge data engineering, data science, and business analytics.

Databricks - What is a Data Lakehouse? <https://www.databricks.com/glossary/data-lakehouse>

Microsoft - Data Lakehouse: <https://learn.microsoft.com/en-us/azure/architecture/patterns/data-lakehouse>

Question: 141

Deep Dumps

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.readStream
  .table("sales")
  .withColumn("avg_price", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .trigger(availableNow=True)
  .table("new_sales")
)
```

The data engineer only wants the query to process all of the available data in as many batches as required.

Which line of code should the data engineer use to fill in the blank?

- A.trigger(availableNow=True)
- B.trigger(processingTime= "once")
- C.trigger(continuous= "once")
- D.trigger(once=True)

Answer: A

Explanation:

trigger(availableNow=True).

Question: 142

Deep Dumps

Data engineer and data analysts are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.

Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

- A. The pipeline can have different notebook sources in SQL & Python
- B. The pipeline will need to be written entirely in SQL
- C. The pipeline will need to use a batch source in place of a streaming source
- D. The pipeline will need to be written entirely in Python

Answer: C

Explanation:

Let's analyze why option C is the correct answer and why the others are not, focusing on the constraints and capabilities of Delta Live Tables (DLT).

Option A: Incorrect. Delta Live Tables pipelines can combine notebooks written in both SQL and Python. This is a key advantage of DLT, allowing teams to leverage the strengths of different languages for different transformations. You can define data flows that involve Python transformations in one notebook feeding data to SQL transformations in another. There's no limitation forcing a single language.

Option B: Incorrect. Similar to A, Delta Live Tables does not force you to use only SQL. Python is fully supported as a language for defining data transformations within a DLT pipeline. This flexibility is crucial for handling complex transformations or leveraging existing Python code.

Option C: Correct. Delta Live Tables primarily works with idempotent batch processing, even when the underlying data source is a streaming source. While DLT can ingest streaming data, it processes it in micro-batches. This means the streaming source will be processed in intervals, creating batches of data that are then ingested and transformed. You cannot directly define a continuous streaming pipeline within DLT in the same way you would with structured streaming. You must define a table (or view) as the input, not a streaming source. The DLT engine manages the ingestion of the stream into that table in a performant and reliable manner. The key takeaway is that the streaming source becomes a source table, which DLT then processes in batches.

Option D: Incorrect. Delta Live Tables support both SQL and Python for defining pipeline logic. The language choice depends on the specific transformation and the expertise of the data engineer or analyst.

Justification for C in Detail:

Delta Live Tables' architecture relies on declarative pipeline definition and automatic infrastructure management. This means you define the what (data transformations) and DLT figures out the how (resource allocation, fault tolerance, etc.). To enable reliable and idempotent updates, DLT breaks down streaming data into manageable batches. While it hides much of the complexity, this batch-oriented approach fundamentally changes how you interact with the streaming data within the pipeline's transformation logic. Essentially, even when feeding in streaming data, your DLT transformations treat the data as a series of micro-batches/tables. The DLT framework itself uses structured streaming behind the scenes, but the pipeline's transformation code operates on tables/batches.

The streaming input needs to be changed to a streaming table in the DLT pipeline. The pipeline will then consume micro-batches from that table.

Authoritative Links for further research:

Delta Live Tables documentation: <https://docs.databricks.com/delta-live-tables/index.html>

Databricks Blog - Delta Live Tables: <https://www.databricks.com/blog/2021/04/29/simplify-data-engineering-with-delta-live-tables.html>

Question: 143

Deep Dumps

Identify a scenario to use an external table.

A Data Engineer needs to create a parquet bronze table and wants to ensure that it gets stored in a specific path in an external location.

Which table can be created in this scenario?

- A. An external table where the location is pointing to specific path in external location.
- B. An external table where the schema has managed location pointing to specific path in external location.
- C. A managed table where the catalog has managed location pointing to specific path in external location.
- D. A managed table where the location is pointing to specific path in external location.

Answer: A

Explanation:

The correct answer is A because it directly addresses the scenario's requirements for a Data Engineer to create a Parquet bronze table stored in a specific external location. Here's a detailed breakdown:

External Tables: External tables in Databricks (and generally in data warehousing systems like Hive and Spark) are ideal when you need to manage the underlying data files separately from the table definition. The table metadata (schema, table name, etc.) is stored in the metastore (e.g., Hive metastore or Databricks Unity Catalog), but the actual data resides in a location you specify, such as cloud storage (AWS S3, Azure Data Lake Storage Gen2, or Google Cloud Storage).

LOCATION Clause: The LOCATION clause in the CREATE EXTERNAL TABLE statement is precisely how you link the table definition to the desired storage path. You explicitly provide the path where the Parquet files should be stored.

Managed Tables (Incorrect): Managed tables (also known as internal tables) are controlled entirely by Databricks. When you create a managed table, Databricks manages both the metadata and the data. The data files for managed tables are typically stored in a default location managed by Databricks, and you generally don't have direct control over the physical storage path. Therefore, options C and D, which suggest managed tables, are not appropriate for this scenario.

Schema's Managed Location (Incorrect): Option B talks about the "schema has managed location." While schemas can have a default location, this doesn't automatically guarantee that an external table's data will be placed there unless explicitly specified in the CREATE EXTERNAL TABLE statement with the LOCATION clause. The crucial point is that the question mandates a specific path to be controlled by the user.

Controlling Data Location: By using an external table with a specified LOCATION, the Data Engineer retains control over where the bronze table data is stored. This is important for data governance, cost management, and integration with other systems that may rely on specific file paths. Bronze tables are typically the initial landing zone for raw data, and controlling their location can be crucial for downstream processing and data lineage.

In summary, using an external table and specifying its location through the LOCATION clause aligns perfectly with the need to store the Parquet bronze table in a precise location, fulfilling the requirements of the

problem.

Authoritative Links:

Databricks Documentation on Tables: <https://docs.databricks.com/en/tables/index.html>

CREATE TABLE SQL Reference (Databricks): <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table.html>

Question: 144

Deep Dumps

Identify the impact of ON VIOLATION DROP ROW and ON VIOLATION FAIL UPDATE for a constraint violation.

A data engineer has created an ETL pipeline using Delta Live table to manage their company travel reimbursement detail, they want to ensure that the if the location details has not been provided by the employee, the pipeline needs to be terminated.

How can the scenario be implemented?

- A.CONSTRAINT valid_location EXPECT (location = NULL)
- B.CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL UPDATE
- C.CONSTRAINT valid_location EXPECT (location != NULL) ON DROP ROW
- D.CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL

Answer: D

Explanation:

Let's break down why option D is the correct choice and why the others aren't. The core objective is to terminate the pipeline when location is NULL. Delta Live Tables (DLT) uses constraints to enforce data quality. The EXPECT clause defines the condition that must be true. The ON VIOLATION clause dictates what happens when the EXPECT condition is not met.

Option A: CONSTRAINT valid_location EXPECT (location = NULL) - This constraint expects location to be NULL. If location is NULL, the pipeline succeeds, which is the opposite of the requirement. This is logically incorrect.

Option B: CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL UPDATE - This constraint correctly expects location to not be NULL. However, ON VIOLATION FAIL UPDATE is not a valid action in DLT. FAIL UPDATE applies to Spark SQL syntax in Databricks, not DLT constraints.

Option C: CONSTRAINT valid_location EXPECT (location != NULL) ON DROP ROW - This constraint correctly expects location to not be NULL. The ON DROP ROW clause means that if location is NULL, the row is simply dropped from the table. While this maintains data quality by removing bad rows, it doesn't terminate the pipeline, which is the key requirement. The pipeline continues processing.

Option D: CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL - This constraint correctly expects location to not be NULL. The ON VIOLATION FAIL clause ensures that if location is NULL, the entire DLT pipeline fails. This immediately stops processing, fulfilling the stated requirement. The FAIL action immediately stops the pipeline execution upon violation.

Therefore, option D is the only one that correctly validates that the location field is not NULL and, crucially, stops the pipeline when a NULL value is encountered. The ON VIOLATION FAIL action makes it the appropriate mechanism for immediately halting the pipeline. DLT leverages this for data quality enforcement and pipeline management.

Authoritative Links for Further Research:

Delta Live Tables Constraints: <https://docs.databricks.com/delta-live-tables/expectations.html>

Delta Live Tables Syntax: <https://docs.databricks.com/delta-live-tables/delta-live-tables-syntax.html>

Question: 145

Deep Dumps

Which two conditions are applicable for governance in Databricks Unity Catalog? (Choose two.)

- A. You can have more than 1 metastore within a databricks account console but only 1 per region.
- B. Both catalog and schema must have a managed location in Unity Catalog provided metastore is not associated with a location
- C. You can have multiple catalogs within metastore and 1 catalog can be associated with multiple metastore
- D. If catalog is not associated with location, it's mandatory to associate schema with managed locations
- E. If metastore is not associated with location, it's mandatory to associate catalog with managed locations

Answer: AE

Explanation:

Here's a detailed justification for why options A and E are the correct answers, and why the others are incorrect, concerning Databricks Unity Catalog governance:

A. You can have more than 1 metastore within a Databricks account console but only 1 per region.

This statement is correct and fundamental to Unity Catalog's architecture. A Databricks account can span multiple regions. To manage data access across these regions in a consistent manner, Unity Catalog allows one metastore per region. This enables regional data governance while maintaining a single pane of glass view across the entire organization's data assets. Having more than one metastore in the same region is not supported, as it would lead to inconsistencies and management overhead.

Supporting Concept: Regional separation, Single pane of glass view.

Authoritative Link: Databricks documentation on Unity Catalog metastores: <https://docs.databricks.com/data-governance/unity-catalog/create-metastore.html>

E. If a metastore is not associated with a location, it's mandatory to associate a catalog with managed locations.

This statement accurately reflects the requirements for managed tables and data ownership within Unity Catalog. A metastore can be configured without a root storage location. If this is the case, individual catalogs must then be associated with managed locations. This is because Unity Catalog needs a location to store the managed tables created within these catalogs. Without either a metastore or a catalog-level managed location, managed tables would have nowhere to store their underlying data files. By setting a location on the catalog, you are giving unity catalog a location to manage data storage in a secure and governed manner.

Supporting Concept: Data Locality, Data Governance, Managed Storage.

Authoritative Link: Databricks documentation on managed locations in Unity Catalog: <https://docs.databricks.com/data-governance/unity-catalog/managed-storage.html>

Why the other options are incorrect:

B. Both catalog and schema must have a managed location in Unity Catalog provided metastore is not associated with a location: While catalog must have a managed location if the metastore lacks one, it is **not mandatory** for schemas to have managed locations. Schemas inherit the managed location from their parent catalog (or the metastore, if the catalog doesn't have one).

C. You can have multiple catalogs within a metastore and 1 catalog can be associated with multiple metastores:

A catalog can exist only within a single metastore. A catalog cannot span multiple metastores. Unity Catalog is designed for a hierarchical structure: Metastore -> Catalog -> Schema -> Table/View.

D. If catalog is not associated with location, it's mandatory to associate schema with managed locations:

The opposite is true. It's mandatory to have a catalog associated with a managed location (if the metastore does not have one), not a schema. Schemas inherit location from their parent catalog or metastore.

Question: 146

Deep Dumps

A data engineer needs to access the view created by the sales team, using a shared cluster. The data engineer has been provided usage permissions on the catalog and schema. In order to access the view created by sales team.

What are the minimum permissions the data engineer would require in addition?

- A.Needs SELECT permission on the VIEW and the underlying TABLE.
- B.Needs SELECT permission only on the VIEW
- C.Needs ALL PRIVILEGES on the VIEW
- D.Needs ALL PRIVILEGES at the SCHEMA level

Answer: A

Explanation:

The correct answer is A: Needs SELECT permission on the VIEW and the underlying TABLE. Here's a detailed justification:

When a user accesses a view in Databricks, the system needs to verify that the user has the necessary permissions not only to access the view itself but also to access the underlying tables or views that the view is based on. This is because a view is essentially a stored query, and executing that query requires reading data from the underlying objects. Without permission on the underlying tables, the data engineer would effectively be bypassing access controls and potentially viewing data they are not authorized to see. The catalog and schema usage permissions only grant access to the container structure, not the data within.

Databricks uses a hierarchical permission model. While usage permission on the catalog and schema allows the user to list and traverse the objects within, it does not grant read access to data. SELECT permission on the view allows the user to execute the query that defines the view. However, that query needs to read data from tables. If the data engineer only has SELECT permission on the view, but lacks SELECT on the underlying table(s), the query execution will fail, even if the user possesses 'USAGE' permission on the parent catalog and schema.

Consider a view that joins data from two tables. To use the view, the data engineer must have SELECT privileges on the view itself and both tables that are referenced by the view. Option B is incorrect because it misses the permission on the underlying table. Option C and D are unnecessarily broad; "ALL PRIVILEGES" grants significantly more permissions than necessary. "ALL PRIVILEGES" at the SCHEMA level would give potentially unwanted permissions on all objects in that schema. Therefore, granting only the necessary permissions to the view and underlying tables follows the principle of least privilege, a key security best practice. Granting only the least amount of privilege reduces the blast radius of a compromised account.

Authoritative Links for Further Research:

Databricks SQL Privileges and Securable Objects: <https://docs.databricks.com/en/security/access-control/table-acls/object-privileges.html>

Databricks Unity Catalog Privileges: <https://docs.databricks.com/en/data-governance/unity-catalog/permissions.html>

Question: 147**Deep Dumps**

Which method should a Data Engineer apply to ensure Workflows are being triggered on schedule?

- A.Scheduled Workflows require an always-running cluster, which is more expensive but reduces processing latency.
- B.Scheduled Workflows process data as it arrives at configured sources.
- C.Scheduled Workflows can reduce resource consumption and expense since the cluster runs only long enough to execute the pipeline.
- D.Scheduled Workflows run continuously until manually stopped.

Answer: C

Explanation:

The correct answer is C: Scheduled Workflows can reduce resource consumption and expense since the cluster runs only long enough to execute the pipeline.

Here's why:

Scheduled Workflows are designed for periodic execution: Databricks Workflows can be configured to run at specific intervals (e.g., hourly, daily, weekly).

On-demand Cluster Provisioning: When a scheduled workflow is triggered, a Databricks cluster is automatically provisioned (if one isn't already available and configured to be reused).

Efficient Resource Utilization: The cluster runs only for the duration required to execute all the tasks defined in the workflow. Once the workflow completes, the cluster is shut down automatically. This "start-and-stop" behavior minimizes resource usage.

Cost Optimization: By only running the cluster when needed, scheduled workflows significantly reduce the overall cost compared to solutions that require always-on clusters.

Alternative A is incorrect: While always-running clusters can reduce latency, they are not required for scheduled workflows. Scheduled Workflows typically spin up a cluster when needed.

Alternative B is incorrect: Scheduled Workflows are triggered at specific times, not based on data arrival at source systems. Workflows triggered by data arrival are referred to as Delta Live Tables with continuous pipeline processing.

Alternative D is incorrect: Scheduled Workflows do not run continuously. They run once each time the schedule is triggered.

In essence, scheduled workflows offer a cost-effective approach to automating data pipelines by leveraging on-demand compute resources that are only active during pipeline execution. This promotes efficient resource management and reduces overall infrastructure expenses.

For further information, please review Databricks official documentation:

[Databricks Workflows](#)

[Scheduling Databricks Jobs](#)

Question: 148**Deep Dumps**

The Delta transaction log for the 'students' tables is shown using the 'DESCRIBE HISTORY students' command. A Data Engineer needs to query the table as it existed before the UPDATE operation listed in the log.

Which command should the Data Engineer use to achieve this? (Choose two.)

	 version	 timestamp	 operation
1	8	2024-04-22T14:33:31.000	OPTIMIZE
2	7	2024-04-22T14:33:16.000	MERGE
3	6	2024-04-22T14:33:06.000	DELETE
4	5	2024-04-22T14:32:58.000	UPDATE
5	4	2024-04-22T14:32:47.000	WRITE
6	3	2024-04-22T14:32:44.000	WRITE
7	2	2024-04-22T14:32:23.000	WRITE
8	1	2024-04-22T14:32:20.000	WRITE
9	0	2024-04-22T14:31:49.000	CREATE TABLE

A.SELECT * FROM students@v4

B.SELECT * FROM students TIMESTAMP AS OF '2024-04-22T 14:32:47.000+00:00'

C.SELECT * FROM students FROM HISTORY VERSION AS OF 3

D.SELECT * FROM students VERSION AS OF 5

E.SELECT * FROM students TIMESTAMP AS OF '2024-04-22T 14:32:58.000+00:00'

Answer: AB

Explanation:

Reference:

<https://docs.databricks.com/en/delta/history.html>

Question: 149

Deep Dumps

An engineering manager uses a Databricks SQL query to monitor ingestion latency for each data source. The manager checks the results of the query every day, but they are manually rerunning the query each day and waiting for the results.

Which of the following approaches can the manager use to ensure the results of the query are updated each day?

- A.They can schedule the query to refresh every 1 day from the SQL endpoint's page in Databricks SQL.
- B.They can schedule the query to refresh every 12 hours from the SQL endpoint's page in Databricks SQL.
- C.They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.
- D.They can schedule the query to run every 12 hours from the Jobs UI.

Answer: C

Explanation:

The correct answer is C because Databricks SQL offers a built-in scheduling feature directly within the query interface, designed specifically for automatically refreshing query results. This eliminates the need for manual intervention or reliance on external scheduling mechanisms. Options A and B are incorrect as scheduling happens at the query level not the SQL endpoint level. While the query is executed using a SQL endpoint, scheduling is configured within the query's settings. Option D is incorrect because the Jobs UI primarily handles notebook and Python script execution, not the scheduling of Databricks SQL queries. Utilizing the query's built-in scheduling capability streamlines the monitoring process. The engineering manager can simply configure the query to refresh every day, ensuring that the latest ingestion latency data is readily available. This automated approach enhances efficiency and eliminates the risk of human error

associated with manual query execution. The feature can be accessed by opening the desired Databricks SQL query and selecting "Schedule" in the interface. This setup allows for precise control over refresh intervals, catering to the manager's specific monitoring needs. It leverages Databricks' managed service capabilities to automate data refresh, simplifying data engineering tasks and improving operational efficiency. The benefit extends beyond mere automation. It enables consistent and timely monitoring, which is crucial for identifying and addressing data ingestion issues promptly.

Reference: <https://docs.databricks.com/sql/user/queries/index.html#schedule-a-query>

Question: 150

Deep Dumps

A data engineer needs to apply custom logic to string column city in table stores for a specific use case. In order to apply this custom logic at scale, the data engineer wants to create a SQL user-defined function (UDF).

Which of the following code blocks creates this SQL UDF?

A.

```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

B.

```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

C.

```
CREATE FUNCTION combine_nyc(city STRING)
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

D.

```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

Answer: A

Explanation:

```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

Question: 151

Deep Dumps

A data engineer has realized that they made a mistake when making a daily update to a table. They need to use Delta time travel to restore the table to a version that is 3 days old. However, when the data engineer attempts to time travel to the older version, they are unable to restore the data because the data files have been deleted.

Which of the following explains why the data files are no longer present?

- A. The VACUUM command was run on the table
- B. The TIME TRAVEL command was run on the table
- C. The DELETE HISTORY command was run on the table
- D. The OPTIMIZE command was run on the table

Answer: A

Explanation:

The correct answer is A, because the VACUUM command removes files from the Delta Lake table's storage location that are no longer needed by the Delta Lake table and are older than the specified retention interval. By default, this retention interval is 7 days. If the VACUUM command was run with the default retention period (or a shorter one), any data files older than that retention period would be deleted. This would prevent time travel to versions older than the retention period, as the necessary data files would no longer be available.

Option B, TIME TRAVEL, is incorrect. TIME TRAVEL allows you to query or restore to older versions of your Delta table, it does not remove files.

Option C, DELETE HISTORY is not a valid Delta Lake command. Delta Lake offers a RESTORE command, but there is no command called "DELETE HISTORY." Furthermore, Delta Lake does not directly expose the deletion of specific history entries in a way that would affect underlying data files in versions.

Option D, OPTIMIZE, is incorrect. The OPTIMIZE command compacts small files into larger files to improve query performance. While OPTIMIZE can rewrite data files, it does not automatically delete older versions of the data necessary for time travel, unless combined with VACUUM.

Therefore, only running VACUUM with a retention period shorter than 3 days could explain why the data engineer is unable to time travel to a version 3 days old.

Authoritative Links:

Delta Lake VACUUM: <https://docs.databricks.com/delta/vacuum.html>

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake Optimize: <https://docs.databricks.com/delta/optimizations/optimize.html>

Question: 152

Deep Dumps

Which of the following describes the relationship between Bronze tables and raw data?

- A. Bronze tables contain less data than raw data files.
- B. Bronze tables contain more truthful data than raw data.
- C. Bronze tables contain raw data with a schema applied.
- D. Bronze tables contain a less refined view of data than raw data.

Answer: C

Explanation:

The correct answer is C: Bronze tables contain raw data with a schema applied. Here's why:

Bronze tables in a Medallion Architecture (Bronze, Silver, Gold) serve as the entry point for data ingestion. Their primary function is to land raw data from various sources directly into the data lake. The data in a Bronze table is essentially a near-exact copy of the raw data, but with an important addition: a schema. This schema, which can be either explicitly defined or inferred, provides structure and meaning to the data. The schema does not refine or transform the data.

Option A is incorrect because Bronze tables do not contain less data. Typically, the raw data is completely loaded into the bronze table.

Option B is incorrect because the truthfulness of the data isn't changed. Raw data, whether truthful or not, persists as is in the bronze table.

Option D is incorrect because Bronze tables impose initial structure to raw data via a schema.

Cloud computing concepts such as data lakes, ETL pipelines, and schema inference are relevant to understanding this. Data lakes are repositories where data in its native format is stored. Bronze tables fit directly into this concept by providing a landing zone. ETL pipelines use the bronze tables as their first stage, where the raw data is extracted. Finally, schema inference helps in initially defining the bronze table when no explicit schema is available. In essence, Bronze tables are a crucial first step in structuring and preparing data for downstream processing.

Here's the link from Databricks official documentation on Medallion Architecture: <https://www.databricks.com/glossary/medallion-architecture>

Question: 153

Deep Dumps

A data engineer only wants to execute the final block of a Python program if the Python variable `day_of_week` is equal to 1 and the Python variable `review_period` is True.

Which of the following control flow statements should the data engineer use to begin this conditionally executed code block?

- A. `if day_of_week = 1 and review_period:`
- B. `if day_of_week = 1 and review_period = "True":`
- C. `if day_of_week = 1 & review_period: = "True":`
- D. `if day_of_week == 1 and review_period:`

Answer: D

Explanation:

The correct if statement is `if day_of_week == 1 and review_period:`. Let's break down why:

if keyword: The if keyword initiates a conditional statement in Python, allowing a block of code to execute only if a specified condition is true.

day_of_week == 1: This part checks if the variable day_of_week is equal to 1. The double equals sign (==) is the equality operator in Python, used for comparing values. Using a single equals sign (=) would be an assignment, which is invalid within a conditional statement.

and operator: The and operator is a logical operator. It requires both conditions on either side of it to be true for the overall condition to be true.

review_period:: This part checks if the variable review_period is True. In Python, non-empty strings, non-zero numbers, and objects that are not None evaluate to True in a boolean context. Therefore, simply including review_period in the if statement is sufficient to check if its value is considered True.

Option A uses the assignment operator (=) instead of the equality operator (==), which is syntactically incorrect and will raise an error. Options B and C incorrectly compare review_period to the string "True". review_period is already specified as a boolean, so explicitly comparing it against the string "True" would lead to incorrect boolean evaluation. Python treats boolean values distinctly from string representations of "True" or "False". Therefore only option D is the correctly formatted code for this scenario.

Refer to these resources for additional information:

Python if statement: <https://docs.python.org/3/tutorial/controlflow.html>

Python Comparison Operators: https://www.w3schools.com/python/python_operators.asp

Python Boolean Values: https://www.w3schools.com/python/python_booleans.asp

Question: 154

Deep Dumps

Which of the following must be specified when creating a new Delta Live Tables pipeline?

- A. A key-value pair configuration
- B. At least one notebook library to be executed
- C. A path to cloud storage location for the written data
- D. A location of a target database for the written data

Answer: B

Explanation:

The correct answer is B: At least one notebook library to be executed.

Delta Live Tables (DLT) pipelines are declarative data processing pipelines where you define the desired transformations on your data, and DLT manages the execution and infrastructure. At the heart of any DLT pipeline lies the logic that transforms the data. This logic is defined in notebooks or Python files that contain data transformations expressed using the DLT API.

A. A key-value pair configuration, while helpful for tuning and customizing pipelines, is not strictly required. You can have a functional DLT pipeline without specifying advanced configuration options.

C. A path to a cloud storage location for the written data (the target location) is important but can be omitted in initial setup or inferred implicitly during development. While best practice is to define your target location explicitly for production, the minimum requirement for creating a pipeline does not depend on it. DLT will utilize a default managed location if a target location is omitted.

D. Similarly, a location of a target database for the written data is also not strictly mandatory. DLT can

function without specifying a target database in initial pipeline definition.

The fundamental requirement for a DLT pipeline is the definition of the data transformations, which are provided by at least one notebook library (or a Python file). Without any transformation logic, the DLT pipeline would essentially be empty and serve no purpose. Therefore, it is the bare minimum component needed to define a DLT pipeline.

For additional research, refer to the official Databricks documentation on Delta Live Tables:

Databricks Delta Live Tables Documentation: <https://docs.databricks.com/delta-live-tables/index.html>

Question: 155

Deep Dumps

In which of the following scenarios should a data engineer select a Task in the Depends On field of a new Databricks Job Task?

- A. When another task needs to be replaced by the new task
- B. When another task needs to successfully complete before the new task begins
- C. When another task has the same dependency libraries as the new task
- D. When another task needs to use as little compute resources as possible

Answer: B

Explanation:

The correct answer is B: When another task needs to successfully complete before the new task begins.

The "Depends On" field in a Databricks Job Task defines task dependencies. This means that a task listed in the "Depends On" field must complete successfully before the current task can start. This mechanism is crucial for orchestrating data pipelines and ensuring data integrity. It allows you to create a directed acyclic graph (DAG) of tasks where the output of one task serves as the input for another.

Option A is incorrect because "Depends On" doesn't mean task replacement. Replacing a task involves modifying or deleting an existing task, not setting up a dependency. Option C is incorrect as shared libraries don't mandate a "Depends On" relationship. While sharing libraries is good practice for efficiency, the tasks can still run independently. Option D is also incorrect; resource consumption by one task doesn't necessitate a dependency relationship with another. The "Depends On" functionality is specifically about ensuring the successful execution and data output of one task before another begins, establishing a workflow order. Without this mechanism, the new task might try to operate on incomplete or inconsistent data, leading to errors or incorrect results. Data pipelines often have stages where data cleaning, transformation, and aggregation are performed sequentially. "Depends On" ensures that the raw data is cleaned before the transformation step, and that transformed data is available before the aggregation step. This ensures the correct flow of data and the successful completion of the data engineering workflow. In essence, "Depends On" ensures that the tasks within a job adhere to a specific order of execution to guarantee correctness and data integrity.

Further research can be conducted on the Databricks documentation site related to "Databricks Jobs" and "Task dependencies":

[Databricks Jobs](#)
[Task dependencies](#)

Question: 156

Deep Dumps

A data engineering team has two tables. The first table `march_transactions` is a collection of all retail transactions in the month of March. The second table `april_transactions` is a collection of all retail transactions in the month of April. There are no duplicate records between the tables.

Which of the following commands should be run to create a new table `all_transactions` that contains all records from `march_transactions` and `april_transactions` without duplicate records?

- A. `CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INNER JOIN SELECT * FROM april_transactions;`
- B. `CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
UNION SELECT * FROM april_transactions;`
- C. `CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
OUTER JOIN SELECT * FROM april_transactions;`
- D. `CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INTERSECT SELECT * FROM april_transactions;`

Answer: B

Explanation:

The correct answer is B: `CREATE TABLE all_transactions AS SELECT FROM march_transactions UNION SELECT FROM april_transactions;`

Here's why:

The goal is to create a new table, `all_transactions`, containing all records from both `march_transactions` and `april_transactions` without duplicates. Each of the other options fails to achieve this.

Option A, using `INNER JOIN`, only returns records that exist in both tables based on some implicit or explicit join condition. This is inappropriate since we want all records from both tables.

Option C, using `OUTER JOIN`, returns all records when there is a match in either left or right table records. This will not remove duplicate records, and we are told that no duplicates exist between the two input tables.

Option D, using `INTERSECT`, returns only the records that are common to both tables. This is the opposite of what we need.

`UNION` combines the result sets of two or more `SELECT` statements into a single result set. Importantly, `UNION` implicitly removes duplicate rows from the final result. Since the question explicitly states that there are no duplicate records between the tables, this default behavior of `UNION` doesn't negatively impact the desired result.

Therefore, `UNION` effectively merges the two tables, resulting in a single table, `all_transactions`, containing all unique records from both `march_transactions` and `april_transactions`, achieving the desired outcome. This is the simplest and most effective way to consolidate these two tables into one in Databricks SQL.

<https://docs.databricks.com/sql/language-manual/sql-ref-syntax-qry-select-union.html>

Question: 157

Deep Dumps

How can Git operations must be performed outside of Databricks Repos?

- A. Commit

- B.Pull
- C.Merge
- D.Clone

Answer: D

Explanation:

The correct answer is **D. Clone**. Here's a detailed justification:

Git operations, in the context of Databricks Repos, primarily involve managing version-controlled code within the Databricks environment. Databricks Repos provides a user interface and tools for committing, pulling, merging, branching, and other common Git operations within the Databricks workspace. However, there are situations where you might need to perform Git operations outside of the Databricks Repos environment.

Clone is the fundamental operation required to initiate working with a remote Git repository. It downloads the entire repository, including all branches and history, from a remote source (like GitHub, GitLab, Azure DevOps, etc.) to your local machine or another environment outside of Databricks. Only after cloning the repo can you perform other operations like committing, merging, and pulling from your local machine. The process is essentially making a copy of the remote repository locally.

Committing, Pulling, and Merging are all actions that typically occur after a repository has already been cloned. You can't commit, pull, or merge if you haven't first obtained a local copy of the repository. Since the question specifically asks for an operation that must be performed outside of Databricks Repos in a specific circumstance (likely the initial setup), cloning fits this condition.

Therefore, if you are setting up a Databricks Repo for the first time from a remote Git repository, you might perform the cloning operation on your local machine or another cloud environment before linking Databricks Repos to that repository. Alternatively, cloning might happen on a CI/CD pipeline for deployment purposes. The key is that a local copy of the code is made, and this operation is very often done outside the Databricks environment.

Git Documentation - Cloning: <https://git-scm.com/docs/git-clone>

Databricks Repos Documentation: <https://docs.databricks.com/repos/index.html> (While not explicitly stating where to clone from, it implies cloning is a precursor to Repos use.)

Question: 158

Deep Dumps

A data engineer has joined an existing project and they see the following query in the project repository:

```
CREATE STREAMING LIVE TABLE loyal_customers AS
```

```
SELECT customer_id -  
FROM STREAM(LIVE.customers)  
WHERE loyalty_level = 'high';
```

Which of the following describes why the STREAM function is included in the query?

- A.The STREAM function is not needed and will cause an error.
- B.The data in the customers table has been updated since its last run.
- C.The customers table is a streaming live table.
- D.The customers table is a reference to a Structured Streaming query on a PySpark DataFrame.

Answer: C

Explanation:

The correct answer is C because the `STREAM` function in the `CREATE STREAMING LIVE TABLE` query is specifically used to read data from another Delta Live Tables (DLT) table that is defined as a streaming live table. Delta Live Tables is a declarative framework for building reliable, maintainable, and testable data processing pipelines. DLT handles incremental data processing and manages the underlying infrastructure. When a table is declared as a streaming live table (as `LIVE.customers` presumably is), it continuously ingests data from a streaming source, such as Apache Kafka or a cloud storage system.

The `STREAM` function indicates that the query should treat the source table (`LIVE.customers`) as a streaming source, enabling incremental processing. Without the `STREAM` function, DLT would treat `LIVE.customers` as a regular Delta table and perform a full refresh each time the pipeline is updated, which defeats the purpose of streaming and incremental updates. This would violate the paradigm of incremental data processing inherent to streaming live tables.

Option A is incorrect because the `STREAM` function is needed when the source table is a streaming live table. It's the mechanism by which DLT understands that data should be read incrementally. Removing it would change the semantics of the query and lead to incorrect results or pipeline failures.

Option B is incorrect because the need for the `STREAM` function isn't directly related to updates since the last run; it's fundamentally tied to how the source table is defined (as a streaming live table). The `STREAM` function ensures new data is processed incrementally and that updates to the source table propagate in a continuous fashion to any downstream DLT table that's reading from it.

Option D is incorrect because while Structured Streaming is a related concept in Apache Spark, `STREAM(LIVE.customers)` does not directly represent a reference to a Structured Streaming query on a PySpark `DataFrame`. DLT provides a higher-level abstraction, and `STREAM` tells DLT that it should leverage its own capabilities to consume data as a stream from the named DLT table. The underlying implementation might use Structured Streaming or other Spark technologies, but the syntax `STREAM(LIVE.customers)` is specifically a DLT construct.

Further research can be conducted on Delta Live Tables in the Databricks documentation:

[What is Delta Live Tables?](#)

[Delta Live Tables SQL Language Reference](#)

Question: 159

Deep Dumps

Which Structured Streaming query is performing a hop from a Silver table to a Gold table?

A.

```
(spark.readStream.load(rawSalesLocation)
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("newSales")
)
```

B.

```
(spark.table("sales")
  .withColumn("avgPrice", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("newSales")
)
```

C.

```
(spark.table("sales")
  .filter(col("units") > 0)
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("newSales")
)
```

D.

```
(spark.table("sales")
  .groupBy("store")
  .agg(sum("sales"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("newSales")
)
```

Answer: D

Explanation:

```
(spark.table("sales")
  .groupBy("store")
  .agg(sum("sales"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("newSales")
)
```

Question: 160

Deep Dumps

A data organization leader is upset about the data analysis team's reports being different from the data engineering team's reports. The leader believes the siloed nature of their organization's data engineering and data analysis architectures is to blame.

Which of the following describes how a data lakehouse could alleviate this issue?

- A. Both teams would respond more quickly to ad-hoc requests
- B. Both teams would use the same source of truth for their work
- C. Both teams would reorganize to report to the same department
- D. Both teams would be able to collaborate on projects in real-time

Answer: B

Explanation:

The correct answer is B: Both teams would use the same source of truth for their work.

A data lakehouse architecture aims to unify the best aspects of data warehouses and data lakes. A key feature of a data lakehouse is its ability to provide a single source of truth for all data within an organization. By establishing a centralized repository with consistent data governance and quality controls, a data lakehouse eliminates data silos. Data engineers and data analysts can then access and utilize the same foundational datasets for their respective tasks. This shared data foundation ensures that both teams are working with the same data, leading to consistent results in their reports and analyses. This consistency resolves discrepancies arising from isolated data pipelines and transformations specific to individual teams. Options A, C, and D are incorrect because they address process or organizational concerns but not the core problem of data inconsistency arising from separate data architectures. A data lakehouse provides a consistent, governed data layer, eliminating data silos and ensuring both teams use the same reliable data source for their work.

Further reading:

Databricks Lakehouse Platform: <https://www.databricks.com/product/data-lakehouse>

What is a Data Lakehouse?: <https://aws.amazon.com/what-is/data-lakehouse/>

Question: 161

Deep Dumps

A data analyst has developed a query that runs against Delta table. They want help from the data engineering team to implement a series of tests to ensure the data returned by the query is clean. However, the data engineering team uses Python for its tests rather than SQL.

Which of the following operations could the data engineering team use to run the query and operate with the results in PySpark?

- A. `SELECT * FROM sales`
- B. `spark.delta.table`
- C. `spark.sql`
- D. `spark.table`

Answer: C

Explanation:

The correct answer is **C. `spark.sql`**. Here's why:

The goal is to execute an existing SQL query within a PySpark environment. The `spark.sql` function in PySpark provides the direct interface for executing SQL queries. It takes a SQL query string as input and returns a PySpark DataFrame containing the results. This allows data engineers to leverage their existing Python-based testing frameworks and data manipulation tools to work with the data returned by the analyst's SQL query. They can then apply various data quality checks, transformations, and assertions using PySpark's DataFrame

API.

Option A (SELECT * FROM sales) is incomplete; it's a SQL query fragment but doesn't provide the context of how to execute it within PySpark. Option B (spark.delta.table) is related to accessing Delta tables specifically but doesn't execute an arbitrary SQL query. It helps load a Delta table into a DataFrame. Option D (spark.table) is used to load an existing table registered in the Spark catalog (metastore) into a DataFrame, it cannot execute an arbitrary SQL query.

Therefore, spark.sql is the only option that directly addresses the requirement of executing a SQL query from within PySpark and making the resulting data available as a DataFrame for further processing and testing. This approach allows seamless integration of SQL-based data extraction with Python-based data validation and transformation workflows. For further research, you can refer to the following links:

Spark SQL, DataFrames and Datasets Guide: <https://spark.apache.org/docs/latest/sql-getting-started.html>
PySpark SQL Functions Documentation:
<https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql.html> (specifically look for spark.sql())

Question: 162

Deep Dumps

A data engineer has a Job that has a complex run schedule, and they want to transfer that schedule to other Jobs.

Rather than manually selecting each value in the scheduling form in Databricks, which of the following tools can the data engineer use to represent and submit the schedule programmatically?

- A. pyspark.sql.types.DateType
- B. datetime
- C. pyspark.sql.types.TimestampType
- D. Cron syntax

Answer: D

Explanation:

The correct answer is Cron syntax because it provides a standard, string-based method for defining complex, recurring schedules. Cron syntax allows users to specify schedules using fields representing minute, hour, day of the month, month, and day of the week. This string representation can then be programmatically stored, transferred, and applied to different Jobs.

Options A, B, and C are incorrect as they are related to data types within PySpark or Python and do not represent a scheduling language or system. pyspark.sql.types.DateType and pyspark.sql.types.TimestampType are used for specifying the data type of columns in a Spark DataFrame, and datetime is a Python module for working with dates and times as data. These types are useful for handling date and time data, but not for scheduling tasks.

Cron syntax, on the other hand, is explicitly designed for defining schedules. By using Cron syntax, the data engineer can define a schedule once and then reuse it across multiple Jobs by simply copying the Cron string and applying it through the Databricks API or UI (if it supports Cron string inputs). This is much more efficient and less error-prone than manually re-entering all the scheduling parameters for each Job. Databricks Jobs often provide a way to define schedules via cron expressions in the job settings.

Therefore, Cron syntax is the appropriate tool for programmatically representing and submitting schedules for Databricks Jobs.

Further research can be done using the following links:

Cron Syntax: <https://en.wikipedia.org/wiki/Cron>

Databricks Jobs (check for cron scheduling support in the API or UI documentation):

<https://docs.databricks.com/workflows/jobs/index.html>

Question: 163

Deep Dumps

A data engineer and data analyst are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.

Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

- A. The pipeline will need to be written entirely in Python
- B. The pipeline will need to stop using the medallion-based multi-hop architecture
- C. The pipeline will need to be written entirely in SQL
- D. The pipeline will need to use a batch source in place of a streaming source

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer, along with supporting concepts and links:

Delta Live Tables (DLT) natively supports both streaming and batch sources. While DLT excels at simplifying incremental data processing, it imposes a specific way of defining the data flow through declarative pipelines. Therefore, option B, that the pipeline will need to stop using the medallion-based multi-hop architecture, is incorrect because DLT's design fits very well with the medallion architecture (Bronze, Silver, Gold layers). The medallion architecture inherently encourages modular data transformation, aligning with the principle of separating concerns that DLT reinforces. Options A and C are incorrect because DLT supports both SQL and Python languages to create pipelines.

The need to use batch sources stems from the inherent processing characteristics of DLT. DLT manages the scheduling, dependencies, and execution of data transformations automatically. To provide reliable guarantees on data completeness and correctness, DLT fundamentally relies on the ability to replay and backfill data in the pipeline as needed. Streaming sources, by their very nature, provide continuous, unbounded data flow. It is usually not possible or easy to replay historic data from a streaming source.

Therefore, if the raw source is truly a streaming input in its purest form (e.g., Kafka topic with no ability to rewind), a direct migration to DLT might not be feasible without introducing a buffering or checkpointing mechanism. The recommendation is to adapt it into a "micro-batch" pattern, meaning the streaming source is treated as a series of small batch jobs. DLT can consume the output of that micro-batch process, but it can not directly consume the original pure streaming data. This is usually done by storing the streaming data into cheap object storage (Azure Data Lake Storage/AWS S3) and pointing the DLT raw data to the cheap object storage instead of the streaming data.

Therefore, the most significant adjustment when moving to DLT would be to transform the existing streaming source into a system that can emit "batches" of data. This might involve buffering the data for a certain period or storing it in a file system/data lake as the streaming data arrives to allow for replayability when DLT pipeline needs to backfill data. This approach ensures that DLT can effectively manage the data flow and provide the data quality guarantees inherent in its design.

Authoritative Links:

Delta Live Tables Documentation: <https://docs.databricks.com/delta-live-tables/index.html> - This provides comprehensive details on DLT, including its features, architecture, and how it handles data processing.

Medallion Architecture: <https://www.databricks.com/blog/2019/08/15/diving-into-delta-lake-unpacking-the-medallion-architecture.html> - This explains the Medallion architecture (Bronze, Silver, Gold) and how it works with Delta Lake.

Question: 164

Deep Dumps

A data engineer needs access to a table `new_table`, but they do not have the correct permissions. They can ask the table owner for permission, but they do not know who the table owner is.

Which of the following approaches can be used to identify the owner of `new_table`?

- A. Review the Permissions tab in the table's page in Data Explorer
- B. There is no way to identify the owner of the table
- C. Review the Owner field in the table's page in Data Explorer
- D. Review the Owner field in the table's page in the cloud storage solution

Answer: C

Explanation:

The correct answer is C, "Review the Owner field in the table's page in Data Explorer."

Databricks Data Explorer is a user interface designed to help data engineers and analysts discover, explore, and understand data assets within the Databricks environment. It provides a centralized location to manage and monitor data objects, including tables. One of the key features of Data Explorer is the ability to view table metadata, including information about the table owner.

The "Owner" field in the table's page within Data Explorer explicitly identifies the principal (user or group) that owns the table. This is crucial for permission management because the owner generally has full control over the table, including the ability to grant permissions to other users.

Option A, "Review the Permissions tab in the table's page in Data Explorer," is partially correct. While the Permissions tab does show which users or groups have specific privileges on the table, it doesn't directly indicate who the owner is. It shows who has what permissions, but not who has the permission to grant permissions.

Option B, "There is no way to identify the owner of the table," is incorrect because Data Explorer is designed to provide this kind of metadata information. Knowing the owner is vital for data governance and access control.

Option D, "Review the Owner field in the table's page in the cloud storage solution," might seem plausible. While cloud storage solutions (like AWS S3 or Azure Blob Storage) underpin Databricks, the ownership within Databricks is managed at the metastore level (typically a Hive metastore or Databricks Unity Catalog). The storage layer might have its own ownership concepts, but those are distinct from the ownership relevant for granting permissions within Databricks SQL or PySpark environments. Data Explorer surfaces the Databricks-level ownership.

Therefore, Data Explorer's "Owner" field directly reveals the table owner, enabling the data engineer to contact the appropriate person for permission grants.

For more information, refer to the Databricks documentation on Data Explorer and access control:

[Databricks Data Explorer](#)

[Databricks Access Control](#)

Question: 165**Deep Dumps**

A data engineer needs to create a table in Databricks using data from their organization's existing SQLite database.

They run the following command:

```
CREATE TABLE jdbc_customer360
USING _____
OPTIONS (
  url "jdbc:sqlite:/customers.db",
  dbtable "customer360"
)
```

Which of the following lines of code fills in the above blank to successfully complete the task?

- A. org.apache.spark.sql.jdbc
- B. autoloader
- C. org.apache.spark.sql.sqlite
- D. sqlite

Answer: A**Explanation:**

org.apache.spark.sql.sqlite: This would be the Java package for interacting with SQLite databases specifically within Spark SQL. SQLite is a lightweight, file-based relational database.

Question: 166**Deep Dumps**

In which of the following scenarios should a data engineer use the MERGE INTO command instead of the INSERT INTO command?

- A. When the location of the data needs to be changed
- B. When the target table is an external table
- C. When the source is not a Delta table
- D. When the target table cannot contain duplicate records

Answer: D**Explanation:**

D. When the target table cannot contain duplicate records.

MERGE INTO is used for upserts (update + insert) and deduplication.

It allows a data engineer to define logic like:

If a matching record exists → UPDATE it.

If no matching record exists → INSERT it.

This is crucial when a table must not contain duplicates, especially when integrating data from sources that may contain overlapping records.

Why the other options are incorrect:

A. When the location of the data needs to be changed

→ This refers more to data movement or path changes, not something handled by MERGE INTO.

B. When the target table is an external table

→ MERGE INTO can be used with external Delta tables, but being "external" alone isn't a reason to choose MERGE over INSERT.

C. When the source is not a Delta table

→ MERGE INTO requires Delta tables as the target; if the source isn't Delta, it still can be used, but the target must be a Delta table.

Question: 167

Deep Dumps

A data engineer is designing a data pipeline. The source system generates files in a shared directory that is also used by other processes. As a result, the files should be kept as is and will accumulate in the directory. The data engineer needs to identify which files are new since the previous run in the pipeline, and set up the pipeline to only ingest those new files with each run.

Which of the following tools can the data engineer use to solve this problem?

- A. Unity Catalog
- B. Delta Lake
- C. Databricks SQL
- D. Auto Loader

Answer: D

Explanation:

The correct answer is **D. Auto Loader**. Here's why:

Auto Loader is specifically designed for incremental data ingestion from cloud storage into Delta Lake. It automatically detects new files as they arrive in a cloud storage location like a shared directory. It leverages cloud infrastructure features (like file notification events) and efficient directory listing to identify new files without requiring manual file tracking or complex logic.

Auto Loader keeps track of which files it has already processed, eliminating the need for the data engineer to build and maintain custom state management. This makes it ideal for scenarios where new files are continuously added to a directory used by other processes, and only the newly added files need to be ingested in each pipeline run. The other options are not as suited to this task. Unity Catalog is for data governance and discovery, not for file ingestion. Delta Lake is a storage layer that benefits from Auto Loader but doesn't solve the new file identification problem on its own. Databricks SQL is a query engine and is not designed for incremental data ingestion.

Auto Loader avoids reprocessing already-ingested data, making the pipeline more efficient. Its schema

inference and evolution capabilities also simplify data pipeline development and maintenance in environments where the source data schema might change over time. It also provides features to handle file format variations automatically.

For further reading on Auto Loader, you can refer to the official Databricks documentation:

<https://docs.databricks.com/ingestion/auto-loader/index.html>

Question: 168

Deep Dumps

What is stored in the Databricks customer's cloud account?

- A. Databricks web application
- B. Cluster management metadata
- C. Notebooks
- D. Data

Answer: D

Explanation:

The correct answer is **D. Data**. Here's why:

Databricks leverages a customer's existing cloud infrastructure (AWS, Azure, or GCP) for its operations. The control plane and compute plane are primarily managed by Databricks, but the data itself resides within the customer's cloud storage.

Let's break down why the other options are incorrect:

A. Databricks web application: The Databricks web application is hosted and managed by Databricks itself, not within the customer's cloud account. Customers access this application through a web browser.

B. Cluster management metadata: While Databricks manages cluster metadata related to cluster configuration, status, and metrics, this metadata is primarily stored within the Databricks control plane for operational efficiency and performance. It's not generally directly exposed or stored within the customer's storage account.

C. Notebooks: While notebook definitions (code, markdown) are stored within the Databricks workspace (backed by cloud storage), the execution context and persistent data generated from those notebooks resides in the customer's cloud storage.

Therefore, the primary component stored in the customer's cloud account is the **data** itself. This includes data stored in data lakes (e.g., S3 on AWS, ADLS Gen2 on Azure, GCS on GCP), databases, and other storage services that the Databricks environment interacts with. Databricks accesses this data for processing, analysis, and transformation. The data is governed and controlled by the customer within their own cloud environment. This architecture enables Databricks to offer its services while maintaining data sovereignty and security for its customers. Databricks is a PaaS (Platform as a Service) on top of IaaS (Infrastructure as a Service) for cloud providers. Databricks processes the data, but the cloud provider is responsible for storing the data itself.

Here are a few authoritative links to support this explanation:

Databricks Architecture: <https://www.databricks.com/wp-content/uploads/2020/07/databricks-architecture.pdf>

Databricks Security and Trust: <https://www.databricks.com/trust>

Question: 169**Deep Dumps**

A data engineer wants to create a relational object by pulling data from two tables. The relational object does not need to be used by other data engineers in other sessions. In order to save on storage costs, the data engineer wants to avoid copying and storing physical data.

Which of the following relational objects should the data engineer create?

- A. Spark SQL Table
- B. View
- C. Delta Table
- D. Temporary view

Answer: D**Explanation:**

The correct answer is **D. Temporary View**. Here's a detailed justification:

A temporary view in Spark SQL is a relational object that exists only within the scope of the current session. This perfectly aligns with the requirement that the relational object doesn't need to be accessible by other data engineers in other sessions. Creating a temporary view avoids persisting the data to storage, satisfying the goal of minimizing storage costs.

A Spark SQL Table (A) and a Delta Table (C) both involve persisting data to storage. Creating either of these would incur storage costs and are not suitable for the given requirements. They are designed for persistent data storage and sharing across sessions. Delta Tables specifically are for reliable data lakes.

A View (B), while not necessarily materializing data immediately, are typically persisted as metadata in the metastore. This means they can be accessed by other sessions and users if the user has appropriate permissions. While views perform query time transformation, the user expects views to be available for any session that can connect to the metastore and database. Thus, a view is not ideal given the stated requirement for temporary, session-specific accessibility.

Temporary views, on the other hand, are not persisted to any metastore. They are created in memory and are automatically dropped when the session ends. They're ideal for scenarios where you need a relational object for intermediate calculations or transformations within a single session without impacting storage costs or other users.

Therefore, the temporary view is the best choice. It provides the required relational object, avoids data duplication, and is confined to the current session, saving on storage costs and preventing unwanted sharing.

Relevant Documentation:

1. **Spark SQL, DataFrames and Datasets Guide:** <https://spark.apache.org/docs/latest/sql-programming-guide.html> (search for "temporary views")
2. **Databricks Views:** <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html>

Question: 170**Deep Dumps**

Which of the following commands will return the number of null values in the member_id column?

- A. `SELECT count(member_id) FROM my_table;`

- B.SELECT count(member_id) - count_null(member_id) FROM my_table;
- C.SELECT count_if(member_id IS NULL) FROM my_table;
- D.SELECT null(member_id) FROM my_table;

Answer: C

Explanation:

C. SELECT count_if(member_id IS NULL) FROM my_table;

count_if(condition) counts the number of rows where the condition is true.

So count_if(member_id IS NULL) returns the number of NULL values in the member_id column.

Why the other options are incorrect:

A. SELECT count(member_id) FROM my_table;

This counts non-null values only, not nulls.

B. SELECT count(member_id) - count_null(member_id) FROM my_table;

count_null() is not a valid SQL function in standard SQL or Databricks/Spark SQL.

D. SELECT null(member_id) FROM my_table;

This is invalid syntax. There's no null() function like this.

Question: 171

Deep Dumps

Which tool is used by Auto Loader to process data incrementally?

- A.Checkpointing
- B.Spark Structured Streaming
- C.Databricks SQL
- D.Unity Catalog

Answer: B

Explanation:

B. Spark Structured Streaming

Auto Loader is a feature in Databricks that allows for incremental loading of data files (like CSV, JSON, Parquet) from cloud storage (e.g., AWS S3, Azure Data Lake) as they arrive.

Auto Loader is built on top of Spark Structured Streaming, which is the underlying engine that:

Continuously processes incoming files

Maintains state information

Supports exact-once semantics

Enables incremental ingestion

Why the other options are incorrect:

A. Checkpointing

Related but not the core tool.

It's a mechanism used by Spark Structured Streaming (including Auto Loader) to track progress. It's a feature, not the tool itself.

C. Databricks SQL

Used for querying and BI dashboards. It's not used for incremental file processing.

D. Unity Catalog

Manages data governance (permissions, metadata), not data ingestion or streaming.

Question: 172

Deep Dumps

Which of the following benefits is provided by the array functions from Spark SQL?

- A. An ability to work with data in a variety of types at once
- B. An ability to work with data within certain partitions and windows
- C. An ability to work with time-related data in specified intervals
- D. An ability to work with complex, nested data ingested from JSON files

Answer: D

Explanation:

D. An ability to work with complex, nested data ingested from JSON files.

This is a core benefit of array functions. When you ingest JSON (or other semi-structured formats like Avro, Parquet with nested structures), you often encounter arrays within the nested data. Spark SQL's array functions (e.g., explode, array_contains, size, get_json_object, from_json, to_json working with arrays, etc.) are specifically designed to manipulate, transform, and query these complex, nested array structures. They allow you to "flatten" arrays, access specific elements, filter based on array content, and more.

Question: 173

Deep Dumps

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

sales

customer_id	spend	units
a1	28.94	7
a3	874.12	23
a4	8.99	1

favorite_stores

customer_id	store_id
a1	s1
a2	s1
a4	s2

The data engineer runs the following query to join these tables together:

```
SELECT
    sales.customer_id,
    sales.spend,
    favorite_stores.store_id
FROM sales
LEFT JOIN favorite_stores
ON sales.customer_id = favorite_stores.customer_id;
```

Which of the following will be returned by the above query?

A.

customer_id	spend	store_id
a1	28.94	s1
a4	8.99	s2

B.

customer_id	spend	store_id
a1	28.94	s1
a2	NULL	s1
a4	8.99	s2

C.

customer_id	spend	store_id
a1	28.94	s1
a3	874.12	NULL
a4	8.99	s2

D.

customer_id	spend	store_id
a1	28.94	s1
a2	NULL	s1
a3	874.12	NULL
a4	8.99	s2

Answer: C

Explanation:

Row 1 (Customer a1):

Spent \$28.94 at store s1.

Both spend and store_id have valid values.

Row 2 (Customer a3):

Spent \$874.12, but store_id is NULL.

This might indicate an online transaction, an unknown store, or a data entry issue.

Row 3 (Customer a4):

Spent \$8.99 at store s2.

Both spend and store_id have valid values.

Question: 174

Deep Dumps

A data engineer has been provided a PySpark DataFrame named df with columns product and revenue. The data engineer needs to compute complex aggregations to determine each product's total revenue, average revenue, and transaction count.

Which code snippet should the data engineer use?

A.

```
from pyspark.sql import functions as F
aggregated_df = df.groupBy("product").agg( F.sum("revenue").alias("total_revenue"),
F.avg("revenue").alias("avg_revenue"), F.count("*").alias("transaction_count") )
```

```
aggregated_df = df.groupby("product").agg(  
    "sum(revenue)",  
    "avg(revenue)",  
    "count(revenue)"
```

B.)

C.

```
from pyspark.sql import functions as F aggregated_df = df.select("product",  
  
"revenue").groupBy("product").agg( F.sum("revenue"), F.mean("revenue") )
```

D.

```
aggregated_df =  
df.groupBy("product").agg( {"revenue": "sum", "revenue": "avg", "revenue": "count"} )
```

Answer: A

Question: 175

Deep Dumps

A Databricks single-task workflow fails at the last task due to an error in a notebook. The data engineer fixes the mistake in the notebook.

What should the data engineer do to rerun the workflow?

- A. Repair the task
- B. Rerun the pipeline
- C. Restart the cluster
- D. Switch the cluster

Answer: A

Explanation:

Correct Answer

Option A: Repair the task

The data engineer should repair the task to rerun the workflow efficiently.

Explanation of the Correct Answer

The correct answer is A. Repair the task.

When a Databricks workflow fails due to an error in a notebook, and the error has been fixed, the "Repair task" functionality allows you to resume execution from the point of failure. This avoids re-running the entire workflow and saves computational resources.

The repair functionality uses the corrected notebook code and preserves the results of previously successful tasks.

Analysis of Each Option

Option A: Repair the task - This is the correct option because it allows resuming the workflow from the failed task after the notebook has been fixed, without re-running the entire workflow.

Option B: Rerun the pipeline - This is incorrect because it restarts the entire workflow from the beginning, wasting resources and time since previous tasks completed successfully.

Option C: Restart the Cluster - This is incorrect because restarting the cluster addresses infrastructure issues, not code errors. The cluster itself wasn't the problem.

Option D: Switch the cluster - This is incorrect because, like restarting the cluster, switching the cluster addresses infrastructure concerns rather than the code issue that was fixed.

Key Takeaways

The "Repair task" functionality in Databricks workflows is the most efficient way to resume a workflow after a task fails due to a code error that has been fixed.

Avoid re-running the entire pipeline or restarting/switching the cluster when the issue is a code error in a specific task.

Always consider the specific cause of the failure when choosing a recovery method.

Question: 176

Deep Dumps

A data engineer needs to provide access to a group named manufacturing-team. The team needs privileges to create tables in the quality schema.

Which set of SQL commands will grant a group named manufacturing-team to create tables in a schema named production with the parent catalog named manufacturing with the least privileges?

- A. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;
- B. GRANT USE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;
- C. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE CATALOG ON CATALOG manufacturing TO manufacturing-team;
- D. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

Answer: A

Explanation:

Option A is correct as it appropriately grants the minimum required privileges for the manufacturing-team to create tables in the specified schema while ensuring they have the necessary access to utilize the schema and catalog.

Reasoning for Option A:

1. **GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team;**

This command directly provides the required permission to create tables within the quality schema.

2. **GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team;**

It allows the team to access the schema to work with existing tables or other objects they may need.

3. **GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;**

This grants access to the parent catalog, necessary for schema operations.

This sequence minimizes privileges while fulfilling the requirement to create tables, adhering to the principle of least privilege.

Evaluation of Other Options:

Option B:

Why Incorrect: Grants USE TABLE, which is not relevant for the task of creating tables. The correct command for table creation should be CREATE TABLE.

Option C:

Why Incorrect: Grants CREATE SCHEMA and CREATE CATALOG, which are unnecessary for the stated requirement. The team only needs to create tables in an existing schema, not create new schemas or catalogs.

Option D:

Why Incorrect: Similar to Option C, it includes CREATE SCHEMA, extending privileges beyond what is required. Additionally, USE CATALOG is appropriate but not the correct combination to limit permissions effectively for the task at hand.

References:

<https://docs.databricks.com/dev-tools/databricks-sql/sql-commands.html>

<https://docs.databricks.com/dev-tools/databricks-sql/access-control.html>

<https://docs.databricks.com/sql/language-manual/sql-commands.html>

Question: 177

Deep Dumps

A data engineer has written a function in a Databricks Notebook to calculate the population of bacteria in a given medium.

```
def calculate_population_of_bacteria (intital_population, exponential_factor)
    return future_population = intital_population ** exponential_factor
```

Analysts use this function in the notebook and sometimes provide input arguments of the wrong data type, which can cause errors during execution.

Which Databricks feature will help the data engineer quickly identify if an incorrect data type has been provided as input?

- A. The Spark User interface has a debug tab that contains the variables that are used in this session.
- B. The Databricks debugger enables breakpoints that will raise an error if the wrong data type is submitted.
- C. The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.

D. The Data Engineer should add print statements to find out what the variable is.

Answer: C

Explanation:

Correct Answer:

C. The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.

Explanation:

When analysts pass the wrong data type to the function (e.g., a string instead of a number), it can cause execution errors. The Databricks debugger's variable explorer is the most efficient way to catch this issue because:

It shows all variables currently in scope.

For each variable, you can see its value and data type immediately.

This makes it very quick to confirm if the wrong type of input (e.g., "ten" instead of 10) has been provided.

This avoids the need for guesswork or manual print statements.

Why the other options are incorrect:

A. Spark UI debug tab → The Spark UI is for job execution monitoring (tasks, stages, DAGs), not for checking variable types in notebook code.

B. Breakpoints → Breakpoints pause execution but don't automatically validate data types. They're useful for step-by-step debugging, but not the fastest way to see type issues.

D. Print statements → A very manual method, requiring code changes each time. Much less efficient than using the variable explorer.

Key Takeaway:

The variable explorer in the Databricks debugger is the right feature because it provides immediate visibility into variables and their types, making it the fastest way to identify incorrect input data types.

Final Answer: C. The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.

Question: 178

Deep Dumps

A data engineer is inspecting an ETL pipeline based on a Pyspark job that consistently encounters performance bottlenecks. Based on developer feedback, the data engineer assumes the job is low on compute resources. To pinpoint the issue, the data engineer observes the Spark UI and finds out the job has a high CPU time vs Task time.

Which course of action should the data engineer take?

A. High CPU time vs Task time means an under-utilized cluster. The data engineer may need to repartition data

to spread the jobs more evenly throughout the cluster.

B. High CPU time vs Task time means efficient use of cluster and no change needed

C. High CPU time vs Task time means a CPU over-utilized job. The data engineer may need to consider executor and core tuning or resizing the cluster

D. High CPU time vs Task time means over-utilized memory and the need to increase parallelism

Answer: C

Explanation:

C is correct because high CPU time relative to task time indicates that the job is CPU-bound, necessitating adjustments to executor and core settings or resizing the cluster.

In distributed computing frameworks like Apache Spark, adequate resource allocation is essential for job performance. A disparity where CPU time significantly exceeds task time suggests that tasks are not completing efficiently, often due to insufficient computing resources to handle the workload. This scenario typically arises when tasks require more processing power than what is currently allocated, leading to CPU over-utilization. Adjusting executor and core configurations can help optimize resource distribution, enabling the pipeline to process data more effectively and ultimately improving throughput. Resizing the cluster may also provide additional compute resources, ensuring that tasks can execute with adequate power.

Evaluation of Incorrect Options:

A. High CPU time vs Task time indicates an under-utilized cluster.

This option misinterprets the performance metrics. An under-utilized cluster would exhibit low CPU times. Here, the imbalance signifies that the current resources are insufficient for the workload, rather than being under-utilized.

B. High CPU time vs Task time means efficient use of the cluster and no change needed.

This assertion is misleading as it ignores the implications of high CPU utilization. Efficiency in resource use would typically manifest as a more balanced ratio of CPU time to task time. A high CPU time indicates that tasks may be taking longer to execute, signaling a need for immediate optimization.

D. High CPU time vs Task time means over-utilized memory and the need to increase parallelism.

This option conflates CPU and memory usage indicators. The issue presented relates to CPU demands, not memory, signaling that increased parallelism alone would not address the CPU bottleneck problem effectively.

References

<https://spark.apache.org/docs/latest/running-on-yarn.html>

<https://databricks.com/blog/2020/02/06/best-practices-for-using-apache-spark-in-yarn.html>

<https://docs.databricks.com/spark/latest/spark-sql/index.html>

Question: 179

Deep Dumps

A data engineer needs to parse only png files in a directory that contains files with different suffixes.

Which code should the data engineer use to achieve this task?

A. `df = spark.readStream.format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \ .append("/*.png")`

B. `df = spark.readstream. format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \ .option("pathGlobfilter", "*.png") \`

```
.load()
```

```
C. df = spark.readStream.format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \  
.option("pathGlobfilter", "*.png") \ .append()
```

```
D. df = spark.readstream.format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \  
.load("/*.png")
```

Answer: D

Explanation:

The correct answer is **D**. Here's a detailed justification:

The task is to read only .png files from a directory using Databricks' cloudFiles source for structured streaming. cloudFiles is designed for efficient processing of files landing in cloud storage like AWS S3 or Azure Blob Storage. It automatically handles file discovery and ensures exactly-once processing.

Option A and C are incorrect because append() is not a valid function when defining a streaming source for cloudFiles. You need to specify where to load the data from.

Option B is close, but the correct syntax for specifying the directory and filtering files is crucial. pathGlobFilter needs to be part of the .load() function's parameters. While the option("pathGlobfilter", "*.png") would correctly filter the files after listing them, the load() function needs a path.

Option D directly uses the directory path along with the glob pattern directly in the load() function: .load("/*.png"). This is the correct way to tell cloudFiles to only consider files matching the .png extension from the specified root directory. This approach avoids listing all files and then filtering, improving efficiency. The underlying streaming engine (Apache Spark) will use this glob pattern when discovering new files. The stream will only process .png files added to the directory.

In summary, the cloudFiles format, combined with specifying the file type as "binaryFile" and correctly specifying the path with the glob filter inside the .load() method ensures that only .png files are processed from the directory, making Option D the most efficient and syntactically accurate solution.

Here are some resources for further reading:

Databricks Auto Loader (cloudFiles format): <https://docs.databricks.com/en/ingestion/auto-loader/index.html>

Structured Streaming Programming Guide: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Question: 180

Deep Dumps

Which languages are supported by Serverless compute clusters? (Choose two.)

- A. SQL
- B. Python
- C. R
- D. Scala
- E. Java

Answer: AB

Explanation:

AB is correct because Serverless compute clusters in Databricks natively support SQL and Python for

interactive data processing and analytics.

The support for SQL allows users to perform complex queries directly on data lakes using familiar SQL syntax, enabling efficient data manipulation and retrieval. Python, as a versatile programming language, facilitates rich data analysis, machine learning workflows, and integration with various data handling libraries like Pandas and NumPy, making it ideal for data engineering tasks. These features are integral to Databricks' framework, which emphasizes ease of use and flexibility in data tasks.

Conversely, the incorrect options can be evaluated as follows:

C. R: Although R is a significant language for statistical analysis and data visualization, its support in Databricks is not as pronounced as SQL or Python within the Serverless platform, mainly due to the ecosystem's focus on data engineering over niche statistical applications.

D. Scala: While Scala is supported in Databricks, especially for Spark-based applications, its inclusion in Serverless compute environments is typically limited compared to the more user-friendly interfaces provided by SQL and Python. Databricks primarily encourages SQL and Python for accessibility and broader usability among data engineers.

E. Java: Similar to Scala, Java is foundational for Spark operations but is less emphasized within the Serverless compute structure, which prioritizes languages that offer greater support for data manipulation and analytics.

In summary, SQL and Python not only align with Databricks' core functionalities but also reflect industry trends where ease of use and community adoption guide technology choices.

References:

<https://docs.databricks.com/cloud-serverless/index.html>

<https://docs.databricks.com/notebooks/index.html>

<https://docs.databricks.com/libraries/requirements-libraries.html>

Question: 181

Deep Dumps

A data engineer is developing an ETL process based on Spark SQL. The execution fails. The data engineer checks the Spark UI and can see the ERRORS as follows:

"java.lang.OutOfMemoryError: Java heap space"

Which two corrective actions should the data engineer perform to resolve this issue? (Choose two.)

- A. Narrow the filters in order to collect less data in the query
- B. Upsize the worker nodes and activate autoshuffle partitions
- C. Upsize the driver node and deactivate autoshuffle partitions
- D. Cache the dataset in order to boost the query performance
- E. Fix the shuffle partitions to 50 to ensure the allocation

Answer: AB

Explanation:

AB is correct as they directly address the memory limitation by reducing data volume and optimizing resource allocation.

Reasoning:

A. Narrowing the filters: Implementing tighter filters effectively reduces the dataset size being processed. In

ETL processes, particularly with Spark SQL, excessive data can strain memory resources, leading to OutOfMemoryError. By limiting the records being fetched, the workload becomes manageable, aligning with Spark's distributed architecture which handles smaller chunks of data more efficiently.

B. Upsizing the worker nodes: Increasing the capacity of worker nodes can enhance memory availability and computational power. Autosshuffle partitions help in redistributing data across available workers more effectively, thus mitigating the risk of exceeding memory limits, especially during operations that require shuffling large amounts of data.

Evaluation of Incorrect Options:

C. Upsize the driver node and deactivate autosshuffle partitions: This option focuses only on the driver node without addressing memory issues among worker nodes where the bulk of processing takes place. Disabling autosshuffle could lead to larger memory consumption as data would not be optimally redistributed, exacerbating the issue.

D. Cache the dataset: While caching can improve performance, it may worsen the OutOfMemoryError if the entire dataset is loaded into memory, especially if it exceeds available resources. On the contrary, minimizing the dataset size reduces the pressure on memory.

E. Fix shuffle partitions to 50: While adjusting shuffle partitions can help in distributing data load, fixing them rigidly without considering actual workload could lead to inefficiencies. The optimal number of partitions depends on the size of the dataset and cluster configuration. Preemptively setting it to 50 may not effectively resolve memory issues if the underlying dataset is still excessively large.

References:

<https://spark.apache.org/docs/latest/performance-tuning.html>

<https://docs.databricks.com/develop/databricks-sql-optimization.html>

<https://www.oreilly.com/library/view/spark-in-action/9781638352152/>

Question: 182

Deep Dumps

A Data Engineer is designing Bronze layer in Databricks Medallion Architecture. The raw data is collected from multiple sources (Clickstream in JSON, Transactions in CSV).

The task is to design the Bronze layer of the Medallion Architecture to ingest and store this raw data for further processing.

Which operation applies to the Bronze layer?

- A. Ingest raw data without transformations, preserving original schemas, and store in Delta format.
- B. Clean and standardize raw data by removing null values and enforcing schemas.
- C. Apply complex business logic to enrich raw data with customer segmentation labels.
- D. Aggregate and transform source data to calculate daily sales performance metrics.

Answer: A

Question: 183

Deep Dumps

What is the primary function of the Silver layer in the Databricks medallion architecture?

- A. Store historical data solely for auditing purposes

- B. Aggregate and enrich data for business analytics
- C. Validate, clean, and deduplicate data for further processing
- D. Ingest raw data in its original state

Answer: C

Question: 184

Deep Dumps

A data engineer needs to combine sales data from an on-premises PostgreSQL database with customer data in Azure Synapse for a comprehensive report. The goal is to avoid data duplication and ensure up-to-date information.

How should the data engineer achieve this using Databricks?

- A. Export data from both sources to CSV files and upload them to Databricks
- B. Use Lakehouse Federation to query both data sources directly
- C. Manually synchronize data from both sources into a single database
- D. Develop custom ETL pipelines to ingest data into Databricks

Answer: B

Question: 185

Deep Dumps

A data engineer is working in a Python notebook on Databricks to process data, but notices that the output is not as expected. The data engineer wants to investigate the issue by stepping through the code and checking the values of certain variables during execution.

Which tool should the data engineer use to inspect the code execution and variables in real-time?

- A. Cluster Logs
- B. Job Execution Dashboard
- C. Python Notebook Interactive Debugger
- D. SQL Analytics

Answer: C

Question: 186

Deep Dumps

A data engineer has inherited a Databricks pipeline from a previous team. The pipeline is missing its SLAs, and an initial investigation has identified memory spills in Spark. These increased runtimes are also driving up costs. The data engineer needs to reduce the runtime without significantly increasing costs.

What should the data engineer do first to address this issue?

- A. Tweak the "spark.sql.shuffle.partitions" configuration.
- B. Enable autoscaling in the cluster to match the requirements
- C. Use a photon enabled execution engine.
- D. Ensure the cluster uses memory optimized node types.

Answer: A

Question: 187**Deep Dumps**

A data engineer is facing performance bottlenecks in an e-commerce transactions Delta table. The table is a managed Unity Catalog table, and it uses partitioning and Z-ordering in its data layout scheme. The predictive optimization for Unity Catalog tables is also enabled. The table has a frequently changing query filter, and the data engineer does not observe a benefit of the Data Layout or the Predictive Optimization.

How should the data engineer fix the data layout bottlenecks?

- A. Re-write the Data Layout with Liquid Clustering and cluster by the Z-Ordered columns.
- B. Enable Delta Caching so that query results can be read through caches.
- C. Tweak the Z-Order columns and run OPTIMIZE manually.
- D. Switch the Data layout from Partition+Z-Ordering to Automatic Liquid Clustering.

Answer: D**Question: 188****Deep Dumps**

A data engineer needs to optimize the data layout and query performance for an e-commerce transactions Delta table. The table is partitioned by "purchase_date" a date column which helps with time-based queries but does not optimize searches on user statistics "customer_id", a high-cardinality column.

The table is usually queried with filters on "customer_id" within specific date ranges, but since this data is spread across multiple files in each partition, it results in full partition scans and increased runtime and costs.

How should the data engineer optimize the Data Layout for efficient reads?

- A. Alter table implementing liquid clustering on "customer_id" while keeping the existing partitioning.
- B. Alter the table implementing liquid clustering by "customer_id" and "purchase_date".
- C. Alter the table to partition by "customer_id".
- D. Enable delta caching on the cluster so that frequent reads are cached for performance.

Answer: B**Question: 189****Deep Dumps**

What is the structure of an Asset Bundle?

- A. A Docker image containing runtime environments and the source code of the assets
- B. A compressed archive (ZIP) that solely contains workspace assets without any accompanying metadata
- C. A single plain text file enumerating the names of assets to be migrated to a new workspace
- D. A YAML configuration file that specifies the artifacts, resources, and configurations for the project

Answer: D**Question: 190****Deep Dumps**

A data engineer is maintaining an ETL pipeline code with a GitHub repository linked to their Databricks account. The data engineer wants to deploy the ETL pipeline to production as a databricks workflow.

Which approach should the data engineer use?

- A. Databricks Asset Bundles (DAB) + GitHub Integration
- B. Maintain workflow_config.json and deploy it using Databricks CLI
- C. Maintain workflow_config.json and deploy it using Terraform
- D. Manually create and manage the workflow in UI

Answer: A

Question: 191

Deep Dumps

A data engineer works for an organization that must meet a stringent Service Level Agreement (SLA) that demands minimal runtime errors and high availability for its data processing pipelines. The data engineer wants to avoid the operational overhead of managing and tuning clusters.

Which architectural solution will meet the requirements?

- A. Use an auto-scaling cluster configured and monitored by the user.
- B. Implement a hybrid approach with scheduled batch jobs on custom cloud VMs.
- C. Deploy a dedicated, manually managed cluster optimized by in-house IT staff.
- D. Utilize Databricks serverless compute that automatically optimizes resources and abstracts cluster management.

Answer: D

Question: 192

Deep Dumps

A data engineer wants to reduce costs and optimize cloud spending. The data engineer has decided to use Databricks Serverless for lowering cloud costs while maintaining existing SLAs.

What is the first step in migrating to Databricks Serverless?

- A. Legacy Ingestion pipelines that include ingestion from sources API's, files, JDBC/ODBC connections
- B. A frequently running and efficient Python-based data transformation pipeline compatible with the latest Databricks runtime and Unity Catalog
- C. A frequently running and efficient Scala-based data transformation pipeline compatible with the latest Databricks runtime and Unity Catalog
- D. Low frequency BI Dashboarding and Adhoc SQL Analytics

Answer: B

Question: 193

Deep Dumps

A data engineering project involves processing large batches of data on a daily schedule using ETL. The jobs are resource-intensive and vary in size, requiring a scalable, cost-efficient compute solution that can automatically scale based on the workload.

Which compute approach will satisfy the needs described?

- A. Job Cluster
- B. Dedicated Cluster

- C. All-Purpose Cluster
- D. Databricks SQL Serverless

Answer: A

Question: 194

Deep Dumps

A data engineer is configuring Delta Sharing for a multi-team project where teams from different departments will need to access shared data. The data engineer has successfully created a Unity Catalog metastore and is now setting up the Delta Share. The goal is to ensure that internal teams can access the shared data with full permissions, while external partners can only read the data.

Which action should the Data Engineer take to configure the sharing correctly?

- A. Create a Delta Share, set up a secure access URL for internal teams and external partners, and distribute the URL to provide them access to the shared data.
- B. Create a Delta Share, add the internal team's tables and views, and assign READ/WRITE permissions to both external partners and internal teams.
- C. Assign READ permissions to external partners through the Delta Share and READ/WRITE permissions to internal teams, and ensure the correct tables and views are shared.
- D. Grant READ permissions to external partners and READ/WRITE permissions to internal teams through the Delta Share.

Answer: C

Question: 195

Deep Dumps

A data engineer manages multiple external tables linked to various data sources. The data engineer wants to manage these external tables efficiently and ensure that only the necessary permissions are granted to users for accessing specific external tables.

How should the data engineer manage access to these external tables?

- A. Set up Azure Blob Storage permissions at the container level, allowing access to all external tables.
- B. Create a single user role with full access to all external tables and assign it to all users.
- C. Grant permissions on the Databricks workspace level, which will automatically apply to all external tables.
- D. Use Unity Catalog to manage access controls and permissions for each external table individually.

Answer: D

Question: 196

Deep Dumps

An organization plans to share a large dataset stored in a Databricks workspace on AWS with a partner organization whose Databricks workspace is hosted on Azure. The data engineer wants to minimize data transfer costs while ensuring secure and efficient data sharing.

Which strategy will reduce data egress costs associated with cross-cloud data sharing?

- A. Migrating the dataset to Cloudflare R2 object storage before sharing
- B. Configure VPN connection between AWS and Azure for faster data sharing
- C. Using Delta Sharing without any additional configurations

D. Sharing data via pre-signed URLs without monitoring egress costs

Answer: C

Question: 197

Deep Dumps

A data engineer is setting up access control in Unity Catalog and needs to ensure that a group of data analysts can query tables but not modify data.

Which permission should the data engineer grant to the data analysts?

- A. ALL PRIVILEGES
- B. MODIFY
- C. SELECT
- D. INSERT

Answer: C

Question: 198

Deep Dumps

What Databricks feature can be used to check the data sources and tables used in a workspace?

- A. Use the lineage feature to visualize a graph that highlights where the table is used only in reports.
- B. Use the lineage feature to visualize a graph that highlights where the table is used only in notebooks.
- C. Use the lineage feature to visualize a graph that shows all dependencies, including where the table is used in notebooks, other tables, and reports.
- D. Do not use the lineage feature as it only tracks activity from the last 3 months and will not provide full details on dependencies.

Answer: C

Question: 199

Deep Dumps

A data engineer is getting a partner organization up to speed with Databricks account. Both the teams share some business use cases. The data engineer has to share some of your Unity-Catalog managed delta tables and the notebook jobs creating those tables with the partner organization.

How can the data engineer seamlessly share the required information?

- A. Zip all the code and share via email and allow data ingestion from your data lake
- B. Share required datasets and notebooks via Delta Sharing. Manage permissions via Unity Catalog.
- C. Data and Notebooks can be shared simply using Unity Catalog.
- D. Share access to codebase via Github and allow them to ingest datasets from your Datalake.

Answer: B

Question: 200

Deep Dumps

A data engineer needs to process SQL queries on a large dataset with fluctuating workloads. The workload requires automatic scaling based on the volume of queries, without the need to manage or provision infrastructure. The solution should be cost-efficient and charge only for the compute resources used during query execution. Which compute option should the data engineer use?

- A. Databricks Runtime for ML
- B. Databricks SQL Analytics
- C. Serverless SQL Warehouse
- D. Databricks Jobs

Answer: C

Explanation:

Correct Answer

Option C: Serverless SQL Warehouse

Explanation of the Correct Answer

Serverless SQL Warehouse is the correct choice because it is specifically designed for SQL query processing, offers automatic scaling based on workload demand, requires no infrastructure management, and uses a pay-per-second billing model, charging only during query execution. This aligns perfectly with the problem's requirements for processing SQL queries on a large dataset with fluctuating workloads in a cost-efficient, serverless manner.

Analysis of Each Option

Option A: Databricks Runtime for ML is incorrect because it is designed for machine learning workloads, requires cluster management, and is not optimized for SQL query processing.

Option B: Databricks SQL Analytics is incorrect because, while it handles SQL queries, it typically requires some infrastructure management and may not provide the same level of automatic scaling as Serverless SQL Warehouse.

Option C: Serverless SQL Warehouse is correct because it is

specifically designed for SQL query processing, provides automatic scaling, requires zero infrastructure management, and offers a pay-per-second billing model.

Option D: Databricks Jobs is incorrect because it is primarily for scheduling and running data processing jobs, requires job configuration and cluster management, and is not specifically optimized for interactive SQL queries.

Key Takeaways

The key takeaway is to understand the different compute options available in Databricks and their specific use cases. Serverless SQL Warehouse is ideal for SQL workloads that require automatic scaling and minimal infrastructure management.

A common mistake is to choose a compute option that requires manual infrastructure management or is not optimized for SQL query processing.

When faced with similar questions, carefully analyze the requirements and choose the compute option that best aligns with those requirements, considering factors such as automatic scaling, infrastructure management, and cost efficiency.

Question: 201

Deep Dumps

How does Unity Catalog's data lineage feature support the visualization of relationships between Delta tables, notebooks, jobs, and dashboards?

- A. Unity Catalog lineage visualizes dependencies between Delta tables, notebooks, and jobs, but does not provide column-level tracing or relationships with dashboards.
- B. Unity Catalog provides an interactive graph that visualizes the dependencies between Delta tables, notebooks, jobs, and dashboards, while also supporting column-level tracking of data transformations.
- C. Unity Catalog lineage only supports visualizing relationships at the table level and does not extend to notebooks, jobs, or dashboards.
- D. Unity Catalog lineage provides an interactive graph that tracks dependencies between tables and notebooks but excludes any job-related dependencies or dashboard visualizations.

Answer: B

Explanation:

Correct Answer

Option B: Unity Catalog provides an interactive graph that visualizes the dependencies between Delta tables, notebooks, jobs, and dashboards, while also supporting column-level tracking of data transformations.

Explanation of the Correct Answer

Option B is correct because Unity Catalog provides a comprehensive data lineage solution. It visualizes dependencies between Delta tables, notebooks, jobs, and dashboards using an interactive graph. It also supports column-level tracking of data transformations, providing a detailed view of how data flows and is modified across the Databricks environment.

Analysis of Each Option

Option A: This option is incorrect because Unity Catalog does not provide column-level tracing and relationships with dashboards.

Option B: This option is correct because it accurately describes the capabilities of Unity Catalog's data lineage feature, including the interactive graph, support for various components, and column-level tracking.

Option C: This option is incorrect because Unity Catalog supports visualizing relationships beyond just the table level, extending to notebooks, jobs, and dashboards.

Option D: This option is incorrect because Unity Catalog does not include job-related dependencies and dashboard visualizations in its lineage tracking.

Key Takeaways

Unity Catalog's data lineage feature is a powerful tool for understanding data dependencies in Databricks.

It tracks relationships between various components like Delta tables, notebooks, jobs, and dashboards.

Column-level lineage tracking provides detailed insights into data transformations.

A common mistake is underestimating the breadth of Unity Catalog's lineage capabilities.

Question: 202

Deep Dumps

A data engineer is using the Databricks OPTIMIZE command on a Delta table. What happens when OPTIMIZE is run twice on the same table with the same data?

- A. It triggers a full liquid clustering process.
- B. It has no effect because it is idempotent.
- C. It changes the number of tuples per file significantly.
- D. It further reduces file sizes by re-clustering the data.

Answer: B

Explanation:

Correct Answer

Option B: It has no effect because it is idempotent.

Explanation of the Correct Answer

The OPTIMIZE command in Databricks Delta Lake is designed to compact small files into larger, optimally sized files. When OPTIMIZE is run on a table, it analyzes the file structure and only performs compaction if it finds files that need optimization. If the data hasn't changed and OPTIMIZE has already been run, a subsequent run will find that the files are already optimally sized and will therefore do nothing. This is because the OPTIMIZE command is idempotent, meaning that running it multiple times on the same data has the same effect as running it once.

Idempotency is a key concept here. An idempotent operation

produces the same result no matter how many times it's executed, assuming the input data remains the same.

Analysis of Each Option

Option A: Liquid clustering is a separate feature in Databricks and is not triggered by running the OPTIMIZE command multiple times. Therefore, this option is incorrect.

Option B: This option is correct because the OPTIMIZE command is idempotent. If the data hasn't changed since the last time OPTIMIZE was run, running it again will have no effect.

Option C: The number of tuples per file might change during the initial OPTIMIZE run, but if the files are already optimally sized, running OPTIMIZE again will not significantly change the number of tuples per file. Therefore, this option is incorrect.

Option D: If the files are already properly sized, running OPTIMIZE again will not further reduce file sizes. Therefore, this option is incorrect.

Key Takeaways

The OPTIMIZE command is idempotent, meaning that running it multiple times on the same data has the same effect as running it once.

Liquid clustering is a separate feature in Databricks and is not triggered by running the OPTIMIZE command.

Understanding idempotency is important for working with Databricks Delta Lake.

A common mistake is to assume that running OPTIMIZE multiple times will further optimize the table, but this is not the case if the data hasn't changed.

A Databricks workflow fails at the last stage due to an error in a notebook. This workflow runs daily. The data engineer fixes the mistake and wants to rerun the pipeline. This workflow is very costly and time-intensive to run. Which action should the data engineer do in order to minimise downtime and cost?

- A. Repair run
- B. Switch to another cluster
- C. Restart the cluster
- D. Re-run the entire workflow

Answer: A

Explanation:

Correct Answer

Option A: Repair run

Explanation of the Correct Answer

The correct answer is Repair run because it allows you to restart the workflow from the point of failure, skipping the successful stages. This saves time and cost, which is important since the workflow is costly and time-intensive.

Repair run is designed to resume execution from the point of failure, maintaining the state of successful upstream tasks.

Analysis of Each Option

Option A: This option is correct because it restarts the workflow from the failed task onwards, skipping previously successful stages and preserving intermediate data.

Option B: This option is incorrect because switching to another cluster would still require running the entire workflow from the beginning and doesn't address avoiding redundant computation.

Option C: This option is incorrect because restarting the cluster forces a complete re-execution of the entire workflow, losing all

intermediate results and progress.

Option D: This option is incorrect because re-running the entire workflow repeats all previously successful stages unnecessarily, leading to the highest cost and longest execution time.

Key Takeaways

The key takeaway is that Databricks has a "Repair run" feature specifically designed to efficiently rerun workflows that have failed, minimizing downtime and cost.

A common mistake is to rerun the entire workflow, which wastes resources and time.

When a workflow fails due to an error that has been fixed, always consider using the "Repair run" option to resume from the point of failure.

Question: 204

Deep Dumps

A data engineer wants to create an external table in Databricks that references data stored in an Azure Data Lake Storage (ADLS) location. The goal is to enable Databricks to access and query this external data without moving it into the Databricks-managed storage. Which step should the data engineer take to successfully create the external table?

- A. Use the CREATE MANAGED TABLE statement and specify the LOCATION clause with the path to the external data.
- B. CREATE UNMANAGED TABLE statement without specifying a LOCATION clause.
- C. CREATE EXTERNAL TABLE statement without specifying a LOCATION clause
- D. Use the CREATE TABLE statement and specify the LOCATION clause with the path to the external data.

Answer: D

Explanation:

Correct Answer

Option D: Use the CREATE TABLE statement and specify the LOCATION clause with the path to the external data.

Explanation of the Correct Answer

The correct way to create an external table in Databricks that references data in Azure Data Lake Storage (ADLS) is to use the `CREATE TABLE` statement along with the `LOCATION` clause. The `LOCATION` clause specifies the path to the external data, allowing Databricks to access and query the data without moving it into Databricks-managed storage.

The key concept here is that specifying the `LOCATION` in the `CREATE TABLE` statement tells Databricks that the table is external, meaning Databricks manages the metadata, but the data resides in the specified external location.

Example:

```
CREATE TABLE my_external_table (  
  column1 INT,  
  column2 STRING  
)  
  
USING DELTA  
  
LOCATION  
'abfss://container@storageaccount.dfs.core.windows.net/path/to/data';
```

Analysis of Each Option

Option A: Incorrect. `CREATE MANAGED TABLE` is not the correct syntax, and managed tables store data within Databricks' managed storage, which contradicts the requirement of accessing external data.

Option B: Incorrect. `CREATE UNMANAGED TABLE` is not standard Databricks SQL syntax. Also, without a `LOCATION` clause, Databricks wouldn't know where to find the external data.

Option C: Incorrect. While `CREATE EXTERNAL TABLE` might be used

in other systems, it's not the standard Databricks syntax. Furthermore, omitting the LOCATION clause means Databricks wouldn't know the path to the external data.

Option D: Correct. CREATE TABLE with the LOCATION clause is the standard Databricks syntax for creating external tables. The LOCATION clause specifies the path to the external data in ADLS.

Key Takeaways

In Databricks, external tables are created using the CREATE TABLE statement combined with the LOCATION clause.

The LOCATION clause is essential for specifying the path to the external data source.

Avoid using CREATE MANAGED TABLE when you want to access data in an external storage location without moving it into Databricks-managed storage.

Remember that Databricks manages the metadata for external tables, but the data itself remains in the external storage location.

Question: 205

Deep Dumps

A data engineer needs to conduct Exploratory Analysis on data residing in a database that is within the company's custom-defined network in the cloud. The data engineer is using SQL for this task. Which type of SQL Warehouse will enable the data engineer to process large numbers of queries quickly and cost-effectively?

- A. Classic SQL Warehouse
- B. Pro SQL Warehouse
- C. Serverless SQL Warehouse
- D. Serverless compute for notebooks

Answer: A

Explanation:

Correct Answer

Option A: Classic SQL Warehouse

Explanation of the Correct Answer

The question asks for the best SQL Warehouse type for exploratory data analysis on data residing in a company's custom-defined network. The data engineer needs to process a large number of queries quickly and cost-effectively using SQL.

Option A, Classic SQL Warehouse, is the correct choice because it provides dedicated compute resources within the customer's VPC/private network. This allows direct access to the data. It's also cost-effective for high-volume query processing, which is needed for exploratory analysis.

Analysis of Each Option

Option A: Classic SQL Warehouse is correct because it runs within the customer's network, allowing access to the data. It's also cost-effective for processing a large number of queries.

Option B: Pro SQL Warehouse is an enhanced version of Classic but is more expensive. It might be overkill for exploratory analysis where cost-effectiveness is a key requirement.

Option C: Serverless SQL Warehouse runs in the cloud provider's network and cannot directly access data in custom-defined private networks. This makes it unsuitable for the scenario.

Option D: Serverless compute for notebooks is designed for notebook-based analytics, not SQL warehousing, so it's not the right tool for SQL-based exploratory analysis.

Key Takeaways

When data resides in a private network, using a Classic SQL Warehouse is often the most straightforward and cost-effective solution for SQL-based analysis.

Serverless SQL Warehouses are not suitable for accessing data in private networks without additional network configurations.

Consider the cost implications of different SQL Warehouse types, especially when dealing with a large number of queries.

Question: 206

Deep Dumps

A data engineer is processing ingested streaming tables and needs to filter out NULL values in the order_datetime column from the raw streaming table orders_raw and store the results in a new table orders_valid using DLT. Which code snippet should the data engineer use?

- A. `SELECT *CREATE OR REPLACE STREAMING TABLE orders_validSELECT *FROM orders_rawWHERE order_datetime IS NOT NULL`
- B. `CREATE OR REPLACE STREAMING TABLE orders_validCONSTRAINT valid_dateORDER_DATETIME IS NOT NULLON VIOLATION DROP ROWAS SELECT * FROM orders_raw`
- C. `SELECTCREATE OR REPLACE STREAMING TABLE orders_validCONSTRAINT valid_date EXPECT (order_datetime IS NOT NULL)ON VIOLATION DROP ROWAS SELECT * FROM orders_raw`
- D. `CREATE OR REPLACE STREAMING TABLE orders_validFILTER (order_datetime IS NOT NULL)AS SELECT * FROM orders_raw`

Answer: B

Explanation:

Correct Answer

Option B: `CREATE OR REFRESH STREAMING TABLE orders_valid(`
`CONSTRAINT valid_date`
`EXPECT (order_datetime IS NOT NULL)`
`ON VIOLATION DROP ROW`
`)`
`AS SELECT * FROM STREAM orders_raw;`

Explanation of the Correct Answer

Option B is selected because it uses the EXPECT constraint, which is the recommended DLT (Delta Live Tables) approach for enforcing

data quality, including filtering out NULL values. The ON VIOLATION DROP ROW clause ensures that rows violating the constraint (i.e., having NULL values in order_datetime) are dropped.

The EXPECT constraint is a key feature of DLT for data validation and quality control.

Analysis of Each Option

Option A: Incorrect. It uses the STREAM(orders_raw) syntax, which is not the correct way to reference a streaming table in DLT. In DLT, streaming tables are referenced directly by name.

Option B: Correct. It uses the EXPECT constraint with ON VIOLATION DROP ROW, which is the recommended DLT approach for data quality enforcement, including filtering out NULL values. Although the original thought process identifies a syntax error (FROM STREAM orders_raw), it still favors this option due to the correct usage of the EXPECT constraint.

Key Takeaways

The EXPECT constraint with ON VIOLATION DROP ROW is the preferred method in DLT for data quality enforcement.

When working with DLT, it's important to use the correct syntax for referencing streaming tables (i.e., directly by name).

The EXPECT constraint allows you to define data quality rules and specify actions to take when those rules are violated.

Question: 207

Deep Dumps

Which languages are supported by Serverless compute clusters?
(Choose 2 answers)

- A. Python
- B. SQL
- C. Java
- D. Scala

E. R

Answer: A, B

Explanation:

Correct Answer

A. Python

B. SQL

Explanation of the Correct Answer

The question asks about languages supported by Serverless compute clusters. The thought process identifies SQL as the primary language due to the design for SQL workloads. Python is also supported through UDFs, connectors, and integration with Python-based workflows.

The reasoning highlights that Serverless compute clusters are optimized for SQL analytics with Python integration, unlike traditional clusters that support a wider range of languages.

Analysis of Each Option

Option A: Python is correct because Serverless compute clusters support Python through UDFs, connectors, and integration with Python-based workflows.

Option B: SQL is correct because Serverless compute clusters are primarily designed for SQL workloads and natively support SQL queries.

Option C: Java is incorrect because Serverless compute clusters don't provide native Java runtime support, although Databricks supports Java on traditional clusters.

Option D: Scala is incorrect because Scala is supported on regular Databricks clusters but not on Serverless compute clusters.

Option E: R is incorrect because R is not natively supported in the Serverless compute environment.

Key Takeaways

Serverless compute clusters are optimized for SQL analytics workloads with Python integration.

It's important to distinguish between languages supported on traditional clusters versus Serverless compute clusters.

Serverless clusters prioritize SQL and provide Python integration, while traditional clusters offer broader language support.

Question: 208

Deep Dumps

How does Databricks Connect enable the engineer to develop, test, and debug code seamlessly on their local machine while interacting with Databricks clusters?

A. By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using their preferred IDE

- B. By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using a specific IDE that is required by Databricks
- C. By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code only through Databricks' own web interface
- D. By allowing direct execution of Spark jobs from the local machine without needing a network connection

Answer: A

Explanation:

Correct Answer

Option A: By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using their preferred IDE

Explanation of the Correct Answer

Option A is correct because Databricks Connect allows developers to use their preferred IDE to connect to a remote Databricks cluster. It simulates the Databricks runtime environment locally, which allows for consistent behavior between local development and cluster execution.

Databricks Connect is a client library that bridges the local machine and the Databricks runtime environment.

Analysis of Each Option

Option A: Correct. This option accurately describes the functionality of Databricks Connect.

Option B: Incorrect. Databricks Connect does not require a specific IDE. It supports various IDEs.

Option C: Incorrect. Databricks Connect enables local development, not just development through the Databricks web interface.

Option D: Incorrect. Databricks Connect requires a network connection to communicate with the remote Databricks cluster.

Key Takeaways

Databricks Connect allows developers to use their local IDE to

develop, test, and debug code that runs on a remote Databricks cluster.

It requires a network connection to the Databricks cluster.

It does not restrict developers to a specific IDE.

Question: 209

Deep Dumps

A data engineer is working on a Databricks project that utilizes cloud storage. The data engineer wants to load several JSON files from containers on a storage account as soon as the file arrives within the storage account. Which syntax should the data engineer follow to first load the files into a dataframe and check that it is working as expected using Python

- A. `df = spark.readStream.format("cloudFiles") .option("cloudFiles.format", "json") .load("/input/path")`
- B. `df = spark.readStream.format("cloud").option("json").load("/input/path")`
- C. `spark.readStream.format("json").load("input/path")`
- D. `spark.read.json("input/path")`

Answer: A

Explanation:

Correct Answer

Option A: `df = spark.readStream.format("cloudFiles")
.option("cloudFiles.format", "json") .load("/input/path")`

Explanation of the Correct Answer

Option A is correct because it uses `spark.readStream` to create a streaming `DataFrame`, which is necessary for processing data as it arrives. It specifies the `cloudFiles` format, which is designed for efficiently reading files from cloud storage as they appear. The `cloudFiles.format` option is set to `"json"` to indicate the file type, and the `load` function specifies the path to the cloud storage directory.

Analysis of Each Option

Option A: Correct. This option correctly uses `spark.readStream` with the `cloudFiles` format and specifies the JSON format.

Option B: Incorrect. The `format("cloud")` is not a valid format option for streaming data from cloud storage. It also lacks the specific `cloudFiles` source needed for efficient file discovery, and the option `option("json")` is incomplete.

Option C: Incorrect. While this reads JSON files in streaming, it doesn't leverage the `cloudFiles` source. Without `cloudFiles`, Spark will not automatically discover new files as they arrive in cloud storage in an efficient and scalable manner.

Option D: Incorrect. This uses `spark.read`, which is for reading a static dataset, not a streaming one. It won't process new files as they arrive.

Key Takeaways

The `spark.readStream` function is essential for creating streaming DataFrames in Spark.

The `cloudFiles` format is designed for efficiently reading files from cloud storage as they appear.

When working with streaming data, it's important to use the appropriate functions and formats to ensure that new data is processed as it arrives.

Avoid using `spark.read` for streaming data, as it is intended for reading static datasets.

Question: 210

Deep Dumps

A data engineer is developing an ETL process based on Spark SQL. The execution fails. The data engineer checks the Spark UI and can see the ERRORS as follows: "java.lang.OutOfMemoryError: Java heap space" Which two corrective actions should the data engineer perform

to resolve this issue? [Choose 2 answers]

- A. Narrow the filters in order to collect less data in the query
- B. Upsize the worker nodes and activate autoshuffle partitions
- C. Upsize the driver node and deactivate autoshuffle partitions
- D. Cache the dataset in order to boost the query performance
- E. Fix the shuffle partitions to 50 to ensure the allocation

Answer: A,B

Explanation:

Correct Answer

Options A and B are correct.

A: Narrow the filters in order to collect less data in the query

B: Upsize the worker nodes and activate autoshuffle partitions

Explanation of the Correct Answer

The error "java.lang.OutOfMemoryError: Java heap space" indicates that the driver or worker nodes are running out of memory during the ETL process.

Option A is correct because reducing the amount of data processed by applying more selective filters early in the query reduces memory pressure on both driver and worker nodes.

Option B is correct because increasing worker node memory provides more heap space to handle data processing. Activating autoshuffle partitions helps Spark automatically optimize partition sizes based on data statistics, which can prevent memory issues caused by data skew and improve memory utilization.

Analysis of Each Option

Option A: Correct. Narrowing filters reduces the amount of data processed, alleviating memory pressure.

Option B: Correct. Upsizing worker nodes increases available memory, and autoshuffle partitions optimize memory utilization.

Option C: Incorrect. Deactivating autoshuffle partitions removes Spark's ability to dynamically optimize partitioning, potentially worsening memory issues.

Option D: Incorrect. Caching increases memory usage, exacerbating the OutOfMemoryError.

Option E: Incorrect. Setting a fixed low number of partitions can create larger partitions that consume more memory per partition, potentially worsening the memory issue.

Key Takeaways

The "java.lang.OutOfMemoryError: Java heap space" error in Spark indicates memory exhaustion on the driver or worker nodes.

Reducing data volume and increasing available memory are effective strategies for resolving this issue.

Avoid caching data when facing OutOfMemoryError, as it will worsen the problem.

Adaptive query execution (autoshuffle partitions) can help optimize memory utilization.

Question: 211

Deep Dumps

An organization has implemented a data pipeline in Databricks and needs to ensure it can scale automatically based on varying workloads without manual cluster management. The goal is to meet the company's Service Level Agreements (SLAs), which require high availability and minimal downtime, while Databricks automatically handles resource allocation and optimization. Which approach fulfills these requirements?

- A. Use Spot Instances to allocate resources dynamically while minimizing costs, with potential interruptions.
- B. Use Serverless compute in Databricks to automatically scale and provision resources with minimal manual intervention.
- C. Use Interactive Clusters in Databricks, adjusting cluster sizes manually based on workload demands.
- D. Deploy Job Clusters with fixed configurations, dedicated to specific tasks, without automatic scaling.

Answer: B

Explanation:

Correct Answer

Option B: Use Serverless compute in Databricks to automatically scale and provision resources with minimal manual intervention.

Explanation of the Correct Answer

Serverless compute in Databricks automatically scales resources based on workload demands without manual intervention.

Databricks handles resource provisioning and optimization, ensuring high availability with automatic failover and recovery, and minimal downtime. This approach perfectly aligns with the requirements of automatic scaling, no manual cluster management, high availability, and automatic resource allocation.

Analysis of Each Option

Option A: Incorrect. Spot Instances can be interrupted, conflicting with the SLA requirements for high availability and minimal downtime.

Option B: Correct. Serverless compute automatically scales resources, requires zero manual cluster management, and ensures high availability with minimal downtime.

Option C: Incorrect. Interactive Clusters require manual adjustment of cluster sizes, contradicting the requirement for no manual cluster management.

Option D: Incorrect. Job Clusters with fixed configurations do not provide automatic scaling, failing to adapt to varying workloads.

Key Takeaways

Serverless compute is ideal for scenarios requiring automatic scaling, minimal manual intervention, and high availability.

Spot Instances are cost-effective but unsuitable for SLA-critical workloads due to potential interruptions.

Manual cluster management is inefficient and error-prone in dynamic environments.

Fixed configurations are not adaptable to varying workloads.

Question: 212

Deep Dumps

A data engineer is configuring Unity Catalog in Databricks and needs to assign a role to a user who should have the ability to grant and revoke privileges on various data objects within a specific schema, but should not have read/write access over the schema or its objects. Which role should the data engineer assign to this user?

- A. Catalog Owner
- B. Schema Owner
- C. USE catalog/schema privilege on the schema
- D. Table Owner

Answer: C

Explanation:

Correct Answer

Option C: USE catalog/schema privilege on the schema

Explanation of the Correct Answer

The USE catalog/schema privilege is the correct answer because it allows a user to access the metadata of a catalog or schema, which is necessary to grant and revoke privileges on objects within that schema.

This privilege does not grant read/write access to the actual data, fulfilling the requirement that the user should not have data access.

The USE catalog/schema privilege enables administrative tasks related to permissions without providing data access.

Analysis of Each Option

Option A: Catalog Owner is incorrect because it grants full administrative rights over the entire catalog, including read/write access, which violates the requirement of no read/write access.

Option B: Schema Owner is incorrect because it provides full control over the schema, including read/write access to all objects, which violates the requirement of no read/write access.

Option C: USE catalog/schema privilege on the schema is correct because it allows a user to manage privileges on objects within the schema without granting read/write access to the data.

Option D: Table Owner is incorrect because it is specific to individual tables and does not provide schema-wide privilege management capabilities.

Key Takeaways

The USE catalog/schema privilege is designed for administrative users who need to manage permissions without accessing the underlying data.

Understanding the specific privileges associated with different roles is crucial for configuring Unity Catalog correctly.

When assigning roles, always consider the principle of least privilege to ensure users only have the necessary permissions.

Question: 213

Deep Dumps

A data engineer streams customer orders into a Kafka topic (orders_topic) and is currently writing the ingestion script of a DLT pipeline. The data engineer needs to ingest the data from Kafka brokers to DLT using Databricks. What is the correct code for ingesting the data?

```
A. import dlt@dlt.table(name = "orders_raw"),def orders_raw():return
(spark.readstream.format("cloudFiles").option("cloudFiles.format", "json").option("cloudFiles.schemaLocation",
```

```
"/schema/location").load("kafka://broker:9092/orders_topic"))
```

```
B. import dlt@dlt.table(name = "orders_raw"),def orders_raw():return  
(spark.readstream.format("kafka").option("kafka.bootstrap.servers", "broker:9092").option("subscribe",  
"orders_topic").load())
```

```
C. import dlt@dlt.table(name = "orders_raw")def orders_raw():return  
(spark.readstream.format("cloudFiles").option("cloudFiles.format",  
"json").load("kafka://broker:9092/orders_topic"))
```

```
D. import dlt@dlt.table(name = "orders_raw")def orders_raw():return  
(spark.readstream.format("kafka").option("kafka.bootstrap.servers", "broker:9092").option("subscribe",  
"orders_topic").option("startingOffsets", "earliest").load())
```

Answer: D

Explanation:

Correct Answer

Option D.

```
import dlt
```

```
@dlt.table(name = "orders_raw")
```

```
def orders_raw():
```

```
return (
```

```
spark.readstream
```

```
.format("kafka")
```

```
.option("kafka.bootstrap.servers", "broker:9092")
```

```
.option("subscribe", "orders_topic")
```

```
.option("startingOffsets", "earliest")
```

```
.load()
```

```
)
```

Explanation of the Correct Answer

Option D is correct because it uses the kafka format to read from Kafka, specifies the bootstrap servers, subscribes to the correct topic, and includes the startingOffsets option, which is important for controlling where to start reading from the Kafka topic. This makes it the most complete and robust implementation for a production

DLT pipeline.

The kafka format is specifically designed for reading data from Kafka streams in Spark. The kafka.bootstrap.servers option tells Spark where to find the Kafka brokers. The subscribe option specifies the Kafka topic to read from. The startingOffsets option allows you to specify where to start reading from the topic (e.g., "earliest" to read from the beginning).

Analysis of Each Option

Option A: Incorrect. It uses the cloudFiles format, which is for reading files from cloud storage, not Kafka streams.

Option B: Correct, but not as complete as Option D. It uses the correct kafka format and specifies the bootstrap servers and topic, but it doesn't include the startingOffsets option.

Option C: Incorrect. It uses the cloudFiles format, which is not appropriate for reading from Kafka.

Option D: Correct. It uses the correct kafka format, specifies the bootstrap servers and topic, and includes the startingOffsets option.

Key Takeaways

When reading from Kafka in Spark, use the kafka format.

Always specify the kafka.bootstrap.servers option to tell Spark where to find the Kafka brokers.

Use the subscribe option to specify the Kafka topic to read from.

Consider using the startingOffsets option to control where to start reading from the topic.

Avoid using the cloudFiles format for reading from Kafka streams.

A data engineer is inspecting an ETL pipeline based on a Pyspark job that consistently encounters performance bottlenecks. Based on developer feedback, the data engineer assumes the job is low on compute resources. To pinpoint the issue, the data engineer observes the Spark UI and finds out the job has a high CPU time vs Task time. Which course of action should the data engineer take?

- A. High CPU time vs Task time means an under-utilized cluster. The data engineer may need to repartition data to spread the jobs more evenly throughout the cluster.
- B. High CPU time vs Task time means efficient use of cluster and no change needed
- C. High CPU time vs Task time means over-utilized memory and the need to increase parallelism
- D. High CPU time vs Task time means a CPU over-utilized job. The data engineer may need to consider executor and core tuning or resizing the cluster

Answer: D

Explanation:

Correct Answer

Option D: High CPU time vs Task time means a CPU over-utilized job. The data engineer may need to consider executor and core tuning or resizing the cluster

Explanation of the Correct Answer

The problem describes a Spark job with a high CPU time relative to task time. This means the CPU is heavily utilized and is the bottleneck. The correct action is to address the CPU over-utilization by tuning executors/cores or resizing the cluster.

High CPU time vs Task time indicates that the majority of the time spent on a task is dedicated to CPU computation, with minimal time spent waiting for I/O or other resources. This signifies a CPU-bound workload.

The thought process correctly identifies that the high CPU time relative to task time means the CPUs are working at or near capacity, and the bottleneck is computational power.

Analysis of Each Option

Option A: Incorrect. High CPU time vs Task time indicates the CPUs

are heavily utilized, not under-utilized. Under-utilization would show low CPU time relative to task time.

Option B: Incorrect. While high CPU utilization might seem efficient, it can still cause performance bottlenecks if the workload exceeds CPU capacity.

Option C: Incorrect. This ratio doesn't directly indicate memory over-utilization. Memory issues would typically manifest differently in Spark metrics.

Option D: Correct. High CPU time vs Task time indicates a CPU-bound workload where CPUs are working at or near capacity. Solutions include adding more CPU cores per executor, increasing the number of executors, or scaling up the cluster.

Key Takeaways

High CPU time vs Task time indicates a CPU-bound workload.

Consider executor and core tuning or resizing the cluster to address CPU over-utilization.

Avoid confusing CPU-bound issues with memory or I/O-related bottlenecks.

Question: 215

Deep Dumps

Which of the following describes the storage organization of a Delta table?

- A. Delta tables are stored in a single file that contains data, history, metadata, and other attributes.
- B. Delta tables store their data in a single file and all metadata in a collection of files in a separate location.
- C. Delta tables are stored in a collection of files that contain data, history, metadata, and other attributes.
- D. Delta tables are stored in a collection of files that contain only the data stored within the table.
- E. Delta tables are stored in a single file that contains only the data stored within the table.

Answer: C

Explanation:

Correct Answer: C

Explanation:

The correct answer is C because Delta Lake, the technology behind Delta tables, is built on top of cloud storage like AWS S3, Azure Blob Storage, or Google Cloud Storage. It leverages the distributed nature of these object storage systems for scalability and reliability.

Delta Lake doesn't store all data in a single file. Instead, data is partitioned into multiple data files (typically Parquet format), and these files are stored in the underlying cloud storage. This allows for parallel processing and efficient querying.

The "history" and "metadata" are critical components of Delta Lake's functionality, enabling features like time travel, ACID transactions, and schema evolution. This metadata isn't stored within each data file but rather as a transaction log alongside the data files. The transaction log is an ordered record of every operation performed on the Delta table.

Specifically, each commit to a Delta table creates a new entry (a metadata file) in the `_delta_log` directory, which is typically located alongside the data files in the same storage location. This log tracks changes to the data, including additions, deletions, and updates. These metadata files (often stored in JSON format) contain information about the data files involved in each transaction, enabling Delta Lake to track the table's state over time.

Therefore, a Delta table isn't a single file. It's a directory containing data files and a `_delta_log` directory containing metadata files. Answer C accurately reflects this distributed storage model, encompassing both the data and the necessary metadata for transaction management and data versioning. The distributed storage enables efficient processing and scaling provided by the underlying cloud storage layer. Options A, B, D, and E are incorrect because they imply a single-file storage or an incomplete view of how metadata is managed within a Delta table.

A data engineer is writing a script that is meant to ingest new data from cloud storage. In the event of the Schema change, the ingestion should fail. It should fail until the changes downstream source can be found and verified as intended changes. Which command will meet the requirements?

- A. addNewColumns
- B. none
- C. rescue
- D. failonNewColumns

Answer: D

Explanation:

Correct Answer

Option D: failonNewColumns

Explanation of the Correct Answer

The question requires a command that will cause the data ingestion process to fail when schema changes (specifically new columns) are detected. This is to ensure that any unintended schema changes are caught and verified before being ingested. The option failonNewColumns directly addresses this requirement by causing the process to fail when new columns are detected.

Analysis of Each Option

Option A: addNewColumns - This option would automatically add new columns, which is the opposite of the desired behavior. Therefore, it is incorrect.

Option B: none - This is not a valid command and does not provide the required functionality. Therefore, it is incorrect.

Option C: rescue - This option is used for error handling and recovery, which would prevent the process from failing. Therefore, it

is incorrect.

Option D: failonNewColumns - This option explicitly causes the process to fail when new columns are detected, which meets the requirement. Therefore, it is correct.

Key Takeaways

When dealing with schema changes in data ingestion, it's important to have mechanisms to handle unexpected changes.

The failonNewColumns command (or similar functionality) can be used to ensure that any schema changes are manually verified before being ingested.

Avoid using commands that automatically add new columns without verification, as this can lead to data quality issues.

Question: 217

Deep Dumps

A data engineer has been provided a PySpark DataFrame named **df** with columns product and revenue. The data engineer needs to compute complex aggregations to determine each product's total revenue, average revenue, and transaction count. Which code snippet should the data engineer use?

- A. `from pyspark.sql import functions as F`
`aggregated_df = df.groupBy("product").agg(F.sum("revenue").alias("total_revenue"), F.avg("revenue").alias("avg_revenue"), F.count("*").alias("transaction_count"))`
- B. `aggregated_df = df.groupby("product").agg("sum(revenue)", "avg(revenue)", "count(revenue)")`
- C. `from pyspark.sql import functions as F`
`aggregated_df = df.select("product", "revenue").groupBy("product").agg(F.sum("revenue"), F.mean("revenue"))`
- D. `aggregated_df = df.groupBy("product").agg("revenue": "sum", "revenue": "avg", "revenue": "count")`

Answer: A

Explanation:

Correct Answer

Option A: `from pyspark.sql import functions as F`
`aggregated_df =`

```
df.groupBy("product").agg(F.sum("revenue").alias("total_revenue"),  
F.avg("revenue").alias("avg_revenue"),  
F.count("*").alias("transaction_count"))
```

Explanation of the Correct Answer

Option A is correct because it uses the `pyspark.sql.functions` module to perform the required aggregations: sum, average, and count. It groups the DataFrame by the "product" column and then calculates the sum of "revenue" (aliased as "total_revenue"), the average of "revenue" (aliased as "avg_revenue"), and the count of all rows in each group (aliased as "transaction_count"). The use of aliases makes the resulting DataFrame more readable.

The `pyspark.sql.functions` module provides functions for performing common data manipulation tasks, including aggregations. The `groupBy` method is used to group rows based on one or more columns, and the `agg` method is used to apply aggregation functions to each group. The `alias` method is used to rename columns in the resulting DataFrame.

Analysis of Each Option

Option A: Correct. It correctly calculates the total revenue, average revenue, and transaction count for each product using PySpark functions and assigns appropriate aliases.

Option B: Incorrect. It uses string-based aggregation, which is less readable and doesn't provide column aliases. Also, `count(revenue)` will only count non-null values of revenue, not the total number of transactions.

Option C: Incorrect. It only calculates the sum and mean of revenue, missing the transaction count. It also includes an unnecessary `select` operation and lacks column aliases.

Option D: Incorrect. Due to the way Python dictionaries work, only the last aggregation (count) would be retained, and the sum and

average aggregations would be lost.

Key Takeaways

When performing aggregations in PySpark, it's best practice to use the functions provided in the `pyspark.sql.functions` module.

Always use aliases to give meaningful names to the resulting aggregation columns.

Be careful when using dictionaries for aggregations, as duplicate keys can lead to unexpected results.

`F.count("*")` is used to count all rows in a group, while `F.count("column_name")` counts only the non-null values in the specified column.

Question: 218

Deep Dumps

A company is collaborating with a partner that does not use Databricks but needs access to a large historical dataset stored in Delta format. The data engineer needs to ensure that the partner can access the data securely, without the need for them to set up an account, and with read-only access. How should the data be shared?

- A. Share the dataset using Unity Catalog, ensuring that both teams have full write access to the data within the same organization.
- B. Share the dataset by exporting it to a CSV file and manually transferring the file to the partner's system.
- C. Share the dataset using Delta Sharing, which allows your partner to access the data using a secure, read-only URL without requiring a Databricks account, ensuring that they cannot modify the data.
- D. Grant your partner access to your Databricks workspace and assign them full write permissions to the Delta table, enabling them to modify the dataset.

Answer: C

Explanation:

Correct Answer

Option C: Share the dataset using Delta Sharing, which allows your partner to access the data using a secure, read-only URL without requiring a Databricks account, ensuring that they cannot modify the data.

Explanation of the Correct Answer

Option C is correct because Delta Sharing is designed for securely sharing Delta tables with external partners who don't use Databricks. It provides read-only access without requiring the partner to have a Databricks account. This aligns perfectly with the requirements of the problem.

Delta Sharing allows sharing data in Delta format while ensuring the partner can only read the data, maintaining security and preventing modifications.

Analysis of Each Option

Option A: Incorrect. Unity Catalog sharing typically requires both parties to be within the Databricks ecosystem, and the mention of "full write access" contradicts the read-only requirement.

Option B: Incorrect. Exporting to CSV loses the benefits of the Delta format and is impractical for large datasets. Manual transfer introduces security and scalability issues.

Option C: Correct. Delta Sharing is designed for sharing Delta tables with external partners who don't use Databricks, providing secure, read-only access without requiring account creation.

Option D: Incorrect. Granting workspace access and write permissions contradicts the requirements for no account setup and read-only access.

Key Takeaways

Delta Sharing is the preferred method for sharing Delta tables with external partners who do not have Databricks accounts.

Always consider the security implications and access requirements when sharing data.

Avoid granting write access unless explicitly required.

Which SQL code snippet will correctly demonstrate a Data Definition Language (DDL) operation used to create a table?

- A. `CREATE TABLE employees (id INT,name STRING);`
- B. `INSERT INTO employees (id, name) VALUES (1, 'Alice');`
- C. `ALTER TABLE employees ADD COLUMN salary DECIMAL(10,2);`
- D. `DROP TABLE employees;`

Answer: A

Explanation:

Correct Answer

Option A: `CREATE TABLE employees ( id INT, name STRING );`

Explanation of the Correct Answer

Option A is correct because the `CREATE TABLE` statement is a Data Definition Language (DDL) command used to define the structure of a new table. It specifies the table name (`employees`) and the columns (`id`, `name`) along with their respective data types (`INT`, `STRING`).

DDL operations are used to define and manage the structure of database objects.

Analysis of Each Option

Option A: This option is correct. The `CREATE TABLE` statement is the standard SQL command for creating a new table.

Option B: This option is incorrect. The `INSERT INTO` statement is a Data Manipulation Language (DML) command used to add data to an existing table, not to create a table.

Option C: This option is incorrect. The `ALTER TABLE` statement is a DDL command used to modify the structure of an existing table (e.g., adding a column), not to create a new table.

Option D: This option is incorrect. The DROP TABLE statement is a DDL command used to delete an existing table, not to create one.

Key Takeaways

The CREATE TABLE statement is fundamental for defining the structure of tables in a relational database.

DDL commands are distinct from DML commands, which are used for manipulating data within tables.

Understanding the difference between DDL and DML is crucial for database management.

Common mistake: Confusing DDL operations like ALTER TABLE or DROP TABLE with the CREATE TABLE operation.

Question: 220

Deep Dumps

What is the maximum output supported by a job cluster to ensure a notebook does not fail?

- A. 10MBs
- B. 25MBs
- C. 15MBs
- D. 30MBs

Answer: C

Explanation:

Correct Answer

Option C: 15MBs

Explanation of the Correct Answer

The question asks for the maximum output size a job cluster can support without causing a notebook to fail. The thought process identifies that Databricks job clusters have a limit to prevent excessive memory usage. While the ideal limit is 20MB (not in the

options), the closest and safest value among the choices is 15MB. This ensures the notebook stays within acceptable bounds and avoids failure.

Analysis of Each Option

Option A: 10MBs - This option is below the typical limit and safe, but not the maximum safe value among the choices. Therefore, it's not the best answer.

Option B: 25MBs - This option exceeds the typical 20MB limit, making it likely to cause a failure. Thus, it's incorrect.

Option C: 15MBs - This option is below the typical limit and represents the highest safe value among the given choices. Therefore, it's the correct answer.

Option D: 30MBs - This option significantly exceeds the limit, making it very likely to cause a failure. Thus, it's incorrect.

Key Takeaways

Job clusters have output limits to prevent memory issues and ensure stability.

When the exact limit isn't an option, choose the next highest value that is still safely below the known limit.

Always consider the consequences of exceeding the limit (notebook failure) when selecting an option.

Question: 221

Deep Dumps

A data engineer needs to provide access to a group named manufacturing-team. The team needs privileges to create tables in the quality schema. Which set of SQL commands will grant a group named manufacturing-team to create tables in a schema named production with the parent catalog named manufacturing with the least privileges?

A. GRANT USE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON

SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

B. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

C. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

D. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE CATALOG ON CATALOG manufacturing TO manufacturing-team;

Answer: C

Explanation:

Correct Answer

Option C: GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

Explanation of the Correct Answer

Option C is correct because it provides the minimum required privileges for the manufacturing-team to create tables in the quality schema within the manufacturing catalog. The team needs USE CATALOG to access the catalog, USE SCHEMA to access the schema, and CREATE TABLE to create tables within that schema.

Analysis of Each Option

Option A: Incorrect because GRANT USE TABLE ON SCHEMA is incorrect syntax and doesn't grant table creation rights. It is also missing the CREATE TABLE privilege.

Option B: Incorrect because GRANT CREATE SCHEMA ON SCHEMA is unnecessary and grants more privileges than needed.

Option C: Correct because it grants CREATE TABLE on the schema, USE SCHEMA on the schema, and USE CATALOG on the catalog, which are the minimum required privileges.

Option D: Incorrect because GRANT CREATE SCHEMA ON SCHEMA and GRANT CREATE CATALOG ON CATALOG are unnecessary and grant far more privileges than needed.

Key Takeaways

When granting privileges, always aim for the principle of least privilege, granting only the necessary permissions.

Understand the hierarchy of permissions: catalog, schema, and then table-level privileges.

Avoid granting CREATE privileges on catalogs or schemas unless explicitly required.

Question: 222

Deep Dumps

A data engineer at a company that uses Databricks with Unity Catalog needs to share a collection of tables with an external partner who also uses a Databricks workspace enabled for Unity Catalog. The data engineer decides to use Delta Sharing to accomplish this. What is the first piece of information the data engineer should request from the external partner to set up Delta Sharing?

- A. Their Databricks account password
- B. The IP address of their Databricks workspace
- C. The name of their Databricks cluster
- D. The sharing identifier of their Unity Catalog metastore

Answer: D

Explanation:

Correct Answer

Option D: The sharing identifier of their Unity Catalog metastore

Explanation of the Correct Answer

The correct answer is D because Delta Sharing with Unity Catalog requires the sharing identifier of the recipient's Unity Catalog

metastore to establish a secure sharing relationship. This identifier uniquely identifies the metastore to which the data will be shared.

Sharing identifiers are essential for directing the shared data to the correct Unity Catalog metastore.

Analysis of Each Option

Option A: Incorrect. Sharing account passwords would be a severe security breach and is not required for Delta Sharing.

Option B: Incorrect. IP addresses are not used for Delta Sharing setup, as the sharing occurs through secure protocols at the metastore level.

Option C: Incorrect. Cluster names are irrelevant for Delta Sharing, which operates at the metastore level, not the compute level.

Option D: Correct. The sharing identifier of the Unity Catalog metastore is the fundamental piece of information needed to establish a Delta Sharing relationship.

Key Takeaways

Delta Sharing relies on secure identifiers to establish sharing relationships between Unity Catalog metastores.

Never share account passwords or rely on IP addresses for secure data sharing.

Understanding the role of the Unity Catalog metastore and its sharing identifier is crucial for setting up Delta Sharing.

Question: 223

Deep Dumps

A company uses Delta Sharing to collaborate with partners across different cloud providers and geographic regions. What will result in additional costs due to cross-region or egress fees?

A. Transferring data via Delta Sharing across clouds and across different geographic regions

- B. Accessing Delta Sharing data using a VPN within the same data center
- C. Sharing data within the same cloud provider and region
- D. Utilizing Delta Sharing for internal data analytics within a single cloud environment

Answer: A

Explanation:

Correct Answer

Option A: Transferring data via Delta Sharing across clouds and across different geographic regions

Explanation of the Correct Answer

Option A is correct because transferring data across different cloud providers (e.g., from AWS to Azure) incurs egress fees, which are charges for data leaving the cloud provider's network. Additionally, transferring data between different geographic regions incurs cross-region transfer costs. Both of these scenarios lead to additional costs.

Analysis of Each Option

Option A: This option is correct. Transferring data across clouds results in egress fees, and transferring across regions results in cross-region fees.

Option B: This option is incorrect. Accessing data via VPN within the same data center typically does not incur additional transfer fees because the data remains within the same network boundary.

Option C: This option is incorrect. Sharing data within the same cloud provider and region usually has minimal or no cost, as there are no cross-region or egress fees.

Option D: This option is incorrect. Utilizing Delta Sharing for internal analytics within a single cloud environment does not involve data leaving the cloud provider's network, so there are no egress fees or cross-region movement.

Key Takeaways

Cloud providers charge for data transfer in scenarios like cross-region transfers, egress fees (data leaving the cloud), and cross-cloud transfers.

When using Delta Sharing in multi-cloud environments, be aware of egress and cross-region fees, as they can significantly impact costs.

Data transfer within the same cloud provider and region is generally less expensive.

Question: 224

Deep Dumps

A data engineer has developed an ETL that produce a Delta managed table with liquid clustering feature activated as output. Several consumers are having issues regarding time delay when reading this table. How could the Data Engineer be sure about the OPTIMIZE command has been executed explicitly?

- A. Use SHOW TABLES EXTENDED to check the partitions columns used
- B. Use DESCRIBE HISTORY table to check if exists any OPTIMIZE operation
- C. Check the system table system.storage.predictive_optimization_operations_history
- D. Use DESCRIBE DETAIL table to see the file size and number of files for the table

Answer: B

Explanation:

Correct Answer

Option B: Use DESCRIBE HISTORY table to check if exists any OPTIMIZE operation

Explanation of the Correct Answer

The correct way to check if the OPTIMIZE command has been executed on a Delta table is to use DESCRIBE HISTORY table. This command provides a history of all operations performed on the table, including OPTIMIZE operations.

The DESCRIBE HISTORY command shows the operation type,

timestamp, user, and details about the optimization. This allows you to confirm if the OPTIMIZE command was executed explicitly.

Analysis of Each Option

Option A: SHOW TABLES EXTENDED provides metadata about tables but does not show operation history or partition information relevant to OPTIMIZE operations. Therefore, this option is incorrect.

Option B: DESCRIBE HISTORY table shows the complete history of operations performed on a Delta table, including OPTIMIZE operations. This allows you to confirm if the OPTIMIZE command was executed explicitly. Therefore, this option is correct.

Option C: system.storage.predictive_optimization_operations_history is related to auto-optimization features, not explicit OPTIMIZE commands executed manually. Therefore, this option is incorrect.

Option D: DESCRIBE DETAIL shows current table statistics (file count, sizes) but does not indicate whether OPTIMIZE was explicitly executed or when it occurred. Therefore, this option is incorrect.

Key Takeaways

The DESCRIBE HISTORY command is crucial for auditing and understanding the operations performed on a Delta table.

It's important to distinguish between explicit OPTIMIZE commands and auto-optimization features.

When troubleshooting performance issues, checking the table's history can provide valuable insights.

A data engineering team is using Kafka to capture event data and then ingest it into Databricks. The team wants to be able to see these historical events. Medallion architecture is already in place. The team

wants to be mindful of costs. Where should this historical event data be stored?

- A. Gold
- B. Silver
- C. Bronze
- D. Raw layer

Answer: C

Explanation:

Correct Answer

Option C: Bronze

Explanation of the Correct Answer

The Bronze layer is the correct place to store historical event data because it's designed to store raw, unprocessed data exactly as it's ingested. This ensures a complete historical record is maintained with full fidelity. It's also the most cost-effective storage option for long-term retention, serving as the source of truth for all downstream processing.

Analysis of Each Option

Option A: Gold: Incorrect. The Gold layer contains highly aggregated, business-ready datasets and is the most expensive due to extensive processing. It typically contains summarized views, not detailed historical events.

Option B: Silver: Incorrect. The Silver layer contains cleaned, validated, and deduplicated data. It's more expensive than the Bronze layer due to processing overhead and may not retain all historical nuances.

Option C: Bronze: Correct. The Bronze layer stores raw, unprocessed data exactly as ingested, maintaining a complete historical record with full fidelity and cost-effective storage for long-term retention.

Option D: Raw layer: Incorrect. While the term "Raw layer" is similar

in concept to the Bronze layer, the question specifically refers to the Medallion architecture, where the standard terminology is Bronze.

Key Takeaways

The Medallion architecture consists of Bronze, Silver, and Gold layers, each serving a specific purpose in data processing and storage.

The Bronze layer is ideal for storing raw, historical data due to its cost-effectiveness and ability to maintain complete data fidelity.

Understanding the purpose of each layer in the Medallion architecture is crucial for making informed decisions about data storage and processing.

Question: 226

Deep Dumps

How are events formatted in Databricks audit logs?

- A. In Databricks, audit logs output events in a plain text format.
- B. In Databricks, audit logs output events in an XML format.
- C. In Databricks, audit logs output events in a CSV format.
- D. In Databricks, audit logs output events in a JSON format.

Answer: D

Explanation:

Correct Answer

Option D: In Databricks, audit logs output events in a JSON format.

Explanation of the Correct Answer

The correct answer is JSON format because Databricks audit logs need to store structured data, support nested data, and be easily parsed by log analysis tools. JSON fulfills these requirements effectively.

JSON (JavaScript Object Notation) is a standard format for data interchange, widely used in APIs and logging systems due to its

human-readable and machine-parsable nature.

Analysis of Each Option

Option A: Plain text format is incorrect because it cannot efficiently handle structured data with multiple fields and nested information.

Option B: XML format is incorrect because, although structured, it is verbose and less commonly used for modern log analysis systems compared to JSON.

Option C: CSV format is incorrect because it is limited in handling nested data structures and complex event metadata that audit logs typically contain.

Option D: JSON format is correct because it provides structured data representation, supports nested objects, is widely supported by log analysis tools, and is the standard format for modern API responses and structured logging.

Key Takeaways

Databricks audit logs use JSON format for structured, machine-readable logging.

JSON is preferred for its flexibility, ease of integration, and support for complex data structures.

When dealing with structured logging in modern systems, JSON is often the correct choice.

Question: 227

Deep Dumps

An organization is looking for an optimized storage layer that supports ACID transactions and schema enforcement. Which technology should the organization use?

- A. Unity Catalog
- B. Data lake
- C. Delta Lake

Answer: C

Explanation:

Correct Answer

Option C: Delta Lake

Explanation of the Correct Answer

Delta Lake is the correct choice because it's designed to bring reliability to data lakes by adding ACID transactions and schema enforcement. It achieves ACID properties through transaction logs and supports schema enforcement and evolution.

The key concepts here are ACID transactions (Atomicity, Consistency, Isolation, Durability) and schema enforcement, which are crucial for data reliability and consistency. Delta Lake is built to provide these features on top of existing data lakes.

Analysis of Each Option

Option A: Unity Catalog is incorrect because it's a governance layer, not a storage layer. It manages metadata and access control but doesn't provide storage capabilities or ACID transactions.

Option B: Data lake is incorrect because it's a generic term for storing raw data and typically doesn't support ACID transactions or have built-in schema enforcement.

Option C: Delta Lake is correct because it's a storage layer built on top of data lakes that provides ACID transactions through transaction logs and supports schema enforcement and evolution.

Option D: Cloud File Storage is incorrect because it's a basic file storage service without ACID transaction support or schema enforcement capabilities.

Key Takeaways

Delta Lake is designed to address the limitations of traditional data lakes by adding ACID transaction support and schema enforcement.

When a question mentions ACID transactions and schema enforcement, Delta Lake is often the correct answer in the context of data storage solutions.

Remember that Unity Catalog is for governance, not storage.

Question: 228

Deep Dumps

A company sells products across multiple categories (e.g., Electronics, Clothing) and regions. The sales team has provided you with a PySpark dataframe named `sales_df` as below, and the team wants the data engineer to analyze the sales data to help make strategic decisions.

`sales_df`

product_id	category	sales_amount	region
1	Electronics	100	North
2	Clothing	200	South
3	Electronics	150	North
4	Clothing	300	South
5	Electronics	250	East
6	Clothing	400	West

Calculate the total sales amount for each region and store the results in a new dataframe called `region_sales`. Given the total sales amount for each region and store the results in a new dataframe called `region_sales`.

Expected result:

Region	1.2 sales_amount
North	250
South	500
East	250
West	400

Which code will generate the expected result?

- A. `region_sales = sales_df.groupby("region").agg(sum("sales_amount").alias("total_sales_amount"))`
- B. `region_sales = sales_df.agg(sum("sales_amount").alias("total_sales_amount")).groupby("region")`
- C. `region_sales = sales_df.groupby("region").agg(sales("total_sales_amount"))`
- D. `region_sales = sales_df.groupby("region").agg(sales_df("total_sales_amount"))`

Answer: A

Explanation:

Correct Answer

Option A: `region_sales = sales_df.groupby("region").agg(sum("sales_amount").alias("total_sales_amount"))`

Explanation of the Correct Answer

Option A is correct because it first groups the `sales_df` by the "region" column using `groupby("region")`. Then, it calculates the sum of the "sales_amount" for each region using the `agg` function with `sum("sales_amount")`. Finally, it renames the resulting aggregated column to "total_sales_amount" using `.alias("total_sales_amount")`. This produces the desired `region_sales` dataframe with the total sales amount for each region.

Analysis of Each Option

Option A: This option is correct as it first groups the data by region

and then calculates the sum of sales amount for each region, aliasing the resulting column as "total_sales_amount".

Option B: This option is incorrect because it attempts to aggregate the total sales amount across the entire dataset before grouping by region, which is the reverse of what's needed. The agg function should be applied after the groupBy function.

Option C: This option is incorrect because it uses sales("total_sales_amount") within the agg function, which is not a valid PySpark aggregation function. The correct function to use for summing values is sum(). Also, "total_sales_amount" is not a column in the original dataframe.

Option D: This option is incorrect because it attempts to select a column named "total_sales_amount" from the sales_df within the agg function without specifying any aggregation function. This is syntactically incorrect and won't produce the desired result.

Key Takeaways

The correct order of operations in PySpark for calculating aggregates by group is to first group the data using groupBy() and then apply the aggregation function using agg().

Ensure that you use the correct aggregation function (e.g., sum(), avg(), min(), max(), count()) based on the desired calculation.

Always check that the column names used in the aggregation function exist in the original dataframe.

Remember to use .alias() to rename the aggregated column to a more descriptive name.

A global retail company sells products across multiple categories (e.g., Electronics, Clothing) and regions (e.g., North, South, East, West). The

sales team has provided the data engineer with a PySpark dataframe named `sales_df` as below and the team wants the data engineer to analyze the sales data to help them make strategic decisions.

`sales_df`

<code>product_id</code>	<code>category</code>	<code>sales_amount</code>	<code>region</code>
1	Electronics	100	North
2	Clothing	200	South
3	Electronics	150	North
4	Clothing	300	South
5	Electronics	250	East
6	Clothing	400	West

Calculate the total sales amount for each product category and store the results in a new dataframe called `category_sales`.

What will generate the expected result of `category_sales`?

<code>category</code>	<code>total_sales_amount</code>
Electronics	500
Clothing	900

- A. `category_sales = sales_df.groupBy("category").agg(sum("sales_amount").alias("total_sales_amount"))`
- B. `category_sales = sales_df.sum("sales_amount").groupBy("category").alias("total_sales_amount")`
- C. `category_sales = sales_df.agg(sum("sales_amount")).groupBy("category").alias("total_sales_amount")`
- D. `category_sales = sales_df.groupBy("region").agg(sum("sales_amount").alias("total_sales_amount"))`

Answer: A

Explanation:

1. We first group by "category".
2. Then, we use `.agg()` with `sum("sales_amount")` to calculate the total sales amount for each category.
3. `.alias("total_sales_amount")` renames the aggregated column.

Analysis:

Option A

This groups the dataframe by category and then aggregates the sum of `sales_amount`. The alias renames the column to `total_sales_amount`. This gives the expected output with Electronics 500 and Clothing 900. This is correct.

Option B

This first applies sum on the whole dataframe so it gives the total sales across all rows. The category column is lost so grouping by category is not possible anymore. This will not give the correct result.

Option C

This aggregates the sum across the entire dataframe first and then tries to group by category. Since category is lost after the aggregation the grouping will fail. This is incorrect.

Option D

This groups by region instead of category. It will give totals for each region not for each category. This does not match the required output.

Question: 230

Deep Dumps

What is the functionality of AutoLoader in Databricks?

- A. Auto Loader automatically ingests and processes new files from cloud storage, handling batch and streaming data with no support for schema evolution.
- B. Auto Loader automatically ingests and processes new files from cloud storage, handling batch data with support for schema evolution.
- C. Auto Loader automatically ingests and processes new files from cloud storage, handling only streaming data with no support for schema evolution.
- D. Auto Loader automatically ingests and processes new files from cloud storage, handling both batch and streaming data with support for schema evolution.

Answer: D

Explanation:

Correct Answer

Option D: Auto Loader automatically ingests and processes new files from cloud storage, handling both batch and streaming data with support for schema evolution.

Explanation of the Correct Answer

Auto Loader in Databricks is designed to automatically ingest and process new files from cloud storage. It can handle both streaming and batch data. A key feature is its support for schema evolution, which means it can adapt to changes in the structure of the incoming data files over time.

The thought process correctly identifies that Auto Loader supports both streaming and batch data processing and includes schema

evolution capabilities.

Analysis of Each Option

Option A: Incorrect because it states that Auto Loader has "no support for schema evolution," which is false. Auto Loader is designed to handle schema evolution.

Option B: Incorrect because it says Auto Loader handles "batch data" only. Auto Loader handles both batch and streaming data.

Option C: Incorrect because it claims Auto Loader handles "only streaming data with no support for schema evolution." Auto Loader handles both batch and streaming data and supports schema evolution.

Option D: Correct because it accurately describes Auto Loader as handling both batch and streaming data and supporting schema evolution.

Key Takeaways

Auto Loader simplifies data ingestion in Databricks by automatically detecting and processing new files.

Understanding that Auto Loader supports both streaming and batch data, along with schema evolution, is crucial.

A common mistake is to overlook the schema evolution feature or to think Auto Loader only supports one type of data processing (either batch or streaming).

When answering questions about Auto Loader, remember its key features: automatic ingestion, support for both batch and streaming, and schema evolution.

answers]

- A. Historical lineage
- B. Read-optimized
- C. Raw Data
- D. De-normalised
- E. Normalised

Answer: B,D

Explanation:

Correct Answer

Options B and D are correct. The Gold Layer is read-optimized and contains de-normalized data.

Explanation of the Correct Answer

The Gold Layer is designed for quick data retrieval and analysis, making read optimization crucial. It also uses denormalization to improve query performance and simplify data access for business users.

The Gold Layer focuses on delivering data in a format that's easy to use for reporting and analytics. This often involves transforming the data into a star schema or other denormalized structure.

Analysis of Each Option

Option A: Incorrect. Historical lineage is a metadata concern across all layers, not specific to the Gold Layer.

Option B: Correct. The Gold Layer is optimized for read operations to ensure fast query performance.

Option C: Incorrect. Raw data is characteristic of the Bronze Layer.

Option D: Correct. Data in the Gold Layer is typically denormalized to optimize query performance.

Option E: Incorrect. Normalization is more characteristic of the Silver Layer.

Key Takeaways

The Gold Layer is the final layer in a data lakehouse, optimized for consumption.

Denormalization is a common technique used in the Gold Layer to improve query performance.

Understanding the purpose of each layer (Bronze, Silver, Gold) is crucial for answering these types of questions.

Question: 232

Deep Dumps

A data engineer is working in a Databricks notebook to design and manage a batch ETL pipeline. The engineer is writing SQL and Python code to clean data, transform it, and join large datasets from different sources. The engineer wants to organize these steps into a structured process that can be run regularly and scheduled as part of a data pipeline. Which Databricks notebook feature is applicable in the use case?

- A. Collaborative editing
- B. Task workflows and job scheduling
- C. Real-time streaming support
- D. Notebook version control

Answer: B

Explanation:

Correct Answer

Option B: Task workflows and job scheduling

Explanation of the Correct Answer

Option B is correct because "Task workflows and job scheduling" in Databricks allows users to organize notebook cells into structured workflows, schedule notebooks to run automatically, chain multiple notebooks together, monitor ETL processes, and set up dependencies between pipeline steps. This directly addresses the need to organize ETL steps, run them regularly, and schedule them

as part of a data pipeline.

Analysis of Each Option

Option A: Collaborative editing allows multiple users to work on the same notebook. While useful for development, it doesn't address the scheduling and automation requirements. Therefore, it's incorrect.

Option B: Task workflows and job scheduling allows organizing notebook cells into structured workflows, scheduling notebooks to run automatically, chaining multiple notebooks, monitoring ETL processes, and setting up dependencies. This directly addresses the requirements, making it correct.

Option C: Real-time streaming support is for processing continuous data streams, but the use case specifies a "batch ETL pipeline," not streaming data. Therefore, it's incorrect.

Option D: Notebook version control helps track changes and manage different versions of notebooks but doesn't provide scheduling or pipeline orchestration capabilities. Therefore, it's incorrect.

Key Takeaways

The key takeaway is to understand the specific features Databricks offers for different data engineering tasks.

A common mistake is to confuse collaborative editing or version control with job scheduling and workflow orchestration.

When faced with similar questions, focus on identifying the feature that directly addresses the requirements of scheduling, automation, and structuring data pipelines.

Question: 233

Deep Dumps

Which Databricks asset bundle format is valid?

A. resources: jobs: hello-job: name: hello-job tasks: _task_key: hello-task existing_cluster_id: 1234-567890-abcde123 notebook_task: notebook_path: ./hello.py

B. "resources": "jobs": "name": "hello-job", "tasks": "task_key: "hello-task", "existing_cluster_id": "1234-567890-abcde123", "notebook_task": "notebook_path": ".hello.py"

C. configuration = "resources": "jobs": "name": "hello-job", "tasks": "task key: "hello-task", "existing_cluster_id": "1234-567890-abcde123", "notebook_task": "notebook_path": ".hello.py"

D. resources (jobs name = "hello-job" tasks task_key = "hello-task" existing cluster_id = "1234-567890-abcde123" notebook_task notebook_path = ".hello.py"

Answer: A

Explanation:

Correct Answer

Option A: resources: jobs: hello-job: name: hello-job tasks: _task_key: hello-task existing_cluster_id: 1234-567890-abcde123 notebook_task: notebook_path: ./hello.py

Explanation of the Correct Answer

Option A is the correct answer because it follows the proper YAML syntax required for Databricks asset bundles. The structure is well-defined with correct indentation, key-value pairs, and a valid file path (./hello.py).

YAML (YAML Ain't Markup Language) is a human-readable data serialization language. Databricks asset bundles use YAML to define configurations.

Analysis of Each Option

Option A: This option is correct. It uses valid YAML syntax, including proper indentation and structure, key-value pairs, and a valid file path notation (./hello.py).

Option B: This option is incorrect. It has a missing closing quote in "task_key: "hello-task" and an invalid path .hello.py (should be ./hello.py).

Option C: This option is incorrect. It contains a space in the key name "task key: and a missing closing quote, as well as an invalid path format.

Option D: This option is incorrect. It uses parentheses instead of

colons/braces, has inconsistent syntax, and is not a valid JSON, YAML, or any standard configuration format.

Key Takeaways

Databricks asset bundles require a specific format, typically YAML.

Correct syntax, including indentation, key-value pairs, and valid file paths, is crucial.

Common mistakes include incorrect indentation, missing quotes, spaces in key names, and invalid file paths.

Always validate the YAML syntax to ensure it is correct.

Question: 234

Deep Dumps

A data engineer has inherited a Databricks pipeline from a previous team. The pipeline is missing SLAs and costs more than the allotted budget. On analysis, it is noted that the cluster is not being fully utilized, and the dataset is getting skewed. How should the data engineer resolve this issue?

- A. Use `coalesce()` on the dataset to merge partitions and reduce skew.
- B. Increase the number of executors for the job.
- C. Repartition the dataset to have it be more optimally spread across all nodes.
- D. Increase the executor memory for the job.

Answer: C

Explanation:

Correct Answer

Option C: Repartition the dataset to have it be more optimally spread across all nodes.

Explanation of the Correct Answer

The problem states that the Databricks cluster is underutilized and the dataset is skewed. This means some partitions have much more data than others, causing some executors to work harder while

others remain idle.

Option C, repartitioning the dataset, directly addresses the skew issue by ensuring balanced data distribution. This improves cluster utilization as all executors get equal work, reducing processing time and costs.

The provided thought process suggests repartitioning on a column that provides good data distribution, such as `df.repartition($"date")`.

Analysis of Each Option

Option A: `coalesce()` only reduces the number of partitions by merging existing ones and does NOT redistribute data evenly. It will not fix the underlying skew problem and may worsen utilization by reducing parallelism. Therefore, it is incorrect.

Option B: Adding more executors won't help if current ones are underutilized due to skew. It increases costs without addressing the root cause, contradicting the budget constraint. Therefore, it is incorrect.

Option C: `repartition()` redistributes data evenly across all partitions, directly addressing the skew issue by ensuring balanced data distribution. This improves cluster utilization as all executors get equal work, reducing processing time and costs. Therefore, it is correct.

Option D: Increasing executor memory addresses memory issues, not data distribution problems. It won't improve cluster utilization and increases costs. Therefore, it is incorrect.

Key Takeaways

Data skew can lead to underutilization of cluster resources and increased processing time.

`repartition()` is used to redistribute data evenly across partitions, while `coalesce()` is used to reduce the number of partitions.

Increasing the number of executors or executor memory does not address data skew.

Always identify the root cause of performance issues before applying solutions.

Question: 235

Deep Dumps

A data engineer is debugging a Python notebook in Databricks that processes a dataset using PySpark. The notebook fails with an error during a DataFrame transformation. The engineer wants to inspect the state of variables, such as the input DataFrame and intermediate results, to identify where the error occurs. Which tool should the engineer use to debug the notebook and inspect the values of variables like DataFrames?

- A. Use the Spark UI to analyze the execution plan and identify stages where the job failed
- B. Use the Ganglia UI to monitor cluster resource usage and identify hardware issues
- C. Use the Python Notebook Interactive Debugger to set breakpoints and inspect variable values in real-time
- D. Use the Databricks CLI to download and analyze driver logs for detailed error messages

Answer: C

Explanation:

Correct Answer

Option C: Use the Python Notebook Interactive Debugger to set breakpoints and inspect variable values in real-time

Explanation of the Correct Answer

The Python Notebook Interactive Debugger is the best tool because it allows you to pause the execution of your code, step through it line by line, and inspect the values of variables, including DataFrames, in real-time. This is crucial for understanding the state of your data and identifying the exact point where an error occurs during a DataFrame transformation.

The key concept here is interactive debugging, which provides a hands-on way to examine the code's behavior and data at various

stages.

Analysis of Each Option

Option A: Incorrect. The Spark UI is useful for analyzing the performance and execution plan of Spark jobs, but it doesn't allow you to inspect the values of variables or DataFrames directly. It shows how the job is executed but not the data within.

Option B: Incorrect. The Ganglia UI is for monitoring cluster resource usage, such as CPU and memory. It doesn't provide any insight into the code or data within the Spark application.

Option C: Correct. The Python Notebook Interactive Debugger is designed for this purpose. It allows you to set breakpoints, step through code, and inspect variable values, including DataFrame contents, in real-time.

Option D: Incorrect. The Databricks CLI and driver logs provide error messages and stack traces, which can help understand what went wrong. However, they don't allow for interactive inspection of variable states during execution.

Key Takeaways

Interactive debuggers are essential for understanding the state of variables during code execution.

Spark UI is for job performance, Ganglia UI is for cluster resource monitoring, and the Databricks CLI is for error logs.

When debugging, focus on tools that allow you to inspect variable values directly.

A Data Engineer is building a simple data pipeline using Delta Live Tables (DLT) in Databricks to ingest customer data. The raw customer data is stored in a cloud storage location in JSON format. The task is to

create a DLT pipeline that reads the raw JSON data and writes it into a Delta table for further processing. Which code snippet will correctly ingest the raw JSON data and create a Delta table using DLT?

- A. `import dlt@dlt.tabledef raw_customers():return spark.read.format("json").load("s3://my-bucket/raw-customers/")`
- B. `import dlt@dlt.tabledef raw_customers():return spark.read.format("delta").load("s3://my-bucket/raw-customers/")`
- C. `@dlt.tabledef raw_customers():return spark.read.json("s3://my-bucket/raw-customers/")`
- D. `import dlt@dlt.tabledef raw_customers():return spark.read.text("s3://my-bucket/raw-customers/")`

Answer: C

Explanation:

Explanation of the Correct Answer

The goal is to ingest raw customer data stored in JSON format from a cloud storage location and create a Delta table using Delta Live Tables (DLT). The correct approach involves using the `dlt` library to define a table and then leveraging Spark's `read.json()` method to parse the JSON data directly.

`import dlt`: This line is crucial for any DLT pipeline as it imports the necessary library.

`@dlt.table`: This decorator marks the function `raw_customers` as a DLT table. The function's return value (a PySpark `DataFrame`) will be materialized as a Delta table.

`spark.read.json("s3://my-bucket/raw-customers/")`: This is the most direct and idiomatic way in PySpark to read data from JSON files. The `json()` method automatically infers the schema from the JSON data and creates a `DataFrame`, which DLT then uses to create the Delta table.

Analysis of Each Option

Option A: ``import dlt`

`@dlt.table`

`def raw_customers():`

`return spark.read.format("json").load("s3://my-bucket/raw-customers/")` This option is technically correct and would work, as `format("json").load()` is a valid way to read JSON data.

However, `spark.read.json()` is a more concise and commonly used shortcut for the same operation.

Option B: ``import dlt`

`@dlt.table`

`def raw_customers():`

`return spark.read.format("delta").load("s3://my-bucket/raw-customers/")` This option is incorrect because the source data is in JSON format, not Delta format. Using `format("delta")` would attempt to read an existing Delta table, which is not the case here.

Option C: `@dlt.table def raw_customers(): return spark.read.json("s3://my-bucket/raw-customers/")`

This is the correct and most efficient way to read JSON data. The `spark.read.json()` method is specifically designed for parsing JSON files, automatically handling schema inference and data parsing.

Option D: ``import dlt`

`@dlt.table`

`def raw_customers():`

`return spark.read.text("s3://my-bucket/raw-customers/")` This option is incorrect. `spark.read.text()` reads each line of the file as a single string, without parsing it as JSON. This would result in a DataFrame with a single string column, not a structured representation of the JSON data.

Key Takeaways

Delta Live Tables (DLT) pipelines are defined using the `import dlt` statement and the `@dlt.table` decorator.

To ingest data from a specific format, use the appropriate `spark.read` method or `format().load()` combination.

For JSON data, `spark.read.json()` is the most direct and recommended method for parsing and loading the data into a `DataFrame`.

C. `@dlt.table`

```
def raw_customers():
```

```
    return spark.read.json("s3://my-bucket/raw-customers/")
```

Question: 237

Deep Dumps

An organization needs to share a dataset stored in its Databricks Unity Catalog with an external partner who uses a different data platform that is not Databricks. The goal is to maintain data security and ensure the partner can access the data efficiently. Which method should the data engineer use to securely share the dataset with the external partner?

- A. Using Delta Sharing with the open sharing protocol
- B. Exporting data as CSV files and emailing them
- C. Using a third-party API to access the Delta table
- D. Databricks-to-Databricks Sharing

Answer: A

Explanation:

Correct Answer

Option A: Using Delta Sharing with the open sharing protocol

Explanation of the Correct Answer

Delta Sharing with the open sharing protocol is the correct choice because it's designed for securely sharing data from Databricks Unity Catalog with external partners, even if they use different data platforms. It ensures secure, governed access without duplicating data.

This method leverages Databricks' open protocol, which is specifically built for cross-platform data sharing, maintaining data lineage and audit trails, and supporting real-time data access.

Analysis of Each Option

Option A: Correct. Delta Sharing is designed for secure, cross-platform data sharing without data duplication.

Option B: Incorrect. Exporting data as CSV files and emailing them is insecure, lacks access controls, and creates stale data copies.

Option C: Incorrect. Using a third-party API adds complexity, may not provide adequate security, and isn't a standard approach.

Option D: Incorrect. Databricks-to-Databricks Sharing only works when both parties use Databricks, which isn't the case here.

Key Takeaways

Delta Sharing is the preferred method for secure, cross-platform data sharing from Databricks.

Avoid insecure methods like emailing CSV files.

Always consider the security and governance implications when sharing data with external partners.

Question: 238

Deep Dumps

A data engineer team has decided to implement a new data platform on Databricks and is currently deciding how to store each kind of data on each data layer. What is the appropriate layer and data pairing for medallion architecture?

- A. Silver Layer - Cleansed master customer data
- B. Gold Layer - Deduplicated money transfer transaction
- C. Silver Layer - Raw data from deposit account application
- D. Bronze Layer - Summary of cash deposit amount for each country and city

Answer: A

Explanation:

Correct Answer

Option A: Silver Layer - Cleansed master customer data

Explanation of the Correct Answer

Option A is correct because the Silver Layer in medallion architecture is designed to store cleansed, validated, and enriched data. Cleansed master customer data fits this description perfectly as it has been processed from its raw form, validated, and standardized.

The key concept here is understanding the role of each layer in the medallion architecture: Bronze (raw data), Silver (cleansed and validated data), and Gold (aggregated and business-ready data).

Analysis of Each Option

Option A: This option is correct. Cleansed master customer data is exactly the type of data that should reside in the Silver Layer.

Option B: This option is incorrect. Deduplicated individual transactions are more suited for the Silver Layer, while the Gold Layer is reserved for aggregated, business-ready datasets.

Option C: This option is incorrect. Raw data should be stored in the Bronze Layer, not the Silver Layer. The Silver Layer contains processed and cleansed data.

Option D: This option is incorrect. Summarized data belongs in the Gold Layer, not the Bronze Layer, which is meant for raw, unprocessed data.

Key Takeaways

The key takeaway is to remember the purpose of each layer in the medallion architecture: Bronze for raw data, Silver for cleansed and validated data, and Gold for aggregated and business-ready data.

A common mistake is confusing the Silver and Gold Layers. Remember that Silver is for cleaned data, while Gold is for aggregated data ready for analysis.

When approaching similar questions, always consider the state of the data (raw, cleansed, or aggregated) and match it to the appropriate layer.

Question: 239

Deep Dumps

Why is Delta Live Tables (DLT) an appropriate choice?

- A. Automatic data quality checks, built-in support for schema evolution, and declarative pipeline development
- B. Manual schema enforcement, high operational overhead, and limited scalability
- C. Requires custom code for data quality checks, no support for streaming data, and complex pipeline maintenance
- D. Supports only batch processing, no data versioning, and high infrastructure costs

Answer: A

Explanation:

Correct Answer

Option A: Automatic data quality checks, built-in support for schema evolution, and declarative pipeline development

Explanation of the Correct Answer

Delta Live Tables (DLT) is a good choice because it automatically checks data quality, supports schema evolution, and allows you to build pipelines in a declarative way. This means you can focus on what you want to achieve with your data, rather than worrying about the details of how to process it. DLT helps ensure your data is reliable and your pipelines are easy to maintain.

Analysis of Each Option

Option A: This option is correct because it accurately describes the key benefits of using Delta Live Tables, such as automatic data quality checks, built-in support for schema evolution, and

declarative pipeline development.

Option B: This option is incorrect because it describes the opposite of what DLT offers. DLT aims to reduce manual work, lower overhead, and provide scalability.

Option C: This option is incorrect because DLT provides built-in data quality checks, supports streaming data, and simplifies pipeline maintenance.

Option D: This option is incorrect because DLT supports both batch and streaming processing, includes data versioning through Delta Lake, and optimizes infrastructure costs.

Key Takeaways

Delta Live Tables simplifies data pipeline development by automating data quality checks and managing schema evolution.

Declarative pipeline development allows users to focus on the desired outcome rather than the implementation details.

When evaluating data pipeline tools, consider factors like data quality, schema management, and ease of maintenance.

A common mistake is to assume that all data pipeline tools require manual data quality checks and complex maintenance.

Question: 240

Deep Dumps

A data engineer needs to develop integration tests for an ETL process and deploy a version-controlled, packaged workflow into production using an external job scheduler. Which tool should the data engineer use for this job?

- A. Databricks Connect
- B. Databricks Command Line Interface
- C. Databricks Asset Bundles
- D. Databricks Software Development Kit

Answer: C

Explanation:

Correct Answer

Option C: Databricks Asset Bundles

Explanation of the Correct Answer

Databricks Asset Bundles are the best choice because they are specifically designed for version control integration, packaging all workflow components into a single deployable unit, environment management, CI/CD pipeline integration, and testing support within the deployment pipeline. This aligns perfectly with the data engineer's needs to develop integration tests, deploy a version-controlled workflow, use an external job scheduler, and deploy into production.

Analysis of Each Option

Option A: Databricks Connect is primarily for local development and testing, not for production deployment and packaging. Therefore, it's not the correct choice.

Option B: Databricks Command Line Interface is useful for automation but doesn't offer a comprehensive deployment solution for packaged workflows. Thus, it's not the best option.

Option C: Databricks Asset Bundles provide a complete solution for modern DataOps practices, combining version control, testing, packaging, and deployment in a single framework that integrates well with external schedulers. This makes it the correct choice.

Option D: Databricks Software Development Kit provides programmatic access to Databricks APIs but doesn't offer the integrated packaging and deployment capabilities needed for this use case. Hence, it's not the optimal solution.

Key Takeaways

Databricks Asset Bundles are designed for managing and deploying Databricks workflows in a structured and repeatable manner.

When choosing a tool for deploying workflows, consider the need for version control, packaging, environment management, and integration with external schedulers.

Avoid using tools primarily intended for local development (like Databricks Connect) for production deployments.

Question: 241

Deep Dumps

A data engineer is attempting to write Python and SQL in the same command cell and is running into an error. The engineer thought that it was possible to use a Python variable in a select statement. Why does the command fail?

- A. Databricks supports multiple languages but only one per notebook.
- B. Databricks supports language interoperability but only if a special character is used.
- C. Databricks supports one language per cell.
- D. Databricks supports language interoperability in the same cell but only between Scala and SQL.

Answer: C

Explanation:

Correct Answer

Option C: Databricks supports one language per cell.

Explanation of the Correct Answer

The correct answer is that Databricks supports one language per cell. This means you can't directly mix Python and SQL code in the same cell without using specific techniques to separate them.

The thought process explains that each cell in a Databricks notebook is designed to execute code in a single language. To use multiple languages, you need to either use separate cells or utilize language-specific "magic commands" to switch between languages within different cells.

The thought process provides examples of how to correctly use Python variables in SQL queries by either using separate cells with the `%sql` magic command or by using Python's string formatting capabilities within a Python cell.

Analysis of Each Option

Option A: Incorrect. Databricks does support multiple languages within a single notebook, but not within the same cell without proper separation.

Option B: Incorrect. While special characters are used in some contexts (like variable substitution), the fundamental issue is the one-language-per-cell rule.

Option C: Correct. Databricks enforces a one-language-per-cell rule, which is why the engineer's attempt to mix Python and SQL in the same cell failed.

Option D: Incorrect. Databricks supports language interoperability between more than just Scala and SQL. It also supports Python and R. The core issue is still the one-language-per-cell rule.

Key Takeaways

Databricks notebooks are designed to execute code in a single language per cell.

To use multiple languages, you must either use separate cells or utilize language-specific "magic commands" (e.g., `%sql`, `%python`).

When working with multiple languages, be mindful of how variables are passed between them. Techniques like string formatting or magic commands with variable substitution can be used.

A common mistake is to assume that you can freely mix languages within a single cell without any special handling.

A data engineer needs to ingest from both streaming and batch sources for a firm that relies on highly accurate data. Occasionally, some of the data picked up by the sensors that provide a streaming input are outside the expected parameters. If this occurs, the data must be dropped, but the stream should not fail. Which feature of Delta Live Tables meets this requirement?

- A. Error Handling
- B. Expectations
- C. Monitoring
- D. Change Data Capture

Answer: B

Explanation:

Correct Answer

Option B: Expectations

Expectations in Delta Live Tables allow defining data quality constraints and specifying actions like dropping rows when data violates these constraints, ensuring the stream continues processing without failing.

Explanation of the Correct Answer

The question asks about a Delta Live Tables feature that allows dropping data outside expected parameters from a streaming source without failing the stream. Expectations in Delta Live Tables are designed for this purpose. They allow defining data quality constraints and specifying actions like DROP ROW when data violates these constraints, ensuring the stream continues processing.

The key concept here is data quality management within a streaming pipeline. Delta Live Tables Expectations provide a mechanism to enforce data quality rules and handle violations gracefully.

The provided SQL example demonstrates how to define expectations with the DROP ROW action, which directly addresses the problem requirement.

Analysis of Each Option

Option A: Error Handling - While important, it's a general concept and not a specific feature for data validation and dropping rows based on defined rules.

Option B: Expectations - This is the correct answer because it allows defining data quality rules and specifying actions like DROP ROW when data violates these rules, ensuring the stream continues.

Option C: Monitoring - This provides insights into the data pipeline but doesn't actively filter or drop invalid data.

Option D: Change Data Capture - This is for tracking changes in data sources, not for data validation or quality control.

Key Takeaways

Delta Live Tables Expectations are crucial for data quality management in streaming pipelines.

The ON VIOLATION clause allows specifying actions like DROP ROW, WARN, or FAIL based on data quality checks.

Understanding the specific features of Delta Live Tables is essential for choosing the right tool for data processing tasks.

A common mistake is confusing general error handling with specific data quality features like Expectations.

Question: 243

Deep Dumps

What are the transformations typically included in building the Bronze layer?

A. Perform extensive data cleansing

- B. Include columns Load date/time, process ID
- C. Business rules and transformations
- D. Aggregate data from multiple sources

Answer: B

Explanation:

Correct Answer

Option B: Include columns Load date/time, process ID

Explanation of the Correct Answer

The Bronze layer in data architecture is the initial landing zone for raw data. The primary goal is to ingest data with minimal transformation, focusing on preserving the original data and adding metadata for tracking and auditing. Including columns like Load date/time and process ID helps in maintaining data lineage and provides an audit trail without altering the original data's content.

The key concept here is the medallion architecture (Bronze-Silver-Gold), where each layer has a specific purpose. The Bronze layer's purpose is to store raw data with minimal changes.

Analysis of Each Option

Option A: Incorrect. Extensive data cleansing is performed in the Silver layer, not the Bronze layer. The Bronze layer preserves raw data without cleansing.

Option B: Correct. Including columns like Load date/time and process ID is a typical transformation in the Bronze layer. This provides an audit trail and data lineage without altering the source data.

Option C: Incorrect. Applying business rules and transformations is typically done in the Silver or Gold layers, not the Bronze layer. The Bronze layer avoids applying business logic.

Option D: Incorrect. Aggregating data from multiple sources is

usually a Gold layer transformation. The Bronze layer maintains source-level granularity.

Key Takeaways

The Bronze layer is for raw data ingestion with minimal transformations.

Common mistakes include applying business logic or performing extensive data cleansing in the Bronze layer.

Remember to focus on preserving the original data and adding metadata for tracking and auditing in the Bronze layer.

Question: 244

Deep Dumps

A Python file is ready to go into production and the client wants to use the cheapest but most efficient type of cluster possible. The workload is quite small, only processing 10GBs of data with only simple joins and no complex aggregations or wide transformations. Which cluster meets the requirement?

- A. Interactive cluster
- B. Job cluster with spot instances disabled
- C. Job cluster with Photon enabled
- D. Job cluster with spot instances enabled

Answer: D

Explanation:

Correct Answer

Option D: Job cluster with spot instances enabled

Explanation of the Correct Answer

Option D is the correct answer because it offers the most cost-effective solution for the given scenario. Spot instances are significantly cheaper than on-demand instances, making them ideal for cost-sensitive production environments.

The workload is small (10GB) and involves only simple joins, meaning the risk of interruption due to spot instance reclamation is manageable. The job cluster will spin up, process the data, and terminate, optimizing resource utilization and cost.

Analysis of Each Option

Option A: Incorrect. Interactive clusters are designed for development and exploration, not for production workloads. They run continuously, making them more expensive than job clusters.

Option B: Incorrect. Job clusters with spot instances disabled use on-demand instances, which are more reliable but also more expensive. This option does not meet the requirement of being the cheapest.

Option C: Incorrect. Photon is beneficial for complex queries and large datasets, adding computational overhead and cost. It is overkill for simple joins on 10GB of data and therefore not the most efficient or cheapest option.

Option D: Correct. Job clusters with spot instances enabled are the cheapest option because spot instances cost significantly less than on-demand instances. They are efficient for this use case as they spin up, process data, and terminate. The risk of interruption is manageable due to the small data size and simple processing requirements.

Key Takeaways

Spot instances are a cost-effective solution for production workloads, especially when cost optimization is a priority.

Job clusters are suitable for batch processing tasks, optimizing resource utilization and cost.

Consider the workload characteristics (data size, complexity) when choosing a cluster type and instance type.

For small, simple workloads, the risk of interruption with spot

instances is often manageable.

Question: 245

Deep Dumps

Which compute option should be chosen in a scenario where small-scale ad-hoc Python scripts need to be run at high frequency and should wind down quickly after these queries have finished running?

- A. All-purpose Cluster
- B. Job Cluster
- C. Serverless Compute
- D. SQL Warehouse

Answer: C

Explanation:

Correct Answer

Option C: Serverless Compute

Explanation of the Correct Answer

Serverless compute is the best option because it automatically scales resources when needed and releases them immediately after tasks are completed. This eliminates the need for manual cluster management and ensures you only pay for the actual compute time used, making it cost-effective for high-frequency, short-duration tasks. Additionally, serverless compute often has fast startup times due to pre-warmed environments, which is ideal for ad-hoc script execution.

Analysis of Each Option

Option A: All-purpose clusters remain running continuously, incurring costs even during idle periods. They require manual termination to avoid unnecessary charges, making them less cost-effective for high-frequency, short-duration tasks.

Option B: Job clusters create a new cluster for each job, leading to longer startup times and higher overhead for frequent executions.

This is less efficient than serverless compute for this use case.

Option C: This is the correct answer, as explained above.

Option D: SQL Warehouses are optimized for SQL workloads, not Python scripts. They are designed for analytical queries rather than general Python execution, making them inappropriate for this scenario.

Key Takeaways

Serverless compute is ideal for scenarios requiring automatic scaling, cost efficiency, and quick wind-down after task completion.

Consider the specific requirements of your workload (e.g., frequency, duration, type of tasks) when choosing a compute option.

Avoid options that incur costs during idle periods or have long startup times if your tasks are short and frequent.

Question: 246

Deep Dumps

A data engineer is developing a small proof of concept in a notebook. When running the entire notebook, the Cluster usage spikes. The data engineer wants to keep the development requirements and get real-time results. Which Cluster meets these requirements?

- A. All-Purpose Cluster with a large fixed memory size
- B. Job cluster with Photon enabled and autoscaling
- C. Job Cluster with autoscaling enabled
- D. All Purpose Cluster with autoscaling

Answer: D

Explanation:

Correct Answer

Option D: All Purpose Cluster with autoscaling

Explanation of the Correct Answer

The correct answer is D because an All-Purpose Cluster is suitable

for interactive development in a notebook, and autoscaling dynamically adjusts the cluster size based on the workload, preventing excessive resource consumption and mitigating usage spikes.

The engineer needs an interactive environment for development and testing, and real-time results, implying the need for an active cluster that can respond quickly to queries and code execution. The solution should efficiently manage resources to prevent unnecessary spikes.

Analysis of Each Option

Option A: This option is incorrect because a large fixed memory size can be wasteful if the cluster is not constantly utilizing all the resources. It doesn't address the spike issue; it might even exacerbate it by providing more resources than needed at times.

Option B: This option is incorrect because Job clusters are primarily designed for running automated, non-interactive jobs. They are not ideal for interactive development in a notebook.

Option C: This option is incorrect because Job clusters are not designed for interactive development.

Option D: This option is correct because it provides an interactive environment for development while efficiently managing resources to prevent usage spikes. Autoscaling can introduce a slight delay when scaling up, but this is generally acceptable for development purposes.

Key Takeaways

All-Purpose Clusters are suitable for interactive development, while Job Clusters are designed for automated, non-interactive jobs.

Autoscaling dynamically adjusts the cluster size based on the workload, preventing excessive resource consumption and mitigating usage spikes.

A fixed memory size can be wasteful if the cluster is not constantly utilizing all the resources.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured
100% Pass Guarantee



24/7 Customer Support
Mail us - support@deepdumps.com



Verified Updates
Lifetime Updates!

Total: **246 Questions**

Link: <https://deepdumps.com/papers/databricks/certified-data-engineer-associate>